Qwen Forecast
Código, arquitetura e inferência
Guia completo comentado

Edição de 6 de outubro de 2026. Leitura linha a linha do projeto atual, com foco na construção de prompts e na inferência local pelo Ollama.

Inclui módulos de dados, três adaptadores de modelos, métricas, CLIs, configurações, scripts, testes e auditoria.

A documentação preserva o comportamento atual e evidencia limitações. Não houve alteração do pipeline ou de resultados para produzir este livro.

Retrato Git: 4dc6d82. Fontes atuais também incluem a auditoria local; hashes individuais constam no manifesto acompanhante.

Sumário navegável

Qwen Forecast — Código, arquitetura e inferência — Guia completo comentado 1

1. Como usar este livro 4

2. Arquitetura: camadas, responsabilidades e padrões 5

3. Mapa de chamadas: de qwen_forecast ao CSV 6

4. Uma janela concreta, sem saltar etapas 7

5. Python aplicado: argumentos, objetos e coleções 8

6. Ollama: onde cada parâmetro age 9

7. Artefatos, reprodutibilidade e limites atuais 10

8. Mini-exemplo: objetos, texto, HTTP e avaliação 11

9. Inventário e dependências entre módulos 12

10. pyproject.toml 14

11. experiments/start_ollama_context_server.sh 15

12. experiments/run_qwen_context_comparison.sh 16

13. experiments/run_benchmark.sh 17

14. configs/benchmark.json 18

15. configs/qwen_context_15_days.json 23

16. configs/qwen_context_expanding.json 28

17. src/pipelines/forecasting_pipeline.py 33

18. src/pipelines/data_pipeline.py 46

19. src/data/ingestion/files.py 48

20. src/data/validation/schema.py 50

21. src/data/preprocessing/resampling.py 53

22. src/data/preprocessing/missing_values.py 54

23. src/evaluation/cross_validation.py 56

24. src/data/features/temporal.py 59

25. src/data/features/covariates.py 60

26. src/data/preprocessing/generate_prompts.py 64

27. src/models/promptcast.py 76

28. src/models/chronos.py 84

29. src/models/timesfm.py 88

30. src/evaluation/metrics.py 92

31. src/pipelines/compare_models.py 95

32. src/pipelines/compare_qwen_contexts.py 102

33. src/training/train.py 111

34. tests/test_forecasting.py 112

35. tests/test_qwen_context_comparison.py 128

36. tests/test_seven_day_metrics.py 135

37. experiments/qwen_context_comparison/audit/audit_run.py 140

Catálogo de funções e métodos de biblioteca 154

Como executar e localizar o resultado 173

Cobertura, fontes e reprodução do documento 174

1. Como usar este livro

Este guia documenta o código existente, não uma arquitetura idealizada. Cada arquivo executável do pipeline, cada teste, os scripts de execução, as configurações e a auditoria são percorridos com numeração de linha. Imports e separadores recebem comentários curtos; as operações que transformam dados, montam prompts, fazem HTTP ou calculam métricas recebem explicações específicas. Os parâmetros apresentados são os usados no projeto: isto não é uma reprodução de todos os parâmetros possíveis de cada biblioteca.

O percurso recomendado é: arquitetura → rastreamento de uma previsão → conceitos de Python → arquivos em ordem de execução → referências das bibliotecas → testes e auditoria. O sumário e os marcadores do PDF permitem abrir diretamente um arquivo. O HTML acompanhante permite procurar nomes com facilidade. Os números de linha e hashes pertencem ao retrato de código indicado no manifesto; se o código mudar, uma linha pode mudar de posição.

Arquivos CSV de dados e milhares de respostas JSON são artefatos, não programas: o guia explica seu esquema e exemplifica sua relação com o código, sem reproduzir todo o dataset. Arquivos vazios __init__.py são listados individualmente. Metadados gerados por instalação, .git, ambientes conda e o próprio gerador deste livro não fazem parte do pipeline de previsão.

2. Arquitetura: camadas, responsabilidades e padrões

A árvore src organiza responsabilidades: ingestion cuida de I/O; validation verifica o contrato dos dados; preprocessing constrói a grade diária e os textos; features separa covariáveis; evaluation define janelas e métricas; models adapta as bibliotecas; pipelines coordena a execução. Configurações ficam em configs; entradas em data/processed; saídas auditáveis em experiments; regressões em tests.

Esse desenho é uma arquitetura em camadas e um pipeline procedural. Não é correto chamar a árvore de MVC ou Clean Architecture completa: não há controladores web, repositórios abstratos nem injeção de dependência por framework. Há separação de responsabilidades e dependências explícitas entre módulos.

create_model funciona como uma fábrica simples: escolhe um adaptador pelo nome qwen, chronos ou timesfm. QwenForecaster, ChronosForecaster e TimesFMForecaster exercem o papel de Adapter: expõem predict, identity e last_audit, escondendo HTTP, predict_df ou forecast_with_covariates. É uma interface por convenção, ou duck typing; não existe classe abstrata obrigando esse contrato.

A escolha de strategy em select_demonstrations é uma estratégia implementada por condicionais, sem hierarquia formal de classes. As dataclasses ForecastWindow, Demonstration e OllamaConfig são objetos de dados com significado explícito. O cache por hash é um mecanismo de reprodutibilidade; não é treinamento nem cache de aprendizado do modelo.

A modularização evita que uma rotina de leitura precise conhecer Ollama, ou que a fórmula do RMSE precise abrir CSV. Em contrapartida, forecasting_pipeline importa vários serviços e conhece o esquema inteiro: é o ponto de orquestração. compare_models importa ROOT desse pipeline; há acoplamento a essa constante. Não existe um módulo de domínio independente de pandas.

api.py, database.py, cleaning.py e moirai.py foram sugeridos na árvore inicial, mas não existem como implementações ativas no retrato atual. Não inventamos chamadas para módulos inexistentes. training/train.py existe como suporte à divisão treino/validação, mas não é chamado no benchmark zero-shot.

3. Mapa de chamadas: de qwen_forecast ao CSV

pyproject.toml registra qwen_forecast = src.pipelines.forecasting_pipeline:main. O comando instalado chama main() desse módulo. Executar python -m src.pipelines.forecasting_pipeline percorre o mesmo módulo e entra no bloco if __name__ == '__main__'. Não há run_qwen.py ativo. O antigo run_qwen_forecasts.py não é o ponto de entrada atual.

experiments/run_benchmark.sh → Python do ambiente promptcast → forecasting_pipeline.main(--model qwen) → run_dataset. Em seguida o shell executa Chronos no ambiente chronos, TimesFM no ambiente timesfm e compare_models. Já run_qwen_context_comparison.sh percorre 15_days e expanding no mesmo checkpoint Qwen3 4B e termina em compare_qwen_contexts. start_ollama_context_server.sh é iniciado separadamente: ele abre o servidor local; não calcula métricas.

main → ArgumentParser/parse_args → JSON de configuração → loop dos datasets → run_dataset. run_dataset → prepare_dataset → read_series → pandas.read_csv → validate_series. Ao voltar: daily_grid → fill_past_only → build_windows. validate_covariates verifica o contrato das colunas.

run_dataset → create_model → OllamaConfig → QwenForecaster.__init__. O construtor consulta /api/tags e /api/show para confirmar instalação, digest e contexto declarado; não faz ainda uma previsão. A identidade resultante participa da procedência e dos hashes de cache.

Para cada ForecastWindow: model_inputs separa history/future; select_demonstrations retorna [] no modo zero; build_tabular_prompt ou build_prompt constrói uma string. O pipeline grava essa string e calcula request_id. Se uma resposta ok tiver o mesmo hash, reaproveita-a. Caso contrário, QwenForecaster.predict → prediction_schema → post_json → urllib.request.Request → urlopen → Ollama /api/generate.

A resposta HTTP contém metadados e uma string response. parse_prediction converte apenas essa string em lista de números, após validar chave, tipo, quantidade, finitude e sinal. O pipeline grava resposta bruta e duração. Só depois seleciona truth = df.iloc[window.target] e emparelha cada número com uma data/lead.

Ao terminar cada dataset: predictions.csv e status.json. O CLI de inferência não calcula automaticamente métricas agregadas: o shell chama um comparador depois. compare_qwen_contexts valida as duas configurações e chama deterministic_metrics; compare_models compara os três modelos do benchmark antigo. A comparação simples com seis colunas foi exportada pelo driver de retomada, que era temporário, e não é uma saída nativa do CLI geral.

```
qwen_forecast (pyproject.toml)
  -> forecasting_pipeline.main()
     -> run_dataset()
        -> prepare_dataset()
           -> read_series() -> pd.read_csv() -> validate_series()
           -> daily_grid() -> fill_past_only() -> build_windows()
        -> create_model() -> OllamaConfig -> QwenForecaster
           -> /api/tags + /api/show
        -> para cada janela:
           model_inputs() -> history + future SEM alvo
           build_tabular_prompt() -> string -> prompt.txt
           hash -> cache valido OU predict()
           predict() -> post_json() -> /api/generate
           parse_prediction() -> sete floats
           zip(truth, predictions) -> registros por lead
        -> predictions.csv + status.json
  -> comparador (invocado separadamente pelo shell)
     -> deterministic_metrics() -> CSVs de metricas
```

4. Uma janela concreta, sem saltar etapas

Em Joinville, a primeira origem é o índice 365: history = df.iloc[0:365], alvo = df.iloc[365:372]. O intervalo da direita é exclusivo: existem 365 observações históricas e sete futuras. As datas históricas vão de 17/06/2023 a 15/06/2024; a primeira previsão é para 16/06/2024. Na origem seguinte, 372, o histórico expansivo passa a df.iloc[0:372]. O começo não muda.

Com qwen_history_days=15, a origem não muda: apenas history vira df.iloc[350:365] na primeira janela. future continua nos mesmos sete dias. Com null no JSON, json.loads produz None e o código não corta o início. context do dataset escolhe a primeira origem; qwen_history_days escolhe quanto desse passado o Qwen recebe; num_ctx é capacidade de tokens do Ollama. São três grandezas distintas.

Em Guaratuba, 730 dias de calendário levam à primeira origem em 31/12/2017. O CSV não contém 29/02/2016: daily_grid materializa o dia. O preenchimento usa passado, mas a auditoria encontrou um problema: fill_past_only também copia atributos de calendário. Esse defeito está documentado nas anotações e não foi corrigido silenciosamente para escrever o guia.

O prompt histórico contém timestamp, water_produced e covariáveis. O futuro contém timestamp e covariáveis, sem water_produced. O usuário confirmou que as covariáveis meteorológicas futuras são previsões disponíveis na origem. A validação estrutural impede demanda futura explícita; a disponibilidade temporal externa dos dados não pode ser deduzida apenas de nomes de colunas.

No walk-forward, alvos de uma janela antiga tornam-se passado observado nas seguintes. Isso é próprio da avaliação expansiva solicitada, não acesso antecipado aos sete alvos da janela atual. Não há ajuste dos pesos do Qwen/Chronos/TimesFM; TimesFM com xreg ajusta regressão auxiliar sobre histórico. A política de 20% para validação está implementada como divisão cronológica, sem busca de hiperparâmetros nesta execução.

5. Python aplicado: argumentos, objetos e coleções

Parâmetro é o nome declarado na assinatura; argumento é o valor fornecido ao chamar. Em model_inputs(df, window, dataset, *, history_days=None), os três primeiros podem ser posicionais; o asterisco isolado obriga history_days a ser nomeado. Ele não coleta argumentos. Já *args numa assinatura coletaria posicionais em uma tupla, e **kwargs coletaria nomeados num dicionário. Nenhuma dessas duas variáveis mágicas é necessária para o padrão funcionar.

O código usa **config['qwen'] na chamada de OllamaConfig: é desempacotamento de um dicionário em argumentos nomeados. {'model':'qwen3:4b...', 'num_ctx':131072} equivale a model='qwen3:4b...', num_ctx=131072. Chave desconhecida ou duplicada com seed=... gera TypeError. **score dentro de dict(...) mescla campos de métricas. Nos mocks dos testes, call.args e call.kwargs são os argumentos capturados, não variáveis do programa real.

args = parser.parse_args() é apenas uma variável que recebe argparse.Namespace: args.config e args.model são atributos; não é *args. Anotações como list[float], str | Path e -> dict descrevem intenção e ajudam ferramentas; Python não valida automaticamente todos esses tipos.

@dataclass(frozen=True) gera __init__, representação e igualdade com base nos campos, e bloqueia reatribuição usual dos campos após construir. __post_init__ executa validações adicionais após o __init__ gerado. frozen não torna recursivamente imutáveis quaisquer objetos internos; aqui os campos centrais são escalares e tuplas. asdict transforma os campos em um dicionário serializável.

@property permite escrever window.history em vez de window.history(). O método calcula e retorna slice, sem criar uma cópia de dados sozinho. self é a instância: self.config persiste entre chamadas, enquanto config local é uma referência para esse objeto. __init__ inicializa a instância. object.__new__ nos testes cria a instância sem executar o construtor que faria I/O ou carregaria pesos.

DataFrame guarda tabela com linhas e colunas; Series representa uma coluna; ndarray é um vetor NumPy. df.iloc usa posições inteiras; df.loc usa rótulos/máscaras. df.iloc[a:b] inclui a e exclui b. Já df.loc[20:, ...] num índice inteiro inclui o rótulo 20. df[['timestamp', 'water_produced']] preserva uma tabela; df['water_produced'] retorna Series. copy evita modificar involuntariamente o objeto de origem.

Compreensões como [c for c in columns if ...] constroem listas filtradas; {c: valor for c in columns} constrói dicionário. any(...) para ao encontrar verdade; all(...) exige todas as condições. Em arrays, & e | combinam máscaras elemento a elemento; and/or combinam condições Python escalares. ~ inverte máscara booleana. Comparações vetoriais não devem ser usadas como um único bool sem redução.

with fecha arquivos/conexões ao sair, inclusive com erro. try/except captura falhas previstas. raise sem argumento relança a exceção corrente; return encerra a função; continue passa à próxima janela. O pipeline captura falhas de inferência para auditá-las e segue, mas um comparador rejeita execução incompleta. time.monotonic mede intervalo sem depender do relógio civil.

Tuplas permitem retorno múltiplo: history, future = model_inputs(...). enumerate(..., 1) numera leads a partir de um. zip emparelha as linhas verdadeiras e previsões; sozinho truncaria ao menor comprimento, por isso o código valida o tamanho antes. O nome _ recebe o índice de iterrows que não será usado.

F-strings avaliam expressões entre chaves: :.2f fixa duas casas, e %.6g na serialização pandas usa seis algarismos significativos. Chaves duplicadas {{ e }} representam chaves literais dentro de f-string. '\n'.join(parts) une blocos com quebras reais; não transmite barras e n como texto literal. JSON é formato de intercâmbio; um dict é o objeto Python em memória.

6. Ollama: onde cada parâmetro age

Ollama é um servidor local separado do processo Python. O ambiente conda fornece pandas/NumPy e o cliente HTTP padrão; o modelo não é carregado pela biblioteca transformers do cliente. base_url aponta para 127.0.0.1:11435 na comparação 4B; 11434 é o padrão antigo. O servidor precisa ter o modelo instalado. O script de previsão não baixa automaticamente o checkpoint.

model escolhe a variante instalada; temperature=0 reduz a aleatoriedade de amostragem; seed=42 fixa uma semente, mas o replay da auditoria mostrou que execução sequencial e isolada não necessariamente produzem números idênticos. num_ctx=131072 limita o contexto em tokens; não dias. num_predict=256 limita tokens gerados, não sete valores. Sete números em JSON consomem vários tokens cada.

timeout=7200 pertence à espera HTTP do cliente, em segundos; retries=1 significa até duas tentativas. keep_alive='10m' pede retenção do modelo carregado após a requisição; não é timeout. stream=False pede uma resposta JSON completa em vez de fluxo de partes. think=False é enviado para desativar o modo de raciocínio onde aplicável. format recebe JSON Schema para orientar a estrutura; parse_prediction continua necessário para verificar a resposta recebida.

truncate=False é enviado como intenção de não cortar entrada; não se deve tratar um campo enviado como prova de que toda versão do servidor o aplica. Há também checagem conservadora de tamanho antes do HTTP, inspeção dos contadores na resposta e registros de runtime na auditoria. Essa implementação não tokeniza exatamente no cliente.

OLLAMA_FLASH_ATTENTION, OLLAMA_KV_CACHE_TYPE=q4_0, OLLAMA_NUM_PARALLEL=1 e LLAMA_ARG_CACHE_RAM=0 são variáveis do processo servidor. Não são parâmetros de pandas ou de OllamaConfig. O cache KV guarda estados da atenção; os arquivos responses/*.json guardam resultados do experimento. Desabilitar snapshots extras não desabilita toda reutilização de prefixo do modelo.

O schema exige objeto com predictions, array de sete números não negativos e nenhuma chave adicional. O envelope HTTP tem response como string: primeiro json.load lê o envelope; depois json.loads lê o JSON gerado pelo modelo. done/done_reason verificam término; prompt_eval_count/eval_count ajudam a auditar limite. O parser rejeita bool explicitamente, pois bool é subtipo de int em Python.

7. Artefatos, reprodutibilidade e limites atuais

Cada pasta <execução>/<dataset>/<modelo> guarda run.json (configuração, modelo, hashes, política temporal), responses/<origem>.json (pedido, resposta, auditoria e números), predictions.csv (uma linha por data prevista) e status.json (contagens e falhas). Os prompts ficam em <execução>/<dataset>/prompts. A previsão de uma janela é armazenada antes de gerar o CSV final do dataset.

predictions.csv relaciona origin (data de início da previsão), timestamp (data-alvo), lead (1 a 7), y_true, y_pred, initial_context (origem inicial da avaliação), context (linhas efetivamente vistas), history_start, horizon, duração e run_id. Contexto inicial da avaliação não é sempre o contexto fornecido ao Qwen: pode haver initial_context=365 e context=15.

Os hashes verificam compatibilidade, não provam ausência de leakage. source_fingerprint percorre todo src/*.py: modificar qualquer fonte invalida caches mesmo que o arquivo não participe daquela execução. Escrever documentação fora de src preserva os hashes. O hash atual não inclui a versão efetiva do servidor por janela nem todos os scripts externos; runtime no JSON é configuração declarada.

As métricas juntam todos os pares dos sete leads e só então calculam RMSE. Não calculam a média dos RMSEs por janela. R^2 pode ser negativo; MAPE usa apenas alvos diferentes de zero e retorna porcentagem. sMAPE trata o caso ambos zero como contribuição zero. O comparador por lead usa os mesmos dados agrupados pela antecedência.

Limitações reais: compare_models assume contexto fixo do Qwen e rotula last 15 days; não é o comparador correto para Qwen expansivo nem uma configuração de sete dias. compare_qwen_contexts foi escrito especificamente para 15_days versus expanding. Não existe configuração pronta de sete dias no retrato, embora AGENTS.md a solicite; não confunda suporte de model_inputs(history_days=7) com uma execução feita.

training_partition é suporte não utilizado na inferência. O pipeline não faz treinamento do Qwen. fill_past_only precisa de tratamento específico para calendário em dias inseridos. Dados meteorológicos foram confirmados pelo usuário como previsões disponíveis na origem. A comparação simples de seis colunas por cidade existe nos resultados da retomada, mas os comparadores permanentes ainda exportam também metadados adicionais.

Este guia é documentação, não alteração de comportamento. Não corrige o bug de calendário, não modifica prompts, não reexecuta benchmarks e não muda as métricas publicadas.

8. Mini-exemplo: objetos, texto, HTTP e avaliação

Exemplo didático reduzido: suponha uma história de dois dias, com demandas 100 e 110, covariável temperature de 20 e 21, e feriado igual a zero. O horizonte fictício deste exemplo é dois, apenas para mostrar a transformação; os experimentos reais usam sete.

model_inputs entrega history com timestamp, water_produced, temperature e holiday. future contém as próximas duas datas e apenas temperature/holiday. Não contém os dois valores reais de demanda que serão usados depois. DataFrame é objeto tabular; build_tabular_prompt o converte em uma única string. Serializar não treina nem ajusta parâmetros do modelo.

Na cópia frame, map(lambda value: f'{float(value):.2f}') transforma 100 em '100.00' e 110 em '110.00'. to_csv(index=False, float_format='%.6g') retorna texto começando por timestamp,water_produced,temperature,holiday, seguido de uma linha por dia. index=False evita um contador 0,1 virar coluna extra. strip remove só o espaço/quebra das bordas. join junta esse CSV com as instruções, delimitadores e tabela futura.

prediction_schema(2) devolve um dict descrevendo minItems=2/maxItems=2. Ele vai no campo format, não dentro de predictions. payload inclui prompt, model e options. json.dumps(payload) transforma o dict em texto JSON; encode('utf-8') transforma esse texto em bytes para a rede. urllib.request.Request monta o POST e urlopen efetivamente envia.

Suponha que o envelope recebido contenha response='{"predictions":[112,114]}', done=True e os contadores de tokens. json.load(response_HTTP) lê o envelope. parse_prediction aplica json.loads na string interna, encontra a lista de dois números, testa finitude e devolve [112.0,114.0]. Ou seja, existem duas decodificações JSON em níveis diferentes: transporte e saída do LLM.

Somente depois o pipeline consulta a demanda real das duas datas: suponha [111,117]. zip associa 112 a 111 e 114 a 117; enumerate(...,1) atribui leads 1 e 2. Os erros são +1 e -3; MAE=(1+3)/2=2; RMSE=sqrt((1+9)/2)=sqrt(5), aproximadamente 2,236. O modelo não recebeu os valores 111 e 117 para produzir a previsão.

Se uma previsão vier como string '112', parse_prediction rejeita em vez de convertê-la permissivamente. Se vierem só um número, rejeita antes do zip. Se a resposta original completa for guardada com request_id válido, uma retomada pode reutilizá-la sem novo HTTP. Esses passos conectam biblioteca, contrato de dados e proteção temporal em um mesmo fluxo.

9. Inventário e dependências entre módulos

pyproject.toml — 18 linhas. Configuração declarativa.

experiments/start_ollama_context_server.sh — 8 linhas. Script de execução.

experiments/run_qwen_context_comparison.sh — 12 linhas. Script de execução.

experiments/run_benchmark.sh — 11 linhas. Script de execução.

configs/benchmark.json — 59 linhas. Configuração declarativa.

configs/qwen_context_15_days.json — 66 linhas. Configuração declarativa.

configs/qwen_context_expanding.json — 66 linhas. Configuração declarativa.

src/pipelines/forecasting_pipeline.py — 142 linhas. Orquestrador principal e CLI. Conecta dados, fábrica de modelos, prompt, cache, inferência e gravação. Executa modelos; não produz sozinho os CSVs agregados de métricas.

src/pipelines/data_pipeline.py — 11 linhas. Composição de leitura, grade diária, preenchimento e janelas. Chamado por run_dataset, testes e auditoria. Retorna o par (DataFrame, lista de ForecastWindow).

src/data/ingestion/files.py — 18 linhas. Entrada/saída de arquivos. Chamado por data_pipeline e forecasting_pipeline. read_series retorna DataFrame validado; write_json grava objetos serializáveis por substituição de arquivo temporário.

src/data/validation/schema.py — 21 linhas. Contrato do CSV: datas diárias únicas e demanda finita, não negativa. Chamado por read_series. Retorna cópia ordenada, preservando covariáveis; erro invalida o dataset antes do modelo.

src/data/preprocessing/resampling.py — 8 linhas. Converte datas observadas em grade diária. Chamado por prepare_dataset antes do preenchimento. Adiciona observed para distinguir alvos reais de linhas inseridas.

src/data/preprocessing/missing_values.py — 11 linhas. Preenchimento causal. Chamado após daily_grid. Retorna tabela sem lacunas iniciais de demanda; o preenchimento indiscriminado do calendário é um defeito confirmado, não uma prática recomendada.

src/evaluation/cross_validation.py — 37 linhas. Define ForecastWindow e origens walk-forward. prepare_dataset chama build_windows. validation_split é usado por training_partition e testes, não para ajustar os modelos neste benchmark.

src/data/features/temporal.py — 6 linhas. Geração determinística das datas-alvo a partir da última data histórica. Chamado pelos dois construtores de prompt; não consulta a demanda futura.

src/data/features/covariates.py — 43 linhas. Contrato comum dos modelos: nomes de covariáveis, validação, separação history/future e conversão para TimesFM. model_inputs é a fronteira que exclui estruturalmente o alvo da entrada futura.

src/data/preprocessing/generate_prompts.py — 127 linhas. Transforma história/covariáveis em texto. forecasting_pipeline usa select_demonstrations e escolhe build_prompt (legado) ou build_tabular_prompt (comparação 4B). Não chama Ollama e não calcula métricas.

src/models/promptcast.py — 99 linhas. Adaptador HTTP do Qwen: configuração, schema de saída, parser, cliente JSON e classe QwenForecaster. forecasting_pipeline chama predict; post_json encapsula transporte, e parse_prediction valida sem corrigir artificialmente valores.

src/models/chronos.py — 34 linhas. Adapter de Chronos2Pipeline. Importações pesadas ficam dentro do construtor, permitindo importar o pipeline em outros ambientes. Usa covariáveis passadas/futuras e devolve o quantil 0,5 como previsão pontual.

src/models/timesfm.py — 41 linhas. Adapter de TimesFM 2.5 com XReg. Prepara capacidade suficiente para todo o histórico a cada origem e devolve somente os primeiros horizon valores. Não corta a história ao

contexto inicial.

src/evaluation/metrics.py — 20 linhas. Função numérica sem I/O. Recebe vetores reais e previstos já alinhados e retorna dicionário de métricas. É chamada pelos comparadores, testes e auditoria.

src/pipelines/compare_models.py — 64 linhas. Comparador do benchmark qwen/chronos/timesfm. Rejeita execuções incompletas e datas incompatíveis; sua lógica específica de contexto do Qwen não cobre o braço expansivo.

src/pipelines/compare_qwen_contexts.py — 90 linhas. Comparador controlado Qwen 15 dias versus expansivo. Exige mesmos alvos, fontes, modelo e configuração exceto qwen_history_days. Produz métricas, melhorias e detalhamento por lead.

src/training/train.py — 11 linhas. Ponto auxiliar para uma extensão de treinamento: oferece slices cronológicos treino/validação. Não é chamado pela inferência atual nem treina pesos.

tests/test_forecasting.py — 158 linhas. Testes automatizados de contratos do pipeline.

tests/test_qwen_context_comparison.py — 57 linhas. Testes automatizados de contratos do pipeline.

tests/test_seven_day_metrics.py — 47 linhas. Testes automatizados de contratos do pipeline.

experiments/qwen_context_comparison/audit/audit_run.py — 99 linhas. Auditoria offline das respostas já salvas. Reconstrói prompts, hashes e alvos; aplica mutação sentinela nos alvos futuros; recalcula erros e produz relatórios por janela. Não altera os resultados originais nem chama Ollama.

src/__init__.py — vazio; identifica/organiza um pacote Python, sem função, efeito de inicialização ou chamada de inferência a explicar.

src/data/__init__.py — vazio; identifica/organiza um pacote Python, sem função, efeito de inicialização ou chamada de inferência a explicar.

src/data/features/__init__.py — vazio; identifica/organiza um pacote Python, sem função, efeito de inicialização ou chamada de inferência a explicar.

src/data/ingestion/__init__.py — vazio; identifica/organiza um pacote Python, sem função, efeito de inicialização ou chamada de inferência a explicar.

src/data/preprocessing/__init__.py — vazio; identifica/organiza um pacote Python, sem função, efeito de inicialização ou chamada de inferência a explicar.

src/data/validation/__init__.py — vazio; identifica/organiza um pacote Python, sem função, efeito de inicialização ou chamada de inferência a explicar.

src/evaluation/__init__.py — vazio; identifica/organiza um pacote Python, sem função, efeito de inicialização ou chamada de inferência a explicar.

src/models/__init__.py — vazio; identifica/organiza um pacote Python, sem função, efeito de inicialização ou chamada de inferência a explicar.

src/pipelines/__init__.py — vazio; identifica/organiza um pacote Python, sem função, efeito de inicialização ou chamada de inferência a explicar.

src/training/__init__.py — vazio; identifica/organiza um pacote Python, sem função, efeito de inicialização ou chamada de inferência a explicar.

10. pyproject.toml

Arquivo de configuração/execução que conecta parâmetros e módulos.

SHA-256: c85a2368ec54a37f42e229907d8c6070958d9fa4dc4b2ae72c3fd89b5c1178f7

Código comentado, linha a linha

```
0001  [build-system]
```

Seção que configura a ferramenta de construção do pacote.

```
0002  requires = ["setuptools>=61"]
```

Dependência de build setuptools>=61; não é comando executado no benchmark.

```
0003  build-backend = "setuptools.build_meta"
```

Backend setuptools.build_meta usado para construir/instalar o pacote.

```
0005  [project]
```

Metadados do projeto instalável.

```
0006  name = "promptcast-forecast"
```

Nome de distribuição, diferente do pacote importado src.

```
0007  version = "0.2.0"
```

Versão declarada do projeto.

```
0008  description = "Auditable rolling-origin water demand forecasting with Qwen, Chronos and
    TimesFM"
```

Descrição textual do pacote.

```
0009  requires-python = ">=3.10"
```

Versão mínima de Python declarada. A sintaxe de união | exige Python 3.10+.

```
0010  dependencies = ["numpy>=1.24", "pandas>=2"]
```

Dependências comuns; PyTorch/Chronos/TimesFM pertencem aos ambientes especializados, não são instalados por esta lista.

```
0012  [project.scripts]
```

Tabela de comandos de console que a instalação disponibiliza.

```
0013  qwen_forecast = "src.pipelines.forecasting_pipeline:main"
```

Cria qwen_forecast apontando para main de forecasting_pipeline. Não procura arquivo qwen_forecast.py.

```
0014  forecast-compare = "src.pipelines.compare_models:main"
```

Cria forecast-compare apontando para main de compare_models.

```
0016  [tool.setuptools.packages.find]
```

Configura descoberta de pacotes do setuptools.

```
0017  where = ["."]
```

Busca pacotes a partir da raiz atual.

```
0018  include = ["src*"]
```

Inclui pacotes cujo nome começa com src. A estrutura real, não nomes de arquivos antigos, determina os imports.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

11. experiments/start_ollama_context_server.sh

Arquivo de configuração/execução que conecta parâmetros e módulos.

SHA-256: 00184c26225c9a45960d3e6a6c9afbf2094f74a251c7cd77fdca6842e9db15ef

Código comentado, linha a linha

```
0001  #!/usr/bin/env bash
```

Escolhe Bash pelo PATH via /usr/bin/env; o sistema usa essa primeira linha quando executa o script diretamente.

```
0002  set -euo pipefail
```

-e encerra em falhas não tratadas; -u rejeita variáveis inexistentes; pipefail propaga falhas de componentes de pipelines shell.

```
0003  export OLLAMA_HOST=127.0.0.1:11435
```

export disponibiliza a variável aos processos filhos. HOST escolhe interface local e porta 11435; não publica o modelo em uma interface externa.

```
0004  export OLLAMA_FLASH_ATTENTION=1
```

Habilita a opção de Flash Attention do servidor, quando suportada pelo backend.

```
0005  export OLLAMA_KV_CACHE_TYPE=q4_0
```

Configura quantização q4_0 para cache de atenção K/V; não é a tag Q4_K_M dos pesos.

```
0006  export OLLAMA_NUM_PARALLEL=1
```

Limita a uma sequência simultânea para reduzir memória e controlar a execução sequencial.

```
0007  export LLAMA_ARG_CACHE_RAM=0
```

Desabilita RAM extra de snapshots de prompts no backend usado; o KV da sequência ativa continua existindo.

```
0008  exec ollama serve
```

exec substitui o processo Bash por ollama serve. O servidor fica ativo e atende HTTP; não chama o pipeline Python por conta própria.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

12. experiments/run_qwen_context_comparison.sh

Arquivo de configuração/execução que conecta parâmetros e módulos.

SHA-256: 72c1804cc384546dced83107f1477f5ee0b6cc3e7865093a11e67f3f42948cda

Código comentado, linha a linha

```
0001  #!/usr/bin/env bash
```

Seleciona Bash para interpretar o arquivo.

```
0002  set -euo pipefail
```

Ativa saída em erro, rejeição de variáveis não definidas e propagação de falhas em pipes.

```
0003  cd "$(dirname "$0")/.."
```

dirname "$0" extrai a pasta do script; $(...) substitui pela saída; /.. sobe à raiz; cd muda o diretório de trabalho para caminhos relativos funcionarem.

```
0004  CONDA_ENVS="${CONDA_ENVS:-/home/wagner-barbosa/miniconda3/envs}"
```

${CONDA_ENVS:-...} conserva variável fornecida ou usa o caminho padrão. Não ativa/modifica ambientes; escolhe executáveis existentes.

```
0005  OUTPUT=experiments/qwen_context_comparison
```

Define raiz de resultados da comparação, sem espaços em volta de = na atribuição shell.

```
0006  mkdir -p "$OUTPUT/logs"
```

mkdir -p cria a pasta de logs e pais se necessário, sem falhar por já existir.

```
0007  for CONTEXT_POLICY in 15_days expanding; do
```

Loop shell com duas políticas: primeiro 15_days, depois expanding. Não inclui sete dias automaticamente.

```
0008      "$CONDA_ENVS/promptcast/bin/python" -m src.pipelines.forecasting_pipeline \
```

Invoca Python do ambiente promptcast com -m para localizar o módulo. Barra invertida prolonga o comando na próxima linha.

```
0009          --model qwen --config "configs/qwen_context_${CONTEXT_POLICY}.json" \
```

Passa --model qwen e JSON cujo nome contém a política atual; a expansão ${CONTEXT_POLICY} ocorre no shell.

```
0010          --output "$OUTPUT/$CONTEXT_POLICY" > "$OUTPUT/logs/${CONTEXT_POLICY}.log" 2>&1
```

--output separa resultados dos braços; > grava stdout no log, sobrescrevendo o log anterior; 2>&1 manda stderr ao mesmo destino. Respostas salvas com hash continuam reutilizáveis.

```
0011  done
```

Encerra o corpo do loop. set -e impede chegar à comparação se alguma execução retornar código não zero.

```
0012  "$CONDA_ENVS/promptcast/bin/python" -m src.pipelines.compare_qwen_contexts --output "$OUTPUT"
```

Executa compare_qwen_contexts após os dois braços completos. O servidor Ollama precisa ter sido iniciado em outro processo.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

13. experiments/run_benchmark.sh

Arquivo de configuração/execução que conecta parâmetros e módulos.

SHA-256: 7c9226c3ec2387aa24ae8b98758c065b32dce9b6595aed63be59765f0870c7bf

Código comentado, linha a linha

```
0001  #!/usr/bin/env bash
```

Seleciona o interpretador Bash.

```
0002  # Use existing conda environments; never install or update packages here.
```

Comentário de política: reutilizar ambientes existentes sem instalar ou atualizar bibliotecas.

```
0003  set -euo pipefail
```

Configura tratamento estrito de falhas e variáveis.

```
0004  cd "$(dirname "$0")/.."
```

Muda à raiz do projeto usando a localização do script, não a pasta de onde o usuário o chamou.

```
0005  CONDA_ENVS="${CONDA_ENVS:-/home/wagner-barbosa/miniconda3/envs}"
```

Escolhe raiz dos ambientes conda por variável opcional ou padrão; mantém os ambientes intactos.

```
0006  mkdir -p experiments/benchmark/logs
```

Cria diretório de logs do benchmark.

```
0007  "$CONDA_ENVS/promptcast/bin/python" -m src.pipelines.forecasting_pipeline --model qwen >
    experiments/benchmark/logs/qwen.log 2>&1
```

Executa Qwen no ambiente promptcast com configuração padrão benchmark.json e redireciona toda saída ao log. Isso usa Qwen3 1.7B, não o JSON do 4B.

```
0008  ollama stop qwen3:1.7b
```

Solicita ao servidor Ollama padrão descarregar o modelo 1.7B para liberar recursos antes das baselines. Não apaga pesos em disco.

```
0009  HF_HUB_OFFLINE=1 "$CONDA_ENVS/chronos/bin/python" -m src.pipelines.forecasting_pipeline
    --model chronos > experiments/benchmark/logs/chronos.log 2>&1
```

Executa Chronos no ambiente chronos; HF_HUB_OFFLINE=1 vale para esse processo e impede acesso ao Hub. Log captura saída/erros.

```
0010  HF_HUB_OFFLINE=1 XLA_PYTHON_CLIENT_PREALLOCATE=false "$CONDA_ENVS/timesfm/bin/python" -m
    src.pipelines.forecasting_pipeline --model timesfm > experiments/benchmark/logs/timesfm.log
    2>&1
```

Executa TimesFM no ambiente timesfm. HF_HUB_OFFLINE vale para o Hub; XLA_PYTHON_CLIENT_PREALLOCATE=false desabilita pré-alocação XLA se esse backend for usado. O caminho principal do modelo aqui é PyTorch.

```
0011  "$CONDA_ENVS/promptcast/bin/python" -m src.pipelines.compare_models
```

Invoca comparador dos três modelos após todas as inferências terminarem com sucesso.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

14. configs/benchmark.json

Arquivo de configuração/execução que conecta parâmetros e módulos.

SHA-256: 56cc63897dcb707bdaa9b2bb10d5df0414cc6977eabf5111f676e1eec4f5319d

Código comentado, linha a linha

```
0001  {
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null representa ausência de corte histórico.

```
0002    "datasets": {
```

Mapeamento dos dois datasets; cada bloco fornece caminho, primeira origem e contrato de covariáveis. JSON é lido por json.loads; não executa chamadas por si só.

```
0003      "joinville": {
```

Bloco da série de Joinville. Contexto inicial de avaliação: 365 dias; total atual 54 janelas completas. JSON é lido por json.loads; não executa chamadas por si só.

```
0004        "path": "data/processed/joinville_processed_1day.csv",
```

Caminho relativo à raiz do projeto para CSV já processado. A extensão não define sozinho esquema; validate_series o verifica. JSON é lido por json.loads; não executa chamadas por si só.

```
0005        "context": 365,
```

Quantidade inicial de dias antes da primeira origem de teste. Expansivo cresce após essa origem; não é limite fixo em todas as janelas. JSON é lido por json.loads; não executa chamadas por si só.

```
0006        "numerical_covariates": [
```

Lista ordenada de colunas numéricas. Nomes passam intactos para prompt/adaptadores; também entram na identidade da execução. JSON é lido por json.loads; não executa chamadas por si só.

```
0007          "precipitation_median",
```

Nome de covariável precipitation_median. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0008          "temperature_2m_previsao_mean",
```

Nome de covariável temperature_2m_previsao_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0009          "shortwave_radiation_previsao_std",
```

Nome de covariável shortwave_radiation_previsao_std. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0010          "day",
```

Nome de covariável day. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0011          "dayofweek_sin",
```

Nome de covariável dayofweek_sin. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0012          "dayofweek_cos",
```

Nome de covariável dayofweek_cos. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0013          "season_sin",
```

Nome de covariável season_sin. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0014          "season_cos"
```

Nome de covariável season_cos. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0015        ],
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null representa ausência de corte histórico.

```
0016        "categorical_covariates": [
```

Lista ordenada de categorias/binários. TimesFM converte para strings; Qwen recebe valores na tabela textual. JSON é lido por json.loads; não executa chamadas por si só.

```
0017          "is_holiday"
```

Nome de covariável is_holiday. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0018        ]
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null representa ausência de corte histórico.

```
0019      },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null representa ausência de corte histórico.

```
0020      "guaratuba": {
```

Bloco da série de Guaratuba. Contexto inicial de avaliação: 730 dias; total atual 104 janelas completas. JSON é lido por json.loads; não executa chamadas por si só.

```
0021        "path": "data/processed/guaratuba_processed_1daysin.csv",
```

Caminho relativo à raiz do projeto para CSV já processado. A extensão não define sozinho esquema; validate_series o verifica. JSON é lido por json.loads; não executa chamadas por si só.

```
0022        "context": 730,
```

Quantidade inicial de dias antes da primeira origem de teste. Expansivo cresce após essa origem; não é limite fixo em todas as janelas. JSON é lido por json.loads; não executa chamadas por si só.

```
0023        "numerical_covariates": [
```

Lista ordenada de colunas numéricas. Nomes passam intactos para prompt/adaptadores; também entram na identidade da execução. JSON é lido por json.loads; não executa chamadas por si só.

```
0024          "relative_humidity_mean",
```

Nome de covariável relative_humidity_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0025          "temperature_mean",
```

Nome de covariável temperature_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0026          "radiation_mean",
```

Nome de covariável radiation_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0027          "precipitation_mean",
```

Nome de covariável precipitation_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0028          "day",
```

Nome de covariável day. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0029         "dayofweek_sin",
```

Nome de covariável dayofweek_sin. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0030         "dayofweek_cos",
```

Nome de covariável dayofweek_cos. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0031         "season_sin",
```

Nome de covariável season_sin. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0032         "season_cos"
```

Nome de covariável season_cos. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0033       ],
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null representa ausência de corte histórico.

```
0034       "categorical_covariates": [
```

Lista ordenada de categorias/binários. TimesFM converte para strings; Qwen recebe valores na tabela textual. JSON é lido por json.loads; não executa chamadas por si só.

```
0035         "is_holiday_ctba_gtba_jve",
```

Nome de covariável is_holiday_ctba_gtba_jve. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0036         "is_weekend",
```

Nome de covariável is_weekend. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0037         "is_carnival",
```

Nome de covariável is_carnival. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0038         "is_school_recess_pr"
```

Nome de covariável is_school_recess_pr. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0039       ]
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null representa ausência de corte histórico.

```
0040     }
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null representa ausência de corte histórico.

```
0041   },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null representa ausência de corte histórico.

```
0042   "horizon": 7,
```

Sete valores futuros por origem; o alvo é [origin, origin+7). JSON é lido por json.loads; não executa chamadas por si só.

```
0043   "step": 7,
```

Avança sete dias entre origens; com horizonte sete as janelas avaliadas não se sobrepõem. JSON é lido por json.loads; não executa chamadas por si só.

```
0044    "validation_fraction": 0.2,
```

Proporção 0,2 para validação em uma extensão de treino. A inferência zero-shot atual não a usa para otimizar parâmetros. JSON é lido por json.loads; não executa chamadas por si só.

```
0045    "seed": 42,
```

Semente global transmitida ao Qwen/seleção de exemplos. Os adaptadores numéricos usam 42 internamente. JSON é lido por json.loads; não executa chamadas por si só.

```
0046    "qwen": {
```

Parâmetros que **config["qwen"] expande em OllamaConfig; chaves precisam corresponder aos campos da dataclass. JSON é lido por json.loads; não executa chamadas por si só.

```
0047      "model": "qwen3:1.7b",
```

Tag exata do checkpoint instalado no Ollama; o digest é consultado no servidor e guardado em run.json. JSON é lido por json.loads; não executa chamadas por si só.

```
0048      "base_url": "http://127.0.0.1:11434",
```

Endpoint local do servidor Ollama. 11435 separa o servidor do 4B; o cliente adiciona /api/tags, /api/show e /api/generate. JSON é lido por json.loads; não executa chamadas por si só.

```
0049      "temperature": 0,
```

Temperatura de geração enviada em options; zero não equivale a treinamento nem a garantia absoluta de determinismo. JSON é lido por json.loads; não executa chamadas por si só.

```
0050      "num_ctx": 8192,
```

Capacidade em tokens do contexto do servidor por sequência. Precisa comportar prompt, template e resposta. JSON é lido por json.loads; não executa chamadas por si só.

```
0051      "num_predict": 256,
```

Limite de tokens gerados pelo Ollama. Não substitui horizon, que é quantidade de números previstos. JSON é lido por json.loads; não executa chamadas por si só.

```
0052      "timeout": 600,
```

Limite de espera HTTP do cliente em segundos. Pode durar mais de uma hora no caso 7200. JSON é lido por json.loads; não executa chamadas por si só.

```
0053      "retries": 2
```

Tentativas adicionais depois da primeira; retries=1 permite duas chamadas para a mesma janela. JSON é lido por json.loads; não executa chamadas por si só.

```
0054    },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null representa ausência de corte histórico.

```
0055    "strategy": "zero",
```

zero retorna nenhuma demonstração; few_random/few_similar só são aceitos no layout legado. JSON é lido por json.loads; não executa chamadas por si só.

```
0056    "few_shot_examples": 3,
```

Quantidade de exemplos para estratégias few-shot. Ignorada efetivamente quando strategy=zero. JSON é lido por json.loads; não executa chamadas por si só.

```
0057    "qwen_history_days": 15,
```

15 seleciona últimos 15 dias; null vira None e mantém todo o histórico desde o início da série. JSON é lido por json.loads; não executa chamadas por si só.

```
0058    "future_covariates_policy": "supplied CSV values, as in original scripts; availability at
    forecast origin is an unverified input assumption"
```

Texto de procedência armazenado com o experimento. A redação antiga diz hipótese não verificada; posteriormente o usuário confirmou disponibilidade na origem. Preservar o JSON preserva o hash histórico. JSON é lido por json.loads; não executa chamadas por si só.

```
0059  }
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null representa ausência de corte histórico.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

15. configs/qwen_context_15_days.json

Arquivo de configuração/execução que conecta parâmetros e módulos.

SHA-256: 8cbfa529dad3c700746354d67360d8c32243e956e3660e4b427cf0c1f2c0be8b

Código comentado, linha a linha

```
0001  {
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null representa ausência de corte histórico.

```
0002    "datasets": {
```

Mapeamento dos dois datasets; cada bloco fornece caminho, primeira origem e contrato de covariáveis. JSON é lido por json.loads; não executa chamadas por si só.

```
0003      "joinville": {
```

Bloco da série de Joinville. Contexto inicial de avaliação: 365 dias; total atual 54 janelas completas. JSON é lido por json.loads; não executa chamadas por si só.

```
0004        "path": "data/processed/joinville_processed_1day.csv",
```

Caminho relativo à raiz do projeto para CSV já processado. A extensão não define sozinho esquema; validate_series o verifica. JSON é lido por json.loads; não executa chamadas por si só.

```
0005        "context": 365,
```

Quantidade inicial de dias antes da primeira origem de teste. Expansivo cresce após essa origem; não é limite fixo em todas as janelas. JSON é lido por json.loads; não executa chamadas por si só.

```
0006        "numerical_covariates": [
```

Lista ordenada de colunas numéricas. Nomes passam intactos para prompt/adaptadores; também entram na identidade da execução. JSON é lido por json.loads; não executa chamadas por si só.

```
0007          "precipitation_median",
```

Nome de covariável precipitation_median. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0008          "temperature_2m_previsao_mean",
```

Nome de covariável temperature_2m_previsao_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0009          "shortwave_radiation_previsao_std",
```

Nome de covariável shortwave_radiation_previsao_std. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0010          "day",
```

Nome de covariável day. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0011          "dayofweek_sin",
```

Nome de covariável dayofweek_sin. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0012          "dayofweek_cos",
```

Nome de covariável dayofweek_cos. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0013          "season_sin",
```

Nome de covariável season_sin. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0014          "season_cos"
```

Nome de covariável season_cos. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0015        ],
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null representa ausência de corte histórico.

```
0016        "categorical_covariates": [
```

Lista ordenada de categorias/binários. TimesFM converte para strings; Qwen recebe valores na tabela textual. JSON é lido por json.loads; não executa chamadas por si só.

```
0017          "is_holiday"
```

Nome de covariável is_holiday. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0018        ]
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null representa ausência de corte histórico.

```
0019      },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null representa ausência de corte histórico.

```
0020      "guaratuba": {
```

Bloco da série de Guaratuba. Contexto inicial de avaliação: 730 dias; total atual 104 janelas completas. JSON é lido por json.loads; não executa chamadas por si só.

```
0021        "path": "data/processed/guaratuba_processed_1daysin.csv",
```

Caminho relativo à raiz do projeto para CSV já processado. A extensão não define sozinho esquema; validate_series o verifica. JSON é lido por json.loads; não executa chamadas por si só.

```
0022        "context": 730,
```

Quantidade inicial de dias antes da primeira origem de teste. Expansivo cresce após essa origem; não é limite fixo em todas as janelas. JSON é lido por json.loads; não executa chamadas por si só.

```
0023        "numerical_covariates": [
```

Lista ordenada de colunas numéricas. Nomes passam intactos para prompt/adaptadores; também entram na identidade da execução. JSON é lido por json.loads; não executa chamadas por si só.

```
0024          "relative_humidity_mean",
```

Nome de covariável relative_humidity_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0025          "temperature_mean",
```

Nome de covariável temperature_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0026          "radiation_mean",
```

Nome de covariável radiation_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0027          "precipitation_mean",
```

Nome de covariável precipitation_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0028          "day",
```

Nome de covariável day. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0029         "dayofweek_sin",
```

Nome de covariável dayofweek_sin. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0030         "dayofweek_cos",
```

Nome de covariável dayofweek_cos. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0031         "season_sin",
```

Nome de covariável season_sin. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0032         "season_cos"
```

Nome de covariável season_cos. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0033       ],
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null representa ausência de corte histórico.

```
0034       "categorical_covariates": [
```

Lista ordenada de categorias/binários. TimesFM converte para strings; Qwen recebe valores na tabela textual. JSON é lido por json.loads; não executa chamadas por si só.

```
0035         "is_holiday_ctba_gtba_jve",
```

Nome de covariável is_holiday_ctba_gtba_jve. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0036         "is_weekend",
```

Nome de covariável is_weekend. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0037         "is_carnival",
```

Nome de covariável is_carnival. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0038         "is_school_recess_pr"
```

Nome de covariável is_school_recess_pr. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0039       ]
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null representa ausência de corte histórico.

```
0040     }
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null representa ausência de corte histórico.

```
0041   },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null representa ausência de corte histórico.

```
0042   "horizon": 7,
```

Sete valores futuros por origem; o alvo é [origin, origin+7). JSON é lido por json.loads; não executa chamadas por si só.

```
0043   "step": 7,
```

Avança sete dias entre origens; com horizonte sete as janelas avaliadas não se sobrepõem. JSON é lido por json.loads; não executa chamadas por si só.

```
0044    "validation_fraction": 0.2,
```

Proporção 0,2 para validação em uma extensão de treino. A inferência zero-shot atual não a usa para otimizar parâmetros. JSON é lido por json.loads; não executa chamadas por si só.

```
0045    "seed": 42,
```

Semente global transmitida ao Qwen/seleção de exemplos. Os adaptadores numéricos usam 42 internamente. JSON é lido por json.loads; não executa chamadas por si só.

```
0046    "qwen": {
```

Parâmetros que **config["qwen"] expande em OllamaConfig; chaves precisam corresponder aos campos da dataclass. JSON é lido por json.loads; não executa chamadas por si só.

```
0047      "model": "qwen3:4b-instruct-2507-q4_K_M",
```

Tag exata do checkpoint instalado no Ollama; o digest é consultado no servidor e guardado em run.json. JSON é lido por json.loads; não executa chamadas por si só.

```
0048      "base_url": "http://127.0.0.1:11435",
```

Endpoint local do servidor Ollama. 11435 separa o servidor do 4B; o cliente adiciona /api/tags, /api/show e /api/generate. JSON é lido por json.loads; não executa chamadas por si só.

```
0049      "temperature": 0,
```

Temperatura de geração enviada em options; zero não equivale a treinamento nem a garantia absoluta de determinismo. JSON é lido por json.loads; não executa chamadas por si só.

```
0050      "num_ctx": 131072,
```

Capacidade em tokens do contexto do servidor por sequência. Precisa comportar prompt, template e resposta. JSON é lido por json.loads; não executa chamadas por si só.

```
0051      "num_predict": 256,
```

Limite de tokens gerados pelo Ollama. Não substitui horizon, que é quantidade de números previstos. JSON é lido por json.loads; não executa chamadas por si só.

```
0052      "timeout": 7200,
```

Limite de espera HTTP do cliente em segundos. Pode durar mais de uma hora no caso 7200. JSON é lido por json.loads; não executa chamadas por si só.

```
0053      "retries": 1
```

Tentativas adicionais depois da primeira; retries=1 permite duas chamadas para a mesma janela. JSON é lido por json.loads; não executa chamadas por si só.

```
0054    },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null representa ausência de corte histórico.

```
0055    "strategy": "zero",
```

zero retorna nenhuma demonstração; few_random/few_similar só são aceitos no layout legado. JSON é lido por json.loads; não executa chamadas por si só.

```
0056    "few_shot_examples": 3,
```

Quantidade de exemplos para estratégias few-shot. Ignorada efetivamente quando strategy=zero. JSON é lido por json.loads; não executa chamadas por si só.

```
0057    "qwen_history_days": 15,
```

15 seleciona últimos 15 dias; null vira None e mantém todo o histórico desde o início da série. JSON é lido por json.loads; não executa chamadas por si só.

```
0058    "future_covariates_policy": "supplied CSV values, as in original scripts; availability at
    forecast origin is an unverified input assumption",
```

Texto de procedência armazenado com o experimento. A redação antiga diz hipótese não verificada; posteriormente o usuário confirmou disponibilidade na origem. Preservar o JSON preserva o hash histórico. JSON é lido por json.loads; não executa chamadas por si só.

```
0059    "runtime": {
```

Registro declarativo das opções pretendidas de servidor. forecasting_pipeline não exporta essas variáveis automaticamente; o script do servidor precisa aplicá-las. JSON é lido por json.loads; não executa chamadas por si só.

```
0060      "ollama_flash_attention": true,
```

Registro da intenção de usar Flash Attention; aplicada por OLLAMA_FLASH_ATTENTION no script do servidor. JSON é lido por json.loads; não executa chamadas por si só.

```
0061      "ollama_kv_cache_type": "q4_0",
```

Registro de cache KV q4_0; diferente da quantização Q4_K_M dos pesos. JSON é lido por json.loads; não executa chamadas por si só.

```
0062      "ollama_num_parallel": 1,
```

Uma sequência simultânea neste experimento para limitar memória e facilitar reutilização de prefixo. JSON é lido por json.loads; não executa chamadas por si só.

```
0063      "llama_arg_cache_ram": 0
```

Zero desabilita cache adicional de snapshots; não desliga estados KV da sequência ativa. JSON é lido por json.loads; não executa chamadas por si só.

```
0064    },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null representa ausência de corte histórico.

```
0065    "prompt_layout": "tabular"
```

tabular escolhe build_tabular_prompt; se chave ausente, o pipeline usa legacy e build_prompt. JSON é lido por json.loads; não executa chamadas por si só.

```
0066  }
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null representa ausência de corte histórico.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

16. configs/qwen_context_expanding.json

Arquivo de configuração/execução que conecta parâmetros e módulos.

SHA-256: 792d9c9a6aa0b2f1e65ba61940f5d7f396cb43181140515c71a708d30a1096e9

Código comentado, linha a linha

```
0001  {
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null representa ausência de corte histórico.

```
0002    "datasets": {
```

Mapeamento dos dois datasets; cada bloco fornece caminho, primeira origem e contrato de covariáveis. JSON é lido por json.loads; não executa chamadas por si só.

```
0003      "joinville": {
```

Bloco da série de Joinville. Contexto inicial de avaliação: 365 dias; total atual 54 janelas completas. JSON é lido por json.loads; não executa chamadas por si só.

```
0004        "path": "data/processed/joinville_processed_1day.csv",
```

Caminho relativo à raiz do projeto para CSV já processado. A extensão não define sozinho esquema; validate_series o verifica. JSON é lido por json.loads; não executa chamadas por si só.

```
0005        "context": 365,
```

Quantidade inicial de dias antes da primeira origem de teste. Expansivo cresce após essa origem; não é limite fixo em todas as janelas. JSON é lido por json.loads; não executa chamadas por si só.

```
0006        "numerical_covariates": [
```

Lista ordenada de colunas numéricas. Nomes passam intactos para prompt/adaptadores; também entram na identidade da execução. JSON é lido por json.loads; não executa chamadas por si só.

```
0007          "precipitation_median",
```

Nome de covariável precipitation_median. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0008          "temperature_2m_previsao_mean",
```

Nome de covariável temperature_2m_previsao_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0009          "shortwave_radiation_previsao_std",
```

Nome de covariável shortwave_radiation_previsao_std. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0010          "day",
```

Nome de covariável day. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0011          "dayofweek_sin",
```

Nome de covariável dayofweek_sin. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0012          "dayofweek_cos",
```

Nome de covariável dayofweek_cos. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0013          "season_sin",
```

Nome de covariável season_sin. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0014          "season_cos"
```

Nome de covariável season_cos. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0015        ],
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null representa ausência de corte histórico.

```
0016        "categorical_covariates": [
```

Lista ordenada de categorias/binários. TimesFM converte para strings; Qwen recebe valores na tabela textual. JSON é lido por json.loads; não executa chamadas por si só.

```
0017          "is_holiday"
```

Nome de covariável is_holiday. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0018        ]
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null representa ausência de corte histórico.

```
0019      },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null representa ausência de corte histórico.

```
0020      "guaratuba": {
```

Bloco da série de Guaratuba. Contexto inicial de avaliação: 730 dias; total atual 104 janelas completas. JSON é lido por json.loads; não executa chamadas por si só.

```
0021        "path": "data/processed/guaratuba_processed_1daysin.csv",
```

Caminho relativo à raiz do projeto para CSV já processado. A extensão não define sozinho esquema; validate_series o verifica. JSON é lido por json.loads; não executa chamadas por si só.

```
0022        "context": 730,
```

Quantidade inicial de dias antes da primeira origem de teste. Expansivo cresce após essa origem; não é limite fixo em todas as janelas. JSON é lido por json.loads; não executa chamadas por si só.

```
0023        "numerical_covariates": [
```

Lista ordenada de colunas numéricas. Nomes passam intactos para prompt/adaptadores; também entram na identidade da execução. JSON é lido por json.loads; não executa chamadas por si só.

```
0024          "relative_humidity_mean",
```

Nome de covariável relative_humidity_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0025          "temperature_mean",
```

Nome de covariável temperature_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0026          "radiation_mean",
```

Nome de covariável radiation_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0027          "precipitation_mean",
```

Nome de covariável precipitation_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0028          "day",
```

Nome de covariável day. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0029         "dayofweek_sin",
```

Nome de covariável dayofweek_sin. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0030         "dayofweek_cos",
```

Nome de covariável dayofweek_cos. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0031         "season_sin",
```

Nome de covariável season_sin. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0032         "season_cos"
```

Nome de covariável season_cos. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0033       ],
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null representa ausência de corte histórico.

```
0034       "categorical_covariates": [
```

Lista ordenada de categorias/binários. TimesFM converte para strings; Qwen recebe valores na tabela textual. JSON é lido por json.loads; não executa chamadas por si só.

```
0035         "is_holiday_ctba_gtba_jve",
```

Nome de covariável is_holiday_ctba_gtba_jve. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0036         "is_weekend",
```

Nome de covariável is_weekend. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0037         "is_carnival",
```

Nome de covariável is_carnival. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0038         "is_school_recess_pr"
```

Nome de covariável is_school_recess_pr. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0039       ]
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null representa ausência de corte histórico.

```
0040     }
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null representa ausência de corte histórico.

```
0041   },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null representa ausência de corte histórico.

```
0042   "horizon": 7,
```

Sete valores futuros por origem; o alvo é [origin, origin+7). JSON é lido por json.loads; não executa chamadas por si só.

```
0043   "step": 7,
```

Avança sete dias entre origens; com horizonte sete as janelas avaliadas não se sobrepõem. JSON é lido por json.loads; não executa chamadas por si só.

```
0044    "validation_fraction": 0.2,
```

Proporção 0,2 para validação em uma extensão de treino. A inferência zero-shot atual não a usa para otimizar parâmetros. JSON é lido por json.loads; não executa chamadas por si só.

```
0045    "seed": 42,
```

Semente global transmitida ao Qwen/seleção de exemplos. Os adaptadores numéricos usam 42 internamente. JSON é lido por json.loads; não executa chamadas por si só.

```
0046    "qwen": {
```

Parâmetros que **config["qwen"] expande em OllamaConfig; chaves precisam corresponder aos campos da dataclass. JSON é lido por json.loads; não executa chamadas por si só.

```
0047      "model": "qwen3:4b-instruct-2507-q4_K_M",
```

Tag exata do checkpoint instalado no Ollama; o digest é consultado no servidor e guardado em run.json. JSON é lido por json.loads; não executa chamadas por si só.

```
0048      "base_url": "http://127.0.0.1:11435",
```

Endpoint local do servidor Ollama. 11435 separa o servidor do 4B; o cliente adiciona /api/tags, /api/show e /api/generate. JSON é lido por json.loads; não executa chamadas por si só.

```
0049      "temperature": 0,
```

Temperatura de geração enviada em options; zero não equivale a treinamento nem a garantia absoluta de determinismo. JSON é lido por json.loads; não executa chamadas por si só.

```
0050      "num_ctx": 131072,
```

Capacidade em tokens do contexto do servidor por sequência. Precisa comportar prompt, template e resposta. JSON é lido por json.loads; não executa chamadas por si só.

```
0051      "num_predict": 256,
```

Limite de tokens gerados pelo Ollama. Não substitui horizon, que é quantidade de números previstos. JSON é lido por json.loads; não executa chamadas por si só.

```
0052      "timeout": 7200,
```

Limite de espera HTTP do cliente em segundos. Pode durar mais de uma hora no caso 7200. JSON é lido por json.loads; não executa chamadas por si só.

```
0053      "retries": 1
```

Tentativas adicionais depois da primeira; retries=1 permite duas chamadas para a mesma janela. JSON é lido por json.loads; não executa chamadas por si só.

```
0054    },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null representa ausência de corte histórico.

```
0055    "strategy": "zero",
```

zero retorna nenhuma demonstração; few_random/few_similar só são aceitos no layout legado. JSON é lido por json.loads; não executa chamadas por si só.

```
0056    "few_shot_examples": 3,
```

Quantidade de exemplos para estratégias few-shot. Ignorada efetivamente quando strategy=zero. JSON é lido por json.loads; não executa chamadas por si só.

```
0057    "qwen_history_days": null,
```

15 seleciona últimos 15 dias; null vira None e mantém todo o histórico desde o início da série. JSON é lido por json.loads; não executa chamadas por si só.

```
0058    "future_covariates_policy": "supplied CSV values, as in original scripts; availability at
    forecast origin is an unverified input assumption",
```

Texto de procedência armazenado com o experimento. A redação antiga diz hipótese não verificada; posteriormente o usuário confirmou disponibilidade na origem. Preservar o JSON preserva o hash histórico. JSON é lido por json.loads; não executa chamadas por si só.

```
0059    "runtime": {
```

Registro declarativo das opções pretendidas de servidor. forecasting_pipeline não exporta essas variáveis automaticamente; o script do servidor precisa aplicá-las. JSON é lido por json.loads; não executa chamadas por si só.

```
0060      "ollama_flash_attention": true,
```

Registro da intenção de usar Flash Attention; aplicada por OLLAMA_FLASH_ATTENTION no script do servidor. JSON é lido por json.loads; não executa chamadas por si só.

```
0061      "ollama_kv_cache_type": "q4_0",
```

Registro de cache KV q4_0; diferente da quantização Q4_K_M dos pesos. JSON é lido por json.loads; não executa chamadas por si só.

```
0062      "ollama_num_parallel": 1,
```

Uma sequência simultânea neste experimento para limitar memória e facilitar reutilização de prefixo. JSON é lido por json.loads; não executa chamadas por si só.

```
0063      "llama_arg_cache_ram": 0
```

Zero desabilita cache adicional de snapshots; não desliga estados KV da sequência ativa. JSON é lido por json.loads; não executa chamadas por si só.

```
0064    },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null representa ausência de corte histórico.

```
0065    "prompt_layout": "tabular"
```

tabular escolhe build_tabular_prompt; se chave ausente, o pipeline usa legacy e build_prompt. JSON é lido por json.loads; não executa chamadas por si só.

```
0066  }
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null representa ausência de corte histórico.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

17. src/pipelines/forecasting_pipeline.py

Orquestrador principal e CLI. Conecta dados, fábrica de modelos, prompt, cache, inferência e gravação. Executa modelos; não produz sozinho os CSVs agregados de métricas.

SHA-256: 2087af35dc26fdefd50c0872fd99f5620dc4cb510930e333e92905c3046ed38c

Funções, contratos e pontos de conexão

```
L18: fingerprint(value) → str
```

value: objeto serializável. Retorna SHA-256 hexadecimal do JSON com chaves ordenadas.

```
L22: source_fingerprint() → str
```

Sem parâmetros. Retorna digest incremental de nomes e bytes de todos os .py de src, em ordem fixa.

```
L31: create_model(name, context, config, dataset)
```

name: qwen/chronos/timesfm; context: contexto inicial; config: configuração global; dataset: configuração da cidade. Retorna adaptador apropriado. A fábrica não prevê por si só.

Chamadas do projeto: ChronosForecaster → src/models/chronos.py:5; OllamaConfig → src/models/promptcast.py:11; QwenForecaster → src/models/promptcast.py:53; TimesFMForecaster → src/models/timesfm.py:7

```
L44: run_dataset(name, dataset, config, model_name, output, limit = 0)
```

name: cidade; dataset: bloco da cidade; config: parâmetros globais; model_name: adaptador; output: raiz de resultados; limit=0: todas as janelas. Retorna número de falhas e grava artefatos.

Chamadas do projeto: build_prompt → src/data/preprocessing/generate_prompts.py:52; build_tabular_prompt → src/data/preprocessing/generate_prompts.py:97; covariate_columns → src/data/features/covariates.py:6; create_model → src/pipelines/forecasting_pipeline.py:31; fingerprint → src/pipelines/forecasting_pipeline.py:18; model.predict → src/models/promptcast.py:71; model.predict → src/models/chronos.py:18; model.predict → src/models/timesfm.py:20; model_inputs → src/data/features/covariates.py:25; prepare_dataset → src/pipelines/data_pipeline.py:8; select_demonstrations → src/data/preprocessing/generate_prompts.py:16; source_fingerprint → src/pipelines/forecasting_pipeline.py:22; validate_covariates → src/data/features/covariates.py:13; write_json → src/data/ingestion/files.py:12

```
L123: main()
```

Sem parâmetros explícitos. Lê argumentos do processo via argparse ou executa entry point do módulo. O corpo de cada arquivo determina qual CLI será iniciado.

Chamadas do projeto: run_dataset → src/pipelines/forecasting_pipeline.py:44

APIs externas usadas neste arquivo

ChronosForecaster, OllamaConfig, Path, QwenForecaster, SystemExit, TimesFMForecaster, ValueError, add_argument, all, append, argparse.ArgumentParser, asdict, dict, encode, enumerate, error, exists, float, get, hashlib.sha256, hexdigest, items, iterrows, json.dumps, json.loads, len, mkdir, np.isfinite, parse_args, pd.DataFrame, predict, print, read_bytes, read_text, relative_to, resolve, rglob, sorted, str, strftime, time.monotonic, to_csv, to_numpy, tolist, type, update, write_text, zip. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Run one model/environment with shared windows and resumable, fingerprinted records."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import argparse
```

Importa argparse. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0003  from dataclasses import asdict
```

Importa asdict de dataclasses. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0004  import hashlib
```

Importa hashlib. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  import json
```

Importa json. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0006  from pathlib import Path
```

Importa Path de pathlib. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0007  import time
```

Importa time. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0008  import numpy as np
```

Importa numpy como np. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0009  import pandas as pd
```

Importa pandas como pd. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0010  from src.data.ingestion.files import write_json
```

Importa write_json de src.data.ingestion.files. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0011  from src.data.preprocessing.generate_prompts import build_prompt, build_tabular_prompt,
    select_demonstrations
```

Importa build_prompt, build_tabular_prompt, select_demonstrations de src.data.preprocessing.generate_prompts. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0012  from src.pipelines.data_pipeline import prepare_dataset
```

Importa prepare_dataset de src.pipelines.data_pipeline. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0013  from src.data.features.covariates import model_inputs, validate_covariates, covariate_columns
```

Importa model_inputs, validate_covariates, covariate_columns de src.data.features.covariates. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0015  ROOT = Path(__file__).resolve().parents[2]
```

ROOT é a raiz obtida a partir da posição deste arquivo; os caminhos de dados e padrões do CLI são ancorados nela.

Path(__file__).resolve — Resolve caminho absoluto e links simbólicos. Argumentos usados: nenhum argumento explícito.

Path — Representa caminho de arquivo com operações portáveis. Argumentos usados: posicional 1: __file__.

```
0018  def fingerprint(value) -> str:
```

Define fingerprint(value) → str. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0019      return hashlib.sha256(json.dumps(value, sort_keys=True).encode()).hexdigest()
```

O JSON de value recebe chaves ordenadas, vira bytes e depois SHA-256 hexadecimal. Ordem de listas permanece significativa: trocar covariáveis de posição muda o hash.

hashlib.sha256(json.dumps(value, sort_keys=True).encode()).hexdigest — Retorna a representação hexadecimal do digest acumulado. Argumentos usados: nenhum argumento explícito.

hashlib.sha256 — Cria calculador de digest SHA-256. Argumentos usados: posicional 1: json.dumps(value, sort_keys=True).encode().

json.dumps(value, sort_keys=True).encode — Codifica string em bytes. Argumentos usados: nenhum argumento explícito.

json.dumps — Serializa objeto Python em string JSON. Argumentos usados: posicional 1: value; sort_keys: True.

```
0022  def source_fingerprint() -> str:
```

Define source_fingerprint() → str. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0023      """Invalidate cached inference when any model/data/pipeline source changes."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0024      digest = hashlib.sha256()
```

O hash de fonte inclui caminho relativo e bytes de cada .py em src, em ordem lexicográfica. Qualquer alteração de fonte pode invalidar todos os caches; relatórios fora de src não entram.

hashlib.sha256 — Cria calculador de digest SHA-256. Argumentos usados: nenhum argumento explícito.

```
0025      for path in sorted((ROOT / 'src').rglob('*.py')):
```

O hash de fonte inclui caminho relativo e bytes de cada .py em src, em ordem lexicográfica. Qualquer alteração de fonte pode invalidar todos os caches; relatórios fora de src não entram.

sorted — Devolve uma lista ordenada. Argumentos usados: posicional 1: (ROOT / 'src').rglob('*.py').

(ROOT / 'src').rglob — Percorre recursivamente caminhos compatíveis com padrão. Argumentos usados: posicional 1: '*.py'.

```
0026          digest.update(str(path.relative_to(ROOT)).encode())
```

O hash de fonte inclui caminho relativo e bytes de cada .py em src, em ordem lexicográfica. Qualquer alteração de fonte pode invalidar todos os caches; relatórios fora de src não entram.

digest.update — Acrescenta conteúdo ao objeto mutável. Argumentos usados: posicional 1: str(path.relative_to(ROOT)).encode().

str(path.relative_to(ROOT)).encode — Codifica string em bytes. Argumentos usados: nenhum argumento explícito.

str — Produz representação textual. Argumentos usados: posicional 1: path.relative_to(ROOT).

path.relative_to — Calcula caminho relativo a um ancestral de um Path. Argumentos usados: posicional 1: ROOT.

```
0027          digest.update(path.read_bytes())
```

O hash de fonte inclui caminho relativo e bytes de cada .py em src, em ordem lexicográfica. Qualquer alteração de fonte pode invalidar todos os caches; relatórios fora de src não entram.

digest.update — Acrescenta conteúdo ao objeto mutável. Argumentos usados: posicional 1: path.read_bytes().

path.read_bytes — Lê bytes integrais do arquivo. Argumentos usados: nenhum argumento explícito.

```
0028      return digest.hexdigest()
```

O hash de fonte inclui caminho relativo e bytes de cada .py em src, em ordem lexicográfica. Qualquer alteração de fonte pode invalidar todos os caches; relatórios fora de src não entram.

digest.hexdigest — Retorna a representação hexadecimal do digest acumulado. Argumentos usados: nenhum argumento explícito.

```
0031  def create_model(name, context, config, dataset):
```

Define create_model(name, context, config, dataset). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0032      if name == 'qwen':
```

Avalia a condição name == 'qwen'. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0033          from src.models.promptcast import OllamaConfig, QwenForecaster
```

Importa apenas o adaptador necessário ao ramo Qwen; dependências dos outros modelos não precisam estar no ambiente promptcast.

```
0034          return QwenForecaster(OllamaConfig(**config['qwen'], seed=config['seed']))
```

**config['qwen'] expande chaves como model/base_url; seed vem do nível global. A dataclass valida limites no __post_init__.

QwenForecaster — Constrói adaptador Qwen e consulta servidor local. Argumentos usados: posicional 1: OllamaConfig(**config['qwen'], seed=config['seed']).

OllamaConfig — Constrói configuração tipada do cliente Qwen. Argumentos usados: **desempacotamento: config['qwen']; seed: config['seed'].

```
0035      if name == 'chronos':
```

Avalia a condição name == 'chronos'. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0036          from src.models.chronos import ChronosForecaster
```

Importações tardias isolam dependências de Chronos e TimesFM; context e dataset são passados a cada adaptador.

```
0037          return ChronosForecaster(context, dataset)
```

Importações tardias isolam dependências de Chronos e TimesFM; context e dataset são passados a cada adaptador.

ChronosForecaster — Constrói adaptador Chronos2 com pesos locais. Argumentos usados: posicional 1: context; posicional 2: dataset.

```
0038      if name == 'timesfm':
```

Importações tardias isolam dependências de Chronos e TimesFM; context e dataset são passados a cada adaptador.

```
0039          from src.models.timesfm import TimesFMForecaster
```

Importações tardias isolam dependências de Chronos e TimesFM; context e dataset são passados a cada adaptador.

```
0040          return TimesFMForecaster(context, dataset)
```

Encerra esta função e devolve TimesFMForecaster(context, dataset) ao chamador; não imprime o resultado por si só.

TimesFMForecaster — Constrói adaptador TimesFM2.5 local. Argumentos usados: posicional 1: context; posicional 2: dataset.

```
0041      raise ValueError(f'Unknown model {name}')
```

Interrompe o caminho atual lançando ValueError(f'Unknown model {name}'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: f'Unknown model {name}'.

```
0044  def run_dataset(name, dataset, config, model_name, output, limit=0):
```

Executa um único par dataset/modelo. output é a raiz de saída; limit=0 significa execução completa, valor positivo limita as janelas para smoke test.

```
0045      path = ROOT / dataset['path']
```

Resolve caminho do CSV relativamente à raiz do projeto, independentemente de qual nome de modelo será executado.

```
0046      df, windows = prepare_dataset(path, dataset['context'], horizon=config['horizon'],
```

prepare_dataset retorna tabela preparada e todas as janelas elegíveis, com horizonte e passo da configuração comum.

prepare_dataset — path: CSV; context: primeira origem; horizon=7 e step=7 nomeados. Retorna df e windows depois da preparação. Argumentos usados: posicional 1: path; posicional 2: dataset['context']; horizon: config['horizon']; step: config['step'].

```
0047                                    step=config['step'])
```

prepare_dataset retorna tabela preparada e todas as janelas elegíveis, com horizonte e passo da configuração comum.

```
0048      validate_covariates(df, dataset)
```

Executa validate_covariates(df, dataset). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

validate_covariates — df: tabela preparada; dataset: contrato de colunas. Retorna None; falha em ausentes/NaN/números não finitos. Argumentos usados: posicional 1: df; posicional 2: dataset.

```
0049      if limit:
```

Um limite positivo retém só as primeiras janelas; o valor entra na procedência para comparadores recusarem métricas parciais como completas.

```
0050          windows = windows[:limit]
```

Um limite positivo retém só as primeiras janelas; o valor entra na procedência para comparadores recusarem métricas parciais como completas.

```
0051      model = create_model(model_name, dataset['context'], config, dataset)
```

Atribui a model o resultado de create_model(model_name, dataset['context'], config, dataset). A expressão à direita é avaliada antes da atribuição.

create_model — name: qwen/chronos/timesfm; context: contexto inicial; config: configuração global; dataset: configuração da cidade. Retorna adaptador apropriado. A fábrica não prevê por si só. Argumentos usados: posicional 1: model_name; posicional 2: dataset['context']; posicional 3: config; posicional 4: dataset.

```
0052      provenance = {'config': config, 'dataset_sha256':
    hashlib.sha256(path.read_bytes()).hexdigest(),
```

Procedência reúne configuração, hashes, identidade do adaptador, datas imputadas e política de covariáveis. É compartilhada pelas janelas desse dataset/modelo.

hashlib.sha256(path.read_bytes()).hexdigest — Retorna a representação hexadecimal do digest acumulado. Argumentos usados: nenhum argumento explícito.

hashlib.sha256 — Cria calculador de digest SHA-256. Argumentos usados: posicional 1: path.read_bytes().

path.read_bytes — Lê bytes integrais do arquivo. Argumentos usados: nenhum argumento explícito.

```
0053                  'source_sha256': source_fingerprint(), 'model': model.identity,
```

Procedência reúne configuração, hashes, identidade do adaptador, datas imputadas e política de covariáveis. É compartilhada pelas janelas desse dataset/modelo.

source_fingerprint — Sem parâmetros. Retorna digest incremental de nomes e bytes de todos os .py de src, em ordem fixa. Argumentos usados: nenhum argumento explícito.

```
0054                  'dataset': name, 'limit': limit, 'n_windows': len(windows),
```

Procedência reúne configuração, hashes, identidade do adaptador, datas imputadas e política de covariáveis. É compartilhada pelas janelas desse dataset/modelo.

len — Conta elementos. Argumentos usados: posicional 1: windows.

```
0055                  'first_test_origin': windows[0].origin,
```

Procedência reúne configuração, hashes, identidade do adaptador, datas imputadas e política de covariáveis. É compartilhada pelas janelas desse dataset/modelo.

```
0056                  'missing_days_forward_filled': df.loc[~df.observed,
    'timestamp'].dt.strftime('%Y-%m-%d').tolist(),
```

Procedência reúne configuração, hashes, identidade do adaptador, datas imputadas e política de covariáveis. É compartilhada pelas janelas desse dataset/modelo.

df.loc[~df.observed, 'timestamp'].dt.strftime('%Y-%m-%d').tolist — Converte array/índice/Series em lista Python. Argumentos usados: nenhum argumento explícito.

df.loc[~df.observed, 'timestamp'].dt.strftime — Formata datas como texto. Argumentos usados: posicional 1: '%Y-%m-%d'.

```
0057                  'protocol': 'expanding history with historical and supplied future
    covariates',
```

Procedência reúne configuração, hashes, identidade do adaptador, datas imputadas e política de covariáveis. É compartilhada pelas janelas desse dataset/modelo.

```
0058                  'covariates': covariate_columns(dataset),
```

Procedência reúne configuração, hashes, identidade do adaptador, datas imputadas e política de covariáveis. É compartilhada pelas janelas desse dataset/modelo.

covariate_columns — dataset: bloco de configuração de uma cidade. Retorna nomes numéricos seguidos dos categóricos, após rejeitar duplicatas e nomes reservados. Argumentos usados: posicional 1: dataset.

```
0059                    'future_covariates_policy': config['future_covariates_policy'],
```

Procedência reúne configuração, hashes, identidade do adaptador, datas imputadas e política de covariáveis. É compartilhada pelas janelas desse dataset/modelo.

```
0060                    'qwen_history_days': config['qwen_history_days']}
```

Procedência reúne configuração, hashes, identidade do adaptador, datas imputadas e política de covariáveis. É compartilhada pelas janelas desse dataset/modelo.

```
0061      run_id = fingerprint(provenance)
```

run_id depende de toda a procedência; um cache de outra configuração não deve passar na checagem mesmo que o nome do arquivo coincida.

fingerprint — value: objeto serializável. Retorna SHA-256 hexadecimal do JSON com chaves ordenadas. Argumentos usados: posicional 1: provenance.

```
0062      destination = output / name / model_name
```

Atribui a destination o resultado de output / name / model_name. A expressão à direita é avaliada antes da atribuição.

```
0063      write_json(destination / 'run.json', dict(provenance, run_id=run_id))
```

Executa write_json(destination / 'run.json', dict(provenance, run_id=run_id)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

write_json — path: destino; value: dict serializável. Sem retorno útil. Cria pais, escreve .tmp e substitui destino. Argumentos usados: posicional 1: destination / 'run.json'; posicional 2: dict(provenance, run_id=run_id).

dict — Cria dicionário. Argumentos usados: posicional 1: provenance; run_id: run_id.

```
0064      # This prefix is frozen for every window, including all few-shot strategies.
```

Comentário de segurança: demonstrações few-shot ficam no prefixo ANTERIOR ao primeiro teste, mesmo quando a história operacional cresce depois.

```
0065      training_values = df.iloc[:windows[0].origin].water_produced.to_numpy()
```

Congela valores para seleção de demonstrações. A série completa continua em df, mas esta variável não inclui alvos da primeira janela nem posteriores.

df.iloc[:windows[0].origin].water_produced.to_numpy — Converte Series/DataFrame em array NumPy. Argumentos usados: nenhum argumento explícito.

```
0066      rows, failures = [], []
```

Atribui a (rows, failures) o resultado de ([], []). A expressão à direita é avaliada antes da atribuição.

```
0067      for index, window in enumerate(windows, 1):
```

Percorre janelas em ordem. index é contador humano começando em 1; window.origin é posição da origem no dataset, outra grandeza.

enumerate — Percorre iterável com contador. Argumentos usados: posicional 1: windows; posicional 2: 1.

```
0068          history, future = model_inputs(df, window, dataset,
```

Apenas Qwen pode receber corte history_days do JSON; None mantém expansão. Chronos e TimesFM recebem None sempre.

model_inputs — df: série completa; window: ForecastWindow; dataset: contrato; history_days: None para expansivo ou inteiro para últimos N dias. Retorna (history, future), excluindo alvo do futuro. Argumentos usados: posicional 1: df; posicional 2: window; posicional 3: dataset; history_days: config['qwen_history_days'] if model_name == 'qwen' else None.

```
0069              history_days=config['qwen_history_days'] if model_name == 'qwen' else None)
```

Apenas Qwen pode receber corte history_days do JSON; None mantém expansão. Chronos e TimesFM recebem None sempre.

```
0070          origin_date = df.timestamp.iloc[window.origin].strftime('%Y-%m-%d')
```

A origem é convertida em YYYY-MM-DD para identificar janela, prompt e resposta; não é data de término do horizonte.

df.timestamp.iloc[window.origin].strftime — Formata datas como texto. Argumentos usados: posicional 1: '%Y-%m-%d'.

```
0071          target = destination / 'responses' / f'{origin_date}.json'
```

Um JSON por origem dentro de responses permite retomar execução sem repetir inferências válidas.

```
0072          prompt = None
```

Atribui a prompt o resultado de None. A expressão à direita é avaliada antes da atribuição.

```
0073          if model_name == 'qwen':
```

Avalia a condição model_name == 'qwen'. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0074              examples = select_demonstrations(training_values,
    history.water_produced.to_numpy(),
```

Seleciona exemplos exclusivamente de training_values congelado; zero-shot retorna [] e ignora count na prática.

select_demonstrations — training_values: prefixo congelado anterior ao primeiro teste; history: demanda histórica corrente; horizon: tamanho de alvo dos exemplos; strategy: zero/few_random/few_similar; count=3: quantidade; seed=42: sorteio. Retorna lista de Demonstration. Argumentos usados: posicional 1: training_values; posicional 2: history.water_produced.to_numpy(); posicional 3: window.horizon; strategy: config['strategy']; count: config['few_shot_examples']; seed: config['seed'].

history.water_produced.to_numpy — Converte Series/DataFrame em array NumPy. Argumentos usados: nenhum argumento explícito.

```
0075                  window.horizon, strategy=config['strategy'],
    count=config['few_shot_examples'], seed=config['seed'])
```

Seleciona exemplos exclusivamente de training_values congelado; zero-shot retorna [] e ignora count na prática.

```
0076              if config.get('prompt_layout', 'legacy') == 'tabular':
```

Layout tabular recusa demonstrações não vazias para manter a comparação controlada zero-shot. O legado é o outro ramo.

config.get — Busca chave de dicionário com padrão se ausente. Argumentos usados: posicional 1: 'prompt_layout'; posicional 2: 'legacy'.

```
0077                  if examples:
```

Layout tabular recusa demonstrações não vazias para manter a comparação controlada zero-shot. O legado é o outro ramo.

```
0078                      raise ValueError('The controlled tabular experiment uses zero-shot
    prompts')
```

Layout tabular recusa demonstrações não vazias para manter a comparação controlada zero-shot. O legado é o outro ramo.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'The controlled tabular experiment uses zero-shot prompts'.

```
0079                  prompt = build_tabular_prompt(history, window.horizon,
    future_covariates=future)
```

Layout tabular recusa demonstrações não vazias para manter a comparação controlada zero-shot. O legado é o outro ramo.

build_tabular_prompt — history: DataFrame histórico; horizon: dias; future_covariates: obrigatório e nomeado. Retorna CSV histórico unificado e tabela futura, com prefixo estável para história expansiva. Argumentos usados: posicional 1: history; posicional 2: window.horizon; future_covariates: future.

```
0080              else:
```

Layout tabular recusa demonstrações não vazias para manter a comparação controlada zero-shot. O legado é o outro ramo.

```
0081                  prompt = build_prompt(history, window.horizon, demonstrations=examples,
    future_covariates=future)
```

Atribui a prompt o resultado de build_prompt(history, window.horizon, demonstrations=examples, future_covariates=future). A expressão à direita é avaliada antes da atribuição.

build_prompt — history: DataFrame histórico; horizon: dias; demonstrations: lista ou None; future_covariates: tabela sem alvo ou None. Retorna prompt legado com lista de demanda e tabelas de covariáveis. Argumentos usados: posicional 1: history; posicional 2: window.horizon; demonstrations: examples; future_covariates: future.

```
0082              prompt_path = output / name / 'prompts' / f'{origin_date}.txt'
```

Atribui a prompt_path o resultado de output / name / 'prompts' / f'{origin_date}.txt'. A expressão à direita é avaliada antes da atribuição.

```
0083              prompt_path.parent.mkdir(parents=True, exist_ok=True)
```

Persiste o prompt antes de chamar o modelo. Se a execução cair no HTTP, pode haver arquivo de prompt sem resposta válida: isso não significa janela concluída.

prompt_path.parent.mkdir — Cria diretório. Argumentos usados: parents: True; exist_ok: True.

```
0084              prompt_path.write_text(prompt)
```

Persiste o prompt antes de chamar o modelo. Se a execução cair no HTTP, pode haver arquivo de prompt sem resposta válida: isso não significa janela concluída.

prompt_path.write_text — Grava uma string em arquivo e retorna a quantidade de caracteres escrita. Argumentos usados: posicional 1: prompt.

```
0085          request_id = fingerprint({'run_id': run_id, 'window': asdict(window), 'prompt':
    prompt})
```

Persiste o prompt antes de chamar o modelo. Se a execução cair no HTTP, pode haver arquivo de prompt sem resposta válida: isso não significa janela concluída.

fingerprint — value: objeto serializável. Retorna SHA-256 hexadecimal do JSON com chaves ordenadas. Argumentos usados: posicional 1: {'run_id': run_id, 'window': asdict(window), 'prompt': prompt}.

asdict — Converte dataclass em dicionário de seus campos. Argumentos usados: posicional 1: window.

```
0086          saved = json.loads(target.read_text()) if target.exists() else {}
```

request_id combina procedência, definição exata da janela e texto completo. Até mudança de espaço no prompt invalida a resposta em cache.

target.exists — Verifica existência do caminho. Argumentos usados: nenhum argumento explícito.

json.loads — Decodifica uma string JSON em objetos Python. Argumentos usados: posicional 1: target.read_text().

target.read_text — Lê todo o arquivo como string. Argumentos usados: nenhum argumento explícito.

```
0087          cached = saved.get('request_id') == request_id and saved.get('status') == 'ok'
```

Só reutiliza resposta com request_id idêntico E status ok. Um JSON de erro anterior será tentado novamente numa retomada.

saved.get — Busca chave de dicionário com padrão se ausente. Argumentos usados: posicional 1: 'request_id'.

saved.get — Busca chave de dicionário com padrão se ausente. Argumentos usados: posicional 1: 'status'.

```
0088          if not cached:
```

Só reutiliza resposta com request_id idêntico E status ok. Um JSON de erro anterior será tentado novamente numa retomada.

```
0089              started = time.monotonic()
```

Atribui a started o resultado de time.monotonic(). A expressão à direita é avaliada antes da atribuição.

time.monotonic — Relógio crescente para medir duração. Argumentos usados: nenhum argumento explícito.

```
0090              saved = {'request_id': request_id, 'window': asdict(window), 'origin':
    origin_date,
```

Relógio monotônico começa antes da inferência e inclui tentativas/esperas; cache não ganha tempo zero, conserva a duração original armazenada.

asdict — Converte dataclass em dicionário de seus campos. Argumentos usados: posicional 1: window.

```
0091                        'history_rows': len(history), 'history_start':
    str(history.timestamp.iloc[0]),
```

Registro inicial guarda posição, limites de história e lista de covariáveis, antes de prever.

len — Conta elementos. Argumentos usados: posicional 1: history.

str — Produz representação textual. Argumentos usados: posicional 1: history.timestamp.iloc[0].

```
0092                        'history_end': str(history.timestamp.iloc[-1]), 'covariates':
    covariate_columns(dataset)}
```

Registro inicial guarda posição, limites de história e lista de covariáveis, antes de prever.

str — Produz representação textual. Argumentos usados: posicional 1: history.timestamp.iloc[-1].

covariate_columns — dataset: bloco de configuração de uma cidade. Retorna nomes numéricos seguidos dos categóricos, após rejeitar duplicatas e nomes reservados. Argumentos usados: posicional 1: dataset.

```
0093             try:
```

Registro inicial guarda posição, limites de história e lista de covariáveis, antes de prever.

```
0094                 predictions = model.predict(history, window.horizon, prompt=prompt,
    future_covariates=future)
```

Chamada comum ao adaptador. Para Qwen, prompt já contém os dados; para Chronos/TimesFM, history e future_covariates alimentam suas APIs numéricas.

model.predict — Interface comum dos adaptadores do projeto. Argumentos usados: posicional 1: history; posicional 2: window.horizon; prompt: prompt; future_covariates: future.

```
0095                 if len(predictions) != window.horizon or not np.isfinite(predictions).all():
```

Exige exatamente horizon números finitos. Impede zip de truncar silenciosamente previsões curtas depois.

len — Conta elementos. Argumentos usados: posicional 1: predictions.

np.isfinite(predictions).all — Exige que todos os elementos sejam True. Argumentos usados: nenhum argumento explícito.

np.isfinite — Testa finitude elemento a elemento. Argumentos usados: posicional 1: predictions.

```
0096                     raise ValueError('Invalid model output shape or finite values')
```

Exige exatamente horizon números finitos. Impede zip de truncar silenciosamente previsões curtas depois.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Invalid model output shape or finite values'.

```
0097                 saved.update(status='ok', predictions=predictions)
```

Marca sucesso somente após validar forma e finitude.

saved.update — Acrescenta conteúdo ao objeto mutável. Argumentos usados: status: 'ok'; predictions: predictions.

```
0098             except Exception as exc:
```

Captura exceções de inferência e registra classe/mensagem, permitindo continuar outras janelas. Não transforma erro em previsão zero.

```
0099                 saved.update(status='error', error=f'{type(exc).__name__}: {exc}')
```

Captura exceções de inferência e registra classe/mensagem, permitindo continuar outras janelas. Não transforma erro em previsão zero.

saved.update — Acrescenta conteúdo ao objeto mutável. Argumentos usados: status: 'error'; error: f'{type(exc).__name__}: {exc}'.

type — Obtém a classe concreta do valor. Argumentos usados: posicional 1: exc.

```
0100             saved.update(elapsed_seconds=time.monotonic()-started, audit=model.last_audit)
```

Acrescenta duração e last_audit do adaptador e grava arquivo por substituição temporária. A resposta bruta do Ollama permanece auditável.

saved.update — Acrescenta conteúdo ao objeto mutável. Argumentos usados: elapsed_seconds: time.monotonic() - started; audit: model.last_audit.

time.monotonic — Relógio crescente para medir duração. Argumentos usados: nenhum argumento explícito.

```
0101             write_json(target, saved)
```

Acrescenta duração e last_audit do adaptador e grava arquivo por substituição temporária. A resposta bruta do Ollama permanece auditável.

write_json — path: destino; value: dict serializável. Sem retorno útil. Cria pais, escreve .tmp e substitui destino. Argumentos usados: posicional 1: target; posicional 2: saved.

```
0102          print(f'{name}/{model_name} {index}/{len(windows)} {origin_date}: {saved["status"]}'
```

Progresso inclui cidade, modelo, contador, origem e status; sufixo cached indica reaproveitamento. flush=True atualiza log prontamente.

print — Escreve representação textual na saída padrão. Argumentos usados: posicional 1: f"{name}/{model_name} {index}/{len(windows)} {origin_date}: {saved['status']}" + (' cached' if cached else ''); flush: True.

len — Conta elementos. Argumentos usados: posicional 1: windows.

```
0103                + (' cached' if cached else ''), flush=True)
```

Progresso inclui cidade, modelo, contador, origem e status; sufixo cached indica reaproveitamento. flush=True atualiza log prontamente.

```
0104          if saved['status'] != 'ok':
```

Janela com falha entra na lista failures e não recebe linhas de previsão. Comparadores rejeitam esse resultado incompleto.

```
0105              failures.append({'origin': origin_date, 'error': saved['error']})
```

Janela com falha entra na lista failures e não recebe linhas de previsão. Comparadores rejeitam esse resultado incompleto.

failures.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: {'origin': origin_date, 'error': saved['error']}.

```
0106              continue
```

Janela com falha entra na lista failures e não recebe linhas de previsão. Comparadores rejeitam esse resultado incompleto.

```
0107          # Ground truth is accessed only AFTER model inference finishes.
```

Comentário de implementação: Ground truth is accessed only AFTER model inference finishes.. Não é executado; as operações efetivas são as linhas seguintes.

```
0108          truth = df.iloc[window.target]
```

Só depois da inferência acessa a tabela de alvos para avaliação; history/future já foram construídos antes sem essa coluna futura.

```
0109          for lead, ((_, observation), prediction) in enumerate(zip(truth.iterrows(),
    saved['predictions']), 1):
```

iterrows fornece (índice, linha); zip associa previsão na mesma posição; enumerate numera lead 1...7; _ descarta o índice pandas.

enumerate — Percorre iterável com contador. Argumentos usados: posicional 1: zip(truth.iterrows(), saved['predictions']); posicional 2: 1.

zip — Emparelha iteráveis pela posição. Argumentos usados: posicional 1: truth.iterrows(); posicional 2: saved['predictions'].

truth.iterrows — Itera pares (índice, Series da linha). Argumentos usados: nenhum argumento explícito.

```
0110              rows.append({'dataset': name, 'model': model_name, 'origin': origin_date,
```

Uma linha de resultado por lead. initial_context é origem inicial da avaliação; context é o histórico efetivamente fornecido. A duração da janela repete nas sete linhas e precisa ser desduplicada ao somar.

rows.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: {'dataset': name, 'model': model_name, 'origin': origin_date, 'timestamp': observation.timestamp.strftime('%Y-%m-%d'), 'lead': lead, 'y_true': float(observation.water_produced), 'y_pred': prediction, 'initial_context': dataset['context'], 'context': len(history), 'history_start': history.timestamp.iloc[0].strftime('%Y-%m-%d'), 'horizon': window.horizon, 'elapsed_seconds_window': saved['elapsed_seconds'], 'run_id': run_id}.

```
0111                            'timestamp': observation.timestamp.strftime('%Y-%m-%d'), 'lead':
    lead,
```

Uma linha de resultado por lead. initial_context é origem inicial da avaliação; context é o histórico efetivamente fornecido. A duração da janela repete nas sete linhas e precisa ser desduplicada ao somar.

observation.timestamp.strftime — Formata datas como texto. Argumentos usados: posicional 1: '%Y-%m-%d'.

```
0112                            'y_true': float(observation.water_produced), 'y_pred': prediction,
```

Uma linha de resultado por lead. initial_context é origem inicial da avaliação; context é o histórico efetivamente fornecido. A duração da janela repete nas sete linhas e precisa ser desduplicada ao somar.

float — Converte escalar para float Python. Argumentos usados: posicional 1: observation.water_produced.

```
0113                            'initial_context': dataset['context'], 'context': len(history),
```

Uma linha de resultado por lead. initial_context é origem inicial da avaliação; context é o histórico efetivamente fornecido. A duração da janela repete nas sete linhas e precisa ser desduplicada ao somar.

len — Conta elementos. Argumentos usados: posicional 1: history.

```
0114                            'history_start': history.timestamp.iloc[0].strftime('%Y-%m-%d'),
```

Uma linha de resultado por lead. initial_context é origem inicial da avaliação; context é o histórico efetivamente fornecido. A duração da janela repete nas sete linhas e precisa ser desduplicada ao somar.

history.timestamp.iloc[0].strftime — Formata datas como texto. Argumentos usados: posicional 1: '%Y-%m-%d'.

```
0115                            'horizon': window.horizon,
```

Uma linha de resultado por lead. initial_context é origem inicial da avaliação; context é o histórico efetivamente fornecido. A duração da janela repete nas sete linhas e precisa ser desduplicada ao somar.

```
0116                            'elapsed_seconds_window': saved['elapsed_seconds'], 'run_id':
    run_id})
```

Uma linha de resultado por lead. initial_context é origem inicial da avaliação; context é o histórico efetivamente fornecido. A duração da janela repete nas sete linhas e precisa ser desduplicada ao somar.

```
0117      pd.DataFrame(rows).to_csv(destination / 'predictions.csv', index=False)
```

Grava o CSV completo do dataset depois do loop. Durante uma execução interrompida, as respostas por janela são a fonte de retomada, não necessariamente esse CSV.

pd.DataFrame(rows).to_csv — Serializa DataFrame como CSV. Argumentos usados: posicional 1: destination / 'predictions.csv'; index: False.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: rows.

```
0118      write_json(destination / 'status.json', {'expected_windows': len(windows),
```

status resume janelas esperadas, concluídas e falhas. successful_windows usa divisão inteira do número de linhas pelo horizonte.

write_json — path: destino; value: dict serializável. Sem retorno útil. Cria pais, escreve .tmp e substitui destino. Argumentos usados: posicional 1: destination / 'status.json'; posicional 2: {'expected_windows': len(windows), 'successful_windows': len(rows) // config['horizon'], 'failures': failures}.

len — Conta elementos. Argumentos usados: posicional 1: windows.

```
0119                 'successful_windows': len(rows)//config['horizon'], 'failures': failures})
```

status resume janelas esperadas, concluídas e falhas. successful_windows usa divisão inteira do número de linhas pelo horizonte.

len — Conta elementos. Argumentos usados: posicional 1: rows.

```
0120      return len(failures)
```

Retorna quantidade de falhas, permitindo main definir o código de saída do processo.

len — Conta elementos. Argumentos usados: posicional 1: failures.

```
0123  def main():
```

Define main(). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0124      parser = argparse.ArgumentParser(description=__doc__)
```

Atribui a parser o resultado de argparse.ArgumentParser(description=__doc__). A expressão à direita é avaliada antes da atribuição.

argparse.ArgumentParser — Cria o parser da interface de linha de comando. Argumentos usados: description: __doc__.

```
0125      parser.add_argument('--model', choices=['qwen', 'chronos', 'timesfm'], default='qwen')
```

--model escolhe adaptador; padrão qwen. Não escolhe automaticamente o tamanho do checkpoint, que vem do JSON.

parser.add_argument — Registra uma opção do CLI. Argumentos usados: posicional 1: '--model'; choices: ['qwen', 'chronos', 'timesfm']; default: 'qwen'.

```
0126      parser.add_argument('--config', type=Path, default=ROOT / 'configs/benchmark.json')
```

--config vira Path e por padrão usa benchmark.json (Qwen1.7B). Para 4B é indispensável fornecer o JSON correspondente.

parser.add_argument — Registra uma opção do CLI. Argumentos usados: posicional 1: '--config'; type: Path; default: ROOT / 'configs/benchmark.json'.

```
0127      parser.add_argument('--dataset', choices=['all', 'joinville', 'guaratuba'],
    default='all')
```

--dataset all executa as duas cidades; uma cidade filtra o loop sem alterar seus parâmetros.

parser.add_argument — Registra uma opção do CLI. Argumentos usados: posicional 1: '--dataset'; choices: ['all', 'joinville', 'guaratuba']; default: 'all'.

```
0128      parser.add_argument('--output', type=Path, default=ROOT / 'experiments/benchmark')
```

--output escolhe pasta de artefatos; separar experimentos evita sobrepor configurações incompatíveis.

parser.add_argument — Registra uma opção do CLI. Argumentos usados: posicional 1: '--output'; type: Path; default: ROOT / 'experiments/benchmark'.

```
0129      parser.add_argument('--limit', type=int, default=0, help='Smoke test only; 0 evaluates
    every window')
```

--limit serve para teste rápido; 0 é todas as janelas. Não use métricas desse recorte como benchmark completo.

parser.add_argument — Registra uma opção do CLI. Argumentos usados: posicional 1: '--limit'; type: int; default: 0; help: 'Smoke test only; 0 evaluates every window'.

```
0130      args = parser.parse_args()
```

Atribui a args o resultado de parser.parse_args(). A expressão à direita é avaliada antes da atribuição.

parser.parse_args — Interpreta argumentos do processo e retorna Namespace. Argumentos usados: nenhum argumento explícito.

```
0131      if args.limit < 0:
```

Avalia a condição args.limit < 0. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0132          parser.error('--limit must be nonnegative')
```

Executa parser.error('--limit must be nonnegative'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

parser.error — Em ArgumentParser, mostra mensagem e encerra com erro de CLI. Argumentos usados: posicional 1: '--limit must be nonnegative'.

```
0133      config = json.loads(args.config.read_text())
```

read_text retorna string; json.loads a transforma em dict. Sem esse segundo passo, acessar config['datasets'] não funcionaria.

json.loads — Decodifica uma string JSON em objetos Python. Argumentos usados: posicional 1: args.config.read_text().

args.config.read_text — Lê todo o arquivo como string. Argumentos usados: nenhum argumento explícito.

```
0134      failures = 0
```

Atribui a failures o resultado de 0. A expressão à direita é avaliada antes da atribuição.

```
0135      for name, dataset in config['datasets'].items():
```

Itera entradas de datasets, aplica filtro all/nome e acumula número de janelas que falharam em run_dataset.

config['datasets'].items — Percorre pares (chave, valor) do dicionário. Argumentos usados: nenhum argumento explícito.

```
0136          if args.dataset in ('all', name):
```

Itera entradas de datasets, aplica filtro all/nome e acumula número de janelas que falharam em run_dataset.

```
0137              failures += run_dataset(name, dataset, config, args.model, args.output,
    args.limit)
```

Itera entradas de datasets, aplica filtro all/nome e acumula número de janelas que falharam em run_dataset.

run_dataset — name: cidade; dataset: bloco da cidade; config: parâmetros globais; model_name: adaptador; output: raiz de resultados; limit=0: todas as janelas. Retorna número de falhas e grava artefatos. Argumentos usados: posicional 1: name; posicional 2: dataset; posicional 3: config; posicional 4: args.model; posicional 5: args.output; posicional 6: args.limit.

```
0138      raise SystemExit(1 if failures else 0)
```

Código 0 indica sucesso; 1 indica alguma falha. Em shells com set -e isso impede executar comparação como se todos os modelos tivessem terminado.

SystemExit — Solicita encerramento do processo Python. Argumentos usados: posicional 1: 1 if failures else 0.

```
0141  if __name__ == '__main__':
```

Guarda de entrada: executa main apenas como programa/módulo -m, não quando outro arquivo importa ROOT ou funções.

```
0142      main()
```

Guarda de entrada: executa main apenas como programa/módulo -m, não quando outro arquivo importa ROOT ou funções.

main — Sem parâmetros explícitos. Lê argumentos do processo via argparse ou executa entry point do módulo. O corpo de cada arquivo determina qual CLI será iniciado. Argumentos usados: nenhum argumento explícito.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

18. src/pipelines/data_pipeline.py

Composição de leitura, grade diária, preenchimento e janelas. Chamado por run_dataset, testes e auditoria. Retorna o par (DataFrame, lista de ForecastWindow).

SHA-256: 126ca72f0c2e882bd547b093cfd1c557f9452c338de59a8db9fd9f60ca8cff7f

Funções, contratos e pontos de conexão

```
L8: prepare_dataset(path, context, *, horizon = 7, step = 7)
```

path: CSV; context: primeira origem; horizon=7 e step=7 nomeados. Retorna df e windows depois da preparação.

Chamadas do projeto: build_windows → src/evaluation/cross_validation.py:20; daily_grid → src/data/preprocessing/resampling.py:5; fill_past_only → src/data/preprocessing/missing_values.py:5; read_series → src/data/ingestion/files.py:8

Código comentado, linha a linha

```
0001  """One data preparation path shared by prompts, models and evaluation."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  from src.data.ingestion.files import read_series
```

Importa read_series de src.data.ingestion.files. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0003  from src.data.preprocessing.resampling import daily_grid
```

Importa daily_grid de src.data.preprocessing.resampling. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0004  from src.data.preprocessing.missing_values import fill_past_only
```

Importa fill_past_only de src.data.preprocessing.missing_values. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  from src.evaluation.cross_validation import build_windows
```

Importa build_windows de src.evaluation.cross_validation. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0008  def prepare_dataset(path, context, *, horizon=7, step=7):
```

Define prepare_dataset(path, context, *, horizon = 7, step = 7). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0009      df = fill_past_only(daily_grid(read_series(path)))
```

Atribui a df o resultado de fill_past_only(daily_grid(read_series(path))). A expressão à direita é avaliada antes da atribuição.

fill_past_only — df: grade com possíveis NaN. Retorno: cópia preenchida com passado. Não reconstrói atributos de calendário, limitação conhecida. Argumentos usados: posicional 1: daily_grid(read_series(path)).

daily_grid — df: tabela com timestamp. Retorno: grade diária com observed. Dias ausentes são expostos, não removidos. Argumentos usados: posicional 1: read_series(path).

read_series — path: caminho CSV; retorno: DataFrame validado. Encadeia pandas.read_csv e validate_series. Argumentos usados: posicional 1: path.

```
0010      windows = build_windows(df, context, horizon, step)
```

Atribui a windows o resultado de build_windows(df, context, horizon, step). A expressão à direita é avaliada antes da atribuição.

build_windows — df: tabela preparada; context: primeira origem; horizon=7: quantidade prevista; step=7: avanço entre origens. Retorna lista ForecastWindow, excluindo janelas com alvo imputado. Argumentos usados: posicional 1: df; posicional 2: context; posicional 3: horizon; posicional 4: step.

```
0011      return df, windows
```

Encerra esta função e devolve (df, windows) ao chamador; não imprime o resultado por si só.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

19. src/data/ingestion/files.py

Entrada/saída de arquivos. Chamado por data_pipeline e forecasting_pipeline. read_series retorna DataFrame validado; write_json grava objetos serializáveis por substituição de arquivo temporário.

SHA-256: 3ace075a801a52dc87b6e29189097bc80204079935544367526c367fdcb27901

Funções, contratos e pontos de conexão

```
L8: read_series(path: str | Path) → pd.DataFrame
```

path: caminho CSV; retorno: DataFrame validado. Encadeia pandas.read_csv e validate_series.

Chamadas do projeto: validate_series → src/data/validation/schema.py:6

```
L12: write_json(path: str | Path, value: dict) → None
```

path: destino; value: dict serializável. Sem retorno útil. Cria pais, escreve .tmp e substitui destino.

APIs externas usadas neste arquivo

Path, json.dumps, mkdir, pd.read_csv, replace, with_suffix, write_text. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Filesystem input and atomic artifact writes."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import json
```

Importa json. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0003  from pathlib import Path
```

Importa Path de pathlib. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0004  import pandas as pd
```

Importa pandas como pd. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  from src.data.validation.schema import validate_series
```

Importa validate_series de src.data.validation.schema. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0008  def read_series(path: str | Path) -> pd.DataFrame:
```

Define read_series(path: str | Path) → pd.DataFrame. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0009      return validate_series(pd.read_csv(path))
```

Encerra esta função e devolve validate_series(pd.read_csv(path)) ao chamador; não imprime o resultado por si só.

validate_series — df: tabela lida. Retorno: cópia ordenada. Rejeita esquema, datas e demanda inválidos antes de qualquer modelo. Argumentos usados: posicional 1: pd.read_csv(path).

pd.read_csv — Lê CSV em um DataFrame, inferindo tipos; datas só são convertidas posteriormente no validador. Argumentos usados: posicional 1: path.

```
0012  def write_json(path: str | Path, value: dict) -> None:
```

Define write_json(path: str | Path, value: dict) → None. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0013      """Replace a complete JSON file atomically; a crash cannot leave half a cache."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0014      path = Path(path)
```

Atribui a path o resultado de Path(path). A expressão à direita é avaliada antes da atribuição.

Path — Representa caminho de arquivo com operações portáveis. Argumentos usados: posicional 1: path.

```
0015      path.parent.mkdir(parents=True, exist_ok=True)
```

Executa path.parent.mkdir(parents=True, exist_ok=True). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

path.parent.mkdir — Cria diretório. Argumentos usados: parents: True; exist_ok: True.

```
0016      temporary = path.with_suffix('.tmp')
```

Atribui a temporary o resultado de path.with_suffix('.tmp'). A expressão à direita é avaliada antes da atribuição.

path.with_suffix — Produz outro Path trocando a extensão. Argumentos usados: posicional 1: '.tmp'.

```
0017      temporary.write_text(json.dumps(value, ensure_ascii=False, indent=2, allow_nan=False) +
    '\n')
```

Executa temporary.write_text(json.dumps(value, ensure_ascii=False, indent=2, allow_nan=False) + '\n'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

temporary.write_text — Grava uma string em arquivo e retorna a quantidade de caracteres escrita. Argumentos usados: posicional 1: json.dumps(value, ensure_ascii=False, indent=2, allow_nan=False) + '\n'.

json.dumps — Serializa objeto Python em string JSON. Argumentos usados: posicional 1: value; ensure_ascii: False; indent: 2; allow_nan: False.

```
0018      temporary.replace(path)
```

Executa temporary.replace(path). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

temporary.replace — Substitui o destino pelo arquivo representado pelo Path. Argumentos usados: posicional 1: path.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

20. src/data/validation/schema.py

Contrato do CSV: datas diárias únicas e demanda finita, não negativa. Chamado por read_series. Retorna cópia ordenada, preservando covariáveis; erro invalida o dataset antes do modelo.

SHA-256: d5c72310716276c63210b694a2e7f0d0cdc7b267b476dac2ab1e412a1436e12c

Funções, contratos e pontos de conexão

```
L6: validate_series(df: pd.DataFrame) → pd.DataFrame
```

df: tabela lida. Retorno: cópia ordenada. Rejeita esquema, datas e demanda inválidos antes de qualquer modelo.

APIs externas usadas neste arquivo

ValueError, all, any, copy, duplicated, eq, isna, issubset, normalize, np.isfinite, pd.to_datetime, pd.to_numeric, reset_index, set, sort_values, to_numpy. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Validate observed data without silently dropping or repairing observations."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import numpy as np
```

Importa numpy como np. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0003  import pandas as pd
```

Importa pandas como pd. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0006  def validate_series(df: pd.DataFrame) -> pd.DataFrame:
```

Define validate_series(df: pd.DataFrame) → pd.DataFrame. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0007      """Require unique daily dates and finite, nonnegative water demand."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0008      required = {'timestamp', 'water_produced'}
```

Atribui a required o resultado de {'timestamp', 'water_produced'}. A expressão à direita é avaliada antes da atribuição.

```
0009      if not required.issubset(df.columns):
```

Avalia a condição not required.issubset(df.columns). O bloco indentado seguinte só é executado quando ela é verdadeira.

required.issubset — Verifica se todos os elementos estão contidos em outro conjunto/iterável. Argumentos usados: posicional 1: df.columns.

```
0010          raise ValueError(f'Missing columns: {required - set(df.columns)}')
```

Interrompe o caminho atual lançando ValueError(f'Missing columns: {required - set(df.columns)}'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: f'Missing columns: {required - set(df.columns)}'.

set — Cria conjunto de elementos únicos. Argumentos usados: posicional 1: df.columns.

```
0011      result = df.copy()
```

Atribui a result o resultado de df.copy(). A expressão à direita é avaliada antes da atribuição.

df.copy — Copia uma tabela/dicionário para não alterar diretamente o objeto anterior. Argumentos usados: nenhum argumento explícito.

```
0012      result['timestamp'] = pd.to_datetime(result['timestamp'], errors='raise')
```

Atribui a result['timestamp'] o resultado de pd.to_datetime(result['timestamp'], errors='raise'). A expressão à direita é avaliada antes da atribuição.

pd.to_datetime — Converte valores em datas pandas. Argumentos usados: posicional 1: result['timestamp']; errors: 'raise'.

```
0013      if result.empty or result.timestamp.isna().any() or result.timestamp.duplicated().any():
```

Avalia a condição result.empty or result.timestamp.isna().any() or result.timestamp.duplicated().any(). O bloco indentado seguinte só é executado quando ela é verdadeira.

result.timestamp.isna().any — Reduz a existência de pelo menos um True. Argumentos usados: nenhum argumento explícito.

result.timestamp.duplicated().any — Reduz a existência de pelo menos um True. Argumentos usados: nenhum argumento explícito.

result.timestamp.isna — Retorna máscara booleana de valores ausentes. Argumentos usados: nenhum argumento explícito.

result.timestamp.duplicated — Marca repetições. Argumentos usados: nenhum argumento explícito.

```
0014          raise ValueError('Dates must be nonempty, nonmissing and unique')
```

Interrompe o caminho atual lançando ValueError('Dates must be nonempty, nonmissing and unique'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Dates must be nonempty, nonmissing and unique'.

```
0015      if not result.timestamp.eq(result.timestamp.dt.normalize()).all():
```

Avalia a condição not result.timestamp.eq(result.timestamp.dt.normalize()).all(). O bloco indentado seguinte só é executado quando ela é verdadeira.

result.timestamp.eq(result.timestamp.dt.normalize()).all — Exige que todos os elementos sejam True. Argumentos usados: nenhum argumento explícito.

result.timestamp.eq — Compara igualdade elemento a elemento. Argumentos usados: posicional 1: result.timestamp.dt.normalize().

result.timestamp.dt.normalize — Zera o horário de cada timestamp, preservando sua data. Argumentos usados: nenhum argumento explícito.

```
0016          raise ValueError('Expected daily timestamps at midnight')
```

Interrompe o caminho atual lançando ValueError('Expected daily timestamps at midnight'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Expected daily timestamps at midnight'.

```
0017      result['water_produced'] = pd.to_numeric(result.water_produced, errors='raise')
```

Atribui a result['water_produced'] o resultado de pd.to_numeric(result.water_produced, errors='raise'). A expressão à direita é avaliada antes da atribuição.

pd.to_numeric — Converte texto/objetos em números. Argumentos usados: posicional 1: result.water_produced; errors: 'raise'.

```
0018      values = result.water_produced.to_numpy()
```

Atribui a values o resultado de result.water_produced.to_numpy(). A expressão à direita é avaliada antes da atribuição.

result.water_produced.to_numpy — Converte Series/DataFrame em array NumPy. Argumentos usados: nenhum argumento explícito.

```
0019      if not np.isfinite(values).all() or (values < 0).any():
```

Avalia a condição not np.isfinite(values).all() or (values < 0).any(). O bloco indentado seguinte só é executado quando ela é verdadeira.

(values < 0).any — Reduz a existência de pelo menos um True. Argumentos usados: nenhum argumento explícito.

np.isfinite(values).all — Exige que todos os elementos sejam True. Argumentos usados: nenhum argumento explícito.

np.isfinite — Testa finitude elemento a elemento. Argumentos usados: posicional 1: values.

```
0020          raise ValueError('Demand must be finite and nonnegative')
```

Interrompe o caminho atual lançando ValueError('Demand must be finite and nonnegative'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Demand must be finite and nonnegative'.

```
0021      return result.sort_values('timestamp').reset_index(drop=True)
```

Encerra esta função e devolve result.sort_values('timestamp').reset_index(drop=True) ao chamador; não imprime o resultado por si só.

result.sort_values('timestamp').reset_index — Transforma o índice atual em coluna ou o descarta e cria índice sequencial. Argumentos usados: drop: True.

result.sort_values — Ordena linhas pelos valores de colunas. Argumentos usados: posicional 1: 'timestamp'.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

21. src/data/preprocessing/resampling.py

Converte datas observadas em grade diária. Chamado por prepare_dataset antes do preenchimento. Adiciona observed para distinguir alvos reais de linhas inseridas.

SHA-256: 33d458e31acaed8a8d826b01d2b7d5b20f45a0825073b66d4092ecf40c4995e0

Funções, contratos e pontos de conexão

```
L5: daily_grid(df: pd.DataFrame) → pd.DataFrame
```

df: tabela com timestamp. Retorno: grade diária com observed. Dias ausentes são expostos, não removidos.

APIs externas usadas neste arquivo

asfreq, notna, rename_axis, reset_index, set_index. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Expose missing calendar days explicitly; do not interpolate future values."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import pandas as pd
```

Importa pandas como pd. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  def daily_grid(df: pd.DataFrame) -> pd.DataFrame:
```

Define daily_grid(df: pd.DataFrame) → pd.DataFrame. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0006      result = df.set_index('timestamp').asfreq('D')
```

Atribui a result o resultado de df.set_index('timestamp').asfreq('D'). A expressão à direita é avaliada antes da atribuição.

df.set_index('timestamp').asfreq — Reindexa a série para uma frequência temporal regular. Argumentos usados: posicional 1: 'D'.

df.set_index — Transforma coluna em índice do DataFrame. Argumentos usados: posicional 1: 'timestamp'.

```
0007      result['observed'] = result.water_produced.notna()
```

Atribui a result['observed'] o resultado de result.water_produced.notna(). A expressão à direita é avaliada antes da atribuição.

result.water_produced.notna — Retorna True onde o valor não é ausente. Argumentos usados: nenhum argumento explícito.

```
0008      return result.rename_axis('timestamp').reset_index()
```

Encerra esta função e devolve result.rename_axis('timestamp').reset_index() ao chamador; não imprime o resultado por si só.

result.rename_axis('timestamp').reset_index — Transforma o índice atual em coluna ou o descarta e cria índice sequencial. Argumentos usados: nenhum argumento explícito.

result.rename_axis — Nomeia o eixo do índice. Argumentos usados: posicional 1: 'timestamp'.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

22. src/data/preprocessing/missing_values.py

Preenchimento causal. Chamado após daily_grid. Retorna tabela sem lacunas iniciais de demanda; o preenchimento indiscriminado do calendário é um defeito confirmado, não uma prática recomendada.

SHA-256: 48d7a521417ef68f727b15797fb8d8c33dfe205f71db9dddd890574ecf1e297a

Funções, contratos e pontos de conexão

```
L5: fill_past_only(df: pd.DataFrame) → pd.DataFrame
```

df: grade com possíveis NaN. Retorno: cópia preenchida com passado. Não reconstrói atributos de calendário, limitação conhecida.

APIs externas usadas neste arquivo

ValueError, any, copy, ffill, isna. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Causal missing-day treatment; imputed targets never enter evaluation."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import pandas as pd
```

Importa pandas como pd. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  def fill_past_only(df: pd.DataFrame) -> pd.DataFrame:
```

Define fill_past_only(df: pd.DataFrame) → pd.DataFrame. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0006      result = df.copy()
```

Atribui a result o resultado de df.copy(). A expressão à direita é avaliada antes da atribuição.

df.copy — Copia uma tabela/dicionário para não alterar diretamente o objeto anterior. Argumentos usados: nenhum argumento explícito.

```
0007      columns = [c for c in result.columns if c not in {'timestamp', 'observed'}]
```

Seleciona TODAS as colunas exceto data e observed. Isso inclui calendário; é a origem do defeito identificado na auditoria de 29/02/2016.

```
0008      result[columns] = result[columns].ffill()
```

Propaga passado sem vazamento futuro, mas copia day/weekday/is_weekend indevidamente para dias inseridos. Preencher demanda causalmente e reconstruir calendário são tarefas distintas.

result[columns].ffill — Propaga o último valor válido anterior para lacunas subsequentes. Argumentos usados: nenhum argumento explícito.

```
0009      if result.water_produced.isna().any():
```

Avalia a condição result.water_produced.isna().any(). O bloco indentado seguinte só é executado quando ela é verdadeira.

result.water_produced.isna().any — Reduz a existência de pelo menos um True. Argumentos usados: nenhum argumento explícito.

result.water_produced.isna — Retorna máscara booleana de valores ausentes. Argumentos usados: nenhum argumento explícito.

```
0010          raise ValueError('Cannot fill initial missing demand without past data')
```

Interrompe o caminho atual lançando ValueError('Cannot fill initial missing demand without past data'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Cannot fill initial missing demand without past data'.

```
0011      return result
```

Encerra esta função e devolve result ao chamador; não imprime o resultado por si só.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

23. src/evaluation/cross_validation.py

Define ForecastWindow e origens walk-forward. prepare_dataset chama build_windows. validation_split é usado por training_partition e testes, não para ajustar os modelos neste benchmark.

SHA-256: fe467e111af3b2907d9aa42f753c01afc8b9aeaab7194583c8542ca9aeab3a70

Funções, contratos e pontos de conexão

```
L12: history(self) → slice
```

Property calculada de ForecastWindow: retorna slice, sem materializar a tabela até o chamador aplicá-lo com .iloc.

```
L16: target(self) → slice
```

Property calculada de ForecastWindow: retorna slice, sem materializar a tabela até o chamador aplicá-lo com .iloc.

```
L20: build_windows(df, context: int, horizon: int = 7, step: int = 7) → list[ForecastWindow]
```

df: tabela preparada; context: primeira origem; horizon=7: quantidade prevista; step=7: avanço entre origens. Retorna lista ForecastWindow, excluindo janelas com alvo imputado.

Chamadas do projeto: ForecastWindow → src/evaluation/cross_validation.py:6

```
L32: validation_split(training_size: int, fraction: float = 0.2) → tuple[slice, slice]
```

training_size: comprimento do prefixo anterior ao teste; fraction=0.2: parte final reservada à validação. Retorna dois slices sem embaralhar.

APIs externas usadas neste arquivo

ForecastWindow, ValueError, all, dataclass, int, len, max, min, range, slice. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Identical, chronological, expanding-history rolling origins for all models."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  from dataclasses import dataclass
```

Importa dataclass de dataclasses. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  @dataclass(frozen=True)
```

Aplica o decorador dataclass(frozen=True) à classe declarada em seguida; gera comportamento de dataclass.

dataclass — Decora a classe para gerar métodos a partir dos campos anotados. Argumentos usados: frozen: True.

```
0006  class ForecastWindow:
```

Declara a classe ForecastWindow. Seus métodos compartilham estado por self; a declaração não cria uma previsão.

```
0007      context_start: int
```

Declara o campo context_start com tipo esperado int, a ser fornecido na construção. A anotação não faz conversão automática.

```
0008      origin: int
```

Declara o campo origin com tipo esperado int, a ser fornecido na construção. A anotação não faz conversão automática.

```
0009      horizon: int
```

Declara o campo horizon com tipo esperado int, a ser fornecido na construção. A anotação não faz conversão automática.

```
0011      @property
```

Aplica o decorador property ao método seguinte; permite acessá-lo como atributo calculado.

```
0012      def history(self) -> slice:
```

Define history(self) → slice. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0013          return slice(self.context_start, self.origin)
```

Encerra esta função e devolve slice(self.context_start, self.origin) ao chamador; não imprime o resultado por si só.

slice — Representa intervalo, não materializa dados. Argumentos usados: posicional 1: self.context_start; posicional 2: self.origin.

```
0015      @property
```

Aplica o decorador property ao método seguinte; permite acessá-lo como atributo calculado.

```
0016      def target(self) -> slice:
```

Define target(self) → slice. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0017          return slice(self.origin, self.origin + self.horizon)
```

Encerra esta função e devolve slice(self.origin, self.origin + self.horizon) ao chamador; não imprime o resultado por si só.

slice — Representa intervalo, não materializa dados. Argumentos usados: posicional 1: self.origin; posicional 2: self.origin + self.horizon.

```
0020  def build_windows(df, context: int, horizon: int = 7, step: int = 7) -> list[ForecastWindow]:
```

Define build_windows(df, context: int, horizon: int = 7, step: int = 7) → list[ForecastWindow]. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0021      """Context is the initial history length. Every later history starts at zero."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0022      if min(context, horizon, step) < 1:
```

Avalia a condição min(context, horizon, step) < 1. O bloco indentado seguinte só é executado quando ela é verdadeira.

min — Retorna menor valor. Argumentos usados: posicional 1: context; posicional 2: horizon; posicional 3: step.

```
0023          raise ValueError('Window parameters must be positive')
```

Interrompe o caminho atual lançando ValueError('Window parameters must be positive'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Window parameters must be positive'.

```
0024      windows = [ForecastWindow(0, origin, horizon)
```

Atribui a windows o resultado de [ForecastWindow(0, origin, horizon) for origin in range(context, len(df) - horizon + 1, step) if df.iloc[origin:origin + horizon].observed.all()]. A expressão à direita é avaliada antes da atribuição.

ForecastWindow — Constrói dataclass de uma origem temporal. Argumentos usados: posicional 1: 0; posicional 2: origin; posicional 3: horizon.

```
0025                  for origin in range(context, len(df) - horizon + 1, step)
```

Cláusula de compreensão: percorre os valores indicados para construir a coleção da instrução anterior, mantendo a ordem desse percurso.

range — Gera inteiros com limite final exclusivo. Argumentos usados: posicional 1: context; posicional 2: len(df) - horizon + 1; posicional 3: step.

len — Conta elementos. Argumentos usados: posicional 1: df.

```
0026                 if df.iloc[origin:origin + horizon].observed.all()]
```

Filtro da compreensão iniciada acima; apenas elementos que atendem à condição são incluídos.

df.iloc[origin:origin + horizon].observed.all — Exige que todos os elementos sejam True. Argumentos usados: nenhum argumento explícito.

```
0027      if not windows:
```

Avalia a condição not windows. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0028          raise ValueError('No complete observed evaluation windows')
```

Interrompe o caminho atual lançando ValueError('No complete observed evaluation windows'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'No complete observed evaluation windows'.

```
0029      return windows
```

Encerra esta função e devolve windows ao chamador; não imprime o resultado por si só.

```
0032  def validation_split(training_size: int, fraction: float = 0.2) -> tuple[slice, slice]:
```

Define validation_split(training_size: int, fraction: float = 0.2) → tuple[slice, slice]. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0033      """Last 20% of a supplied training prefix; never split a test interval."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0034      if training_size < 2 or not 0 < fraction < 1:
```

Avalia a condição training_size < 2 or not 0 < fraction < 1. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0035          raise ValueError('Invalid training size or validation fraction')
```

Interrompe o caminho atual lançando ValueError('Invalid training size or validation fraction'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Invalid training size or validation fraction'.

```
0036      size = max(1, int(training_size * fraction))
```

Atribui a size o resultado de max(1, int(training_size * fraction)). A expressão à direita é avaliada antes da atribuição.

max — Retorna maior valor. Argumentos usados: posicional 1: 1; posicional 2: int(training_size * fraction).

int — Converte para inteiro, truncando parte fracionária numérica. Argumentos usados: posicional 1: training_size * fraction.

```
0037      return slice(0, training_size - size), slice(training_size - size, training_size)
```

Encerra esta função e devolve (slice(0, training_size - size), slice(training_size - size, training_size)) ao chamador; não imprime o resultado por si só.

slice — Representa intervalo, não materializa dados. Argumentos usados: posicional 1: 0; posicional 2: training_size - size.

slice — Representa intervalo, não materializa dados. Argumentos usados: posicional 1: training_size - size; posicional 2: training_size.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

24. src/data/features/temporal.py

Geração determinística das datas-alvo a partir da última data histórica. Chamado pelos dois construtores de prompt; não consulta a demanda futura.

SHA-256: 05e1165987546b2320e6dff5a8709cf646b98db9f8f5281c0a8092bce8144c24

Funções, contratos e pontos de conexão

```
L5: future_dates(last_observed: pd.Timestamp, horizon: int) → pd.DatetimeIndex
```

last_observed: último Timestamp histórico; horizon: quantidade de dias. Retorna DatetimeIndex dos próximos dias.

APIs externas usadas neste arquivo

pd.Timedelta, pd.date_range. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Only deterministic calendar data may be computed for future dates."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import pandas as pd
```

Importa pandas como pd. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  def future_dates(last_observed: pd.Timestamp, horizon: int) -> pd.DatetimeIndex:
```

Define future_dates(last_observed: pd.Timestamp, horizon: int) → pd.DatetimeIndex. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0006      return pd.date_range(last_observed + pd.Timedelta(days=1), periods=horizon, freq='D')
```

Encerra esta função e devolve pd.date_range(last_observed + pd.Timedelta(days=1), periods=horizon, freq='D') ao chamador; não imprime o resultado por si só.

pd.date_range — Produz uma sequência ordenada de timestamps. Argumentos usados: posicional 1: last_observed + pd.Timedelta(days=1); periods: horizon; freq: 'D'.

pd.Timedelta — Representa uma duração temporal. Argumentos usados: days: 1.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

25. src/data/features/covariates.py

Contrato comum dos modelos: nomes de covariáveis, validação, separação history/future e conversão para TimesFM. model_inputs é a fronteira que exclui estruturalmente o alvo da entrada futura.

SHA-256: 265c84d324e98cc1db784ef3d155806d742adb6e360735207cc6f9881fe9eddd

Funções, contratos e pontos de conexão

```
L6: covariate_columns(dataset: dict) → list[str]
```

dataset: bloco de configuração de uma cidade. Retorna nomes numéricos seguidos dos categóricos, após rejeitar duplicatas e nomes reservados.

```
L13: validate_covariates(df: pd.DataFrame, dataset: dict) → None
```

df: tabela preparada; dataset: contrato de colunas. Retorna None; falha em ausentes/NaN/números não finitos.

Chamadas do projeto: covariate_columns → src/data/features/covariates.py:6

```
L25: model_inputs(df, window, dataset, *, history_days = None)
```

df: série completa; window: ForecastWindow; dataset: contrato; history_days: None para expansivo ou inteiro para últimos N dias. Retorna (history, future), excluindo alvo do futuro.

Chamadas do projeto: covariate_columns → src/data/features/covariates.py:6

```
L36: timesfm_covariates(history, future, dataset)
```

history/future: tabelas separadas; dataset: listas numéricas/categóricas. Retorna dois dicts com listas de séries cobrindo passado e futuro, sem demanda futura.

Chamadas do projeto: covariate_columns → src/data/features/covariates.py:6

APIs externas usadas neste arquivo

ValueError, all, any, astype, copy, isna, len, max, min, np.isfinite, pd.concat, pd.to_numeric, set, to_numpy, tolist. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """One explicit covariate contract, shared by Chronos, TimesFM and Qwen."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import numpy as np
```

Importa numpy como np. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0003  import pandas as pd
```

Importa pandas como pd. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0006  def covariate_columns(dataset: dict) -> list[str]:
```

Define covariate_columns(dataset: dict) → list[str]. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0007      columns = dataset['numerical_covariates'] + dataset['categorical_covariates']
```

Atribui a columns o resultado de dataset['numerical_covariates'] + dataset['categorical_covariates']. A expressão à direita é avaliada antes da atribuição.

```
0008      if len(columns) != len(set(columns)) or {'timestamp', 'water_produced', 'observed',
    'unique_id'} & set(columns):
```

Avalia a condição len(columns) != len(set(columns)) or {'timestamp', 'water_produced', 'observed', 'unique_id'} & set(columns). O bloco indentado seguinte só é executado quando ela é verdadeira.

len — Conta elementos. Argumentos usados: posicional 1: columns.

len — Conta elementos. Argumentos usados: posicional 1: set(columns).

set — Cria conjunto de elementos únicos. Argumentos usados: posicional 1: columns.

set — Cria conjunto de elementos únicos. Argumentos usados: posicional 1: columns.

```
0009          raise ValueError('Covariates must be unique and cannot contain the target or
    metadata')
```

Interrompe o caminho atual lançando ValueError('Covariates must be unique and cannot contain the target or metadata'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Covariates must be unique and cannot contain the target or metadata'.

```
0010      return columns
```

Encerra esta função e devolve columns ao chamador; não imprime o resultado por si só.

```
0013  def validate_covariates(df: pd.DataFrame, dataset: dict) -> None:
```

Define validate_covariates(df: pd.DataFrame, dataset: dict) → None. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0014      columns = covariate_columns(dataset)
```

Atribui a columns o resultado de covariate_columns(dataset). A expressão à direita é avaliada antes da atribuição.

covariate_columns — dataset: bloco de configuração de uma cidade. Retorna nomes numéricos seguidos dos categóricos, após rejeitar duplicatas e nomes reservados. Argumentos usados: posicional 1: dataset.

```
0015      missing = set(columns) - set(df.columns)
```

Atribui a missing o resultado de set(columns) - set(df.columns). A expressão à direita é avaliada antes da atribuição.

set — Cria conjunto de elementos únicos. Argumentos usados: posicional 1: columns.

set — Cria conjunto de elementos únicos. Argumentos usados: posicional 1: df.columns.

```
0016      if missing:
```

Avalia a condição missing. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0017          raise ValueError(f'Missing configured covariates: {missing}')
```

Interrompe o caminho atual lançando ValueError(f'Missing configured covariates: {missing}'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: f'Missing configured covariates: {missing}'.

```
0018      if df[columns].isna().any().any():
```

Avalia a condição df[columns].isna().any().any(). O bloco indentado seguinte só é executado quando ela é verdadeira.

df[columns].isna().any().any — Reduz a existência de pelo menos um True. Argumentos usados: nenhum argumento explícito.

df[columns].isna().any — Reduz a existência de pelo menos um True. Argumentos usados: nenhum argumento explícito.

df[columns].isna — Retorna máscara booleana de valores ausentes. Argumentos usados: nenhum argumento explícito.

```
0019          raise ValueError('Covariates contain missing values')
```

Interrompe o caminho atual lançando ValueError('Covariates contain missing values'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Covariates contain missing values'.

```
0020      for column in dataset['numerical_covariates']:
```

Percorre dataset['numerical_covariates']; em cada iteração associa o item a column. O corpo indentado aplica a operação a esse item/grupo.

```
0021         if not np.isfinite(pd.to_numeric(df[column], errors='raise').to_numpy()).all():
```

Avalia a condição not np.isfinite(pd.to_numeric(df[column], errors='raise').to_numpy()).all(). O bloco indentado seguinte só é executado quando ela é verdadeira.

np.isfinite(pd.to_numeric(df[column], errors='raise').to_numpy()).all — Exige que todos os elementos sejam True. Argumentos usados: nenhum argumento explícito.

np.isfinite — Testa finitude elemento a elemento. Argumentos usados: posicional 1: pd.to_numeric(df[column], errors='raise').to_numpy().

pd.to_numeric(df[column], errors='raise').to_numpy — Converte Series/DataFrame em array NumPy. Argumentos usados: nenhum argumento explícito.

pd.to_numeric — Converte texto/objetos em números. Argumentos usados: posicional 1: df[column]; errors: 'raise'.

```
0022             raise ValueError(f'Nonfinite covariate: {column}')
```

Interrompe o caminho atual lançando ValueError(f'Nonfinite covariate: {column}'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: f'Nonfinite covariate: {column}'.

```
0025  def model_inputs(df, window, dataset, *, history_days=None):
```

Define model_inputs(df, window, dataset, *, history_days = None). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0026     """Exclude target structurally from the future frame, including for Qwen."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0027     columns = covariate_columns(dataset)
```

Atribui a columns o resultado de covariate_columns(dataset). A expressão à direita é avaliada antes da atribuição.

covariate_columns — dataset: bloco de configuração de uma cidade. Retorna nomes numéricos seguidos dos categóricos, após rejeitar duplicatas e nomes reservados. Argumentos usados: posicional 1: dataset.

```
0028     start = window.context_start if history_days is None else max(window.context_start,
    window.origin-history_days)
```

Atribui a start o resultado de window.context_start if history_days is None else max(window.context_start, window.origin - history_days). A expressão à direita é avaliada antes da atribuição.

max — Retorna maior valor. Argumentos usados: posicional 1: window.context_start; posicional 2: window.origin - history_days.

```
0029     history = df.iloc[start:window.origin][['timestamp', 'water_produced'] + columns].copy()
```

Atribui a history o resultado de df.iloc[start:window.origin][['timestamp', 'water_produced'] + columns].copy(). A expressão à direita é avaliada antes da atribuição.

df.iloc[start:window.origin][['timestamp', 'water_produced'] + columns].copy — Copia uma tabela/dicionário para não alterar diretamente o objeto anterior. Argumentos usados: nenhum argumento explícito.

```
0030     future = df.iloc[window.target][['timestamp'] + columns].copy()
```

Atribui a future o resultado de df.iloc[window.target][['timestamp'] + columns].copy(). A expressão à direita é avaliada antes da atribuição.

df.iloc[window.target][['timestamp'] + columns].copy — Copia uma tabela/dicionário para não alterar diretamente o objeto anterior. Argumentos usados: nenhum argumento explícito.

```
0031     if len(future) != window.horizon or history.timestamp.max() >= future.timestamp.min():
```

Avalia a condição len(future) != window.horizon or history.timestamp.max() >= future.timestamp.min(). O bloco indentado seguinte só é executado quando ela é verdadeira.

len — Conta elementos. Argumentos usados: posicional 1: future.

history.timestamp.max — Retorna maior valor. Argumentos usados: nenhum argumento explícito.

future.timestamp.min — Retorna menor valor. Argumentos usados: nenhum argumento explícito.

```
0032          raise ValueError('History and future covariates must be temporally disjoint')
```

Interrompe o caminho atual lançando ValueError('History and future covariates must be temporally disjoint'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'History and future covariates must be temporally disjoint'.

```
0033      return history, future
```

Encerra esta função e devolve (history, future) ao chamador; não imprime o resultado por si só.

```
0036  def timesfm_covariates(history, future, dataset):
```

Define timesfm_covariates(history, future, dataset). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0037      if 'water_produced' in future:
```

Avalia a condição 'water_produced' in future. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0038          raise ValueError('Future targets cannot enter model inputs')
```

Interrompe o caminho atual lançando ValueError('Future targets cannot enter model inputs'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Future targets cannot enter model inputs'.

```
0039      columns = covariate_columns(dataset)
```

Atribui a columns o resultado de covariate_columns(dataset). A expressão à direita é avaliada antes da atribuição.

covariate_columns — dataset: bloco de configuração de uma cidade. Retorna nomes numéricos seguidos dos categóricos, após rejeitar duplicatas e nomes reservados. Argumentos usados: posicional 1: dataset.

```
0040      full = pd.concat([history[columns], future[columns]], ignore_index=True)
```

Atribui a full o resultado de pd.concat([history[columns], future[columns]], ignore_index=True). A expressão à direita é avaliada antes da atribuição.

pd.concat — Concatena tabelas. Argumentos usados: posicional 1: [history[columns], future[columns]]; ignore_index: True.

```
0041      numerical = {c: [full[c].to_numpy(dtype=float)] for c in dataset['numerical_covariates']}
```

Atribui a numerical o resultado de {c: [full[c].to_numpy(dtype=float)] for c in dataset['numerical_covariates']}. A expressão à direita é avaliada antes da atribuição.

full[c].to_numpy — Converte Series/DataFrame em array NumPy. Argumentos usados: dtype: float.

```
0042      categorical = {c: [full[c].astype(str).tolist()] for c in
    dataset['categorical_covariates']}
```

Atribui a categorical o resultado de {c: [full[c].astype(str).tolist()] for c in dataset['categorical_covariates']}. A expressão à direita é avaliada antes da atribuição.

full[c].astype(str).tolist — Converte array/índice/Series em lista Python. Argumentos usados: nenhum argumento explícito.

full[c].astype — Converte o tipo dos valores. Argumentos usados: posicional 1: str.

```
0043      return numerical, categorical
```

Encerra esta função e devolve (numerical, categorical) ao chamador; não imprime o resultado por si só.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

26. src/data/preprocessing/generate_prompts.py

Transforma história/covariáveis em texto. forecasting_pipeline usa select_demonstrations e escolhe build_prompt (legado) ou build_tabular_prompt (comparação 4B). Não chama Ollama e não calcula métricas.

SHA-256: 1305b1f127f4a9de567eb7006e88172fb4181a795af3a469021c5e541354c546

Funções, contratos e pontos de conexão

```
L16: select_demonstrations(training_values, history, horizon: int, *, strategy: str = 'zero',
    count: int = 3, seed: int = 42) → list[Demonstration]
```

training_values: prefixo congelado anterior ao primeiro teste; history: demanda histórica corrente; horizon: tamanho de alvo dos exemplos; strategy: zero/few_random/few_similar; count=3: quantidade; seed=42: sorteio. Retorna lista de Demonstration.

Chamadas do projeto: Demonstration → src/data/preprocessing/generate_prompts.py:10; standardized → src/data/preprocessing/generate_prompts.py:38

```
L38: standardized(x)
```

x: vetor de exemplo/consulta histórica. Retorna valores centrados e divididos por desvio-padrão com piso 1e-8; apenas auxilia seleção por forma.

```
L47: format_values(values) → str
```

values: números históricos. Retorna string separada por vírgulas com duas casas decimais.

```
L52: build_prompt(history: pd.DataFrame, horizon: int, *, demonstrations: list[Demonstration] |
    None = None, future_covariates: pd.DataFrame | None = None) → str
```

history: DataFrame histórico; horizon: dias; demonstrations: lista ou None; future_covariates: tabela sem alvo ou None. Retorna prompt legado com lista de demanda e tabelas de covariáveis.

Chamadas do projeto: format_values → src/data/preprocessing/generate_prompts.py:47; future_dates → src/data/features/temporal.py:5

```
L97: build_tabular_prompt(history: pd.DataFrame, horizon: int, *, future_covariates: pd.DataFrame
    [obrigatório]) → str
```

history: DataFrame histórico; horizon: dias; future_covariates: obrigatório e nomeado. Retorna CSV histórico unificado e tabela futura, com prefixo estável para história expansiva.

Chamadas do projeto: future_dates → src/data/features/temporal.py:5

APIs externas usadas neste arquivo

Demonstration, ValueError, all, append, choice, copy, dataclass, date, equals, extend, float, int, issubset, join, json.dumps, len, list, map, max, np.arange, np.argsort, np.asarray, np.isfinite, np.mean, np.random.default_rng, np.std, pd.DatetimeIndex, set, sorted, strftime, strip, to_csv, to_numpy, tuple. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """PromptCast prompts with demand and the same covariates used by the baselines; future
    targets are absent by construction."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import json
```

Importa json. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0003  from dataclasses import dataclass
```

Importa dataclass de dataclasses. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0004  import numpy as np
```

Importa numpy como np. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  import pandas as pd
```

Importa pandas como pd. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0006  from src.data.features.temporal import future_dates
```

Importa future_dates de src.data.features.temporal. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0009  @dataclass(frozen=True)
```

Aplica o decorador dataclass(frozen=True) à classe declarada em seguida; gera comportamento de dataclass.

dataclass — Decora a classe para gerar métodos a partir dos campos anotados. Argumentos usados: frozen: True.

```
0010  class Demonstration:
```

Declara a classe Demonstration. Seus métodos compartilham estado por self; a declaração não cria uma previsão.

```
0011      history: tuple[float, ...]
```

Tupla com o histórico de um exemplo few-shot; ... na anotação significa comprimento variável.

```
0012      target: tuple[float, ...]
```

Tupla com valores de saída do exemplo histórico, anterior ao teste; não é o alvo da janela corrente.

```
0013      target_end: int
```

Fim exclusivo do alvo do exemplo; permite verificar que o exemplo termina dentro do prefixo de treinamento.

```
0016  def select_demonstrations(training_values, history, horizon: int, *,
```

training_values e history são posicionais; strategy, count e seed são nomeados. O retorno é lista de objetos Demonstration. history define o tamanho de cada exemplo.

```
0017                            strategy: str = 'zero', count: int = 3,
```

training_values e history são posicionais; strategy, count e seed são nomeados. O retorno é lista de objetos Demonstration. history define o tamanho de cada exemplo.

```
0018                            seed: int = 42) -> list[Demonstration]:
```

training_values e history são posicionais; strategy, count e seed são nomeados. O retorno é lista de objetos Demonstration. history define o tamanho de cada exemplo.

```
0019      """Select only from the frozen prefix preceding the FIRST evaluated target.
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0021      Similarity uses historical inputs only. The caller passes an already
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0022      restricted training prefix; test observations can never become examples.
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0023      """
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0024      if strategy not in {'zero', 'few_random', 'few_similar'}:
```

Avalia a condição strategy not in {'zero', 'few_random', 'few_similar'}. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0025          raise ValueError(f'Unknown strategy: {strategy}')
```

Interrompe o caminho atual lançando ValueError(f'Unknown strategy: {strategy}'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: f'Unknown strategy: {strategy}'.

```
0026      if strategy == 'zero':
```

Zero-shot não inclui demonstrações: retorna lista vazia antes de construir origens. Esse é o caminho efetivamente usado no experimento 4B.

```
0027          return []
```

Zero-shot não inclui demonstrações: retorna lista vazia antes de construir origens. Esse é o caminho efetivamente usado no experimento 4B.

```
0028      if count < 1 or horizon < 1:
```

Avalia a condição count < 1 or horizon < 1. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0029          raise ValueError('count and horizon must be positive')
```

Interrompe o caminho atual lançando ValueError('count and horizon must be positive'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'count and horizon must be positive'.

```
0030      values = np.asarray(training_values, dtype=float)
```

Atribui a values o resultado de np.asarray(training_values, dtype=float). A expressão à direita é avaliada antes da atribuição.

np.asarray — Representa dados como ndarray, evitando cópia quando possível. Argumentos usados: posicional 1: training_values; dtype: float.

```
0031      width = len(history)
```

Atribui a width o resultado de len(history). A expressão à direita é avaliada antes da atribuição.

len — Conta elementos. Argumentos usados: posicional 1: history.

```
0032      origins = np.arange(width, len(values) - horizon + 1)
```

Uma demonstração precisa de width pontos passados e horizon pontos posteriores ainda dentro do prefixo permitido. O +1 faz a última origem válida entrar no arange de fim exclusivo.

np.arange — Gera progressão de valores com fim exclusivo. Argumentos usados: posicional 1: width; posicional 2: len(values) - horizon + 1.

len — Conta elementos. Argumentos usados: posicional 1: values.

```
0033      if len(origins) < count:
```

Avalia a condição len(origins) < count. O bloco indentado seguinte só é executado quando ela é verdadeira.

len — Conta elementos. Argumentos usados: posicional 1: origins.

```
0034          raise ValueError('Insufficient pre-test data for requested demonstrations')
```

Interrompe o caminho atual lançando ValueError('Insufficient pre-test data for requested demonstrations'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Insufficient pre-test data for requested demonstrations'.

```
0035      if strategy == 'few_random':
```

Avalia a condição strategy == 'few_random'. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0036          selected = np.random.default_rng(seed).choice(origins, size=count, replace=False)
```

Atribui a selected o resultado de np.random.default_rng(seed).choice(origins, size=count, replace=False). A expressão à direita é avaliada antes da atribuição.

np.random.default_rng(seed).choice — Sorteia elementos do gerador NumPy. Argumentos usados: posicional 1: origins; size: count; replace: False.

np.random.default_rng — Cria um gerador pseudoaleatório local. Argumentos usados: posicional 1: seed.

```
0037      else:
```

Ramo alternativo da condição imediatamente anterior: executa quando o teste anterior não foi satisfeito.

```
0038          def standardized(x):
```

Define standardized(x). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0039              return (x - np.mean(x)) / max(float(np.std(x)), 1e-8)
```

Centraliza e escala o vetor para comparar forma; max(...,1e-8) evita divisão por desvio zero. Não aplica normalização à previsão entregue pelo modelo.

max — Retorna maior valor. Argumentos usados: posicional 1: float(np.std(x)); posicional 2: 1e-08.

np.mean — Calcula média aritmética. Argumentos usados: posicional 1: x.

float — Converte escalar para float Python. Argumentos usados: posicional 1: np.std(x).

np.std — Calcula desvio-padrão. Argumentos usados: posicional 1: x.

```
0040          query = standardized(np.asarray(history))
```

Atribui a query o resultado de standardized(np.asarray(history)). A expressão à direita é avaliada antes da atribuição.

standardized — x: vetor de exemplo/consulta histórica. Retorna valores centrados e divididos por desvio-padrão com piso 1e-8; apenas auxilia seleção por forma. Argumentos usados: posicional 1: np.asarray(history).

np.asarray — Representa dados como ndarray, evitando cópia quando possível. Argumentos usados: posicional 1: history.

```
0041          distances = [np.mean((standardized(values[o-width:o]) - query) ** 2) for o in
    origins]
```

Para cada origem candidata, mede MSE entre a janela histórica padronizada e a consulta padronizada. O alvo do exemplo não participa da escolha por similaridade.

np.mean — Calcula média aritmética. Argumentos usados: posicional 1: (standardized(values[o - width:o]) - query) ** 2.

standardized — x: vetor de exemplo/consulta histórica. Retorna valores centrados e divididos por desvio-padrão com piso 1e-8; apenas auxilia seleção por forma. Argumentos usados: posicional 1: values[o - width:o].

```
0042          selected = origins[np.argsort(distances, kind='stable')[:count]]
```

Atribui a selected o resultado de origins[np.argsort(distances, kind='stable')[:count]]. A expressão à direita é avaliada antes da atribuição.

np.argsort — Retorna índices que ordenariam os valores. Argumentos usados: posicional 1: distances; kind: 'stable'.

```
0043      return [Demonstration(tuple(values[o-width:o]), tuple(values[o:o+horizon]),
    int(o+horizon))
```

Empacota histórico, saída e posição final dos exemplos escolhidos, em ordem cronológica; converte arrays para tuplas e índice para int nativo.

Demonstration — Constrói dataclass de exemplo few-shot. Argumentos usados: posicional 1: tuple(values[o - width:o]); posicional 2: tuple(values[o:o + horizon]); posicional 3: int(o + horizon).

tuple — Materializa sequência imutável. Argumentos usados: posicional 1: values[o - width:o].

tuple — Materializa sequência imutável. Argumentos usados: posicional 1: values[o:o + horizon].

int — Converte para inteiro, truncando parte fracionária numérica. Argumentos usados: posicional 1: o + horizon.

```
0044              for o in sorted(selected)]
```

Empacota histórico, saída e posição final dos exemplos escolhidos, em ordem cronológica; converte arrays para tuplas e índice para int nativo.

sorted — Devolve uma lista ordenada. Argumentos usados: posicional 1: selected.

```
0047  def format_values(values) -> str:
```

Define format_values(values) → str. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0048      """Round only the text representation, retaining raw values in model baselines."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0049      return ','.join(f'{float(value):.2f}' for value in values)
```

Encerra esta função e devolve ','.join((f'{float(value):.2f}' for value in values)) ao chamador; não imprime o resultado por si só.

','.join — Une textos com o separador que recebe o método. Argumentos usados: posicional 1: (f'{float(value):.2f}' for value in values).

float — Converte escalar para float Python. Argumentos usados: posicional 1: value.

```
0052  def build_prompt(history: pd.DataFrame, horizon: int, *,
```

Prompt legado aceita demonstrações e covariáveis opcionais; tabela completa nunca é parâmetro. A responsabilidade de passar apenas história pertence ao chamador.

```
0053                   demonstrations: list[Demonstration] | None = None,
```

Prompt legado aceita demonstrações e covariáveis opcionais; tabela completa nunca é parâmetro. A responsabilidade de passar apenas história pertence ao chamador.

```
0054                   future_covariates: pd.DataFrame | None = None) -> str:
```

Prompt legado aceita demonstrações e covariáveis opcionais; tabela completa nunca é parâmetro. A responsabilidade de passar apenas história pertence ao chamador.

```
0055      """Accept history only, not the complete dataframe or a target dataframe."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0056      if horizon < 1 or history.empty:
```

Avalia a condição horizon < 1 or history.empty. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0057          raise ValueError('History and positive horizon are required')
```

Interrompe o caminho atual lançando ValueError('History and positive horizon are required'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'History and positive horizon are required'.

```
0058      values = history.water_produced.to_numpy(dtype=float)
```

Atribui a values o resultado de history.water_produced.to_numpy(dtype=float). A expressão à direita é avaliada antes da atribuição.

history.water_produced.to_numpy — Converte Series/DataFrame em array NumPy. Argumentos usados: dtype: float.

```
0059      if not np.isfinite(values).all():
```

Avalia a condição not np.isfinite(values).all(). O bloco indentado seguinte só é executado quando ela é verdadeira.

np.isfinite(values).all — Exige que todos os elementos sejam True. Argumentos usados: nenhum argumento explícito.

np.isfinite — Testa finitude elemento a elemento. Argumentos usados: posicional 1: values.

```
0060          raise ValueError('History contains invalid values')
```

Interrompe o caminho atual lançando ValueError('History contains invalid values'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'History contains invalid values'.

```
0061      dates = future_dates(history.timestamp.iloc[-1], horizon)
```

Atribui a dates o resultado de future_dates(history.timestamp.iloc[-1], horizon). A expressão à direita é avaliada antes da atribuição.

future_dates — last_observed: último Timestamp histórico; horizon: quantidade de dias. Retorna DatetimeIndex dos próximos dias. Argumentos usados: posicional 1: history.timestamp.iloc[-1]; posicional 2: horizon.

```
0062      parts = [
```

Atribui a parts o resultado de ['Forecast daily water production in cubic metres.', 'Use the chronological daily history below to infer level, trend and seasonality.', f'Return ONLY a JSON object with key "predictions" and exactly {horizon} nonnegative numbers.', 'Do not explain. Do not repeat the historical values as an answer.']. A expressão à direita é avaliada antes da atribuição.

```
0063          'Forecast daily water production in cubic metres.',
```

Instrução da tarefa e unidade esperada. É texto dirigido ao LLM; não converte a unidade dos números no CSV.

```
0064          'Use the chronological daily history below to infer level, trend and seasonality.',
```

Pede inferência de nível, tendência e sazonalidade; isso orienta o modelo, mas não executa um estimador estatístico explícito.

```
0065          f'Return ONLY a JSON object with key "predictions" and exactly {horizon} nonnegative
    numbers.',
```

Interpolação de horizon informa número exato de saídas e chave JSON. É reforçada pelo schema do adaptador e pelo parser.

```
0066          'Do not explain. Do not repeat the historical values as an answer.'
```

Instrui a não explicar nem simplesmente copiar a história; não impede matematicamente uma previsão constante ou repetida.

```
0067      ]
```

Fecha a coleção/chamada iniciada acima. Não inicia uma nova transformação de dados; a indentação seguinte determina a continuidade do bloco.

```
0068      for example in demonstrations or []:
```

O operador or substitui None por lista vazia; nenhuma iteração é feita sem demonstrações.

```
0069          parts.append('Historical example input: ' + format_values(example.history))
```

Adiciona um par histórico entrada/saída ao texto por demonstração. json.dumps formata o exemplo de resposta no mesmo envelope predictions.

parts.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: 'Historical example input: ' + format_values(example.history).

format_values — values: números históricos. Retorna string separada por vírgulas com duas casas decimais. Argumentos usados: posicional 1: example.history.

```
0070          parts.append('Historical example output: ' + json.dumps({'predictions':
    list(example.target)}))
```

Adiciona um par histórico entrada/saída ao texto por demonstração. json.dumps formata o exemplo de resposta no mesmo envelope predictions.

parts.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: 'Historical example output: ' + json.dumps({'predictions': list(example.target)}).

json.dumps — Serializa objeto Python em string JSON. Argumentos usados: posicional 1: {'predictions': list(example.target)}.

list — Materializa um iterável em lista. Argumentos usados: posicional 1: example.target.

```
0071      parts.extend([
```

Acrescenta metadados, lista de demanda, datas-alvo e lembrete de schema. extend acrescenta os blocos separadamente; chaves duplas no f-string representam chaves literais.

parts.extend — Adiciona cada elemento de um iterável à lista. Argumentos usados: posicional 1: [f'History: {len(history)} consecutive daily observations, from {history.timestamp.iloc[0].date()} to {history.timestamp.iloc[-1].date()}.', 'Values (oldest to newest): ' + format_values(values), 'Forecast dates in output order: ' + ','.join(dates.strftime('%Y-%m-%d')), f'Output schema: {{"predictions": [number, ...]}}; array length must be {horizon}.'].

```
0072          f'History: {len(history)} consecutive daily observations, '
```

Acrescenta metadados, lista de demanda, datas-alvo e lembrete de schema. extend acrescenta os blocos separadamente; chaves duplas no f-string representam chaves literais.

len — Conta elementos. Argumentos usados: posicional 1: history.

```
0073          f'from {history.timestamp.iloc[0].date()} to {history.timestamp.iloc[-1].date()}.',
```

Acrescenta metadados, lista de demanda, datas-alvo e lembrete de schema. extend acrescenta os blocos separadamente; chaves duplas no f-string representam chaves literais.

history.timestamp.iloc[0].date — Extrai a parte civil da data de um Timestamp. Argumentos usados: nenhum argumento explícito.

history.timestamp.iloc[-1].date — Extrai a parte civil da data de um Timestamp. Argumentos usados: nenhum argumento explícito.

```
0074          'Values (oldest to newest): ' + format_values(values),
```

Acrescenta metadados, lista de demanda, datas-alvo e lembrete de schema. extend acrescenta os blocos separadamente; chaves duplas no f-string representam chaves literais.

format_values — values: números históricos. Retorna string separada por vírgulas com duas casas decimais. Argumentos usados: posicional 1: values.

```
0075          'Forecast dates in output order: ' + ','.join(dates.strftime('%Y-%m-%d')),
```

Acrescenta metadados, lista de demanda, datas-alvo e lembrete de schema. extend acrescenta os blocos separadamente; chaves duplas no f-string representam chaves literais.

','.join — Une textos com o separador que recebe o método. Argumentos usados: posicional 1: dates.strftime('%Y-%m-%d').

dates.strftime — Formata datas como texto. Argumentos usados: posicional 1: '%Y-%m-%d'.

```
0076          f'Output schema: {{"predictions": [number, ...]}}; array length must be {horizon}.'
```

Acrescenta metadados, lista de demanda, datas-alvo e lembrete de schema. extend acrescenta os blocos separadamente; chaves duplas no f-string representam chaves literais.

```
0077      ])
```

Acrescenta metadados, lista de demanda, datas-alvo e lembrete de schema. extend acrescenta os blocos separadamente; chaves duplas no f-string representam chaves literais.

```
0078      if future_covariates is not None:
```

Avalia a condição future_covariates is not None. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0079          if 'water_produced' in future_covariates.columns:
```

Rejeita tabela futura com coluna water_produced, mesmo que os valores sejam nulos. Defesa estrutural contra passagem acidental do alvo.

```
0080              raise ValueError('Future target must never appear in a prompt')
```

Rejeita tabela futura com coluna water_produced, mesmo que os valores sejam nulos. Defesa estrutural contra passagem acidental do alvo.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Future target must never appear in a prompt'.

```
0081          if len(future_covariates) != horizon or not
    dates.equals(pd.DatetimeIndex(future_covariates.timestamp)):
```

Confere quantidade e igualdade ordenada das datas; não basta ter sete linhas quaisquer.

len — Conta elementos. Argumentos usados: posicional 1: future_covariates.

dates.equals — Compara conteúdo e estrutura do receptor com outro objeto pandas. Argumentos usados: posicional 1: pd.DatetimeIndex(future_covariates.timestamp).

pd.DatetimeIndex — Constrói um índice temporal para comparação. Argumentos usados: posicional 1: future_covariates.timestamp.

```
0082              raise ValueError('Future covariate dates must match the requested horizon')
```

Interrompe o caminho atual lançando ValueError('Future covariate dates must match the requested horizon'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Future covariate dates must match the requested horizon'.

```
0083          columns = [c for c in future_covariates.columns if c != 'timestamp']
```

Atribui a columns o resultado de [c for c in future_covariates.columns if c != 'timestamp']. A expressão à direita é avaliada antes da atribuição.

```
0084          if not set(columns).issubset(history.columns):
```

Avalia a condição not set(columns).issubset(history.columns). O bloco indentado seguinte só é executado quando ela é verdadeira.

set(columns).issubset — Verifica se todos os elementos estão contidos em outro conjunto/iterável. Argumentos usados: posicional 1: history.columns.

set — Cria conjunto de elementos únicos. Argumentos usados: posicional 1: columns.

```
0085              raise ValueError('Historical and future covariates must match')
```

Interrompe o caminho atual lançando ValueError('Historical and future covariates must match'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Historical and future covariates must match'.

```
0086          parts.extend([
```

Executa parts.extend(['Use the historical and supplied future covariates below in this forecast.', 'Covariate values for forecast dates are supplied by the dataset.', 'Historical covariates (CSV):', history[['timestamp'] + columns].to_csv(index=False, float_format='%.6g').strip(), 'Future covariates (CSV, no future water production):', future_covariates.to_csv(index=False, float_format='%.6g').strip()]). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

parts.extend — Adiciona cada elemento de um iterável à lista. Argumentos usados: posicional 1: ['Use the historical and supplied future covariates below in this forecast.', 'Covariate values for forecast dates are supplied by the dataset.', 'Historical covariates (CSV):', history[['timestamp'] + columns].to_csv(index=False, float_format='%.6g').strip(), 'Future covariates (CSV, no future water production):', future_covariates.to_csv(index=False, float_format='%.6g').strip()].

```
0087              'Use the historical and supplied future covariates below in this forecast.',
```

Rótulos que explicam ao modelo a utilização das covariáveis e marcam a tabela histórica. Não são cálculos nem chamadas de biblioteca.

```
0088              'Covariate values for forecast dates are supplied by the dataset.',
```

Rótulos que explicam ao modelo a utilização das covariáveis e marcam a tabela histórica. Não são cálculos nem chamadas de biblioteca.

```
0089              'Historical covariates (CSV):',
```

Rótulos que explicam ao modelo a utilização das covariáveis e marcam a tabela histórica. Não são cálculos nem chamadas de biblioteca.

```
0090              history[['timestamp'] + columns].to_csv(index=False,
    float_format='%.6g').strip(),
```

No layout legado esta tabela contém data e covariáveis; a demanda já foi enviada na lista Values acima. index=False impede enviar índice pandas como variável extra.

history[['timestamp'] + columns].to_csv(index=False, float_format='%.6g').strip — Remove espaços e quebras nas bordas do texto. Argumentos usados: nenhum argumento explícito.

history[['timestamp'] + columns].to_csv — Serializa DataFrame como CSV. Argumentos usados: index: False; float_format: '%.6g'.

```
0091              'Future covariates (CSV, no future water production):',
```

Tabela futura é serializada sem demanda; %.6g reduz apenas representação textual de floats. strip elimina a quebra final do CSV.

```
0092              future_covariates.to_csv(index=False, float_format='%.6g').strip()
```

Tabela futura é serializada sem demanda; %.6g reduz apenas representação textual de floats. strip elimina a quebra final do CSV.

future_covariates.to_csv(index=False, float_format='%.6g').strip — Remove espaços e quebras nas bordas do texto. Argumentos usados: nenhum argumento explícito.

future_covariates.to_csv — Serializa DataFrame como CSV. Argumentos usados: index: False; float_format: '%.6g'.

```
0093          ])
```

Fecha a coleção/chamada iniciada acima. Não inicia uma nova transformação de dados; a indentação seguinte determina a continuidade do bloco.

```
0094      return '\n'.join(parts)
```

Encerra esta função e devolve '\n'.join(parts) ao chamador; não imprime o resultado por si só.

'\n'.join — Une textos com o separador que recebe o método. Argumentos usados: posicional 1: parts.

```
0097  def build_tabular_prompt(history: pd.DataFrame, horizon: int, *, future_covariates:
    pd.DataFrame) -> str:
```

Layout tabular usado em AMBOS os braços do 4B. future_covariates é obrigatório e nomeado; não há parâmetros de demonstração aqui.

```
0098      """Full daily rows with a stable historical prefix for Ollama cache reuse.
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0100      Both arms of the context experiment use this exact layout and precision.
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0101      Metadata that changes with the origin follows the history, never preceding it.
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0102      """
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0103      if horizon < 1 or history.empty or 'water_produced' in future_covariates:
```

Rejeita horizonte não positivo, história vazia ou alvo na tabela futura. Esta função não verifica sozinha toda a origem externa dos dados.

```
0104          raise ValueError('Require historical demand and future covariates without targets')
```

Rejeita horizonte não positivo, história vazia ou alvo na tabela futura. Esta função não verifica sozinha toda a origem externa dos dados.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Require historical demand and future covariates without targets'.

```
0105      dates = future_dates(history.timestamp.iloc[-1], horizon)
```

Atribui a dates o resultado de future_dates(history.timestamp.iloc[-1], horizon). A expressão à direita é avaliada antes da atribuição.

future_dates — last_observed: último Timestamp histórico; horizon: quantidade de dias. Retorna DatetimeIndex dos próximos dias. Argumentos usados: posicional 1: history.timestamp.iloc[-1]; posicional 2: horizon.

```
0106      if len(future_covariates) != horizon or not
    dates.equals(pd.DatetimeIndex(future_covariates.timestamp)):
```

Avalia a condição len(future_covariates) != horizon or not dates.equals(pd.DatetimeIndex(future_covariates.timestamp)). O bloco indentado seguinte só é executado quando ela é verdadeira.

len — Conta elementos. Argumentos usados: posicional 1: future_covariates.

dates.equals — Compara conteúdo e estrutura do receptor com outro objeto pandas. Argumentos usados: posicional 1: pd.DatetimeIndex(future_covariates.timestamp).

pd.DatetimeIndex — Constrói um índice temporal para comparação. Argumentos usados: posicional 1: future_covariates.timestamp.

```
0107          raise ValueError('Future covariate dates do not match the horizon')
```

Interrompe o caminho atual lançando ValueError('Future covariate dates do not match the horizon'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Future covariate dates do not match the horizon'.

```
0108      columns = [c for c in future_covariates if c != 'timestamp']
```

Atribui a columns o resultado de [c for c in future_covariates if c != 'timestamp']. A expressão à direita é avaliada antes da atribuição.

```
0109      if not set(columns).issubset(history):
```

Avalia a condição not set(columns).issubset(history). O bloco indentado seguinte só é executado quando ela é verdadeira.

set(columns).issubset — Verifica se todos os elementos estão contidos em outro conjunto/iterável. Argumentos usados: posicional 1: history.

set — Cria conjunto de elementos únicos. Argumentos usados: posicional 1: columns.

```
0110          raise ValueError('Historical covariates are missing')
```

Interrompe o caminho atual lançando ValueError('Historical covariates are missing'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Historical covariates are missing'.

```
0111      frame = history[['timestamp','water_produced'] + columns].copy()
```

Ordem explícita das colunas garante que water_produced apareça junto à respectiva data antes das covariáveis. A cópia protege a tabela original.

history[['timestamp', 'water_produced'] + columns].copy — Copia uma tabela/dicionário para não alterar diretamente o objeto anterior. Argumentos usados: nenhum argumento explícito.

```
0112      if not np.isfinite(frame.water_produced.to_numpy(dtype=float)).all():
```

Avalia a condição not np.isfinite(frame.water_produced.to_numpy(dtype=float)).all(). O bloco indentado seguinte só é executado quando ela é verdadeira.

np.isfinite(frame.water_produced.to_numpy(dtype=float)).all — Exige que todos os elementos sejam True. Argumentos usados: nenhum argumento explícito.

np.isfinite — Testa finitude elemento a elemento. Argumentos usados: posicional 1: frame.water_produced.to_numpy(dtype=float).

frame.water_produced.to_numpy — Converte Series/DataFrame em array NumPy. Argumentos usados: dtype: float.

```
0113          raise ValueError('Nonfinite demand in history')
```

Interrompe o caminho atual lançando ValueError('Nonfinite demand in history'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Nonfinite demand in history'.

```
0114      frame['water_produced'] = frame.water_produced.map(lambda value: f'{float(value):.2f}')
```

Converte a coluna da cópia em strings com duas casas. map aplica a lambda em cada célula. As demais colunas numéricas usam seis algarismos no to_csv.

frame.water_produced.map — Aplica uma função a cada valor da Series. Argumentos usados: posicional 1: lambda value: f'{float(value):.2f}'.

float — Converte escalar para float Python. Argumentos usados: posicional 1: value.

```
0115      return '\n'.join([
```

Encerra esta função e devolve '\n'.join(['Forecast daily water production in cubic metres from the complete historical table below.', 'Use demand, weather and calendar covariates to infer level, trend and seasonality.', f'Return ONLY a JSON object with key "predictions" and exactly {horizon} nonnegative numbers.', 'Do not explain. Do not repeat the historical values as an answer.', 'HISTORICAL DAILY OBSERVATIONS (CSV, oldest to newest):', frame.to_csv(index=False, float_format='%.6g').strip(), f'END HISTORY: {len(frame)} observations from {frame.timestamp.iloc[0].date()} to {frame.timestamp.iloc[-1].date()}.', 'SUPPLIED FUTURE COVARIATES (CSV, no future water production):', future_covariates.to_csv(index=False, float_format='%.6g').strip(), 'Forecast dates in output order: ' + ','.join(dates.strftime('%Y-%m-%d')), f'Output schema: {{"predictions": [number, ...]}}; array length must be {horizon}.']) ao chamador; não imprime o resultado por si só.

'\n'.join — Une textos com o separador que recebe o método. Argumentos usados: posicional 1: ['Forecast daily water production in cubic metres from the complete historical table below.', 'Use demand, weather and calendar covariates to infer level, trend and seasonality.', f'Return ONLY a JSON object with key "predictions" and exactly {horizon} nonnegative numbers.', 'Do not explain. Do not repeat the historical values as an answer.', 'HISTORICAL DAILY OBSERVATIONS (CSV, oldest to newest):', frame.to_csv(index=False, float_format='%.6g').strip(), f'END HISTORY: {len(frame)} observations from {frame.timestamp.iloc[0].date()} to {frame.timestamp.iloc[-1].date()}.', 'SUPPLIED FUTURE COVARIATES (CSV, no future water production):', future_covariates.to_csv(index=False, float_format='%.6g').strip(), 'Forecast dates in output order: ' + ','.join(dates.strftime('%Y-%m-%d')), f'Output schema: {{"predictions": [number, ...]}}; array length must be {horizon}.'].

```
0116          'Forecast daily water production in cubic metres from the complete historical table
    below.',
```

Declara que o histórico enviado é completo para a entrada recebida; no braço curto significa todos os 15 dias selecionados, não todo o dataset.

```
0117          'Use demand, weather and calendar covariates to infer level, trend and seasonality.',
```

Pede combinação de demanda, clima e calendário; não há seleção automática de variáveis ou pesos definidos pelo Python.

```
0118          f'Return ONLY a JSON object with key "predictions" and exactly {horizon} nonnegative
    numbers.',
```

Reforça formato e restrições de resposta, idênticos nos dois contextos do experimento 4B.

```
0119          'Do not explain. Do not repeat the historical values as an answer.',
```

Reforça formato e restrições de resposta, idênticos nos dois contextos do experimento 4B.

```
0120          'HISTORICAL DAILY OBSERVATIONS (CSV, oldest to newest):',
```

Marcador literal que delimita o início do CSV histórico e também é usado no teste de preservação do prefixo.

```
0121          frame.to_csv(index=False, float_format='%.6g').strip(),
```

Insere todas as linhas de history na string, em ordem cronológica. Não amostra nem resume valores.

frame.to_csv(index=False, float_format='%.6g').strip — Remove espaços e quebras nas bordas do texto. Argumentos usados: nenhum argumento explícito.

frame.to_csv — Serializa DataFrame como CSV. Argumentos usados: index: False; float_format: '%.6g'.

```
0122          f'END HISTORY: {len(frame)} observations from {frame.timestamp.iloc[0].date()} to
    {frame.timestamp.iloc[-1].date()}.',
```

Metadados que mudam por janela vêm DEPOIS da tabela. Isso mantém o prefixo histórico anterior igual entre janelas expansivas, favorecendo reutilização de cache KV.

len — Conta elementos. Argumentos usados: posicional 1: frame.

frame.timestamp.iloc[0].date — Extrai a parte civil da data de um Timestamp. Argumentos usados: nenhum argumento explícito.

frame.timestamp.iloc[-1].date — Extrai a parte civil da data de um Timestamp. Argumentos usados: nenhum argumento explícito.

```
0123          'SUPPLIED FUTURE COVARIATES (CSV, no future water production):',
```

Marca e serializa as sete linhas futuras de covariáveis. As datas aparecem aqui, mas os sete valores water_produced não.

```
0124          future_covariates.to_csv(index=False, float_format='%.6g').strip(),
```

Marca e serializa as sete linhas futuras de covariáveis. As datas aparecem aqui, mas os sete valores water_produced não.

future_covariates.to_csv(index=False, float_format='%.6g').strip — Remove espaços e quebras nas bordas do texto. Argumentos usados: nenhum argumento explícito.

future_covariates.to_csv — Serializa DataFrame como CSV. Argumentos usados: index: False; float_format: '%.6g'.

```
0125          'Forecast dates in output order: ' + ','.join(dates.strftime('%Y-%m-%d')),
```

Define explicitamente a associação por posição entre números do array e datas; o modelo não devolve timestamps individuais.

','.join — Une textos com o separador que recebe o método. Argumentos usados: posicional 1: dates.strftime('%Y-%m-%d').

dates.strftime — Formata datas como texto. Argumentos usados: posicional 1: '%Y-%m-%d'.

```
0126          f'Output schema: {{"predictions": [number, ...]}}; array length must be {horizon}.'
```

Lembrete final do schema; number é um marcador textual de formato, não um número que deva ser copiado na saída.

```
0127      ])
```

Fecha a coleção/chamada iniciada acima. Não inicia uma nova transformação de dados; a indentação seguinte determina a continuidade do bloco.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

27. src/models/promptcast.py

Adaptador HTTP do Qwen: configuração, schema de saída, parser, cliente JSON e classe QwenForecaster. forecasting_pipeline chama predict; post_json encapsula transporte, e parse_prediction valida sem corrigir artificialmente valores.

SHA-256: b222c85acd066aaed968da9b511fe19f49ee6f98e7dbc602ed1abda79792e9ee

Funções, contratos e pontos de conexão

```
L21: __post_init__(self)
```

Valida limites após a inicialização automática da dataclass; não é chamado manualmente no pipeline.

```
L26: prediction_schema(horizon: int) → dict
```

horizon: comprimento exigido do array. Retorna JSON Schema para objeto predictions, números não negativos e ausência de chaves extras.

```
L33: parse_prediction(text: str, horizon: int) → list[float]
```

text: string JSON do modelo; horizon: quantidade exata de valores. Retorna list[float] ou lança erro; não remove prose nem aceita resposta parcial.

```
L46: post_json(url: str, payload: dict, timeout: int) → dict
```

url: endpoint; payload: dict do pedido; timeout: segundos de espera. Retorna dict decodificado do envelope HTTP, não ainda a lista de previsões.

```
L54: __init__(self, config: OllamaConfig)
```

Construtor da classe: self recebe a instância e os demais argumentos inicializam configuração, backend ou estado. Confira o corpo abaixo para I/O e carregamento de modelo.

Chamadas do projeto: post_json → src/models/promptcast.py:46

```
L71: predict(self, history, horizon: int, *, prompt: str [obrigatório], future_covariates = None) →
     list[float]
```

history é tabela histórica; horizon é quantidade prevista; future_covariates é tabela sem alvo; prompt é texto já construído para Qwen. Retorna lista de floats; cada classe escolhe backend. O método pertence à classe adaptadora deste arquivo.

Chamadas do projeto: parse_prediction → src/models/promptcast.py:33; post_json → src/models/promptcast.py:46; prediction_schema → src/models/promptcast.py:26

APIs externas usadas neste arquivo

RuntimeError, ValueError, any, append, asdict, dataclass, encode, float, get, int, isinstance, json.dumps, json.load, json.loads, len, math.isfinite, min, range, set, str, time.sleep, type, urllib.request.Request, urllib.request.urlopen. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Qwen forecasting through Ollama, with strict output validation and audit data."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import json
```

Importa json. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0003  import math
```

Importa math. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0004  import time
```

Importa time. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  import urllib.error
```

Importa urllib.error. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0006  import urllib.request
```

Importa urllib.request. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0007  from dataclasses import asdict, dataclass
```

Importa asdict, dataclass de dataclasses. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0010  @dataclass(frozen=True)
```

Aplica o decorador dataclass(frozen=True) à classe declarada em seguida; gera comportamento de dataclass.

dataclass — Decora a classe para gerar métodos a partir dos campos anotados. Argumentos usados: frozen: True.

```
0011  class OllamaConfig:
```

Declara a classe OllamaConfig. Seus métodos compartilham estado por self; a declaração não cria uma previsão.

```
0012      model: str = 'qwen3:1.7b'
```

Nome padrão legado. Configurações do experimento 4B substituem esse valor ao construir OllamaConfig; alterar apenas este default não muda JSON já fornecido.

```
0013      base_url: str = 'http://127.0.0.1:11434'
```

URL padrão legado, porta 11434. O JSON do 4B usa servidor dedicado local na porta 11435.

```
0014      temperature: float = 0
```

Temperatura padrão zero. É configuração de amostragem, não um normalizador dos dados; não garante igualdade bit a bit entre runtimes.

```
0015      num_ctx: int = 8192
```

Capacidade padrão de 8192 tokens, sobrescrita para 131072 no experimento 4B. Não corresponde ao número de dias.

```
0016      num_predict: int = 256
```

Máximo de tokens novos, contando pontuação e estrutura JSON, não quantidade de valores previstos.

```
0017      timeout: int = 600
```

Espera HTTP padrão de 600 segundos; configuração 4B sobrescreve para 7200.

```
0018      retries: int = 2
```

Número de tentativas adicionais, não total; retries=2 permite três chamadas no loop.

```
0019      seed: int = 42
```

Semente transmitida às opções de geração do Ollama.

```
0021      def __post_init__(self):
```

Gancho da dataclass: executado automaticamente depois da inicialização gerada, para rejeitar limites inválidos.

```
0022          if min(self.num_ctx, self.num_predict, self.timeout) < 1 or self.retries < 0:
```

Avalia a condição min(self.num_ctx, self.num_predict, self.timeout) < 1 or self.retries < 0. O bloco indentado seguinte só é executado quando ela é verdadeira.

min — Retorna menor valor. Argumentos usados: posicional 1: self.num_ctx; posicional 2: self.num_predict; posicional 3: self.timeout.

```
0023              raise ValueError('Invalid Ollama limits')
```

Interrompe o caminho atual lançando ValueError('Invalid Ollama limits'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Invalid Ollama limits'.

```
0026  def prediction_schema(horizon: int) -> dict:
```

Define prediction_schema(horizon: int) → dict. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0027      return {'type': 'object', 'properties': {'predictions': {
```

Abre schema de objeto JSON. properties descreve a chave predictions, sem inserir previsões reais no pedido.

```
0028          'type': 'array', 'items': {'type': 'number', 'minimum': 0},
```

Cada item deve ser número com limite inferior zero; aqui mínimo é uma restrição de saída, não clipping posterior.

```
0029          'minItems': horizon, 'maxItems': horizon}},
```

minItems e maxItems iguais a horizon impõem exatamente sete valores quando horizon=7.

```
0030          'required': ['predictions'], 'additionalProperties': False}
```

required torna predictions obrigatória; additionalProperties=False proíbe outras chaves no objeto.

```
0033  def parse_prediction(text: str, horizon: int) -> list[float]:
```

Define parse_prediction(text: str, horizon: int) → list[float]. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0034      """Reject prose, strings, booleans, NaN, missing values and wrong horizons."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0035      parsed = json.loads(text)
```

Atribui a parsed o resultado de json.loads(text). A expressão à direita é avaliada antes da atribuição.

json.loads — Decodifica uma string JSON em objetos Python. Argumentos usados: posicional 1: text.

```
0036      if not isinstance(parsed, dict) or set(parsed) != {'predictions'}:
```

Avalia a condição not isinstance(parsed, dict) or set(parsed) != {'predictions'}. O bloco indentado seguinte só é executado quando ela é verdadeira.

isinstance — Verifica se um objeto é instância de um tipo ou tupla de tipos. Argumentos usados: posicional 1: parsed; posicional 2: dict.

set — Cria conjunto de elementos únicos. Argumentos usados: posicional 1: parsed.

```
0037          raise ValueError('Expected exactly the predictions key')
```

Interrompe o caminho atual lançando ValueError('Expected exactly the predictions key'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Expected exactly the predictions key'.

```
0038      values = parsed['predictions']
```

Atribui a values o resultado de parsed['predictions']. A expressão à direita é avaliada antes da atribuição.

```
0039      if not isinstance(values, list) or len(values) != horizon:
```

Avalia a condição not isinstance(values, list) or len(values) != horizon. O bloco indentado seguinte só é executado quando ela é verdadeira.

isinstance — Verifica se um objeto é instância de um tipo ou tupla de tipos. Argumentos usados: posicional 1: values; posicional 2: list.

len — Conta elementos. Argumentos usados: posicional 1: values.

```
0040          raise ValueError(f'Expected {horizon} predictions')
```

Interrompe o caminho atual lançando ValueError(f'Expected {horizon} predictions'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: f'Expected {horizon} predictions'.

```
0041      if any(type(v) not in (int, float) or not math.isfinite(v) or v < 0 for v in values):
```

O teste type(v), diferente de isinstance(v,int), impede que true/false sejam aceitos como 1/0. O gerador rejeita também infinitos e negativos.

any — Reduz a existência de pelo menos um True. Argumentos usados: posicional 1: (type(v) not in (int, float) or not math.isfinite(v) or v < 0 for v in values).

type — Obtém a classe concreta do valor. Argumentos usados: posicional 1: v.

math.isfinite — Confere se um único número é finito. Argumentos usados: posicional 1: v.

```
0042          raise ValueError('Predictions must be finite, nonnegative numbers')
```

Interrompe o caminho atual lançando ValueError('Predictions must be finite, nonnegative numbers'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Predictions must be finite, nonnegative numbers'.

```
0043      return [float(v) for v in values]
```

Encerra esta função e devolve [float(v) for v in values] ao chamador; não imprime o resultado por si só.

float — Converte escalar para float Python. Argumentos usados: posicional 1: v.

```
0046  def post_json(url: str, payload: dict, timeout: int) -> dict:
```

Define post_json(url: str, payload: dict, timeout: int) → dict. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0047      request = urllib.request.Request(url, data=json.dumps(payload).encode('utf-8'),
```

Atribui a request o resultado de urllib.request.Request(url, data=json.dumps(payload).encode('utf-8'), headers={'Content-Type': 'application/json'}). A expressão à direita é avaliada antes da atribuição.

urllib.request.Request — Monta uma requisição HTTP. Argumentos usados: posicional 1: url; data: json.dumps(payload).encode('utf-8'); headers: {'Content-Type': 'application/json'}.

json.dumps(payload).encode — Codifica string em bytes. Argumentos usados: posicional 1: 'utf-8'.

json.dumps — Serializa objeto Python em string JSON. Argumentos usados: posicional 1: payload.

```
0048                                      headers={'Content-Type': 'application/json'})
```

Define 'Content-Type': campo do registro/coleção em construção; recebe 'application/json'. Os valores são avaliados nesta janela/iteração.

```
0049      with urllib.request.urlopen(request, timeout=timeout) as response:
```

Abre o contexto gerenciado urllib.request.urlopen(request, timeout=timeout) como response. Ao sair, libera o recurso ou restaura a simulação/redirecionamento, inclusive se ocorrer erro.

urllib.request.urlopen — Abre conexão HTTP e devolve resposta como fluxo. Argumentos usados: posicional 1: request; timeout: timeout.

```
0050          return json.load(response)
```

Encerra esta função e devolve json.load(response) ao chamador; não imprime o resultado por si só.

json.load — Decodifica JSON lido de um arquivo/response. Argumentos usados: posicional 1: response.

```
0053  class QwenForecaster:
```

Declara a classe QwenForecaster. Seus métodos compartilham estado por self; a declaração não cria uma previsão.

```
0054      def __init__(self, config: OllamaConfig):
```

Define __init__(self, config: OllamaConfig). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0055          self.config = config
```

Atribui a self.config o resultado de config. A expressão à direita é avaliada antes da atribuição.

```
0056          self.last_audit = {}
```

Atribui a self.last_audit o resultado de {}. A expressão à direita é avaliada antes da atribuição.

```
0057          with urllib.request.urlopen(config.base_url + '/api/tags', timeout=10) as response:
```

Abre o contexto gerenciado urllib.request.urlopen(config.base_url + '/api/tags', timeout=10) como response. Ao sair, libera o recurso ou restaura a simulação/redirecionamento, inclusive se ocorrer erro.

urllib.request.urlopen — Abre conexão HTTP e devolve resposta como fluxo. Argumentos usados: posicional 1: config.base_url + '/api/tags'; timeout: 10.

```
0058              tags = json.load(response)
```

Atribui a tags o resultado de json.load(response). A expressão à direita é avaliada antes da atribuição.

json.load — Decodifica JSON lido de um arquivo/response. Argumentos usados: posicional 1: response.

```
0059          matches = [m for m in tags['models'] if m['name'] == config.model]
```

Filtra os modelos instalados por nome exato, incluindo tag/quantização. Não escolhe uma variante aproximada silenciosamente.

```
0060          if not matches:
```

Avalia a condição not matches. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0061              raise ValueError(f'Model {config.model} is not installed in Ollama')
```

Interrompe o caminho atual lançando ValueError(f'Model {config.model} is not installed in Ollama'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: f'Model {config.model} is not installed in Ollama'.

```
0062          metadata = post_json(config.base_url + '/api/show', {'model': config.model}, 30)
```

Consulta metadados do checkpoint instalado; timeout posicional de 30 segundos só vale para esta consulta.

post_json — url: endpoint; payload: dict do pedido; timeout: segundos de espera. Retorna dict decodificado do envelope HTTP, não ainda a lista de previsões. Argumentos usados: posicional 1: config.base_url + '/api/show'; posicional 2: {'model': config.model}; posicional 3: 30.

```
0063          info = metadata['model_info']
```

Atribui a info o resultado de metadata['model_info']. A expressão à direita é avaliada antes da atribuição.

```
0064          architecture = info['general.architecture']
```

Obtém a arquitetura declarada e compõe a chave de metadados de contexto correspondente, em vez de fixar um nome de arquitetura.

```
0065          maximum_context = int(info[f'{architecture}.context_length'])
```

Obtém a arquitetura declarada e compõe a chave de metadados de contexto correspondente, em vez de fixar um nome de arquitetura.

int — Converte para inteiro, truncando parte fracionária numérica. Argumentos usados: posicional 1: info[f'{architecture}.context_length'].

```
0066          if config.num_ctx > maximum_context:
```

Avalia a condição config.num_ctx > maximum_context. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0067              raise ValueError(f'Requested {config.num_ctx} tokens exceeds model limit
    {maximum_context}')
```

Interrompe o caminho atual lançando ValueError(f'Requested {config.num_ctx} tokens exceeds model limit {maximum_context}'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: f'Requested {config.num_ctx} tokens exceeds model limit {maximum_context}'.

```
0068          self.identity = {'model': config.model, 'digest': matches[0]['digest'],
```

Registra modelo, digest dos pesos, contexto declarado e configuração efetivamente construída. Este dict participa do hash de cache do pipeline.

```
0069                          'maximum_context_tokens': maximum_context, 'config': asdict(config)}
```

Registra modelo, digest dos pesos, contexto declarado e configuração efetivamente construída. Este dict participa do hash de cache do pipeline.

asdict — Converte dataclass em dicionário de seus campos. Argumentos usados: posicional 1: config.

```
0071      def predict(self, history, horizon: int, *, prompt: str, future_covariates=None) ->
    list[float]:
```

Mantém interface dos adaptadores: history e future_covariates são aceitos, mas este método usa o prompt já montado. horizon determina schema e parser; prompt deve ser nomeado após *.

```
0072          """Send only a prebuilt, target-free prompt; retain every attempted response."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0073          config = self.config
```

Atribui a config o resultado de self.config. A expressão à direita é avaliada antes da atribuição.

```
0074          # UTF-8 byte count is a conservative upper bound for byte-level tokenizers.
```

Comentário de implementação: UTF-8 byte count is a conservative upper bound for byte-level tokenizers.. Não é executado; as operações efetivas são as linhas seguintes.

```
0075          # Reserve space for the Ollama model template and the entire generated answer.
```

Comentário de implementação: Reserve space for the Ollama model template and the entire generated answer.. Não é executado; as operações efetivas são as linhas seguintes.

```
0076          if len(prompt.encode('utf-8')) + config.num_predict + 512 > config.num_ctx:
```

Compara bytes UTF-8 do prompt + reserva de geração + 512 bytes/tokens de margem com num_ctx. É limite conservador, não tokenizer exato. Um prompt legítimo pode ser rejeitado antes do limite real.

len — Conta elementos. Argumentos usados: posicional 1: prompt.encode('utf-8').

prompt.encode — Codifica string em bytes. Argumentos usados: posicional 1: 'utf-8'.

```
0077              raise ValueError('Prompt exceeds conservative context budget; increase num_ctx')
```

Interrompe o caminho atual lançando ValueError('Prompt exceeds conservative context budget; increase num_ctx'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Prompt exceeds conservative context budget; increase num_ctx'.

```
0078          payload = {'model': config.model, 'prompt': prompt, 'stream': False,
```

Monta corpo HTTP: nome do modelo, texto completo e stream=False para resposta única. Isso não envia ainda a requisição.

```
0079                      'think': False, 'keep_alive': '10m', 'truncate': False, 'format':
    prediction_schema(horizon),
```

think=False solicita geração sem modo de raciocínio; keep_alive='10m' controla retenção do modelo; truncate=False expressa intenção de não cortar; format recebe o schema. A auditoria do servidor continua necessária.

prediction_schema — horizon: comprimento exigido do array. Retorna JSON Schema para objeto predictions, números não negativos e ausência de chaves extras. Argumentos usados: posicional 1: horizon.

```
0080                      'options': {'temperature': config.temperature, 'seed': config.seed,
```

options agrupa configuração de amostragem e limites; são opções do Ollama, diferentes das opções do cliente urllib, como timeout.

```
0081                                 'num_ctx': config.num_ctx, 'num_predict': config.num_predict}}
```

options agrupa configuração de amostragem e limites; são opções do Ollama, diferentes das opções do cliente urllib, como timeout.

```
0082         self.last_audit = {'request': payload, 'attempts': []}
```

Reinicia auditoria da janela, guardando pedido exato e lista de tentativas. Não reutiliza uma resposta da janela anterior aqui.

```
0083         for attempt in range(config.retries + 1):
```

range(retries+1) inclui a tentativa inicial: com retries=1, attempt assume 0 e 1.

range — Gera inteiros com limite final exclusivo. Argumentos usados: posicional 1: config.retries + 1.

```
0084             audit = {'attempt': attempt + 1}
```

Cria dict mutável por tentativa e o anexa antes do HTTP. Quando response/error são adicionados depois, a lista aponta para o mesmo dict e preserva o registro.

```
0085             self.last_audit['attempts'].append(audit)
```

Cria dict mutável por tentativa e o anexa antes do HTTP. Quando response/error são adicionados depois, a lista aponta para o mesmo dict e preserva o registro.

self.last_audit['attempts'].append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: audit.

```
0086             try:
```

Inicia região protegida de execução. Exceções compatíveis serão tratadas pelo except correspondente; sucesso continua normalmente.

```
0087                 answer = post_json(config.base_url + '/api/generate', payload,
    config.timeout)
```

Atribui a answer o resultado de post_json(config.base_url + '/api/generate', payload, config.timeout). A expressão à direita é avaliada antes da atribuição.

post_json — url: endpoint; payload: dict do pedido; timeout: segundos de espera. Retorna dict decodificado do envelope HTTP, não ainda a lista de previsões. Argumentos usados: posicional 1: config.base_url + '/api/generate'; posicional 2: payload; posicional 3: config.timeout.

```
0088                 audit['response'] = answer
```

Atribui a audit['response'] o resultado de answer. A expressão à direita é avaliada antes da atribuição.

```
0089                 if not answer.get('done') or answer.get('done_reason') == 'length':
```

done ausente/False ou done_reason='length' rejeita resposta incompleta, mesmo que haja texto parseável.

answer.get — Busca chave de dicionário com padrão se ausente. Argumentos usados: posicional 1: 'done'.

answer.get — Busca chave de dicionário com padrão se ausente. Argumentos usados: posicional 1: 'done_reason'.

```
0090                     raise ValueError('Incomplete or token-limited Ollama response')
```

Interrompe o caminho atual lançando ValueError('Incomplete or token-limited Ollama response'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Incomplete or token-limited Ollama response'.

```
0091                 if answer.get('prompt_eval_count', 0) + answer.get('eval_count', 0) >=
    config.num_ctx:
```

Soma tokens de prompt e saída para rejeitar saturação. get(...,0) tolera campos ausentes, então esta checagem sozinha não certifica contadores válidos em toda versão.

answer.get — Busca chave de dicionário com padrão se ausente. Argumentos usados: posicional 1: 'prompt_eval_count'; posicional 2: 0.

answer.get — Busca chave de dicionário com padrão se ausente. Argumentos usados: posicional 1: 'eval_count'; posicional 2: 0.

```
0092                     raise ValueError('Context limit reached')
```

Interrompe o caminho atual lançando ValueError('Context limit reached'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Context limit reached'.

```
0093                 return parse_prediction(answer.get('response', ''), horizon)
```

Extrai apenas a string response do envelope e valida sete números. Não converte metadados do Ollama em previsão.

parse_prediction — text: string JSON do modelo; horizon: quantidade exata de valores. Retorna list[float] ou lança erro; não remove prose nem aceita resposta parcial. Argumentos usados: posicional 1: answer.get('response', ''); posicional 2: horizon.

answer.get — Busca chave de dicionário com padrão se ausente. Argumentos usados: posicional 1: 'response'; posicional 2: ''.

```
0094             except (ValueError, urllib.error.URLError, TimeoutError) as exc:
```

Captura erros de validação, transporte e timeout; não captura toda exceção possível dentro deste método.

```
0095                 audit['error'] = str(exc)
```

Atribui a audit['error'] o resultado de str(exc). A expressão à direita é avaliada antes da atribuição.

str — Produz representação textual. Argumentos usados: posicional 1: exc.

```
0096                 if attempt == config.retries:
```

Avalia a condição attempt == config.retries. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0097                     raise
```

Relança a última exceção preservando seu tipo; o pipeline superior registrará status='error'.

```
0098                 time.sleep(min(2 ** attempt, 4))
```

Executa time.sleep(min(2 ** attempt, 4)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

time.sleep — Suspende o processo por uma duração. Argumentos usados: posicional 1: min(2 ** attempt, 4).

min — Retorna menor valor. Argumentos usados: posicional 1: 2 ** attempt; posicional 2: 4.

```
0099         raise RuntimeError('Unreachable retry state')
```

Guarda contra término inesperado do loop; caminhos normais retornam lista válida ou relançam erro antes daqui.

RuntimeError — Cria exceção para estado de execução inesperado. Argumentos usados: posicional 1: 'Unreachable retry state'.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

28. src/models/chronos.py

Adapter de Chronos2Pipeline. Importações pesadas ficam dentro do construtor, permitindo importar o pipeline em outros ambientes. Usa covariáveis passadas/futuras e devolve o quantil 0,5 como previsão pontual.

SHA-256: b87428a5ee2bc106c38203330df3866cfdc3a1b4b4e9ad7b5920a4907eff917c

Funções, contratos e pontos de conexão

```
L6: __init__(self, context: int, dataset: dict)
```

Construtor da classe: self recebe a instância e os demais argumentos inicializam configuração, backend ou estado. Confira o corpo abaixo para I/O e carregamento de modelo.

```
L18: predict(self, history, horizon: int, *, future_covariates [obrigatório], prompt = None) →
    list[float]
```

history é tabela histórica; horizon é quantidade prevista; future_covariates é tabela sem alvo; prompt é texto já construído para Qwen. Retorna lista de floats; cada classe escolhe backend. O método pertence à classe adaptadora deste arquivo.

Chamadas do projeto: covariate_columns → src/data/features/covariates.py:6

APIs externas usadas neste arquivo

ValueError, copy, from_pretrained, importlib.metadata.version, len, predict_df, str, to_numpy, tolist, torch.cuda.is_available, torch.manual_seed. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Chronos-2 with original historical/future covariates and growing context."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import importlib.metadata
```

Importa importlib.metadata. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  class ChronosForecaster:
```

Declara a classe ChronosForecaster. Seus métodos compartilham estado por self; a declaração não cria uma previsão.

```
0006      def __init__(self, context: int, dataset: dict):
```

Define __init__(self, context: int, dataset: dict). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0007          import torch
```

Importa torch. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0008          from chronos import Chronos2Pipeline
```

Importa Chronos2Pipeline de chronos. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0009          self.dataset = dataset
```

Atribui a self.dataset o resultado de dataset. A expressão à direita é avaliada antes da atribuição.

```
0010          torch.manual_seed(42)
```

Executa torch.manual_seed(42). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

torch.manual_seed — Define semente do gerador PyTorch. Argumentos usados: posicional 1: 42.

```
0011          self.pipeline = Chronos2Pipeline.from_pretrained(
```

Atribui a self.pipeline o resultado de Chronos2Pipeline.from_pretrained('amazon/chronos-2', device_map='cuda' if torch.cuda.is_available() else 'cpu', torch_dtype=torch.float32, local_files_only=True). A expressão à direita é avaliada antes da atribuição.

Chronos2Pipeline.from_pretrained — Carrega pesos/configuração de um checkpoint. Argumentos usados: posicional 1: 'amazon/chronos-2'; device_map: 'cuda' if torch.cuda.is_available() else 'cpu'; torch_dtype: torch.float32; local_files_only: True.

```
0012              'amazon/chronos-2', device_map='cuda' if torch.cuda.is_available() else 'cpu',
```

Continua os argumentos nomeados: device_map = 'cuda' if torch.cuda.is_available() else 'cpu'. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

torch.cuda.is_available — Consulta disponibilidade do backend CUDA. Argumentos usados: nenhum argumento explícito.

```
0013              torch_dtype=torch.float32, local_files_only=True)
```

Continua os argumentos nomeados: torch_dtype = torch.float32; local_files_only = True. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0014          self.identity = {'model': 'amazon/chronos-2', 'initial_context': context,
```

Atribui a self.identity o resultado de {'model': 'amazon/chronos-2', 'initial_context': context, 'version': importlib.metadata.version('chronos-forecasting')}. A expressão à direita é avaliada antes da atribuição.

```
0015                            'version': importlib.metadata.version('chronos-forecasting')}
```

Define 'version': campo do registro/coleção em construção; recebe importlib.metadata.version('chronos-forecasting'). Os valores são avaliados nesta janela/iteração.

importlib.metadata.version — Lê versão instalada de uma distribuição Python. Argumentos usados: posicional 1: 'chronos-forecasting'.

```
0016          self.last_audit = {}
```

Atribui a self.last_audit o resultado de {}. A expressão à direita é avaliada antes da atribuição.

```
0018      def predict(self, history, horizon: int, *, future_covariates, prompt=None) ->
    list[float]:
```

Define predict(self, history, horizon: int, *, future_covariates [obrigatório], prompt = None) → list[float]. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0019          from src.data.features.covariates import covariate_columns
```

Importa covariate_columns de src.data.features.covariates. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0020          if 'water_produced' in future_covariates:
```

Avalia a condição 'water_produced' in future_covariates. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0021              raise ValueError('Future targets cannot enter Chronos')
```

Interrompe o caminho atual lançando ValueError('Future targets cannot enter Chronos'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Future targets cannot enter Chronos'.

```
0022          columns = covariate_columns(self.dataset)
```

Atribui a columns o resultado de covariate_columns(self.dataset). A expressão à direita é avaliada antes da atribuição.

covariate_columns — dataset: bloco de configuração de uma cidade. Retorna nomes numéricos seguidos dos categóricos, após rejeitar duplicatas e nomes reservados. Argumentos usados: posicional 1: self.dataset.

```
0023          frame = history[['timestamp', 'water_produced'] + columns].copy()
```

Atribui a frame o resultado de history[['timestamp', 'water_produced'] + columns].copy(). A expressão à direita é avaliada antes da atribuição.

history[['timestamp', 'water_produced'] + columns].copy — Copia uma tabela/dicionário para não alterar diretamente o objeto anterior. Argumentos usados: nenhum argumento explícito.

```
0024          future = future_covariates[['timestamp'] + columns].copy()
```

Atribui a future o resultado de future_covariates[['timestamp'] + columns].copy(). A expressão à direita é avaliada antes da atribuição.

future_covariates[['timestamp'] + columns].copy — Copia uma tabela/dicionário para não alterar diretamente o objeto anterior. Argumentos usados: nenhum argumento explícito.

```
0025          frame['unique_id'] = future['unique_id'] = 'water'
```

Atribui a frame['unique_id'], future['unique_id'] o resultado de 'water'. A expressão à direita é avaliada antes da atribuição.

```
0026          self.last_audit = {'history_rows': len(frame), 'context_length': len(frame),
```

Atribui a self.last_audit o resultado de {'history_rows': len(frame), 'context_length': len(frame), 'history_start': str(frame.timestamp.iloc[0]), 'history_end': str(frame.timestamp.iloc[-1]), 'future_columns': future.columns.tolist(), 'covariates': columns}. A expressão à direita é avaliada antes da atribuição.

len — Conta elementos. Argumentos usados: posicional 1: frame.

len — Conta elementos. Argumentos usados: posicional 1: frame.

```
0027                              'history_start': str(frame.timestamp.iloc[0]),
```

Define 'history_start': primeira data histórica efetivamente fornecida; recebe str(frame.timestamp.iloc[0]). Os valores são avaliados nesta janela/iteração.

str — Produz representação textual. Argumentos usados: posicional 1: frame.timestamp.iloc[0].

```
0028                              'history_end': str(frame.timestamp.iloc[-1]),
```

Define 'history_end': última data conhecida antes da origem; recebe str(frame.timestamp.iloc[-1]). Os valores são avaliados nesta janela/iteração.

str — Produz representação textual. Argumentos usados: posicional 1: frame.timestamp.iloc[-1].

```
0029                              'future_columns': future.columns.tolist(), 'covariates': columns}
```

Define 'future_columns': campo do registro/coleção em construção; recebe future.columns.tolist(); 'covariates': nomes das variáveis auxiliares; recebe columns. Os valores são avaliados nesta janela/iteração.

future.columns.tolist — Converte array/índice/Series em lista Python. Argumentos usados: nenhum argumento explícito.

```
0030          result = self.pipeline.predict_df(
```

Atribui a result o resultado de self.pipeline.predict_df(frame, future_df=future, prediction_length=horizon, freq='D', batch_size=1, id_column='unique_id', timestamp_column='timestamp', target='water_produced', quantile_levels=[0.1, 0.5, 0.9], context_length=len(frame)). A expressão à direita é avaliada antes da atribuição.

self.pipeline.predict_df — Chronos: prevê uma tabela temporal com covariáveis. Argumentos usados: posicional 1: frame; future_df: future; prediction_length: horizon; freq: 'D'; batch_size: 1; id_column: 'unique_id'; timestamp_column: 'timestamp'; target: 'water_produced'; quantile_levels: [0.1, 0.5, 0.9]; context_length: len(frame).

```
0031              frame, future_df=future, prediction_length=horizon, freq='D', batch_size=1,
```

Continua os argumentos nomeados: future_df = future; prediction_length = horizon; freq = 'D'; batch_size = 1. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0032              id_column='unique_id', timestamp_column='timestamp', target='water_produced',
```

Continua os argumentos nomeados: id_column = 'unique_id'; timestamp_column = 'timestamp'; target = 'water_produced'. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0033              quantile_levels=[0.1, 0.5, 0.9], context_length=len(frame))
```

Continua os argumentos nomeados: quantile_levels = [0.1, 0.5, 0.9]; context_length = len(frame). Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

len — Conta elementos. Argumentos usados: posicional 1: frame.

```
0034          return result['0.5'].to_numpy(dtype=float).tolist()
```

Encerra esta função e devolve result['0.5'].to_numpy(dtype=float).tolist() ao chamador; não imprime o resultado por si só.

result['0.5'].to_numpy(dtype=float).tolist — Converte array/índice/Series em lista Python. Argumentos usados: nenhum argumento explícito.

result['0.5'].to_numpy — Converte Series/DataFrame em array NumPy. Argumentos usados: dtype: float.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

29. src/models/timesfm.py

Adapter de TimesFM 2.5 com XReg. Prepara capacidade suficiente para todo o histórico a cada origem e devolve somente os primeiros horizon valores. Não corta a história ao contexto inicial.

SHA-256: 508356fc7f9b65ed2ef610fa1214374f15cc57dca933f767015cceac03892ff0

Funções, contratos e pontos de conexão

```
L8: __init__(self, context: int, dataset: dict)
```

Construtor da classe: self recebe a instância e os demais argumentos inicializam configuração, backend ou estado. Confira o corpo abaixo para I/O e carregamento de modelo.

```
L20: predict(self, history, horizon: int, *, future_covariates [obrigatório], prompt = None) →
    list[float]
```

history é tabela histórica; horizon é quantidade prevista; future_covariates é tabela sem alvo; prompt é texto já construído para Qwen. Retorna lista de floats; cada classe escolhe backend. O método pertence à classe adaptadora deste arquivo.

Chamadas do projeto: covariate_columns → src/data/features/covariates.py:6; timesfm_covariates → src/data/features/covariates.py:36

APIs externas usadas neste arquivo

compile, forecast_with_covariates, from_pretrained, importlib.metadata.version, len, np.asarray, str, timesfm.ForecastConfig, to_numpy, tolist, torch.manual_seed, torch.set_float32_matmul_precision. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Original TimesFM XReg + TimesFM path, with context growing at each origin."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import importlib.metadata
```

Importa importlib.metadata. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0003  import numpy as np
```

Importa numpy como np. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0004  from src.data.features.covariates import timesfm_covariates, covariate_columns
```

Importa timesfm_covariates, covariate_columns de src.data.features.covariates. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0007  class TimesFMForecaster:
```

Declara a classe TimesFMForecaster. Seus métodos compartilham estado por self; a declaração não cria uma previsão.

```
0008      def __init__(self, context: int, dataset: dict):
```

Define __init__(self, context: int, dataset: dict). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0009          import torch
```

Importa torch. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0010          import timesfm
```

Importa timesfm. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0011          self.dataset = dataset
```

Atribui a self.dataset o resultado de dataset. A expressão à direita é avaliada antes da atribuição.

```
0012          torch.manual_seed(42)
```

Executa torch.manual_seed(42). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

torch.manual_seed — Define semente do gerador PyTorch. Argumentos usados: posicional 1: 42.

```
0013          torch.set_float32_matmul_precision('high')
```

Executa torch.set_float32_matmul_precision('high'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

torch.set_float32_matmul_precision — Configura precisão permitida para multiplicações de matrizes float32. Argumentos usados: posicional 1: 'high'.

```
0014          self.model = timesfm.TimesFM_2p5_200M_torch.from_pretrained(
```

Atribui a self.model o resultado de timesfm.TimesFM_2p5_200M_torch.from_pretrained('google/timesfm-2.5-200m-pytorch', local_files_only=True). A expressão à direita é avaliada antes da atribuição.

timesfm.TimesFM_2p5_200M_torch.from_pretrained — Carrega pesos/configuração de um checkpoint. Argumentos usados: posicional 1: 'google/timesfm-2.5-200m-pytorch'; local_files_only: True.

```
0015              'google/timesfm-2.5-200m-pytorch', local_files_only=True)
```

Continua os argumentos nomeados: local_files_only = True. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0016          self.identity = {'model': 'google/timesfm-2.5-200m-pytorch', 'initial_context':
    context,
```

Atribui a self.identity o resultado de {'model': 'google/timesfm-2.5-200m-pytorch', 'initial_context': context, 'version': importlib.metadata.version('timesfm'), 'xreg_mode': 'xreg + timesfm'}. A expressão à direita é avaliada antes da atribuição.

```
0017                          'version': importlib.metadata.version('timesfm'), 'xreg_mode': 'xreg
     + timesfm'}
```

Define 'version': campo do registro/coleção em construção; recebe importlib.metadata.version('timesfm'); 'xreg_mode': campo do registro/coleção em construção; recebe 'xreg + timesfm'. Os valores são avaliados nesta janela/iteração.

importlib.metadata.version — Lê versão instalada de uma distribuição Python. Argumentos usados: posicional 1: 'timesfm'.

```
0018          self.last_audit = {}
```

Atribui a self.last_audit o resultado de {}. A expressão à direita é avaliada antes da atribuição.

```
0020      def predict(self, history, horizon: int, *, future_covariates, prompt=None) ->
    list[float]:
```

Define predict(self, history, horizon: int, *, future_covariates [obrigatório], prompt = None) → list[float]. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0021          import timesfm
```

Importa timesfm. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0022          context = len(history)
```

Atribui a context o resultado de len(history). A expressão à direita é avaliada antes da atribuição.

len — Conta elementos. Argumentos usados: posicional 1: history.

```
0023          self.model.compile(timesfm.ForecastConfig(
```

Executa self.model.compile(timesfm.ForecastConfig(max_context=(context + 31) // 32 * 32, max_horizon=(horizon + 127) // 128 * 128, normalize_inputs=True, use_continuous_quantile_head=True, force_flip_invariance=True, infer_is_positive=True, fix_quantile_crossing=True, return_backcast=True)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.model.compile — Prepara a configuração de previsão do TimesFM. Argumentos usados: posicional 1: timesfm.ForecastConfig(max_context=(context + 31) // 32 * 32, max_horizon=(horizon + 127) // 128 * 128, normalize_inputs=True, use_continuous_quantile_head=True, force_flip_invariance=True, infer_is_positive=True, fix_quantile_crossing=True, return_backcast=True).

timesfm.ForecastConfig — Configura os limites e opções de inferência do TimesFM. Argumentos usados: max_context: (context + 31) // 32 * 32; max_horizon: (horizon + 127) // 128 * 128; normalize_inputs: True; use_continuous_quantile_head: True; force_flip_invariance: True; infer_is_positive: True; fix_quantile_crossing: True; return_backcast: True.

```
0024              max_context=((context + 31) // 32) * 32,
```

Continua os argumentos nomeados: max_context = (context + 31) // 32 * 32. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0025              max_horizon=((horizon + 127) // 128) * 128,
```

Continua os argumentos nomeados: max_horizon = (horizon + 127) // 128 * 128. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0026              normalize_inputs=True, use_continuous_quantile_head=True,
```

Continua os argumentos nomeados: normalize_inputs = True; use_continuous_quantile_head = True. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0027              force_flip_invariance=True, infer_is_positive=True,
```

Continua os argumentos nomeados: force_flip_invariance = True; infer_is_positive = True. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0028              fix_quantile_crossing=True, return_backcast=True))
```

Continua os argumentos nomeados: fix_quantile_crossing = True; return_backcast = True. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0029          numerical, categorical = timesfm_covariates(history, future_covariates, self.dataset)
```

Atribui a (numerical, categorical) o resultado de timesfm_covariates(history, future_covariates, self.dataset). A expressão à direita é avaliada antes da atribuição.

timesfm_covariates — history/future: tabelas separadas; dataset: listas numéricas/categóricas. Retorna dois dicts com listas de séries cobrindo passado e futuro, sem demanda futura. Argumentos usados: posicional 1: history; posicional 2: future_covariates; posicional 3: self.dataset.

```
0030          self.last_audit = {'history_rows': context,
```

Atribui a self.last_audit o resultado de {'history_rows': context, 'history_start': str(history.timestamp.iloc[0]), 'history_end': str(history.timestamp.iloc[-1]), 'max_context': (context + 31) // 32 * 32, 'covariates': covariate_columns(self.dataset), 'future_rows': len(future_covariates), 'xreg_mode': 'xreg + timesfm'}. A expressão à direita é avaliada antes da atribuição.

```
0031                            'history_start': str(history.timestamp.iloc[0]),
```

Define 'history_start': primeira data histórica efetivamente fornecida; recebe str(history.timestamp.iloc[0]). Os valores são avaliados nesta janela/iteração.

str — Produz representação textual. Argumentos usados: posicional 1: history.timestamp.iloc[0].

```
0032                            'history_end': str(history.timestamp.iloc[-1]),
```

Define 'history_end': última data conhecida antes da origem; recebe str(history.timestamp.iloc[-1]). Os valores são avaliados nesta janela/iteração.

str — Produz representação textual. Argumentos usados: posicional 1: history.timestamp.iloc[-1].

```
0033                            'max_context': ((context+31)//32)*32,
```

Define 'max_context': campo do registro/coleção em construção; recebe (context + 31) // 32 * 32. Os valores são avaliados nesta janela/iteração.

```
0034                            'covariates': covariate_columns(self.dataset),
```

Define 'covariates': nomes das variáveis auxiliares; recebe covariate_columns(self.dataset). Os valores são avaliados nesta janela/iteração.

covariate_columns — dataset: bloco de configuração de uma cidade. Retorna nomes numéricos seguidos dos categóricos, após rejeitar duplicatas e nomes reservados. Argumentos usados: posicional 1: self.dataset.

```
0035                            'future_rows': len(future_covariates), 'xreg_mode': 'xreg +
    timesfm'}
```

Define 'future_rows': campo do registro/coleção em construção; recebe len(future_covariates); 'xreg_mode': campo do registro/coleção em construção; recebe 'xreg + timesfm'. Os valores são avaliados nesta janela/iteração.

len — Conta elementos. Argumentos usados: posicional 1: future_covariates.

```
0036          point, _ = self.model.forecast_with_covariates(
```

Atribui a (point, _) o resultado de self.model.forecast_with_covariates(inputs=[history.water_produced.to_numpy(dtype=float)], dynamic_numerical_covariates=numerical, dynamic_categorical_covariates=categorical, xreg_mode='xreg + timesfm'). A expressão à direita é avaliada antes da atribuição.

self.model.forecast_with_covariates — Executa regressão auxiliar e TimesFM com variáveis exógenas. Argumentos usados: inputs: [history.water_produced.to_numpy(dtype=float)]; dynamic_numerical_covariates: numerical; dynamic_categorical_covariates: categorical; xreg_mode: 'xreg + timesfm'.

```
0037              inputs=[history.water_produced.to_numpy(dtype=float)],
```

Continua os argumentos nomeados: inputs = [history.water_produced.to_numpy(dtype=float)]; dtype = float. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

history.water_produced.to_numpy — Converte Series/DataFrame em array NumPy. Argumentos usados: dtype: float.

```
0038              dynamic_numerical_covariates=numerical,
```

Continua os argumentos nomeados: dynamic_numerical_covariates = numerical. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0039              dynamic_categorical_covariates=categorical,
```

Continua os argumentos nomeados: dynamic_categorical_covariates = categorical. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0040              xreg_mode='xreg + timesfm')
```

Continua os argumentos nomeados: xreg_mode = 'xreg + timesfm'. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0041          return np.asarray(point[0], dtype=float)[:horizon].tolist()
```

Encerra esta função e devolve np.asarray(point[0], dtype=float)[:horizon].tolist() ao chamador; não imprime o resultado por si só.

np.asarray(point[0], dtype=float)[:horizon].tolist — Converte array/índice/Series em lista Python. Argumentos usados: nenhum argumento explícito.

np.asarray — Representa dados como ndarray, evitando cópia quando possível. Argumentos usados: posicional 1: point[0]; dtype: float.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

30. src/evaluation/metrics.py

Função numérica sem I/O. Recebe vetores reais e previstos já alinhados e retorna dicionário de métricas. É chamada pelos comparadores, testes e auditoria.

SHA-256: f4141db5d243c7b5e939064ef2f4c022545b6d99fc6846ce36a6aeee4875d2a2

Funções, contratos e pontos de conexão

```
L5: deterministic_metrics(ytrue, yhat) → dict
```

ytrue/yhat: vetores de mesmo formato, não vazios e finitos. Retorna MAE, RMSE, MAPE%, sMAPE%, R² e contagem de zeros excluídos do MAPE.

APIs externas usadas neste arquivo

ValueError, all, any, float, int, np.abs, np.asarray, np.divide, np.isfinite, np.mean, np.sqrt, np.sum, np.zeros_like, sum. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Explicit metric definitions, with no independent dropping of true/predicted rows."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import numpy as np
```

Importa numpy como np. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  def deterministic_metrics(ytrue, yhat) -> dict:
```

Define deterministic_metrics(ytrue, yhat) → dict. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0006      actual, predicted = np.asarray(ytrue, dtype=float), np.asarray(yhat, dtype=float)
```

Converte ambos os vetores para floats sem remover qualquer linha. Mesma posição deve representar a mesma data e lead.

np.asarray — Representa dados como ndarray, evitando cópia quando possível. Argumentos usados: posicional 1: ytrue; dtype: float.

np.asarray — Representa dados como ndarray, evitando cópia quando possível. Argumentos usados: posicional 1: yhat; dtype: float.

```
0007      if actual.shape != predicted.shape or actual.size == 0:
```

Avalia a condição actual.shape != predicted.shape or actual.size == 0. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0008          raise ValueError('Aligned nonempty targets and predictions are required')
```

Interrompe o caminho atual lançando ValueError('Aligned nonempty targets and predictions are required'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Aligned nonempty targets and predictions are required'.

```
0009      if not np.isfinite(actual).all() or not np.isfinite(predicted).all():
```

Avalia a condição not np.isfinite(actual).all() or not np.isfinite(predicted).all(). O bloco indentado seguinte só é executado quando ela é verdadeira.

np.isfinite(actual).all — Exige que todos os elementos sejam True. Argumentos usados: nenhum argumento explícito.

np.isfinite(predicted).all — Exige que todos os elementos sejam True. Argumentos usados: nenhum argumento explícito.

np.isfinite — Testa finitude elemento a elemento. Argumentos usados: posicional 1: actual.

np.isfinite — Testa finitude elemento a elemento. Argumentos usados: posicional 1: predicted.

```
0010          raise ValueError('Nonfinite values are not silently removed')
```

Interrompe o caminho atual lançando ValueError('Nonfinite values are not silently removed'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Nonfinite values are not silently removed'.

```
0011      error = predicted - actual
```

Convenção: previsão menos observado. O sinal desaparece em MAE/RMSE, mas é útil para viés na auditoria.

```
0012      nonzero = actual != 0
```

Máscara apenas para MAPE; os zeros continuam incluídos em MAE/RMSE/sMAPE/R².

```
0013      denom = np.abs(actual) + np.abs(predicted)
```

Atribui a denom o resultado de np.abs(actual) + np.abs(predicted). A expressão à direita é avaliada antes da atribuição.

np.abs — Módulo elemento a elemento. Argumentos usados: posicional 1: actual.

np.abs — Módulo elemento a elemento. Argumentos usados: posicional 1: predicted.

```
0014      smape = np.divide(2 * np.abs(error), denom, out=np.zeros_like(error), where=denom != 0)
```

Numerador 2*abs(erro), denominador abs(real)+abs(previsto). Onde ambos são zero, out inicializado com zero conserva contribuição zero.

np.divide — Divide vetores com máscara e destino explícitos. Argumentos usados: posicional 1: 2 * np.abs(error); posicional 2: denom; out: np.zeros_like(error); where: denom != 0.

np.abs — Módulo elemento a elemento. Argumentos usados: posicional 1: error.

np.zeros_like — Cria zeros com formato e tipo semelhantes ao array. Argumentos usados: posicional 1: error.

```
0015      total = float(np.sum((actual - np.mean(actual)) ** 2))
```

Soma dos quadrados em relação à média real; é denominador de R², não variância já dividida por N.

float — Converte escalar para float Python. Argumentos usados: posicional 1: np.sum((actual - np.mean(actual)) ** 2).

np.sum — Soma elementos. Argumentos usados: posicional 1: (actual - np.mean(actual)) ** 2.

np.mean — Calcula média aritmética. Argumentos usados: posicional 1: actual.

```
0016      return {'mae': float(np.mean(np.abs(error))), 'rmse': float(np.sqrt(np.mean(error **
    2))),
```

MAE é média do módulo; RMSE é raiz da média dos quadrados em TODOS os pares fornecidos. Não é média de RMSEs por janela.

float — Converte escalar para float Python. Argumentos usados: posicional 1: np.mean(np.abs(error)).

float — Converte escalar para float Python. Argumentos usados: posicional 1: np.sqrt(np.mean(error ** 2)).

np.mean — Calcula média aritmética. Argumentos usados: posicional 1: np.abs(error).

np.sqrt — Raiz quadrada elemento a elemento ou escalar. Argumentos usados: posicional 1: np.mean(error ** 2).

np.abs — Módulo elemento a elemento. Argumentos usados: posicional 1: error.

np.mean — Calcula média aritmética. Argumentos usados: posicional 1: error ** 2.

```
0017              'mape_percent': float(np.mean(np.abs(error[nonzero] / actual[nonzero])) * 100) if
     nonzero.any() else None,
```

MAPE considera apenas alvos não zero, multiplica por 100 e retorna None se nenhum alvo é elegível.

nonzero.any — Reduz a existência de pelo menos um True. Argumentos usados: nenhum argumento explícito.

float — Converte escalar para float Python. Argumentos usados: posicional 1: np.mean(np.abs(error[nonzero] / actual[nonzero])) * 100.

np.mean — Calcula média aritmética. Argumentos usados: posicional 1: np.abs(error[nonzero] / actual[nonzero]).

np.abs — Módulo elemento a elemento. Argumentos usados: posicional 1: error[nonzero] / actual[nonzero].

```
0018             'smape_percent': float(np.mean(smape) * 100),
```

sMAPE é apresentado em porcentagem; nesta definição com fator dois pode chegar a 200%.

float — Converte escalar para float Python. Argumentos usados: posicional 1: np.mean(smape) * 100.

np.mean — Calcula média aritmética. Argumentos usados: posicional 1: smape.

```
0019             'r2': 1 - float(np.sum(error ** 2)) / total if total else None,
```

R^2 = 1 - SSE/SST. Se os alvos forem constantes, total=0 e retorna None. Valores negativos são possíveis e não são erro do parser.

float — Converte escalar para float Python. Argumentos usados: posicional 1: np.sum(error ** 2).

np.sum — Soma elementos. Argumentos usados: posicional 1: error ** 2.

```
0020             'mape_zero_targets_excluded': int((~nonzero).sum())}
```

Conta zeros excluídos do MAPE para não esconder redução do conjunto dessa métrica.

int — Converte para inteiro, truncando parte fracionária numérica. Argumentos usados: posicional 1: (~nonzero).sum().

(~nonzero).sum — Soma elementos de iterável. Argumentos usados: nenhum argumento explícito.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

31. src/pipelines/compare_models.py

Comparador do benchmark qwen/chronos/timesfm. Rejeita execuções incompletas e datas incompatíveis; sua lógica específica de contexto do Qwen não cobre o braço expansivo.

SHA-256: 7b2a4b81739aeda24d12693f71da1ddaf13e685b9cc753b6d6477b4a35bd63b1

Funções, contratos e pontos de conexão

```
L10: compare(output: Path)
```

output: pasta com resultados dos três modelos/cidades. Sem retorno útil; valida e grava métricas globais e por lead.

Chamadas do projeto: deterministic_metrics → src/evaluation/metrics.py:5

```
L57: main()
```

Sem parâmetros explícitos. Lê argumentos do processo via argparse ou executa entry point do módulo. O corpo de cada arquivo determina qual CLI será iniciado.

Chamadas do projeto: compare → src/pipelines/compare_models.py:10

APIs externas usadas neste arquivo

ValueError, add_argument, any, append, argparse.ArgumentParser, dict, drop_duplicates, duplicated, equals, float, groupby, int, join, json.loads, len, max, mean, min, nunique, parse_args, pd.DataFrame, pd.read_csv, print, range, read_text, reset_index, sort_values, sum, to_csv, to_string, tolist. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Refuse incomplete or mismatched comparisons; report aggregate and per-lead metrics."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import argparse
```

Importa argparse. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0003  import json
```

Importa json. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0004  from pathlib import Path
```

Importa Path de pathlib. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  import pandas as pd
```

Importa pandas como pd. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0006  from src.evaluation.metrics import deterministic_metrics
```

Importa deterministic_metrics de src.evaluation.metrics. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0007  from src.pipelines.forecasting_pipeline import ROOT
```

Importa ROOT de src.pipelines.forecasting_pipeline. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0010  def compare(output: Path):
```

Define compare(output: Path). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0011      summaries, per_lead = [], []
```

Atribui a (summaries, per_lead) o resultado de ([], []). A expressão à direita é avaliada antes da atribuição.

```
0012      for dataset in ('joinville', 'guaratuba'):
```

Percorre ('joinville', 'guaratuba'); em cada iteração associa o item a dataset. O corpo indentado aplica a operação a esse item/grupo.

```
0013          reference = None
```

Atribui a reference o resultado de None. A expressão à direita é avaliada antes da atribuição.

```
0014          reference_run = None
```

Atribui a reference_run o resultado de None. A expressão à direita é avaliada antes da atribuição.

```
0015          for model in ('qwen', 'chronos', 'timesfm'):
```

Percorre ('qwen', 'chronos', 'timesfm'); em cada iteração associa o item a model. O corpo indentado aplica a operação a esse item/grupo.

```
0016              folder = output / dataset / model
```

Atribui a folder o resultado de output / dataset / model. A expressão à direita é avaliada antes da atribuição.

```
0017              status = json.loads((folder / 'status.json').read_text())
```

Atribui a status o resultado de json.loads((folder / 'status.json').read_text()). A expressão à direita é avaliada antes da atribuição.

json.loads — Decodifica uma string JSON em objetos Python. Argumentos usados: posicional 1: (folder / 'status.json').read_text().

(folder / 'status.json').read_text — Lê todo o arquivo como string. Argumentos usados: nenhum argumento explícito.

```
0018              run = json.loads((folder / 'run.json').read_text())
```

Atribui a run o resultado de json.loads((folder / 'run.json').read_text()). A expressão à direita é avaliada antes da atribuição.

json.loads — Decodifica uma string JSON em objetos Python. Argumentos usados: posicional 1: (folder / 'run.json').read_text().

(folder / 'run.json').read_text — Lê todo o arquivo como string. Argumentos usados: nenhum argumento explícito.

```
0019              if status['failures'] or status['successful_windows'] !=
    status['expected_windows'] or run['limit']:
```

Avalia a condição status['failures'] or status['successful_windows'] != status['expected_windows'] or run['limit']. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0020                  raise ValueError(f'Incomplete benchmark: {dataset}/{model}')
```

Interrompe o caminho atual lançando ValueError(f'Incomplete benchmark: {dataset}/{model}'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: f'Incomplete benchmark: {dataset}/{model}'.

```
0021              frame = pd.read_csv(folder / 'predictions.csv').sort_values(['origin',
    'lead']).reset_index(drop=True)
```

Atribui a frame o resultado de pd.read_csv(folder / 'predictions.csv').sort_values(['origin', 'lead']).reset_index(drop=True). A expressão à direita é avaliada antes da atribuição.

pd.read_csv(folder / 'predictions.csv').sort_values(['origin', 'lead']).reset_index — Transforma o índice atual em coluna ou o descarta e cria índice sequencial. Argumentos usados: drop: True.

pd.read_csv(folder / 'predictions.csv').sort_values — Ordena linhas pelos valores de colunas. Argumentos usados: posicional 1: ['origin', 'lead'].

pd.read_csv — Lê CSV em um DataFrame, inferindo tipos; datas só são convertidas posteriormente no validador. Argumentos usados: posicional 1: folder / 'predictions.csv'.

```
0022              keys = frame[['origin', 'timestamp', 'lead', 'y_true', 'initial_context',
    'horizon']]
```

Atribui a keys o resultado de frame[['origin', 'timestamp', 'lead', 'y_true', 'initial_context', 'horizon']]. A expressão à direita é avaliada antes da atribuição.

```
0023             origins = frame.drop_duplicates('origin')
```

Atribui a origins o resultado de frame.drop_duplicates('origin'). A expressão à direita é avaliada antes da atribuição.

frame.drop_duplicates — Retém uma linha por chave repetida. Argumentos usados: posicional 1: 'origin'.

```
0024             initial = int(frame.initial_context.iloc[0])
```

Atribui a initial o resultado de int(frame.initial_context.iloc[0]). A expressão à direita é avaliada antes da atribuição.

int — Converte para inteiro, truncando parte fracionária numérica. Argumentos usados: posicional 1: frame.initial_context.iloc[0].

```
0025             expected_contexts = ([run['qwen_history_days']] * len(origins) if model == 'qwen'
```

Limitação: assume que todo Qwen tem história fixa igual a qwen_history_days. Se for None para expansivo, espera uma lista de None e rejeita comprimentos reais. Use compare_qwen_contexts para o experimento expansivo existente.

len — Conta elementos. Argumentos usados: posicional 1: origins.

```
0026                                 else [initial + 7*i for i in range(len(origins))])
```

Limitação: assume que todo Qwen tem história fixa igual a qwen_history_days. Se for None para expansivo, espera uma lista de None e rejeita comprimentos reais. Use compare_qwen_contexts para o experimento expansivo existente.

range — Gera inteiros com limite final exclusivo. Argumentos usados: posicional 1: len(origins).

len — Conta elementos. Argumentos usados: posicional 1: origins.

```
0027             if origins.context.tolist() != expected_contexts:
```

Avalia a condição origins.context.tolist() != expected_contexts. O bloco indentado seguinte só é executado quando ela é verdadeira.

origins.context.tolist — Converte array/índice/Series em lista Python. Argumentos usados: nenhum argumento explícito.

```
0028                 raise ValueError(f'Incorrect history lengths: {dataset}/{model}')
```

Interrompe o caminho atual lançando ValueError(f'Incorrect history lengths: {dataset}/{model}'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: f'Incorrect history lengths: {dataset}/{model}'.

```
0029             if model != 'qwen' and origins.history_start.nunique() != 1:
```

Avalia a condição model != 'qwen' and origins.history_start.nunique() != 1. O bloco indentado seguinte só é executado quando ela é verdadeira.

origins.history_start.nunique — Conta valores distintos. Argumentos usados: nenhum argumento explícito.

```
0030                 raise ValueError('Expanding history must always start on the first date')
```

Interrompe o caminho atual lançando ValueError('Expanding history must always start on the first date'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Expanding history must always start on the first date'.

```
0031             if frame.duplicated(['origin', 'lead']).any() or len(frame) !=
    status['expected_windows'] * 7:
```

Avalia a condição frame.duplicated(['origin', 'lead']).any() or len(frame) != status['expected_windows'] * 7. O bloco indentado seguinte só é executado quando ela é verdadeira.

frame.duplicated(['origin', 'lead']).any — Reduz a existência de pelo menos um True. Argumentos usados: nenhum argumento explícito.

len — Conta elementos. Argumentos usados: posicional 1: frame.

frame.duplicated — Marca repetições. Argumentos usados: posicional 1: ['origin', 'lead'].

```
0032                 raise ValueError('Duplicate or missing forecasts')
```

Interrompe o caminho atual lançando ValueError('Duplicate or missing forecasts'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Duplicate or missing forecasts'.

```
0033             comparable = {key: run[key] for key in ('config', 'dataset_sha256',
    'source_sha256', 'n_windows')}
```

Atribui a comparable o resultado de {key: run[key] for key in ('config', 'dataset_sha256', 'source_sha256', 'n_windows')}. A expressão à direita é avaliada antes da atribuição.

```
0034             if reference is not None and (not keys.equals(reference) or comparable !=
    reference_run):
```

Avalia a condição reference is not None and (not keys.equals(reference) or comparable != reference_run). O bloco indentado seguinte só é executado quando ela é verdadeira.

keys.equals — Compara conteúdo e estrutura do receptor com outro objeto pandas. Argumentos usados: posicional 1: reference.

```
0035                 raise ValueError(f'Incomparable evaluation for {dataset}/{model}')
```

Interrompe o caminho atual lançando ValueError(f'Incomparable evaluation for {dataset}/{model}'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: f'Incomparable evaluation for {dataset}/{model}'.

```
0036             reference, reference_run = keys, comparable
```

Atribui a (reference, reference_run) o resultado de (keys, comparable). A expressão à direita é avaliada antes da atribuição.

```
0037             common = {'dataset': dataset, 'model': model, 'model_id': run['model']['model'],
```

Atribui a common o resultado de {'dataset': dataset, 'model': model, 'model_id': run['model']['model'], 'initial_context_days': int(frame.initial_context.iloc[0]), 'context_days_first': int(frame.context.iloc[0]), 'context_days_last': int(frame.context.iloc[-1]), 'history_policy': 'last 15 days' if model == 'qwen' else 'expanding', 'covariates': '|'.join(run['covariates']), 'horizon_days': 7, 'step_days': 7, 'strategy': run['config']['strategy'] if model == 'qwen' else 'pretrained', 'n_windows': status['expected_windows'], 'n_predictions': len(frame), 'test_start': frame.timestamp.min(), 'test_end': frame.timestamp.max()}. A expressão à direita é avaliada antes da atribuição.

```
0038                         'initial_context_days': int(frame.initial_context.iloc[0]),
```

Define 'initial_context_days': campo do registro/coleção em construção; recebe int(frame.initial_context.iloc[0]). Os valores são avaliados nesta janela/iteração.

int — Converte para inteiro, truncando parte fracionária numérica. Argumentos usados: posicional 1: frame.initial_context.iloc[0].

```
0039                         'context_days_first': int(frame.context.iloc[0]),
```

Define 'context_days_first': campo do registro/coleção em construção; recebe int(frame.context.iloc[0]). Os valores são avaliados nesta janela/iteração.

int — Converte para inteiro, truncando parte fracionária numérica. Argumentos usados: posicional 1: frame.context.iloc[0].

```
0040                         'context_days_last': int(frame.context.iloc[-1]),
```

Define 'context_days_last': campo do registro/coleção em construção; recebe int(frame.context.iloc[-1]). Os valores são avaliados nesta janela/iteração.

int — Converte para inteiro, truncando parte fracionária numérica. Argumentos usados: posicional 1: frame.context.iloc[-1].

```
0041                         'history_policy': 'last 15 days' if model == 'qwen' else 'expanding',
```

Rótulo hardcoded last 15 days: não descreve corretamente um futuro teste Qwen com sete dias. Este guia expõe a limitação sem modificar o comparador.

```
0042                        'covariates': '|'.join(run['covariates']), 'horizon_days': 7,
    'step_days': 7,
```

Define 'covariates': nomes das variáveis auxiliares; recebe '|'.join(run['covariates']); 'horizon_days': campo do registro/coleção em construção; recebe 7; 'step_days': campo do registro/coleção em construção; recebe 7. Os valores são avaliados nesta janela/iteração.

'|'.join — Une textos com o separador que recebe o método. Argumentos usados: posicional 1: run['covariates'].

```
0043                        'strategy': run['config']['strategy'] if model == 'qwen' else
    'pretrained',
```

Define 'strategy': campo do registro/coleção em construção; recebe run['config']['strategy'] if model == 'qwen' else 'pretrained'. Os valores são avaliados nesta janela/iteração.

```
0044                        'n_windows': status['expected_windows'], 'n_predictions': len(frame),
```

Define 'n_windows': campo do registro/coleção em construção; recebe status['expected_windows']; 'n_predictions': campo do registro/coleção em construção; recebe len(frame). Os valores são avaliados nesta janela/iteração.

len — Conta elementos. Argumentos usados: posicional 1: frame.

```
0045                        'test_start': frame.timestamp.min(), 'test_end': frame.timestamp.max()}
```

Define 'test_start': campo do registro/coleção em construção; recebe frame.timestamp.min(); 'test_end': campo do registro/coleção em construção; recebe frame.timestamp.max(). Os valores são avaliados nesta janela/iteração.

frame.timestamp.min — Retorna menor valor. Argumentos usados: nenhum argumento explícito.

frame.timestamp.max — Retorna maior valor. Argumentos usados: nenhum argumento explícito.

```
0046              timing = frame.drop_duplicates('origin').elapsed_seconds_window
```

Atribui a timing o resultado de frame.drop_duplicates('origin').elapsed_seconds_window. A expressão à direita é avaliada antes da atribuição.

frame.drop_duplicates — Retém uma linha por chave repetida. Argumentos usados: posicional 1: 'origin'.

```
0047              summaries.append(dict(common, **deterministic_metrics(frame.y_true,
    frame.y_pred),
```

Executa summaries.append(dict(common, **deterministic_metrics(frame.y_true, frame.y_pred), inference_seconds=float(timing.sum()), mean_window_seconds=float(timing.mean()))). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

summaries.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: dict(common, **deterministic_metrics(frame.y_true, frame.y_pred), inference_seconds=float(timing.sum()), mean_window_seconds=float(timing.mean())).

dict — Cria dicionário. Argumentos usados: posicional 1: common; **desempacotamento: deterministic_metrics(frame.y_true, frame.y_pred); inference_seconds: float(timing.sum()); mean_window_seconds: float(timing.mean()).

deterministic_metrics — ytrue/yhat: vetores de mesmo formato, não vazios e finitos. Retorna MAE, RMSE, MAPE%, sMAPE%, R^2 e contagem de zeros excluídos do MAPE. Argumentos usados: posicional 1: frame.y_true; posicional 2: frame.y_pred.

```
0048                                  inference_seconds=float(timing.sum()),
    mean_window_seconds=float(timing.mean())))
```

Continua os argumentos nomeados: inference_seconds = float(timing.sum()); mean_window_seconds = float(timing.mean()). Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

float — Converte escalar para float Python. Argumentos usados: posicional 1: timing.sum().

float — Converte escalar para float Python. Argumentos usados: posicional 1: timing.mean().

timing.sum — Soma elementos de iterável. Argumentos usados: nenhum argumento explícito.

timing.mean — Calcula média dos valores do objeto receptor. Argumentos usados: nenhum argumento explícito.

```
0049              for lead, subset in frame.groupby('lead'):
```

Percorre frame.groupby('lead'); em cada iteração associa o item a (lead, subset). O corpo indentado aplica a operação a esse item/grupo.

frame.groupby — Cria grupos sem calcular métricas por si só. Argumentos usados: posicional 1: 'lead'.

```
0050                 per_lead.append(dict(dataset=dataset, model=model, lead=int(lead),
```

Executa per_lead.append(dict(dataset=dataset, model=model, lead=int(lead), **deterministic_metrics(subset.y_true, subset.y_pred))). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

per_lead.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: dict(dataset=dataset, model=model, lead=int(lead), **deterministic_metrics(subset.y_true, subset.y_pred)).

dict — Cria dicionário. Argumentos usados: dataset: dataset; model: model; lead: int(lead); **desempacotamento: deterministic_metrics(subset.y_true, subset.y_pred).

int — Converte para inteiro, truncando parte fracionária numérica. Argumentos usados: posicional 1: lead.

```
0051                                     **deterministic_metrics(subset.y_true, subset.y_pred)))
```

Continua os argumentos nomeados: ** = deterministic_metrics(subset.y_true, subset.y_pred). Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

deterministic_metrics — ytrue/yhat: vetores de mesmo formato, não vazios e finitos. Retorna MAE, RMSE, MAPE%, sMAPE%, R² e contagem de zeros excluídos do MAPE. Argumentos usados: posicional 1: subset.y_true; posicional 2: subset.y_pred.

```
0052     pd.DataFrame(summaries).to_csv(output / 'metrics_comparison.csv', index=False)
```

Executa pd.DataFrame(summaries).to_csv(output / 'metrics_comparison.csv', index=False). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

pd.DataFrame(summaries).to_csv — Serializa DataFrame como CSV. Argumentos usados: posicional 1: output / 'metrics_comparison.csv'; index: False.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: summaries.

```
0053     pd.DataFrame(per_lead).to_csv(output / 'metrics_by_horizon.csv', index=False)
```

Executa pd.DataFrame(per_lead).to_csv(output / 'metrics_by_horizon.csv', index=False). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

pd.DataFrame(per_lead).to_csv — Serializa DataFrame como CSV. Argumentos usados: posicional 1: output / 'metrics_by_horizon.csv'; index: False.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: per_lead.

```
0054     print(pd.DataFrame(summaries)[['dataset', 'model', 'mae', 'rmse', 'mape_percent',
    'r2']].to_string(index=False))
```

Executa print(pd.DataFrame(summaries)[['dataset', 'model', 'mae', 'rmse', 'mape_percent', 'r2']].to_string(index=False)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

print — Escreve representação textual na saída padrão. Argumentos usados: posicional 1: pd.DataFrame(summaries)[['dataset', 'model', 'mae', 'rmse', 'mape_percent', 'r2']].to_string(index=False).

pd.DataFrame(summaries)[['dataset', 'model', 'mae', 'rmse', 'mape_percent', 'r2']].to_string — Formata tabela como texto para terminal/log. Argumentos usados: index: False.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: summaries.

```
0057  def main():
```

Define main(). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0058     parser = argparse.ArgumentParser(description=__doc__)
```

Atribui a parser o resultado de argparse.ArgumentParser(description=__doc__). A expressão à direita é avaliada antes da atribuição.

argparse.ArgumentParser — Cria o parser da interface de linha de comando. Argumentos usados: description: __doc__.

```
0059     parser.add_argument('--output', type=Path, default=ROOT / 'experiments/benchmark')
```

Executa parser.add_argument('--output', type=Path, default=ROOT / 'experiments/benchmark'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

parser.add_argument — Registra uma opção do CLI. Argumentos usados: posicional 1: '--output'; type: Path; default: ROOT / 'experiments/benchmark'.

```
0060      compare(parser.parse_args().output)
```

Executa compare(parser.parse_args().output). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

compare — output: pasta com resultados dos três modelos/cidades. Sem retorno útil; valida e grava métricas globais e por lead. Argumentos usados: posicional 1: parser.parse_args().output.

parser.parse_args — Interpreta argumentos do processo e retorna Namespace. Argumentos usados: nenhum argumento explícito.

```
0063  if __name__ == '__main__':
```

Avalia a condição __name__ == '__main__'. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0064      main()
```

Executa main(). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

main — Sem parâmetros explícitos. Lê argumentos do processo via argparse ou executa entry point do módulo. O corpo de cada arquivo determina qual CLI será iniciado. Argumentos usados: nenhum argumento explícito.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

32. src/pipelines/compare_qwen_contexts.py

Comparador controlado Qwen 15 dias versus expansivo. Exige mesmos alvos, fontes, modelo e configuração exceto qwen_history_days. Produz métricas, melhorias e detalhamento por lead.

SHA-256: d4b47c183d1b8dd1b0ef8dd83add939a1a09093bb1c501181b479eadbecc7247

Funções, contratos e pontos de conexão

```
L10: relative_reduction(short, long)
```

short: erro de 15 dias; long: erro expansivo. Retorna 100*(short-long)/short ou None se baseline zero. Positivo é melhoria, negativo piora.

```
L15: load_completed_run(root, dataset, label)
```

root: raiz de comparação Qwen; dataset: cidade; label: 15_days ou expanding. Retorna (frame ordenado, run) após validar completude e história.

```
L36: compare_contexts(output)
```

output: pasta que contém os dois braços. Sem retorno útil. Exige comparação controlada e grava quatro CSVs.

Chamadas do projeto: deterministic_metrics → src/evaluation/metrics.py:5; load_completed_run → src/pipelines/compare_qwen_contexts.py:15; relative_reduction → src/pipelines/compare_qwen_contexts.py:10

```
L83: main()
```

Sem parâmetros explícitos. Lê argumentos do processo via argparse ou executa entry point do módulo. O corpo de cada arquivo determina qual CLI será iniciado.

Chamadas do projeto: compare_contexts → src/pipelines/compare_qwen_contexts.py:36

APIs externas usadas neste arquivo

ValueError, add_argument, all, any, append, apply, argparse.ArgumentParser, dict, drop_duplicates, duplicated, float, groupby, int, json.loads, len, list, nunique, parse_args, pd.DataFrame, pd.read_csv, pd.testing.assert_frame_equal, pop, print, range, read_text, reset_index, sort_values, sorted, sum, to_csv, to_string, tolist, values. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Compare 15 days versus expanding history for the SAME Qwen checkpoint/runtime."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import argparse
```

Importa argparse. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0003  import json
```

Importa json. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0004  from pathlib import Path
```

Importa Path de pathlib. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  import pandas as pd
```

Importa pandas como pd. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0006  from src.evaluation.metrics import deterministic_metrics
```

Importa deterministic_metrics de src.evaluation.metrics. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0007  from src.pipelines.forecasting_pipeline import ROOT
```

Importa ROOT de src.pipelines.forecasting_pipeline. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0010  def relative_reduction(short, long):
```

Define relative_reduction(short, long). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0011      """Positive is an improvement; do not invent a percentage for zero baseline error."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0012      return 100 * (short-long) / short if short else None
```

Encerra esta função e devolve 100 * (short - long) / short if short else None ao chamador; não imprime o resultado por si só.

```
0015  def load_completed_run(root, dataset, label):
```

Define load_completed_run(root, dataset, label). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0016      folder = root / label / dataset / 'qwen'
```

Atribui a folder o resultado de root / label / dataset / 'qwen'. A expressão à direita é avaliada antes da atribuição.

```
0017      status = json.loads((folder/'status.json').read_text())
```

Atribui a status o resultado de json.loads((folder / 'status.json').read_text()). A expressão à direita é avaliada antes da atribuição.

json.loads — Decodifica uma string JSON em objetos Python. Argumentos usados: posicional 1: (folder / 'status.json').read_text().

(folder / 'status.json').read_text — Lê todo o arquivo como string. Argumentos usados: nenhum argumento explícito.

```
0018      run = json.loads((folder/'run.json').read_text())
```

Atribui a run o resultado de json.loads((folder / 'run.json').read_text()). A expressão à direita é avaliada antes da atribuição.

json.loads — Decodifica uma string JSON em objetos Python. Argumentos usados: posicional 1: (folder / 'run.json').read_text().

(folder / 'run.json').read_text — Lê todo o arquivo como string. Argumentos usados: nenhum argumento explícito.

```
0019      if run['limit'] or status['failures'] or status['successful_windows'] !=
    status['expected_windows']:
```

Avalia a condição run['limit'] or status['failures'] or status['successful_windows'] != status['expected_windows']. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0020          raise ValueError(f'Incomplete run: {dataset}/{label}')
```

Interrompe o caminho atual lançando ValueError(f'Incomplete run: {dataset}/{label}'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: f'Incomplete run: {dataset}/{label}'.

```
0021      frame =
    pd.read_csv(folder/'predictions.csv').sort_values(['origin','lead']).reset_index(drop=True)
```

Atribui a frame o resultado de pd.read_csv(folder / 'predictions.csv').sort_values(['origin', 'lead']).reset_index(drop=True). A expressão à direita é avaliada antes da atribuição.

pd.read_csv(folder / 'predictions.csv').sort_values(['origin', 'lead']).reset_index — Transforma o índice atual em coluna ou o descarta e cria índice sequencial. Argumentos usados: drop: True.

pd.read_csv(folder / 'predictions.csv').sort_values — Ordena linhas pelos valores de colunas. Argumentos usados: posicional 1: ['origin', 'lead'].

pd.read_csv — Lê CSV em um DataFrame, inferindo tipos; datas só são convertidas posteriormente no validador. Argumentos usados: posicional 1: folder / 'predictions.csv'.

```
0022      if len(frame) != status['expected_windows']*7 or
    frame.duplicated(['origin','lead']).any():
```

Avalia a condição len(frame) != status['expected_windows'] * 7 or frame.duplicated(['origin', 'lead']).any(). O bloco indentado seguinte só é executado quando ela é verdadeira.

frame.duplicated(['origin', 'lead']).any — Reduz a existência de pelo menos um True. Argumentos usados: nenhum argumento explícito.

len — Conta elementos. Argumentos usados: posicional 1: frame.

frame.duplicated — Marca repetições. Argumentos usados: posicional 1: ['origin', 'lead'].

```
0023          raise ValueError('Missing or duplicated forecasts')
```

Interrompe o caminho atual lançando ValueError('Missing or duplicated forecasts'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Missing or duplicated forecasts'.

```
0024      if not frame.groupby('origin').lead.apply(lambda x: sorted(x)==list(range(1,8))).all():
```

Avalia a condição not frame.groupby('origin').lead.apply(lambda x: sorted(x) == list(range(1, 8))).all(). O bloco indentado seguinte só é executado quando ela é verdadeira.

frame.groupby('origin').lead.apply(lambda x: sorted(x) == list(range(1, 8))).all — Exige que todos os elementos sejam True. Argumentos usados: nenhum argumento explícito.

frame.groupby('origin').lead.apply — Aplica uma função por grupo/objeto. Argumentos usados: posicional 1: lambda x: sorted(x) == list(range(1, 8)).

frame.groupby — Cria grupos sem calcular métricas por si só. Argumentos usados: posicional 1: 'origin'.

sorted — Devolve uma lista ordenada. Argumentos usados: posicional 1: x.

list — Materializa um iterável em lista. Argumentos usados: posicional 1: range(1, 8).

range — Gera inteiros com limite final exclusivo. Argumentos usados: posicional 1: 1; posicional 2: 8.

```
0025          raise ValueError('Every forecast must contain days 1 through 7')
```

Interrompe o caminho atual lançando ValueError('Every forecast must contain days 1 through 7'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Every forecast must contain days 1 through 7'.

```
0026      origins = frame.drop_duplicates('origin')
```

Atribui a origins o resultado de frame.drop_duplicates('origin'). A expressão à direita é avaliada antes da atribuição.

frame.drop_duplicates — Retém uma linha por chave repetida. Argumentos usados: posicional 1: 'origin'.

```
0027      initial = int(frame.initial_context.iloc[0])
```

Validação especializada nos nomes 15_days/expanding; uma terceira configuração de sete dias exigirá generalização explícita, não só mudar o nome da pasta.

int — Converte para inteiro, truncando parte fracionária numérica. Argumentos usados: posicional 1: frame.initial_context.iloc[0].

```
0028      expected = [15]*len(origins) if label=='15_days' else
    list(range(initial,initial+7*len(origins),7))
```

Atribui a expected o resultado de [15] * len(origins) if label == '15_days' else list(range(initial, initial + 7 * len(origins), 7)). A expressão à direita é avaliada antes da atribuição.

list — Materializa um iterável em lista. Argumentos usados: posicional 1: range(initial, initial + 7 * len(origins), 7).

len — Conta elementos. Argumentos usados: posicional 1: origins.

range — Gera inteiros com limite final exclusivo. Argumentos usados: posicional 1: initial; posicional 2: initial + 7 * len(origins); posicional 3: 7.

len — Conta elementos. Argumentos usados: posicional 1: origins.

```
0029      if origins.context.tolist() != expected:
```

Avalia a condição origins.context.tolist() != expected. O bloco indentado seguinte só é executado quando ela é verdadeira.

origins.context.tolist — Converte array/índice/Series em lista Python. Argumentos usados: nenhum argumento explícito.

```
0030          raise ValueError('Unexpected historical context lengths')
```

Interrompe o caminho atual lançando ValueError('Unexpected historical context lengths'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Unexpected historical context lengths'.

```
0031      if label=='expanding' and origins.history_start.nunique()!=1:
```

Avalia a condição label == 'expanding' and origins.history_start.nunique() != 1. O bloco indentado seguinte só é executado quando ela é verdadeira.

origins.history_start.nunique — Conta valores distintos. Argumentos usados: nenhum argumento explícito.

```
0032          raise ValueError('Expanding history must start on the first date')
```

Interrompe o caminho atual lançando ValueError('Expanding history must start on the first date'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Expanding history must start on the first date'.

```
0033      return frame, run
```

Encerra esta função e devolve (frame, run) ao chamador; não imprime o resultado por si só.

```
0036  def compare_contexts(output):
```

Define compare_contexts(output). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0037      metrics, changes, lead_metrics, lead_changes = [], [], [], []
```

Atribui a (metrics, changes, lead_metrics, lead_changes) o resultado de ([], [], [], []). A expressão à direita é avaliada antes da atribuição.

```
0038      for dataset in ('joinville','guaratuba'):
```

Percorre ('joinville', 'guaratuba'); em cada iteração associa o item a dataset. O corpo indentado aplica a operação a esse item/grupo.

```
0039          frames, runs, scores = {}, {}, {}
```

Atribui a (frames, runs, scores) o resultado de ({}, {}, {}). A expressão à direita é avaliada antes da atribuição.

```
0040          for label in ('15_days','expanding'):
```

Percorre ('15_days', 'expanding'); em cada iteração associa o item a label. O corpo indentado aplica a operação a esse item/grupo.

```
0041              frame, run = load_completed_run(output,dataset,label)
```

Atribui a (frame, run) o resultado de load_completed_run(output, dataset, label). A expressão à direita é avaliada antes da atribuição.

load_completed_run — root: raiz de comparação Qwen; dataset: cidade; label: 15_days ou expanding. Retorna (frame ordenado, run) após validar completude e história. Argumentos usados: posicional 1: output; posicional 2: dataset; posicional 3: label.

```
0042              frames[label], runs[label] = frame, run
```

Atribui a (frames[label], runs[label]) o resultado de (frame, run). A expressão à direita é avaliada antes da atribuição.

```
0043              score = deterministic_metrics(frame.y_true,frame.y_pred)
```

Atribui a score o resultado de deterministic_metrics(frame.y_true, frame.y_pred). A expressão à direita é avaliada antes da atribuição.

deterministic_metrics — ytrue/yhat: vetores de mesmo formato, não vazios e finitos. Retorna MAE, RMSE, MAPE%, sMAPE%, R^2 e contagem de zeros excluídos do MAPE. Argumentos usados: posicional 1: frame.y_true; posicional 2: frame.y_pred.

```
0044              scores[label] = score
```

Atribui a scores[label] o resultado de score. A expressão à direita é avaliada antes da atribuição.

```
0045
    metrics.append(dict(dataset=dataset,context_policy=label,model=run['model']['model'],
```

Executa metrics.append(dict(dataset=dataset, context_policy=label, model=run['model']['model'], context_first=int(frame.context.iloc[0]), context_last=int(frame.context.iloc[-1]), num_ctx=run['config']['qwen']['num_ctx'], n_windows=frame.origin.nunique(), n_predictions=len(frame), **score, inference_seconds=float(frame.drop_duplicates('origin').elapsed_seconds_window.sum()))). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

metrics.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: dict(dataset=dataset, context_policy=label, model=run['model']['model'], context_first=int(frame.context.iloc[0]), context_last=int(frame.context.iloc[-1]), num_ctx=run['config']['qwen']['num_ctx'], n_windows=frame.origin.nunique(), n_predictions=len(frame), **score, inference_seconds=float(frame.drop_duplicates('origin').elapsed_seconds_window.sum())).

dict — Cria dicionário. Argumentos usados: dataset: dataset; context_policy: label; model: run['model']['model']; context_first: int(frame.context.iloc[0]); context_last: int(frame.context.iloc[-1]); num_ctx: run['config']['qwen']['num_ctx']; n_windows: frame.origin.nunique(); n_predictions: len(frame); **desempacotamento: score; inference_seconds: float(frame.drop_duplicates('origin').elapsed_seconds_window.sum()).

```
0046
    context_first=int(frame.context.iloc[0]),context_last=int(frame.context.iloc[-1]),
```

Continua os argumentos nomeados: context_first = int(frame.context.iloc[0]); context_last = int(frame.context.iloc[-1]). Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

int — Converte para inteiro, truncando parte fracionária numérica. Argumentos usados: posicional 1: frame.context.iloc[0].

int — Converte para inteiro, truncando parte fracionária numérica. Argumentos usados: posicional 1: frame.context.iloc[-1].

```
0047
    num_ctx=run['config']['qwen']['num_ctx'],n_windows=frame.origin.nunique(),n_predictions=len(fra
    me),
```

Continua os argumentos nomeados: num_ctx = run['config']['qwen']['num_ctx']; n_windows = frame.origin.nunique(); n_predictions = len(frame). Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

frame.origin.nunique — Conta valores distintos. Argumentos usados: nenhum argumento explícito.

len — Conta elementos. Argumentos usados: posicional 1: frame.

```
0048
    **score,inference_seconds=float(frame.drop_duplicates('origin').elapsed_seconds_window.sum())))
```

Continua os argumentos nomeados: ** = score; inference_seconds = float(frame.drop_duplicates('origin').elapsed_seconds_window.sum()). Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

float — Converte escalar para float Python. Argumentos usados: posicional 1: frame.drop_duplicates('origin').elapsed_seconds_window.sum().

frame.drop_duplicates('origin').elapsed_seconds_window.sum — Soma elementos de iterável. Argumentos usados: nenhum argumento explícito.

frame.drop_duplicates — Retém uma linha por chave repetida. Argumentos usados: posicional 1: 'origin'.

```
0049            for lead,subset in frame.groupby('lead'):
```

Percorre frame.groupby('lead'); em cada iteração associa o item a (lead, subset). O corpo indentado aplica a operação a esse item/grupo.

frame.groupby — Cria grupos sem calcular métricas por si só. Argumentos usados: posicional 1: 'lead'.

```
0050                lead_metrics.append(dict(dataset=dataset,context_policy=label,lead=int(lead),
```

Executa lead_metrics.append(dict(dataset=dataset, context_policy=label, lead=int(lead), **deterministic_metrics(subset.y_true, subset.y_pred))). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

lead_metrics.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: dict(dataset=dataset, context_policy=label, lead=int(lead), **deterministic_metrics(subset.y_true, subset.y_pred)).

dict — Cria dicionário. Argumentos usados: dataset: dataset; context_policy: label; lead: int(lead); **desempacotamento: deterministic_metrics(subset.y_true, subset.y_pred).

int — Converte para inteiro, truncando parte fracionária numérica. Argumentos usados: posicional 1: lead.

```
0051
    **deterministic_metrics(subset.y_true,subset.y_pred)))
```

Continua os argumentos nomeados: ** = deterministic_metrics(subset.y_true, subset.y_pred). Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

deterministic_metrics — ytrue/yhat: vetores de mesmo formato, não vazios e finitos. Retorna MAE, RMSE, MAPE%, sMAPE%, R^2 e contagem de zeros excluídos do MAPE. Argumentos usados: posicional 1: subset.y_true; posicional 2: subset.y_pred.

```
0052          columns=['origin','timestamp','lead','y_true','initial_context','horizon']
```

Atribui a columns o resultado de ['origin', 'timestamp', 'lead', 'y_true', 'initial_context', 'horizon']. A expressão à direita é avaliada antes da atribuição.

```
0053
    pd.testing.assert_frame_equal(frames['15_days'][columns],frames['expanding'][columns])
```

Executa pd.testing.assert_frame_equal(frames['15_days'][columns], frames['expanding'][columns]). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

pd.testing.assert_frame_equal — Verifica igualdade detalhada entre dois DataFrames e lança AssertionError se diferirem. Argumentos usados: posicional 1: frames['15_days'][columns]; posicional 2: frames['expanding'][columns].

```
0054          for field in
    ['dataset_sha256','source_sha256','model','covariates','first_test_origin']:
```

Percorre ['dataset_sha256', 'source_sha256', 'model', 'covariates', 'first_test_origin']; em cada iteração associa o item a field. O corpo indentado aplica a operação a esse item/grupo.

```
0055              if runs['15_days'][field] != runs['expanding'][field]:
```

Avalia a condição runs['15_days'][field] != runs['expanding'][field]. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0056                  raise ValueError(f'Comparison changes {field} in addition to history')
```

Interrompe o caminho atual lançando ValueError(f'Comparison changes {field} in addition to history'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: f'Comparison changes {field} in addition to history'.

```
0057          configs=[]
```

Atribui a configs o resultado de []. A expressão à direita é avaliada antes da atribuição.

```
0058          for run in runs.values():
```

Percorre runs.values(); em cada iteração associa o item a run. O corpo indentado aplica a operação a esse item/grupo.

runs.values — Percorre valores do dicionário. Argumentos usados: nenhum argumento explícito.

```
0059              c=dict(run['config']);c.pop('qwen_history_days');configs.append(c)
```

A cópia superficial de config permite remover qwen_history_days para comparar todo o resto; não altera o dict original de run. pop exige que a chave exista.

dict — Cria dicionário. Argumentos usados: posicional 1: run['config'].

c.pop — Remove e devolve valor de chave. Argumentos usados: posicional 1: 'qwen_history_days'.

configs.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: c.

```
0060          if configs[0]!=configs[1]:
```

Avalia a condição configs[0] != configs[1]. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0061             raise ValueError('Only qwen_history_days may differ between configurations')
```

Interrompe o caminho atual lançando ValueError('Only qwen_history_days may differ between configurations'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Only qwen_history_days may differ between configurations'.

```
0062         short,long=scores['15_days'],scores['expanding']
```

Atribui a (short, long) o resultado de (scores['15_days'], scores['expanding']). A expressão à direita é avaliada antes da atribuição.

```
0063         change={'dataset':dataset,'n_predictions':len(frames['15_days']),
```

Atribui a change o resultado de {'dataset': dataset, 'n_predictions': len(frames['15_days']), 'model': runs['15_days']['model']['model']}. A expressão à direita é avaliada antes da atribuição.

len — Conta elementos. Argumentos usados: posicional 1: frames['15_days'].

```
0064                 'model':runs['15_days']['model']['model']}
```

Define 'model': identidade do checkpoint/adaptador; recebe runs['15_days']['model']['model']. Os valores são avaliados nesta janela/iteração.

```
0065         for metric in ('rmse','mae','mape_percent','smape_percent'):
```

Percorre ('rmse', 'mae', 'mape_percent', 'smape_percent'); em cada iteração associa o item a metric. O corpo indentado aplica a operação a esse item/grupo.

```
0066             change[f'{metric}_15_days']=short[metric]
```

Atribui a change[f'{metric}_15_days'] o resultado de short[metric]. A expressão à direita é avaliada antes da atribuição.

```
0067             change[f'{metric}_expanding']=long[metric]
```

Atribui a change[f'{metric}_expanding'] o resultado de long[metric]. A expressão à direita é avaliada antes da atribuição.

```
0068
    change[f'{metric}_reduction_percent']=relative_reduction(short[metric],long[metric])
```

Atribui a change[f'{metric}_reduction_percent'] o resultado de relative_reduction(short[metric], long[metric]). A expressão à direita é avaliada antes da atribuição.

relative_reduction — short: erro de 15 dias; long: erro expansivo. Retorna 100*(short-long)/short ou None se baseline zero. Positivo é melhoria, negativo piora. Argumentos usados: posicional 1: short[metric]; posicional 2: long[metric].

```
0069         change['r2_15_days']=short['r2'];change['r2_expanding']=long['r2']
```

Atribui a change['r2_15_days'] o resultado de short['r2']. A expressão à direita é avaliada antes da atribuição. Atribui a change['r2_expanding'] o resultado de long['r2']. A expressão à direita é avaliada antes da atribuição.

```
0070         change['r2_difference']=long['r2']-short['r2']
```

Atribui a change['r2_difference'] o resultado de long['r2'] - short['r2']. A expressão à direita é avaliada antes da atribuição.

```
0071         changes.append(change)
```

Executa changes.append(change). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

changes.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: change.

```
0072         for lead in range(1,8):
```

Percorre range(1, 8); em cada iteração associa o item a lead. O corpo indentado aplica a operação a esse item/grupo.

range — Gera inteiros com limite final exclusivo. Argumentos usados: posicional 1: 1; posicional 2: 8.

```
0073             scores_by_label={r['context_policy']:r for r in lead_metrics if
    r['dataset']==dataset and r['lead']==lead}
```

Atribui a scores_by_label o resultado de {r['context_policy']: r for r in lead_metrics if r['dataset'] == dataset and r['lead'] == lead}. A expressão à direita é avaliada antes da atribuição.

```
0074             short,long=scores_by_label['15_days'],scores_by_label['expanding']
```

Atribui a (short, long) o resultado de (scores_by_label['15_days'], scores_by_label['expanding']). A expressão à direita é avaliada antes da atribuição.

```
0075             lead_changes.append({'dataset':dataset,'lead':lead,'rmse_15_days':short['rmse'],
```

Executa lead_changes.append({'dataset': dataset, 'lead': lead, 'rmse_15_days': short['rmse'], 'rmse_expanding': long['rmse'], 'rmse_reduction_percent': relative_reduction(short['rmse'], long['rmse'])}). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

lead_changes.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: {'dataset': dataset, 'lead': lead, 'rmse_15_days': short['rmse'], 'rmse_expanding': long['rmse'], 'rmse_reduction_percent': relative_reduction(short['rmse'], long['rmse'])}.

```
0076                 'rmse_expanding':long['rmse'],'rmse_reduction_percent':relative_reduction(sho
    rt['rmse'],long['rmse'])})
```

Define 'rmse_expanding': campo do registro/coleção em construção; recebe long['rmse']; 'rmse_reduction_percent': campo do registro/coleção em construção; recebe relative_reduction(short['rmse'], long['rmse']). Os valores são avaliados nesta janela/iteração.

relative_reduction — short: erro de 15 dias; long: erro expansivo. Retorna 100*(short-long)/short ou None se baseline zero. Positivo é melhoria, negativo piora. Argumentos usados: posicional 1: short['rmse']; posicional 2: long['rmse'].

```
0077     for filename,rows in [('metrics.csv',metrics),('improvement.csv',changes),
```

Percorre [('metrics.csv', metrics), ('improvement.csv', changes), ('metrics_by_horizon.csv', lead_metrics), ('improvement_by_horizon.csv', lead_changes)]; em cada iteração associa o item a (filename, rows). O corpo indentado aplica a operação a esse item/grupo.

```
0078
    ('metrics_by_horizon.csv',lead_metrics),('improvement_by_horizon.csv',lead_changes)]:
```

Continuação da instrução na linha 77: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0079         pd.DataFrame(rows).to_csv(output/filename,index=False)
```

Executa pd.DataFrame(rows).to_csv(output / filename, index=False). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

pd.DataFrame(rows).to_csv — Serializa DataFrame como CSV. Argumentos usados: posicional 1: output / filename; index: False.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: rows.

```
0080     print(pd.DataFrame(changes)[['dataset','rmse_15_days','rmse_expanding','rmse_reduction_pe
    rcent']].to_string(index=False))
```

Executa print(pd.DataFrame(changes)[['dataset', 'rmse_15_days', 'rmse_expanding', 'rmse_reduction_percent']].to_string(index=False)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

print — Escreve representação textual na saída padrão. Argumentos usados: posicional 1: pd.DataFrame(changes)[['dataset', 'rmse_15_days', 'rmse_expanding', 'rmse_reduction_percent']].to_string(index=False).

pd.DataFrame(changes)[['dataset', 'rmse_15_days', 'rmse_expanding', 'rmse_reduction_percent']].to_string — Formata tabela como texto para terminal/log. Argumentos usados: index: False.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: changes.

```
0083  def main():
```

Define main(). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0084      parser=argparse.ArgumentParser(description=__doc__)
```

Atribui a parser o resultado de argparse.ArgumentParser(description=__doc__). A expressão à direita é avaliada antes da atribuição.

argparse.ArgumentParser — Cria o parser da interface de linha de comando. Argumentos usados: description: __doc__.

```
0085      parser.add_argument('--
    output',type=Path,default=ROOT/'experiments/qwen_context_comparison')
```

Executa parser.add_argument('--output', type=Path, default=ROOT / 'experiments/qwen_context_comparison'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

parser.add_argument — Registra uma opção do CLI. Argumentos usados: posicional 1: '--output'; type: Path; default: ROOT / 'experiments/qwen_context_comparison'.

```
0086      compare_contexts(parser.parse_args().output)
```

Executa compare_contexts(parser.parse_args().output). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

compare_contexts — output: pasta que contém os dois braços. Sem retorno útil. Exige comparação controlada e grava quatro CSVs. Argumentos usados: posicional 1: parser.parse_args().output.

parser.parse_args — Interpreta argumentos do processo e retorna Namespace. Argumentos usados: nenhum argumento explícito.

```
0089  if __name__=='__main__':
```

Avalia a condição __name__ == '__main__'. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0090      main()
```

Executa main(). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

main — Sem parâmetros explícitos. Lê argumentos do processo via argparse ou executa entry point do módulo. O corpo de cada arquivo determina qual CLI será iniciado. Argumentos usados: nenhum argumento explícito.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

33. src/training/train.py

Ponto auxiliar para uma extensão de treinamento: oferece slices cronológicos treino/validação. Não é chamado pela inferência atual nem treina pesos.

SHA-256: bdff53809e5067f833bbec7698ea9b0be09af475a4f6a893f5465c905e04df30

Funções, contratos e pontos de conexão

```
L5: training_partition(first_test_origin: int, validation_fraction: float = 0.2)
```

first_test_origin: tamanho de prefixo de treino; validation_fraction=0.2: proporção final de validação. Retorna os slices de validation_split; não faz fit.

Chamadas do projeto: validation_split → src/evaluation/cross_validation.py:32

Código comentado, linha a linha

```
0001  """Explicit experiment policy: pretrained inference only, with no fit on test data."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  from src.evaluation.cross_validation import validation_split
```

Importa validation_split de src.evaluation.cross_validation. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  def training_partition(first_test_origin: int, validation_fraction: float = 0.2):
```

Define training_partition(first_test_origin: int, validation_fraction: float = 0.2). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0006      """Provide chronological train/validation slices for a future training extension.
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0008      This benchmark does not optimize weights or hyperparameters. Callers must
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0009      slice the dataset at first_test_origin before fitting any future estimator.
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0010      """
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0011      return validation_split(first_test_origin, validation_fraction)
```

Encerra esta função e devolve validation_split(first_test_origin, validation_fraction) ao chamador; não imprime o resultado por si só.

validation_split — training_size: comprimento do prefixo anterior ao teste; fraction=0.2: parte final reservada à validação. Retorna dois slices sem embaralhar. Argumentos usados: posicional 1: first_test_origin; posicional 2: validation_fraction.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

34. tests/test_forecasting.py

Testes de regressão. São executados por unittest discover, não durante a inferência.

SHA-256: 9f6b38ebb1f83bdcb0a0f73e8b3bc37e74f095a2a713a6532fcff659a0e24402

Funções, contratos e pontos de conexão

`L20: test_real_datasets_shared_protocol(self)`

Caso de teste: real datasets shared protocol. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: prepare_dataset → src/pipelines/data_pipeline.py:8

`L32: test_future_target_mutation_does_not_change_prompt(self)`

Caso de teste: future target mutation does not change prompt. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: build_prompt → src/data/preprocessing/generate_prompts.py:52

`L41: test_demonstrations_remain_before_frozen_test(self)`

Caso de teste: demonstrations remain before frozen test. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: select_demonstrations → src/data/preprocessing/generate_prompts.py:16

`L48: test_missing_day_is_filled_from_past_only(self)`

Caso de teste: missing day is filled from past only. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: daily_grid → src/data/preprocessing/resampling.py:5; fill_past_only → src/data/preprocessing/missing_values.py:5

`L54: test_imputed_test_days_not_scored(self)`

Caso de teste: imputed test days not scored. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: build_windows → src/evaluation/cross_validation.py:20

`L61: test_covariates_and_history_grow_without_future_targets(self)`

Caso de teste: covariates and history grow without future targets. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: build_prompt → src/data/preprocessing/generate_prompts.py:52; covariate_columns → src/data/features/covariates.py:6; model_inputs → src/data/features/covariates.py:25; prepare_dataset → src/pipelines/data_pipeline.py:8; timesfm_covariates → src/data/features/covariates.py:36; validate_covariates → src/data/features/covariates.py:13

`L88: test_future_target_column_is_rejected(self)`

Caso de teste: future target column is rejected. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: build_prompt → src/data/preprocessing/generate_prompts.py:52

`L94: test_chronos_receives_entire_growing_history_and_covariates(self)`

Caso de teste: chronos receives entire growing history and covariates. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: adapter.predict → src/models/promptcast.py:71; adapter.predict → src/models/chronos.py:18; adapter.predict → src/models/timesfm.py:20; covariate_columns → src/data/features/covariates.py:6; model_inputs → src/data/features/covariates.py:25; prepare_dataset → src/pipelines/data_pipeline.py:8

`L113: test_invalid_model_responses(self)`

Caso de teste: invalid model responses. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: parse_prediction → src/models/promptcast.py:33

```
L122: test_retry_and_raw_response_audit(self)
```

Caso de teste: retry and raw response audit. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: OllamaConfig → src/models/promptcast.py:11; model.predict → src/models/promptcast.py:71; model.predict → src/models/chronos.py:18; model.predict → src/models/timesfm.py:20

```
L132: test_context_budget_rejects_silent_truncation(self)
```

Caso de teste: context budget rejects silent truncation. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: OllamaConfig → src/models/promptcast.py:11; model.predict → src/models/promptcast.py:71; model.predict → src/models/chronos.py:18; model.predict → src/models/timesfm.py:20

```
L138: test_metric_units(self)
```

Caso de teste: metric units. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: deterministic_metrics → src/evaluation/metrics.py:5

```
L148: test_validation_uses_training_prefix(self)
```

Caso de teste: validation uses training prefix. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: validation_split → src/evaluation/cross_validation.py:32

```
L152: test_duplicate_dates_rejected(self)
```

Caso de teste: duplicate dates rejected. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: validate_series → src/data/validation/schema.py:6

APIs externas usadas neste arquivo

Mock, OllamaConfig, all, assertAlmostEqual, assertEqual, assertFalse, assertGreaterEqual, assertIn, assertLess, assertNotIn, assertRaises, assertRaisesRegex, assertTrue, copy, enumerate, float, issubset, json.loads, len, list, max, min, np.arange, object.__new__, patch, pd.DataFrame, pd.date_range, pd.testing.assert_frame_equal, pd.to_datetime, predict, read_text, set, subTest, unittest.main, values, zip. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Leakage, evaluation and malformed-output regression tests; no model downloads."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import json
```

Importa json. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0003  import unittest
```

Importa unittest. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0004  from unittest.mock import patch
```

Importa patch de unittest.mock. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  import numpy as np
```

Importa numpy como np. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0006  import pandas as pd
```

Importa pandas como pd. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0007  from src.data.preprocessing.generate_prompts import build_prompt, select_demonstrations
```

Importa build_prompt, select_demonstrations de src.data.preprocessing.generate_prompts. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0008  from src.data.preprocessing.missing_values import fill_past_only
```

Importa fill_past_only de src.data.preprocessing.missing_values. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0009  from src.data.preprocessing.resampling import daily_grid
```

Importa daily_grid de src.data.preprocessing.resampling. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0010  from src.data.validation.schema import validate_series
```

Importa validate_series de src.data.validation.schema. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0011  from src.evaluation.cross_validation import build_windows, validation_split
```

Importa build_windows, validation_split de src.evaluation.cross_validation. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0012  from src.evaluation.metrics import deterministic_metrics
```

Importa deterministic_metrics de src.evaluation.metrics. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0013  from src.models.promptcast import OllamaConfig, QwenForecaster, parse_prediction
```

Importa OllamaConfig, QwenForecaster, parse_prediction de src.models.promptcast. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0014  from src.pipelines.data_pipeline import prepare_dataset
```

Importa prepare_dataset de src.pipelines.data_pipeline. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0015  from src.pipelines.forecasting_pipeline import ROOT
```

Importa ROOT de src.pipelines.forecasting_pipeline. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0016  from src.data.features.covariates import model_inputs, covariate_columns, timesfm_covariates,
     validate_covariates
```

Importa model_inputs, covariate_columns, timesfm_covariates, validate_covariates de src.data.features.covariates. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0019  class ForecastTests(unittest.TestCase):
```

Declara a classe ForecastTests(unittest.TestCase). Seus métodos compartilham estado por self; a declaração não cria uma previsão.

```
0020      def test_real_datasets_shared_protocol(self):
```

Define test_real_datasets_shared_protocol(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0021          for name, context in [('joinville_processed_1day',365),
    ('guaratuba_processed_1daysin',730)]:
```

Percorre [('joinville_processed_1day', 365), ('guaratuba_processed_1daysin', 730)]; em cada iteração associa o item a (name, context). O corpo indentado aplica a operação a esse item/grupo.

```
0022             df, windows = prepare_dataset(ROOT / f'data/processed/{name}.csv', context)
```

Atribui a (df, windows) o resultado de prepare_dataset(ROOT / f'data/processed/{name}.csv', context). A expressão à direita é avaliada antes da atribuição.

prepare_dataset — path: CSV; context: primeira origem; horizon=7 e step=7 nomeados. Retorna df e windows depois da preparação. Argumentos usados: posicional 1: ROOT / f'data/processed/{name}.csv'; posicional 2: context.

```
0023             for window in windows:
```

Percorre windows; em cada iteração associa o item a window. O corpo indentado aplica a operação a esse item/grupo.

```
0024                 self.assertEqual(window.context_start, 0)
```

Executa self.assertEqual(window.context_start, 0). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: window.context_start; posicional 2: 0.

```
0025                 self.assertGreaterEqual(window.origin, context)
```

Executa self.assertGreaterEqual(window.origin, context). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertGreaterEqual — Exige first >= second. Argumentos usados: posicional 1: window.origin; posicional 2: context.

```
0026                 self.assertEqual(window.history.stop, window.target.start)
```

Executa self.assertEqual(window.history.stop, window.target.start). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: window.history.stop; posicional 2: window.target.start.

```
0027                 self.assertEqual(window.horizon, 7)
```

Executa self.assertEqual(window.horizon, 7). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: window.horizon; posicional 2: 7.

```
0028                 self.assertTrue(df.iloc[window.target].observed.all())
```

Executa self.assertTrue(df.iloc[window.target].observed.all()). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertTrue — Asserção de condição verdadeira. Argumentos usados: posicional 1: df.iloc[window.target].observed.all().

df.iloc[window.target].observed.all — Exige que todos os elementos sejam True. Argumentos usados: nenhum argumento explícito.

```
0029                 self.assertLess(df.iloc[window.history].timestamp.max(),
    df.iloc[window.target].timestamp.min())
```

Executa self.assertLess(df.iloc[window.history].timestamp.max(), df.iloc[window.target].timestamp.min()). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertLess — Exige first < second. Argumentos usados: posicional 1: df.iloc[window.history].timestamp.max(); posicional 2: df.iloc[window.target].timestamp.min().

df.iloc[window.history].timestamp.max — Retorna maior valor. Argumentos usados: nenhum argumento explícito.

df.iloc[window.target].timestamp.min — Retorna menor valor. Argumentos usados: nenhum argumento explícito.

```
0030             self.assertTrue(all(b.origin-a.origin == 7 for a,b in zip(windows,windows[1:])))
```

Executa self.assertTrue(all((b.origin - a.origin == 7 for a, b in zip(windows, windows[1:])))). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertTrue — Asserção de condição verdadeira. Argumentos usados: posicional 1: all((b.origin - a.origin == 7 for a, b in zip(windows, windows[1:]))).

all — Exige que todos os elementos sejam True. Argumentos usados: posicional 1: (b.origin - a.origin == 7 for a, b in zip(windows, windows[1:])).

zip — Emparelha iteráveis pela posição. Argumentos usados: posicional 1: windows; posicional 2: windows[1:].

```
0032      def test_future_target_mutation_does_not_change_prompt(self):
```

Define test_future_target_mutation_does_not_change_prompt(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0033          df = pd.DataFrame({'timestamp': pd.date_range('2020-01-01',periods=30),
```

Atribui a df o resultado de pd.DataFrame({'timestamp': pd.date_range('2020-01-01', periods=30), 'water_produced': np.arange(30, dtype=float)}). A expressão à direita é avaliada antes da atribuição.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: {'timestamp': pd.date_range('2020-01-01', periods=30), 'water_produced': np.arange(30, dtype=float)}.

pd.date_range — Produz uma sequência ordenada de timestamps. Argumentos usados: posicional 1: '2020-01-01'; periods: 30.

```
0034                              'water_produced': np.arange(30,dtype=float)})
```

Define 'water_produced': campo do registro/coleção em construção; recebe np.arange(30, dtype=float). Os valores são avaliados nesta janela/iteração.

np.arange — Gera progressão de valores com fim exclusivo. Argumentos usados: posicional 1: 30; dtype: float.

```
0035          before = build_prompt(df.iloc[:20],7)
```

Atribui a before o resultado de build_prompt(df.iloc[:20], 7). A expressão à direita é avaliada antes da atribuição.

build_prompt — history: DataFrame histórico; horizon: dias; demonstrations: lista ou None; future_covariates: tabela sem alvo ou None. Retorna prompt legado com lista de demanda e tabelas de covariáveis. Argumentos usados: posicional 1: df.iloc[:20]; posicional 2: 7.

```
0036          df.loc[20:, 'water_produced'] = 987654321
```

Atribui a df.loc[20:, 'water_produced'] o resultado de 987654321. A expressão à direita é avaliada antes da atribuição.

```
0037          self.assertEqual(before,build_prompt(df.iloc[:20],7))
```

Executa self.assertEqual(before, build_prompt(df.iloc[:20], 7)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: before; posicional 2: build_prompt(df.iloc[:20], 7).

build_prompt — history: DataFrame histórico; horizon: dias; demonstrations: lista ou None; future_covariates: tabela sem alvo ou None. Retorna prompt legado com lista de demanda e tabelas de covariáveis. Argumentos usados: posicional 1: df.iloc[:20]; posicional 2: 7.

```
0038          self.assertNotIn('987654321',before)
```

Executa self.assertNotIn('987654321', before). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertNotIn — Exige item ausente de container. Argumentos usados: posicional 1: '987654321'; posicional 2: before.

```
0039          self.assertIn('2020-01-21',before)
```

Executa self.assertIn('2020-01-21', before). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertIn — Exige item presente em container. Argumentos usados: posicional 1: '2020-01-21'; posicional 2: before.

```
0041      def test_demonstrations_remain_before_frozen_test(self):
```

Define test_demonstrations_remain_before_frozen_test(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0042          for strategy in ['few_random','few_similar']:
```

Percorre ['few_random', 'few_similar']; em cada iteração associa o item a strategy. O corpo indentado aplica a operação a esse item/grupo.

```
0043              examples = select_demonstrations(np.arange(80.), np.arange(20.), 7,
    strategy=strategy)
```

Atribui a examples o resultado de select_demonstrations(np.arange(80.0), np.arange(20.0), 7, strategy=strategy). A expressão à direita é avaliada antes da atribuição.

select_demonstrations — training_values: prefixo congelado anterior ao primeiro teste; history: demanda histórica corrente; horizon: tamanho de alvo dos exemplos; strategy: zero/few_random/few_similar; count=3: quantidade; seed=42: sorteio. Retorna lista de Demonstration. Argumentos usados: posicional 1: np.arange(80.0); posicional 2: np.arange(20.0); posicional 3: 7; strategy: strategy.

np.arange — Gera progressão de valores com fim exclusivo. Argumentos usados: posicional 1: 80.0.

np.arange — Gera progressão de valores com fim exclusivo. Argumentos usados: posicional 1: 20.0.

```
0044              self.assertEqual(len(examples),3)
```

Executa self.assertEqual(len(examples), 3). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: len(examples); posicional 2: 3.

len — Conta elementos. Argumentos usados: posicional 1: examples.

```
0045              self.assertTrue(all(e.target_end <= 80 for e in examples))
```

Executa self.assertTrue(all((e.target_end <= 80 for e in examples))). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertTrue — Asserção de condição verdadeira. Argumentos usados: posicional 1: all((e.target_end <= 80 for e in examples)).

all — Exige que todos os elementos sejam True. Argumentos usados: posicional 1: (e.target_end <= 80 for e in examples).

```
0046              self.assertTrue(all(max(e.target)<80 for e in examples))
```

Executa self.assertTrue(all((max(e.target) < 80 for e in examples))). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertTrue — Asserção de condição verdadeira. Argumentos usados: posicional 1: all((max(e.target) < 80 for e in examples)).

all — Exige que todos os elementos sejam True. Argumentos usados: posicional 1: (max(e.target) < 80 for e in examples).

max — Retorna maior valor. Argumentos usados: posicional 1: e.target.

```
0048      def test_missing_day_is_filled_from_past_only(self):
```

Define test_missing_day_is_filled_from_past_only(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0049          df = pd.DataFrame({'timestamp':pd.to_datetime(['2020-01-01','2020-01-03']),
    'water_produced':[10.,999.]})
```

Atribui a df o resultado de pd.DataFrame({'timestamp': pd.to_datetime(['2020-01-01', '2020-01-03']), 'water_produced': [10.0, 999.0]}). A expressão à direita é avaliada antes da atribuição.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: {'timestamp': pd.to_datetime(['2020-01-01', '2020-01-03']), 'water_produced': [10.0, 999.0]}.

pd.to_datetime — Converte valores em datas pandas. Argumentos usados: posicional 1: ['2020-01-01', '2020-01-03'].

```
0050          result = fill_past_only(daily_grid(df))
```

Atribui a result o resultado de fill_past_only(daily_grid(df)). A expressão à direita é avaliada antes da atribuição.

fill_past_only — df: grade com possíveis NaN. Retorno: cópia preenchida com passado. Não reconstrói atributos de calendário, limitação conhecida. Argumentos usados: posicional 1: daily_grid(df).

daily_grid — df: tabela com timestamp. Retorno: grade diária com observed. Dias ausentes são expostos, não removidos. Argumentos usados: posicional 1: df.

```
0051          self.assertEqual(result.water_produced.iloc[1],10.)
```

Executa self.assertEqual(result.water_produced.iloc[1], 10.0). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: result.water_produced.iloc[1]; posicional 2: 10.0.

```
0052          self.assertFalse(result.observed.iloc[1])
```

Executa self.assertFalse(result.observed.iloc[1]). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertFalse — Asserção de condição falsa. Argumentos usados: posicional 1: result.observed.iloc[1].

```
0054      def test_imputed_test_days_not_scored(self):
```

Define test_imputed_test_days_not_scored(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0055          df = pd.DataFrame({'timestamp':pd.date_range('2020-01-01',periods=30),
```

Atribui a df o resultado de pd.DataFrame({'timestamp': pd.date_range('2020-01-01', periods=30), 'water_produced': np.arange(30.0), 'observed': True}). A expressão à direita é avaliada antes da atribuição.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: {'timestamp': pd.date_range('2020-01-01', periods=30), 'water_produced': np.arange(30.0), 'observed': True}.

pd.date_range — Produz uma sequência ordenada de timestamps. Argumentos usados: posicional 1: '2020-01-01'; periods: 30.

```
0056                                'water_produced':np.arange(30.), 'observed':True})
```

Define 'water_produced': campo do registro/coleção em construção; recebe np.arange(30.0); 'observed': campo do registro/coleção em construção; recebe True. Os valores são avaliados nesta janela/iteração.

np.arange — Gera progressão de valores com fim exclusivo. Argumentos usados: posicional 1: 30.0.

```
0057          df.loc[12,'observed'] = False
```

Atribui a df.loc[12, 'observed'] o resultado de False. A expressão à direita é avaliada antes da atribuição.

```
0058          windows=build_windows(df,10,horizon=7,step=7)
```

Atribui a windows o resultado de build_windows(df, 10, horizon=7, step=7). A expressão à direita é avaliada antes da atribuição.

build_windows — df: tabela preparada; context: primeira origem; horizon=7: quantidade prevista; step=7: avanço entre origens. Retorna lista ForecastWindow, excluindo janelas com alvo imputado. Argumentos usados: posicional 1: df; posicional 2: 10; horizon: 7; step: 7.

```
0059          self.assertTrue(all(not(w.origin<=12<w.origin+7) for w in windows))
```

Executa self.assertTrue(all((not w.origin <= 12 < w.origin + 7 for w in windows))). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertTrue — Asserção de condição verdadeira. Argumentos usados: posicional 1: all((not w.origin <= 12 < w.origin + 7 for w in windows)).

all — Exige que todos os elementos sejam True. Argumentos usados: posicional 1: (not w.origin <= 12 < w.origin + 7 for w in windows).

```
0061      def test_covariates_and_history_grow_without_future_targets(self):
```

Define test_covariates_and_history_grow_without_future_targets(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0062          config = json.loads((ROOT / 'configs/benchmark.json').read_text())
```

Atribui a config o resultado de json.loads((ROOT / 'configs/benchmark.json').read_text()). A expressão à direita é avaliada antes da atribuição.

json.loads — Decodifica uma string JSON em objetos Python. Argumentos usados: posicional 1: (ROOT / 'configs/benchmark.json').read_text().

(ROOT / 'configs/benchmark.json').read_text — Lê todo o arquivo como string. Argumentos usados: nenhum argumento explícito.

```
0063          for dataset in config['datasets'].values():
```

Percorre config['datasets'].values(); em cada iteração associa o item a dataset. O corpo indentado aplica a operação a esse item/grupo.

config['datasets'].values — Percorre valores do dicionário. Argumentos usados: nenhum argumento explícito.

```
0064              df, windows = prepare_dataset(ROOT / dataset['path'], dataset['context'])
```

Atribui a (df, windows) o resultado de prepare_dataset(ROOT / dataset['path'], dataset['context']). A expressão à direita é avaliada antes da atribuição.

prepare_dataset — path: CSV; context: primeira origem; horizon=7 e step=7 nomeados. Retorna df e windows depois da preparação. Argumentos usados: posicional 1: ROOT / dataset['path']; posicional 2: dataset['context'].

```
0065            validate_covariates(df, dataset)
```

Executa validate_covariates(df, dataset). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

validate_covariates — df: tabela preparada; dataset: contrato de colunas. Retorna None; falha em ausentes/NaN/números não finitos. Argumentos usados: posicional 1: df; posicional 2: dataset.

```
0066            self.assertEqual(windows[0].origin, dataset['context'])
```

Executa self.assertEqual(windows[0].origin, dataset['context']). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: windows[0].origin; posicional 2: dataset['context'].

```
0067            columns = covariate_columns(dataset)
```

Atribui a columns o resultado de covariate_columns(dataset). A expressão à direita é avaliada antes da atribuição.

covariate_columns — dataset: bloco de configuração de uma cidade. Retorna nomes numéricos seguidos dos categóricos, após rejeitar duplicatas e nomes reservados. Argumentos usados: posicional 1: dataset.

```
0068            for offset, window in enumerate(windows[:3]):
```

Percorre enumerate(windows[:3]); em cada iteração associa o item a (offset, window). O corpo indentado aplica a operação a esse item/grupo.

enumerate — Percorre iterável com contador. Argumentos usados: posicional 1: windows[:3].

```
0069                history, future = model_inputs(df, window, dataset)
```

Atribui a (history, future) o resultado de model_inputs(df, window, dataset). A expressão à direita é avaliada antes da atribuição.

model_inputs — df: série completa; window: ForecastWindow; dataset: contrato; history_days: None para expansivo ou inteiro para últimos N dias. Retorna (history, future), excluindo alvo do futuro. Argumentos usados: posicional 1: df; posicional 2: window; posicional 3: dataset.

```
0070                self.assertEqual(len(history), dataset['context'] + offset*7)
```

Executa self.assertEqual(len(history), dataset['context'] + offset * 7). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: len(history); posicional 2: dataset['context'] + offset * 7.

len — Conta elementos. Argumentos usados: posicional 1: history.

```
0071                self.assertEqual(history.timestamp.iloc[0], df.timestamp.iloc[0])
```

Executa self.assertEqual(history.timestamp.iloc[0], df.timestamp.iloc[0]). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: history.timestamp.iloc[0]; posicional 2: df.timestamp.iloc[0].

```
0072                self.assertEqual(list(future.columns), ['timestamp'] + columns)
```

Executa self.assertEqual(list(future.columns), ['timestamp'] + columns). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: list(future.columns); posicional 2: ['timestamp'] + columns.

list — Materializa um iterável em lista. Argumentos usados: posicional 1: future.columns.

```
0073                self.assertNotIn('water_produced', future)
```

Executa self.assertNotIn('water_produced', future). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertNotIn — Exige item ausente de container. Argumentos usados: posicional 1: 'water_produced'; posicional 2: future.

```
0074                 numerical, categorical = timesfm_covariates(history, future, dataset)
```

Atribui a (numerical, categorical) o resultado de timesfm_covariates(history, future, dataset). A expressão à direita é avaliada antes da atribuição.

timesfm_covariates — history/future: tabelas separadas; dataset: listas numéricas/categóricas. Retorna dois dicts com listas de séries cobrindo passado e futuro, sem demanda futura. Argumentos usados: posicional 1: history; posicional 2: future; posicional 3: dataset.

```
0075                 self.assertEqual(set(numerical) | set(categorical), set(columns))
```

Executa self.assertEqual(set(numerical) | set(categorical), set(columns)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: set(numerical) | set(categorical); posicional 2: set(columns).

set — Cria conjunto de elementos únicos. Argumentos usados: posicional 1: columns.

set — Cria conjunto de elementos únicos. Argumentos usados: posicional 1: numerical.

set — Cria conjunto de elementos únicos. Argumentos usados: posicional 1: categorical.

```
0076                 for values in {**numerical, **categorical}.values():
```

Percorre {**numerical, **categorical}.values(); em cada iteração associa o item a values. O corpo indentado aplica a operação a esse item/grupo.

{**numerical, **categorical}.values — Percorre valores do dicionário. Argumentos usados: nenhum argumento explícito.

```
0077                     self.assertEqual(len(values[0]), len(history)+7)
```

Executa self.assertEqual(len(values[0]), len(history) + 7). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: len(values[0]); posicional 2: len(history) + 7.

len — Conta elementos. Argumentos usados: posicional 1: values[0].

len — Conta elementos. Argumentos usados: posicional 1: history.

```
0078                 short, same_future = model_inputs(df, window, dataset, history_days=15)
```

Atribui a (short, same_future) o resultado de model_inputs(df, window, dataset, history_days=15). A expressão à direita é avaliada antes da atribuição.

model_inputs — df: série completa; window: ForecastWindow; dataset: contrato; history_days: None para expansivo ou inteiro para últimos N dias. Retorna (history, future), excluindo alvo do futuro. Argumentos usados: posicional 1: df; posicional 2: window; posicional 3: dataset; history_days: 15.

```
0079                 self.assertEqual(len(short), 15)
```

Executa self.assertEqual(len(short), 15). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: len(short); posicional 2: 15.

len — Conta elementos. Argumentos usados: posicional 1: short.

```
0080                 pd.testing.assert_frame_equal(future, same_future)
```

Executa pd.testing.assert_frame_equal(future, same_future). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

pd.testing.assert_frame_equal — Verifica igualdade detalhada entre dois DataFrames e lança AssertionError se diferirem. Argumentos usados: posicional 1: future; posicional 2: same_future.

```
0081                 prompt = build_prompt(short, 7, future_covariates=future)
```

Atribui a prompt o resultado de build_prompt(short, 7, future_covariates=future). A expressão à direita é avaliada antes da atribuição.

build_prompt — history: DataFrame histórico; horizon: dias; demonstrations: lista ou None; future_covariates: tabela sem alvo ou None. Retorna prompt legado com lista de demanda e tabelas de covariáveis. Argumentos usados: posicional 1: short; posicional 2: 7; future_covariates: future.

```
0082                 self.assertTrue(all(c in prompt for c in columns))
```

Executa self.assertTrue(all((c in prompt for c in columns))). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertTrue — Asserção de condição verdadeira. Argumentos usados: posicional 1: all((c in prompt for c in columns)).

all — Exige que todos os elementos sejam True. Argumentos usados: posicional 1: (c in prompt for c in columns).

```
0083                 poisoned = df.copy()
```

Atribui a poisoned o resultado de df.copy(). A expressão à direita é avaliada antes da atribuição.

df.copy — Copia uma tabela/dicionário para não alterar diretamente o objeto anterior. Argumentos usados: nenhum argumento explícito.

```
0084                 poisoned.loc[window.origin:, 'water_produced'] = 987654321
```

Atribui a poisoned.loc[window.origin:, 'water_produced'] o resultado de 987654321. A expressão à direita é avaliada antes da atribuição.

```
0085                 other_history, other_future = model_inputs(poisoned, window, dataset,
    history_days=15)
```

Atribui a (other_history, other_future) o resultado de model_inputs(poisoned, window, dataset, history_days=15). A expressão à direita é avaliada antes da atribuição.

model_inputs — df: série completa; window: ForecastWindow; dataset: contrato; history_days: None para expansivo ou inteiro para últimos N dias. Retorna (history, future), excluindo alvo do futuro. Argumentos usados: posicional 1: poisoned; posicional 2: window; posicional 3: dataset; history_days: 15.

```
0086                 self.assertEqual(prompt, build_prompt(other_history, 7,
    future_covariates=other_future))
```

Executa self.assertEqual(prompt, build_prompt(other_history, 7, future_covariates=other_future)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: prompt; posicional 2: build_prompt(other_history, 7, future_covariates=other_future).

build_prompt — history: DataFrame histórico; horizon: dias; demonstrations: lista ou None; future_covariates: tabela sem alvo ou None. Retorna prompt legado com lista de demanda e tabelas de covariáveis. Argumentos usados: posicional 1: other_history; posicional 2: 7; future_covariates: other_future.

```
0088     def test_future_target_column_is_rejected(self):
```

Define test_future_target_column_is_rejected(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0089         history = pd.DataFrame({'timestamp': pd.date_range('2020-01-01', periods=3),
    'water_produced':[1.,2.,3.]})
```

Atribui a history o resultado de pd.DataFrame({'timestamp': pd.date_range('2020-01-01', periods=3), 'water_produced': [1.0, 2.0, 3.0]}). A expressão à direita é avaliada antes da atribuição.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: {'timestamp': pd.date_range('2020-01-01', periods=3), 'water_produced': [1.0, 2.0, 3.0]}.

pd.date_range — Produz uma sequência ordenada de timestamps. Argumentos usados: posicional 1: '2020-01-01'; periods: 3.

```
0090         future = pd.DataFrame({'timestamp':pd.date_range('2020-01-04',periods=7),
    'water_produced':99.})
```

Atribui a future o resultado de pd.DataFrame({'timestamp': pd.date_range('2020-01-04', periods=7), 'water_produced': 99.0}). A expressão à direita é avaliada antes da atribuição.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: {'timestamp': pd.date_range('2020-01-04', periods=7), 'water_produced': 99.0}.

pd.date_range — Produz uma sequência ordenada de timestamps. Argumentos usados: posicional 1: '2020-01-04'; periods: 7.

```
0091         with self.assertRaisesRegex(ValueError, 'Future target'):
```

Abre o contexto gerenciado self.assertRaisesRegex(ValueError, 'Future target'). Ao sair, libera o recurso ou restaura a simulação/redirecionamento, inclusive se ocorrer erro.

self.assertRaisesRegex — Exige exceção e mensagem compatível com expressão regular. Argumentos usados: posicional 1: ValueError; posicional 2: 'Future target'.

```
0092              build_prompt(history, 7, future_covariates=future)
```

Executa build_prompt(history, 7, future_covariates=future). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

build_prompt — history: DataFrame histórico; horizon: dias; demonstrations: lista ou None; future_covariates: tabela sem alvo ou None. Retorna prompt legado com lista de demanda e tabelas de covariáveis. Argumentos usados: posicional 1: history; posicional 2: 7; future_covariates: future.

```
0094      def test_chronos_receives_entire_growing_history_and_covariates(self):
```

Define test_chronos_receives_entire_growing_history_and_covariates(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0095          from src.models.chronos import ChronosForecaster
```

Importa ChronosForecaster de src.models.chronos. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0096          from unittest.mock import Mock
```

Importa Mock de unittest.mock. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0097          config = json.loads((ROOT / 'configs/benchmark.json').read_text())
```

Atribui a config o resultado de json.loads((ROOT / 'configs/benchmark.json').read_text()). A expressão à direita é avaliada antes da atribuição.

json.loads — Decodifica uma string JSON em objetos Python. Argumentos usados: posicional 1: (ROOT / 'configs/benchmark.json').read_text().

(ROOT / 'configs/benchmark.json').read_text — Lê todo o arquivo como string. Argumentos usados: nenhum argumento explícito.

```
0098          dataset = config['datasets']['joinville']
```

Atribui a dataset o resultado de config['datasets']['joinville']. A expressão à direita é avaliada antes da atribuição.

```
0099          df, windows = prepare_dataset(ROOT / dataset['path'], dataset['context'])
```

Atribui a (df, windows) o resultado de prepare_dataset(ROOT / dataset['path'], dataset['context']). A expressão à direita é avaliada antes da atribuição.

prepare_dataset — path: CSV; context: primeira origem; horizon=7 e step=7 nomeados. Retorna df e windows depois da preparação. Argumentos usados: posicional 1: ROOT / dataset['path']; posicional 2: dataset['context'].

```
0100          adapter = object.__new__(ChronosForecaster)
```

Atribui a adapter o resultado de object.__new__(ChronosForecaster). A expressão à direita é avaliada antes da atribuição.

object.__new__ — Cria instância sem executar seu __init__. Argumentos usados: posicional 1: ChronosForecaster.

```
0101          adapter.dataset = dataset
```

Atribui a adapter.dataset o resultado de dataset. A expressão à direita é avaliada antes da atribuição.

```
0102          adapter.pipeline = Mock()
```

Atribui a adapter.pipeline o resultado de Mock(). A expressão à direita é avaliada antes da atribuição.

Mock — Cria objeto simulado para registrar chamadas. Argumentos usados: nenhum argumento explícito.

```
0103          adapter.pipeline.predict_df.return_value = pd.DataFrame({'0.5': [1.]*7})
```

Atribui a adapter.pipeline.predict_df.return_value o resultado de pd.DataFrame({'0.5': [1.0] * 7}). A expressão à direita é avaliada antes da atribuição.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: {'0.5': [1.0] * 7}.

```
0104         for window in windows[:3]:
```

Percorre windows[:3]; em cada iteração associa o item a window. O corpo indentado aplica a operação a esse item/grupo.

```
0105             history, future = model_inputs(df, window, dataset)
```

Atribui a (history, future) o resultado de model_inputs(df, window, dataset). A expressão à direita é avaliada antes da atribuição.

model_inputs — df: série completa; window: ForecastWindow; dataset: contrato; history_days: None para expansivo ou inteiro para últimos N dias. Retorna (history, future), excluindo alvo do futuro. Argumentos usados: posicional 1: df; posicional 2: window; posicional 3: dataset.

```
0106             adapter.predict(history, 7, future_covariates=future)
```

Executa adapter.predict(history, 7, future_covariates=future). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

adapter.predict — Interface comum dos adaptadores do projeto. Argumentos usados: posicional 1: history; posicional 2: 7; future_covariates: future.

```
0107             call = adapter.pipeline.predict_df.call_args
```

Atribui a call o resultado de adapter.pipeline.predict_df.call_args. A expressão à direita é avaliada antes da atribuição.

```
0108             self.assertEqual(call.kwargs['context_length'], window.origin)
```

Executa self.assertEqual(call.kwargs['context_length'], window.origin). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: call.kwargs['context_length']; posicional 2: window.origin.

```
0109             self.assertEqual(len(call.args[0]), window.origin)
```

Executa self.assertEqual(len(call.args[0]), window.origin). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: len(call.args[0]); posicional 2: window.origin.

len — Conta elementos. Argumentos usados: posicional 1: call.args[0].

```
0110             self.assertNotIn('water_produced', call.kwargs['future_df'])
```

Executa self.assertNotIn('water_produced', call.kwargs['future_df']). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertNotIn — Exige item ausente de container. Argumentos usados: posicional 1: 'water_produced'; posicional 2: call.kwargs['future_df'].

```
0111
    self.assertTrue(set(covariate_columns(dataset)).issubset(call.kwargs['future_df']))
```

Executa self.assertTrue(set(covariate_columns(dataset)).issubset(call.kwargs['future_df'])). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertTrue — Asserção de condição verdadeira. Argumentos usados: posicional 1: set(covariate_columns(dataset)).issubset(call.kwargs['future_df']).

set(covariate_columns(dataset)).issubset — Verifica se todos os elementos estão contidos em outro conjunto/iterável. Argumentos usados: posicional 1: call.kwargs['future_df'].

set — Cria conjunto de elementos únicos. Argumentos usados: posicional 1: covariate_columns(dataset).

covariate_columns — dataset: bloco de configuração de uma cidade. Retorna nomes numéricos seguidos dos categóricos, após rejeitar duplicatas e nomes reservados. Argumentos usados: posicional 1: dataset.

```
0113     def test_invalid_model_responses(self):
```

Define test_invalid_model_responses(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0114          for text in ['{"predictions":[true]}','{"predictions":["1"]}',
```

Percorre ['{"predictions":[true]}', '{"predictions":["1"]}', '{"predictions":[NaN]}', '{"predictions":[-1]}', '{"predictions":[1,2]}', '[1]', 'answer: 1', '{"predictions":[1],"extra":0}']; em cada iteração associa o item a text. O corpo indentado aplica a operação a esse item/grupo.

```
0115                        '{"predictions":[NaN]}','{"predictions":[-1]}',
```

Continuação da instrução na linha 114: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0116                        '{"predictions":[1,2]}','[1]','answer: 1',
```

Continuação da instrução na linha 114: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0117                        '{"predictions":[1],"extra":0}']:
```

Continuação da instrução na linha 114: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0118              with self.subTest(text=text), self.assertRaises((ValueError,TypeError)):
```

Abre o contexto gerenciado self.subTest(text=text), self.assertRaises((ValueError, TypeError)). Ao sair, libera o recurso ou restaura a simulação/redirecionamento, inclusive se ocorrer erro.

self.subTest — Identifica um caso dentro de um loop de teste. Argumentos usados: text: text.

self.assertRaises — Context manager que exige uma exceção. Argumentos usados: posicional 1: (ValueError, TypeError).

```
0119                  parse_prediction(text,1)
```

Executa parse_prediction(text, 1). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

parse_prediction — text: string JSON do modelo; horizon: quantidade exata de valores. Retorna list[float] ou lança erro; não remove prose nem aceita resposta parcial. Argumentos usados: posicional 1: text; posicional 2: 1.

```
0120          self.assertEqual(parse_prediction('{"predictions":[1.5]}',1),[1.5])
```

Executa self.assertEqual(parse_prediction('{"predictions":[1.5]}', 1), [1.5]). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: parse_prediction('{"predictions":[1.5]}', 1); posicional 2: [1.5].

parse_prediction — text: string JSON do modelo; horizon: quantidade exata de valores. Retorna list[float] ou lança erro; não remove prose nem aceita resposta parcial. Argumentos usados: posicional 1: '{"predictions":[1.5]}'; posicional 2: 1.

```
0122      def test_retry_and_raw_response_audit(self):
```

Define test_retry_and_raw_response_audit(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0123          model = object.__new__(QwenForecaster)
```

Atribui a model o resultado de object.__new__(QwenForecaster). A expressão à direita é avaliada antes da atribuição.

object.__new__ — Cria instância sem executar seu __init__. Argumentos usados: posicional 1: QwenForecaster.

```
0124          model.config = OllamaConfig(retries=1)
```

Atribui a model.config o resultado de OllamaConfig(retries=1). A expressão à direita é avaliada antes da atribuição.

OllamaConfig — Constrói configuração tipada do cliente Qwen. Argumentos usados: retries: 1.

```
0125          model.last_audit = {}
```

Atribui a model.last_audit o resultado de {}. A expressão à direita é avaliada antes da atribuição.

```
0126          answers=[{'done':True,'response':'{}'},
    {'done':True,'response':'{"predictions":[2]}'}]
```

Atribui a answers o resultado de [{'done': True, 'response': '{}'}, {'done': True, 'response': '{"predictions":[2]}'}]. A expressão à direita é avaliada antes da atribuição.

```
0127          with patch('src.models.promptcast.post_json',side_effect=answers),
    patch('src.models.promptcast.time.sleep'):
```

Abre o contexto gerenciado patch('src.models.promptcast.post_json', side_effect=answers), patch('src.models.promptcast.time.sleep'). Ao sair, libera o recurso ou restaura a simulação/redirecionamento, inclusive se ocorrer erro.

patch — Substitui temporariamente uma referência durante o bloco with. Argumentos usados: posicional 1: 'src.models.promptcast.post_json'; side_effect: answers.

patch — Substitui temporariamente uma referência durante o bloco with. Argumentos usados: posicional 1: 'src.models.promptcast.time.sleep'.

```
0128              self.assertEqual(model.predict(None,1,prompt='history'),[2.])
```

Executa self.assertEqual(model.predict(None, 1, prompt='history'), [2.0]). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: model.predict(None, 1, prompt='history'); posicional 2: [2.0].

model.predict — Interface comum dos adaptadores do projeto. Argumentos usados: posicional 1: None; posicional 2: 1; prompt: 'history'.

```
0129          self.assertEqual(len(model.last_audit['attempts']),2)
```

Executa self.assertEqual(len(model.last_audit['attempts']), 2). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: len(model.last_audit['attempts']); posicional 2: 2.

len — Conta elementos. Argumentos usados: posicional 1: model.last_audit['attempts'].

```
0130          self.assertIn('error',model.last_audit['attempts'][0])
```

Executa self.assertIn('error', model.last_audit['attempts'][0]). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertIn — Exige item presente em container. Argumentos usados: posicional 1: 'error'; posicional 2: model.last_audit['attempts'][0].

```
0132      def test_context_budget_rejects_silent_truncation(self):
```

Define test_context_budget_rejects_silent_truncation(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0133          model=object.__new__(QwenForecaster)
```

Atribui a model o resultado de object.__new__(QwenForecaster). A expressão à direita é avaliada antes da atribuição.

object.__new__ — Cria instância sem executar seu __init__. Argumentos usados: posicional 1: QwenForecaster.

```
0134          model.config=OllamaConfig(num_ctx=800)
```

Atribui a model.config o resultado de OllamaConfig(num_ctx=800). A expressão à direita é avaliada antes da atribuição.

OllamaConfig — Constrói configuração tipada do cliente Qwen. Argumentos usados: num_ctx: 800.

```
0135          with self.assertRaises(ValueError):
```

Abre o contexto gerenciado self.assertRaises(ValueError). Ao sair, libera o recurso ou restaura a simulação/redirecionamento, inclusive se ocorrer erro.

self.assertRaises — Context manager que exige uma exceção. Argumentos usados: posicional 1: ValueError.

```
0136              model.predict(None,7,prompt='x'*1000)
```

Executa model.predict(None, 7, prompt='x' * 1000). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

model.predict — Interface comum dos adaptadores do projeto. Argumentos usados: posicional 1: None; posicional 2: 7; prompt: 'x' * 1000.

```
0138      def test_metric_units(self):
```

Define test_metric_units(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0139          metrics=deterministic_metrics([10.,20.],[12.,18.])
```

Atribui a metrics o resultado de deterministic_metrics([10.0, 20.0], [12.0, 18.0]). A expressão à direita é avaliada antes da atribuição.

deterministic_metrics — ytrue/yhat: vetores de mesmo formato, não vazios e finitos. Retorna MAE, RMSE, MAPE%, sMAPE%, R² e contagem de zeros excluídos do MAPE. Argumentos usados: posicional 1: [10.0, 20.0]; posicional 2: [12.0, 18.0].

```
0140          self.assertEqual(metrics['mae'],2.)
```

Executa self.assertEqual(metrics['mae'], 2.0). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: metrics['mae']; posicional 2: 2.0.

```
0141          self.assertEqual(metrics['rmse'],2.)
```

Executa self.assertEqual(metrics['rmse'], 2.0). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: metrics['rmse']; posicional 2: 2.0.

```
0142          self.assertAlmostEqual(metrics['mape_percent'],15.)
```

Executa self.assertAlmostEqual(metrics['mape_percent'], 15.0). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertAlmostEqual — Compara números com aproximação decimal. Argumentos usados: posicional 1: metrics['mape_percent']; posicional 2: 15.0.

```
0143          self.assertAlmostEqual(metrics['r2'],0.84)
```

Executa self.assertAlmostEqual(metrics['r2'], 0.84). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertAlmostEqual — Compara números com aproximação decimal. Argumentos usados: posicional 1: metrics['r2']; posicional 2: 0.84.

```
0144
    self.assertEqual(deterministic_metrics([0.,10.],[0.,12.])['mape_zero_targets_excluded'],1)
```

Executa self.assertEqual(deterministic_metrics([0.0, 10.0], [0.0, 12.0])['mape_zero_targets_excluded'], 1). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: deterministic_metrics([0.0, 10.0], [0.0, 12.0])['mape_zero_targets_excluded']; posicional 2: 1.

deterministic_metrics — ytrue/yhat: vetores de mesmo formato, não vazios e finitos. Retorna MAE, RMSE, MAPE%, sMAPE%, R² e contagem de zeros excluídos do MAPE. Argumentos usados: posicional 1: [0.0, 10.0]; posicional 2: [0.0, 12.0].

```
0145          with self.assertRaises(ValueError):
```

Abre o contexto gerenciado self.assertRaises(ValueError). Ao sair, libera o recurso ou restaura a simulação/redirecionamento, inclusive se ocorrer erro.

self.assertRaises — Context manager que exige uma exceção. Argumentos usados: posicional 1: ValueError.

```
0146              deterministic_metrics([1.,2.],[1.,float('nan')])
```

Executa deterministic_metrics([1.0, 2.0], [1.0, float('nan')]). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

deterministic_metrics — ytrue/yhat: vetores de mesmo formato, não vazios e finitos. Retorna MAE, RMSE, MAPE%, sMAPE%, R² e contagem de zeros excluídos do MAPE. Argumentos usados: posicional 1: [1.0, 2.0]; posicional 2: [1.0, float('nan')].

float — Converte escalar para float Python. Argumentos usados: posicional 1: 'nan'.

```
0148      def test_validation_uses_training_prefix(self):
```

Define test_validation_uses_training_prefix(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0149          train,validation=validation_split(100)
```

Atribui a (train, validation) o resultado de validation_split(100). A expressão à direita é avaliada antes da atribuição.

validation_split — training_size: comprimento do prefixo anterior ao teste; fraction=0.2: parte final reservada à validação. Retorna dois slices sem embaralhar. Argumentos usados: posicional 1: 100.

```
0150
    self.assertEqual((train.start,train.stop,validation.start,validation.stop),(0,80,80,100))
```

Executa self.assertEqual((train.start, train.stop, validation.start, validation.stop), (0, 80, 80, 100)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: (train.start, train.stop, validation.start, validation.stop); posicional 2: (0, 80, 80, 100).

```
0152      def test_duplicate_dates_rejected(self):
```

Define test_duplicate_dates_rejected(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0153          with self.assertRaises(ValueError):
```

Abre o contexto gerenciado self.assertRaises(ValueError). Ao sair, libera o recurso ou restaura a simulação/redirecionamento, inclusive se ocorrer erro.

self.assertRaises — Context manager que exige uma exceção. Argumentos usados: posicional 1: ValueError.

```
0154
    validate_series(pd.DataFrame({'timestamp':['2020-01-01']*2,'water_produced':[1,2]}))
```

Executa validate_series(pd.DataFrame({'timestamp': ['2020-01-01'] * 2, 'water_produced': [1, 2]})). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

validate_series — df: tabela lida. Retorno: cópia ordenada. Rejeita esquema, datas e demanda inválidos antes de qualquer modelo. Argumentos usados: posicional 1: pd.DataFrame({'timestamp': ['2020-01-01'] * 2, 'water_produced': [1, 2]}).

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: {'timestamp': ['2020-01-01'] * 2, 'water_produced': [1, 2]}.

```
0157  if __name__ == '__main__':
```

Avalia a condição __name__ == '__main__'. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0158      unittest.main()
```

Executa unittest.main(). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

unittest.main — Executa testes quando o arquivo é ponto de entrada. Argumentos usados: nenhum argumento explícito.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

35. tests/test_qwen_context_comparison.py

Testes de regressão. São executados por unittest discover, não durante a inferência.

SHA-256: 6dc522d0f7a2bee4467031a185a15f8089970c5a1f88c279a4f5ef4ee29ad954

Funções, contratos e pontos de conexão

```
L13: test_positive_and_negative_improvement(self)
```

Caso de teste: positive and negative improvement. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: relative_reduction → src/pipelines/compare_qwen_contexts.py:10

```
L18: test_comparison_uses_same_checkpoint_and_all_seven_days(self)
```

Caso de teste: comparison uses same checkpoint and all seven days. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: compare_contexts → src/pipelines/compare_qwen_contexts.py:36

```
L43: test_tabular_prompt_preserves_values_and_expanding_prefix(self)
```

Caso de teste: tabular prompt preserves values and expanding prefix. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: build_tabular_prompt → src/data/preprocessing/generate_prompts.py:97

APIs externas usadas neste arquivo

Path, all, assertEqual, assertIsNone, assertRaisesRegex, assertTrue, contextlib.redirect_stdout, copy, drop, eq, io.StringIO, json.dumps, json.loads, len, mkdir, np.arange, np.testing.assert_allclose, pd.DataFrame, pd.date_range, pd.read_csv, range, read_text, split, startswith, strftime, tempfile.TemporaryDirectory, to_csv, write_text. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Ensure the context ablation changes only history and reports signed improvement."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import contextlib
```

Importa contextlib. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0003  import io
```

Importa io. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0004  import json
```

Importa json. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  from pathlib import Path
```

Importa Path de pathlib. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0006  import tempfile
```

Importa tempfile. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0007  import unittest
```

Importa unittest. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0008  import pandas as pd
```

Importa pandas como pd. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0009  from src.pipelines.compare_qwen_contexts import compare_contexts,relative_reduction
```

Importa compare_contexts, relative_reduction de src.pipelines.compare_qwen_contexts. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0012  class QwenContextComparisonTests(unittest.TestCase):
```

Declara a classe QwenContextComparisonTests(unittest.TestCase). Seus métodos compartilham estado por self; a declaração não cria uma previsão.

```
0013      def test_positive_and_negative_improvement(self):
```

Define test_positive_and_negative_improvement(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0014          self.assertEqual(relative_reduction(100,80),20)
```

Executa self.assertEqual(relative_reduction(100, 80), 20). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: relative_reduction(100, 80); posicional 2: 20.

relative_reduction — short: erro de 15 dias; long: erro expansivo. Retorna 100*(short-long)/short ou None se baseline zero. Positivo é melhoria, negativo piora. Argumentos usados: posicional 1: 100; posicional 2: 80.

```
0015          self.assertEqual(relative_reduction(100,120),-20)
```

Executa self.assertEqual(relative_reduction(100, 120), -20). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: relative_reduction(100, 120); posicional 2: -20.

relative_reduction — short: erro de 15 dias; long: erro expansivo. Retorna 100*(short-long)/short ou None se baseline zero. Positivo é melhoria, negativo piora. Argumentos usados: posicional 1: 100; posicional 2: 120.

```
0016          self.assertIsNone(relative_reduction(0,1))
```

Executa self.assertIsNone(relative_reduction(0, 1)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertIsNone — Exige que o resultado seja o objeto None. Argumentos usados: posicional 1: relative_reduction(0, 1).

relative_reduction — short: erro de 15 dias; long: erro expansivo. Retorna 100*(short-long)/short ou None se baseline zero. Positivo é melhoria, negativo piora. Argumentos usados: posicional 1: 0; posicional 2: 1.

```
0018      def test_comparison_uses_same_checkpoint_and_all_seven_days(self):
```

Define test_comparison_uses_same_checkpoint_and_all_seven_days(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0019          with tempfile.TemporaryDirectory() as directory:
```

Abre o contexto gerenciado tempfile.TemporaryDirectory() como directory. Ao sair, libera o recurso ou restaura a simulação/redirecionamento, inclusive se ocorrer erro.

tempfile.TemporaryDirectory — Cria diretório temporário gerenciado. Argumentos usados: nenhum argumento explícito.

```
0020              root=Path(directory)
```

Atribui a root o resultado de Path(directory). A expressão à direita é avaliada antes da atribuição.

Path — Representa caminho de arquivo com operações portáveis. Argumentos usados: posicional 1: directory.

```
0021              for dataset,initial in [('joinville',365),('guaratuba',730)]:
```

Percorre [('joinville', 365), ('guaratuba', 730)]; em cada iteração associa o item a (dataset, initial). O corpo indentado aplica a operação a esse item/grupo.

```
0022                  for policy,context,error in [('15_days',15,10),('expanding',initial,5)]:
```

Percorre [('15_days', 15, 10), ('expanding', initial, 5)]; em cada iteração associa o item a (policy, context, error). O corpo indentado aplica a operação a esse item/grupo.

```
0023                        folder=root/policy/dataset/'qwen';folder.mkdir(parents=True)
```

Atribui a folder o resultado de root / policy / dataset / 'qwen'. A expressão à direita é avaliada antes da atribuição. Executa folder.mkdir(parents=True). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

folder.mkdir — Cria diretório. Argumentos usados: parents: True.

```
0024                        run={'limit':0,'dataset_sha256':dataset,'source_sha256':'same',
```

Atribui a run o resultado de {'limit': 0, 'dataset_sha256': dataset, 'source_sha256': 'same', 'model': {'model': 'same-qwen', 'digest': 'same-digest'}, 'covariates': ['weather'], 'first_test_origin': initial, 'config': {'qwen': {'num_ctx': 131072}, 'qwen_history_days': 15 if policy == '15_days' else None}}. A expressão à direita é avaliada antes da atribuição.

```
0025                            'model':{'model':'same-qwen','digest':'same-
    digest'},'covariates':['weather'],
```

Define 'model': identidade do checkpoint/adaptador; recebe {'model': 'same-qwen', 'digest': 'same-digest'}; 'covariates': nomes das variáveis auxiliares; recebe ['weather']; 'model': identidade do checkpoint/adaptador; recebe 'same-qwen'; 'digest': campo do registro/coleção em construção; recebe 'same-digest'. Os valores são avaliados nesta janela/iteração.

```
0026                            'first_test_origin':initial,'config':{'qwen':{'num_ctx':131072},
```

Define 'first_test_origin': campo do registro/coleção em construção; recebe initial; 'config': campo do registro/coleção em construção; recebe {'qwen': {'num_ctx': 131072}, 'qwen_history_days': 15 if policy == '15_days' else None}; 'qwen': campo do registro/coleção em construção; recebe {'num_ctx': 131072}; 'num_ctx': campo do registro/coleção em construção; recebe 131072. Os valores são avaliados nesta janela/iteração.

```
0027                            'qwen_history_days':15 if policy=='15_days' else None}}
```

Define 'qwen_history_days': campo do registro/coleção em construção; recebe 15 if policy == '15_days' else None. Os valores são avaliados nesta janela/iteração.

```
0028                        (folder/'run.json').write_text(json.dumps(run))
```

Executa (folder / 'run.json').write_text(json.dumps(run)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

(folder / 'run.json').write_text — Grava uma string em arquivo e retorna a quantidade de caracteres escrita. Argumentos usados: posicional 1: json.dumps(run).

json.dumps — Serializa objeto Python em string JSON. Argumentos usados: posicional 1: run.

```
0029                        (folder/'status.json').write_text(json.dumps({'expected_windows':1,'succe
    ssful_windows':1,'failures':[]}))
```

Executa (folder / 'status.json').write_text(json.dumps({'expected_windows': 1, 'successful_windows': 1, 'failures': []})). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

(folder / 'status.json').write_text — Grava uma string em arquivo e retorna a quantidade de caracteres escrita. Argumentos usados: posicional 1: json.dumps({'expected_windows': 1, 'successful_windows': 1, 'failures': []}).

json.dumps — Serializa objeto Python em string JSON. Argumentos usados: posicional 1: {'expected_windows': 1, 'successful_windows': 1, 'failures': []}.

```
0030                        pd.DataFrame({'origin':['2020-01-01']*7,'timestamp':pd.date_range('2020-
    01-01',periods=7).strftime('%Y-%m-%d'),
```

Executa pd.DataFrame({'origin': ['2020-01-01'] * 7, 'timestamp': pd.date_range('2020-01-01', periods=7).strftime('%Y-%m-%d'), 'lead': range(1, 8), 'y_true': range(10, 17), 'y_pred': range(10 + error, 17 + error), 'initial_context': [initial] * 7, 'context': [context] * 7, 'history_start': ['2018-01-01'] * 7, 'horizon': [7] * 7, 'elapsed_seconds_window': [1.0] * 7}).to_csv(folder / 'predictions.csv', index=False). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

pd.DataFrame({'origin': ['2020-01-01'] * 7, 'timestamp': pd.date_range('2020-01-01', periods=7).strftime('%Y-%m-%d'), 'lead': range(1, 8), 'y_true': range(10, 17), 'y_pred': range(10 + error, 17 + error), 'initial_context': [initial] * 7, 'context': [context] * 7, 'history_start': ['2018-01-01'] * 7, 'horizon': [7] * 7, 'elapsed_seconds_window': [1.0] * 7}).to_csv — Serializa DataFrame como CSV. Argumentos usados: posicional 1: folder / 'predictions.csv'; index: False.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: {'origin': ['2020-01-01'] * 7, 'timestamp': pd.date_range('2020-01-01', periods=7).strftime('%Y-%m-%d'), 'lead': range(1, 8), 'y_true': range(10, 17), 'y_pred': range(10 + error, 17 + error), 'initial_context': [initial] * 7, 'context': [context] * 7, 'history_start': ['2018-01-01'] * 7, 'horizon': [7] * 7, 'elapsed_seconds_window': [1.0] * 7}.

pd.date_range('2020-01-01', periods=7).strftime — Formata datas como texto. Argumentos usados: posicional 1: '%Y-%m-%d'.

pd.date_range — Produz uma sequência ordenada de timestamps. Argumentos usados: posicional 1: '2020-01-01'; periods: 7.

```
0031
    'lead':range(1,8),'y_true':range(10,17),'y_pred':range(10+error,17+error),
```

Define 'lead': posição de 1 a 7 dentro do horizonte; recebe range(1, 8); 'y_true': demanda observada usada somente na avaliação; recebe range(10, 17); 'y_pred': valor retornado pelo modelo para a data correspondente; recebe range(10 + error, 17 + error). Os valores são avaliados nesta janela/iteração.

range — Gera inteiros com limite final exclusivo. Argumentos usados: posicional 1: 1; posicional 2: 8.

range — Gera inteiros com limite final exclusivo. Argumentos usados: posicional 1: 10; posicional 2: 17.

range — Gera inteiros com limite final exclusivo. Argumentos usados: posicional 1: 10 + error; posicional 2: 17 + error.

```
0032
    'initial_context':[initial]*7,'context':[context]*7,'history_start':['2018-01-01']*7,
```

Define 'initial_context': origem inicial do protocolo de avaliação; recebe [initial] * 7; 'context': número de linhas históricas efetivamente fornecidas; recebe [context] * 7; 'history_start': primeira data histórica efetivamente fornecida; recebe ['2018-01-01'] * 7. Os valores são avaliados nesta janela/iteração.

```
0033
    'horizon':[7]*7,'elapsed_seconds_window':[1.]*7}).to_csv(folder/'predictions.csv',index=False)
```

Define 'horizon': campo do registro/coleção em construção; recebe [7] * 7; 'elapsed_seconds_window': duração da janela, repetida em seus sete registros; recebe [1.0] * 7. Os valores são avaliados nesta janela/iteração.

```
0034            with contextlib.redirect_stdout(io.StringIO()):compare_contexts(root)
```

Abre o contexto gerenciado contextlib.redirect_stdout(io.StringIO()). Ao sair, libera o recurso ou restaura a simulação/redirecionamento, inclusive se ocorrer erro. Executa compare_contexts(root). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

contextlib.redirect_stdout — Redireciona print temporariamente. Argumentos usados: posicional 1: io.StringIO().

compare_contexts — output: pasta que contém os dois braços. Sem retorno útil. Exige comparação controlada e grava quatro CSVs. Argumentos usados: posicional 1: root.

io.StringIO — Buffer de texto em memória com interface de arquivo. Argumentos usados: nenhum argumento explícito.

```
0035            scores=pd.read_csv(root/'improvement.csv')
```

Atribui a scores o resultado de pd.read_csv(root / 'improvement.csv'). A expressão à direita é avaliada antes da atribuição.

pd.read_csv — Lê CSV em um DataFrame, inferindo tipos; datas só são convertidas posteriormente no validador. Argumentos usados: posicional 1: root / 'improvement.csv'.

```
0036            self.assertTrue(scores.rmse_reduction_percent.eq(50).all())
```

Executa self.assertTrue(scores.rmse_reduction_percent.eq(50).all()). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertTrue — Asserção de condição verdadeira. Argumentos usados: posicional 1: scores.rmse_reduction_percent.eq(50).all().

scores.rmse_reduction_percent.eq(50).all — Exige que todos os elementos sejam True. Argumentos usados: nenhum argumento explícito.

scores.rmse_reduction_percent.eq — Compara igualdade elemento a elemento. Argumentos usados: posicional 1: 50.

```
0037            self.assertEqual(len(pd.read_csv(root/'metrics_by_horizon.csv')),28)
```

Executa self.assertEqual(len(pd.read_csv(root / 'metrics_by_horizon.csv')), 28). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: len(pd.read_csv(root / 'metrics_by_horizon.csv')); posicional 2: 28.

len — Conta elementos. Argumentos usados: posicional 1: pd.read_csv(root / 'metrics_by_horizon.csv').

pd.read_csv — Lê CSV em um DataFrame, inferindo tipos; datas só são convertidas posteriormente no validador. Argumentos usados: posicional 1: root / 'metrics_by_horizon.csv'.

```
0038              changed=root/'expanding/guaratuba/qwen/run.json'
```

Atribui a changed o resultado de root / 'expanding/guaratuba/qwen/run.json'. A expressão à direita é avaliada antes da atribuição.

```
0039              run=json.loads(changed.read_text());run['model']['digest']='different';changed.wr
    ite_text(json.dumps(run))
```

Atribui a run o resultado de json.loads(changed.read_text()). A expressão à direita é avaliada antes da atribuição. Atribui a run['model']['digest'] o resultado de 'different'. A expressão à direita é avaliada antes da atribuição. Executa changed.write_text(json.dumps(run)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

json.loads — Decodifica uma string JSON em objetos Python. Argumentos usados: posicional 1: changed.read_text().

changed.write_text — Grava uma string em arquivo e retorna a quantidade de caracteres escrita. Argumentos usados: posicional 1: json.dumps(run).

changed.read_text — Lê todo o arquivo como string. Argumentos usados: nenhum argumento explícito.

json.dumps — Serializa objeto Python em string JSON. Argumentos usados: posicional 1: run.

```
0040              with self.assertRaisesRegex(ValueError,'changes model'):
```

Abre o contexto gerenciado self.assertRaisesRegex(ValueError, 'changes model'). Ao sair, libera o recurso ou restaura a simulação/redirecionamento, inclusive se ocorrer erro.

self.assertRaisesRegex — Exige exceção e mensagem compatível com expressão regular. Argumentos usados: posicional 1: ValueError; posicional 2: 'changes model'.

```
0041                  compare_contexts(root)
```

Executa compare_contexts(root). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

compare_contexts — output: pasta que contém os dois braços. Sem retorno útil. Exige comparação controlada e grava quatro CSVs. Argumentos usados: posicional 1: root.

```
0043      def test_tabular_prompt_preserves_values_and_expanding_prefix(self):
```

Define test_tabular_prompt_preserves_values_and_expanding_prefix(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0044          import numpy as np
```

Importa numpy como np. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0045          from src.data.preprocessing.generate_prompts import build_tabular_prompt
```

Importa build_tabular_prompt de src.data.preprocessing.generate_prompts. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0046          frame=pd.DataFrame({'timestamp':pd.date_range('2020-01-01',periods=40),
```

Atribui a frame o resultado de pd.DataFrame({'timestamp': pd.date_range('2020-01-01', periods=40), 'water_produced': np.arange(40.0) + 100, 'weather': np.arange(40.0) / 2, 'holiday': [0] * 40}). A expressão à direita é avaliada antes da atribuição.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: {'timestamp': pd.date_range('2020-01-01', periods=40), 'water_produced': np.arange(40.0) + 100, 'weather': np.arange(40.0) / 2, 'holiday': [0] * 40}.

pd.date_range — Produz uma sequência ordenada de timestamps. Argumentos usados: posicional 1: '2020-01-01'; periods: 40.

```
0047
    'water_produced':np.arange(40.)+100,'weather':np.arange(40.)/2,'holiday':[0]*40})
```

Define 'water_produced': campo do registro/coleção em construção; recebe np.arange(40.0) + 100; 'weather': campo do registro/coleção em construção; recebe np.arange(40.0) / 2; 'holiday': campo do registro/coleção em construção; recebe [0] * 40. Os valores são avaliados nesta janela/iteração.

np.arange — Gera progressão de valores com fim exclusivo. Argumentos usados: posicional 1: 40.0.

np.arange — Gera progressão de valores com fim exclusivo. Argumentos usados: posicional 1: 40.0.

```
0048          first=build_tabular_prompt(frame.iloc[:20],7,future_covariates=frame.iloc[20:27].drop
    (columns='water_produced'))
```

Atribui a first o resultado de build_tabular_prompt(frame.iloc[:20], 7, future_covariates=frame.iloc[20:27].drop(columns='water_produced')). A expressão à direita é avaliada antes da atribuição.

build_tabular_prompt — history: DataFrame histórico; horizon: dias; future_covariates: obrigatório e nomeado. Retorna CSV histórico unificado e tabela futura, com prefixo estável para história expansiva. Argumentos usados: posicional 1: frame.iloc[:20]; posicional 2: 7; future_covariates: frame.iloc[20:27].drop(columns='water_produced').

frame.iloc[20:27].drop — Retorna tabela sem as linhas/colunas indicadas. Argumentos usados: columns: 'water_produced'.

```
0049          second=build_tabular_prompt(frame.iloc[:27],7,future_covariates=frame.iloc[27:34].dro
    p(columns='water_produced'))
```

Atribui a second o resultado de build_tabular_prompt(frame.iloc[:27], 7, future_covariates=frame.iloc[27:34].drop(columns='water_produced')). A expressão à direita é avaliada antes da atribuição.

build_tabular_prompt — history: DataFrame histórico; horizon: dias; future_covariates: obrigatório e nomeado. Retorna CSV histórico unificado e tabela futura, com prefixo estável para história expansiva. Argumentos usados: posicional 1: frame.iloc[:27]; posicional 2: 7; future_covariates: frame.iloc[27:34].drop(columns='water_produced').

frame.iloc[27:34].drop — Retorna tabela sem as linhas/colunas indicadas. Argumentos usados: columns: 'water_produced'.

```
0050          prefix=first.split('\nEND HISTORY:')[0]
```

Atribui a prefix o resultado de first.split('\nEND HISTORY:')[0]. A expressão à direita é avaliada antes da atribuição.

first.split — Divide uma string por separador. Argumentos usados: posicional 1: '\nEND HISTORY:'.

```
0051          self.assertTrue(second.startswith(prefix+'\n'))
```

Executa self.assertTrue(second.startswith(prefix + '\n')). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertTrue — Asserção de condição verdadeira. Argumentos usados: posicional 1: second.startswith(prefix + '\n').

second.startswith — Testa prefixo textual exato. Argumentos usados: posicional 1: prefix + '\n'.

```
0052          table=prefix.split('(CSV, oldest to newest):\n')[1]
```

Atribui a table o resultado de prefix.split('(CSV, oldest to newest):\n')[1]. A expressão à direita é avaliada antes da atribuição.

prefix.split — Divide uma string por separador. Argumentos usados: posicional 1: '(CSV, oldest to newest):\n'.

```
0053          restored=pd.read_csv(io.StringIO(table))
```

Atribui a restored o resultado de pd.read_csv(io.StringIO(table)). A expressão à direita é avaliada antes da atribuição.

pd.read_csv — Lê CSV em um DataFrame, inferindo tipos; datas só são convertidas posteriormente no validador. Argumentos usados: posicional 1: io.StringIO(table).

io.StringIO — Buffer de texto em memória com interface de arquivo. Argumentos usados: posicional 1: table.

```
0054          np.testing.assert_allclose(restored.water_produced,frame.iloc[:20].water_produced)
```

Executa np.testing.assert_allclose(restored.water_produced, frame.iloc[:20].water_produced). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

np.testing.assert_allclose — Asserção numérica vetorial tolerante. Argumentos usados: posicional 1: restored.water_produced; posicional 2: frame.iloc[:20].water_produced.

```
0055          np.testing.assert_allclose(restored.weather,frame.iloc[:20].weather)
```

Executa np.testing.assert_allclose(restored.weather, frame.iloc[:20].weather). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

np.testing.assert_allclose — Asserção numérica vetorial tolerante. Argumentos usados: posicional 1: restored.weather; posicional 2: frame.iloc[:20].weather.

```
0056          changed=frame.copy();changed.loc[20:,'water_produced']=999999
```

Atribui a changed o resultado de frame.copy(). A expressão à direita é avaliada antes da atribuição. Atribui a changed.loc[20:, 'water_produced'] o resultado de 999999. A expressão à direita é avaliada antes da atribuição.

frame.copy — Copia uma tabela/dicionário para não alterar diretamente o objeto anterior. Argumentos usados: nenhum argumento explícito.

```
0057          self.assertEqual(first,build_tabular_prompt(changed.iloc[:20],7,future_covariates=cha
    nged.iloc[20:27].drop(columns='water_produced')))
```

Executa self.assertEqual(first, build_tabular_prompt(changed.iloc[:20], 7, future_covariates=changed.iloc[20:27].drop(columns='water_produced'))). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: first; posicional 2: build_tabular_prompt(changed.iloc[:20], 7, future_covariates=changed.iloc[20:27].drop(columns='water_produced')).

build_tabular_prompt — history: DataFrame histórico; horizon: dias; future_covariates: obrigatório e nomeado. Retorna CSV histórico unificado e tabela futura, com prefixo estável para história expansiva. Argumentos usados: posicional 1: changed.iloc[:20]; posicional 2: 7; future_covariates: changed.iloc[20:27].drop(columns='water_produced').

changed.iloc[20:27].drop — Retorna tabela sem as linhas/colunas indicadas. Argumentos usados: columns: 'water_produced'.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

36. tests/test_seven_day_metrics.py

Testes de regressão. São executados por unittest discover, não durante a inferência.

SHA-256: fff364c2855fb53d1cbdec0bb8e08dea740b2b6c3df90c5e8cfaed79e43b8c63

Funções, contratos e pontos de conexão

```
L14: test_comparison_includes_errors_after_day_two(self)
```

Caso de teste: comparison includes errors after day two. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: compare → src/pipelines/compare_models.py:10

APIs externas usadas neste arquivo

Path, assertEqual, contextlib.redirect_stdout, io.StringIO, json.dumps, len, mkdir, np.sqrt, np.testing.assert_allclose, pd.DataFrame, pd.date_range, pd.read_csv, range, set, strftime, tempfile.TemporaryDirectory, to_csv, write_text. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Regression test: days 3–7 must contribute to the actual comparison CSV."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import contextlib
```

Importa contextlib. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0003  import io
```

Importa io. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0004  import json
```

Importa json. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  from pathlib import Path
```

Importa Path de pathlib. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0006  import tempfile
```

Importa tempfile. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0007  import unittest
```

Importa unittest. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0008  import numpy as np
```

Importa numpy como np. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0009  import pandas as pd
```

Importa pandas como pd. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0010  from src.pipelines.compare_models import compare
```

Importa compare de src.pipelines.compare_models. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0013  class SevenDayMetricsTests(unittest.TestCase):
```

Declara a classe SevenDayMetricsTests(unittest.TestCase). Seus métodos compartilham estado por self; a declaração não cria uma previsão.

```
0014      def test_comparison_includes_errors_after_day_two(self):
```

Define test_comparison_includes_errors_after_day_two(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0015          with tempfile.TemporaryDirectory() as temporary:
```

Abre o contexto gerenciado tempfile.TemporaryDirectory() como temporary. Ao sair, libera o recurso ou restaura a simulação/redirecionamento, inclusive se ocorrer erro.

tempfile.TemporaryDirectory — Cria diretório temporário gerenciado. Argumentos usados: nenhum argumento explícito.

```
0016              root = Path(temporary)
```

Atribui a root o resultado de Path(temporary). A expressão à direita é avaliada antes da atribuição.

Path — Representa caminho de arquivo com operações portáveis. Argumentos usados: posicional 1: temporary.

```
0017              for dataset, initial in [('joinville', 365), ('guaratuba', 730)]:
```

Percorre [('joinville', 365), ('guaratuba', 730)]; em cada iteração associa o item a (dataset, initial). O corpo indentado aplica a operação a esse item/grupo.

```
0018                  for model in ['qwen', 'chronos', 'timesfm']:
```

Percorre ['qwen', 'chronos', 'timesfm']; em cada iteração associa o item a model. O corpo indentado aplica a operação a esse item/grupo.

```
0019                      folder = root / dataset / model
```

Atribui a folder o resultado de root / dataset / model. A expressão à direita é avaliada antes da atribuição.

```
0020                      folder.mkdir(parents=True)
```

Executa folder.mkdir(parents=True). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

folder.mkdir — Cria diretório. Argumentos usados: parents: True.

```
0021                      run = {'limit': 0, 'qwen_history_days': 15,
```

Atribui a run o resultado de {'limit': 0, 'qwen_history_days': 15, 'config': {'strategy': 'zero'}, 'dataset_sha256': dataset, 'source_sha256': 'same-code', 'n_windows': 1, 'model': {'model': model}, 'covariates': ['temperature']}. A expressão à direita é avaliada antes da atribuição.

```
0022                              'config': {'strategy': 'zero'}, 'dataset_sha256': dataset,
```

Define 'config': campo do registro/coleção em construção; recebe {'strategy': 'zero'}; 'dataset_sha256': hash dos bytes do CSV de entrada; recebe dataset; 'strategy': campo do registro/coleção em construção; recebe 'zero'. Os valores são avaliados nesta janela/iteração.

```
0023                              'source_sha256': 'same-code', 'n_windows': 1,
```

Define 'source_sha256': hash de todos os .py sob src; recebe 'same-code'; 'n_windows': campo do registro/coleção em construção; recebe 1. Os valores são avaliados nesta janela/iteração.

```
0024                              'model': {'model': model}, 'covariates': ['temperature']}
```

Define 'model': identidade do checkpoint/adaptador; recebe {'model': model}; 'covariates': nomes das variáveis auxiliares; recebe ['temperature']; 'model': identidade do checkpoint/adaptador; recebe model. Os valores são avaliados nesta janela/iteração.

```
0025                      status = {'failures': [], 'successful_windows': 1, 'expected_windows': 1}
```

Atribui a status o resultado de {'failures': [], 'successful_windows': 1, 'expected_windows': 1}. A expressão à direita é avaliada antes da atribuição.

```
0026                      (folder/'run.json').write_text(json.dumps(run))
```

Executa (folder / 'run.json').write_text(json.dumps(run)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

(folder / 'run.json').write_text — Grava uma string em arquivo e retorna a quantidade de caracteres escrita. Argumentos usados: posicional 1: json.dumps(run).

json.dumps — Serializa objeto Python em string JSON. Argumentos usados: posicional 1: run.

```
0027                     (folder/'status.json').write_text(json.dumps(status))
```

Executa (folder / 'status.json').write_text(json.dumps(status)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

(folder / 'status.json').write_text — Grava uma string em arquivo e retorna a quantidade de caracteres escrita. Argumentos usados: posicional 1: json.dumps(status).

json.dumps — Serializa objeto Python em string JSON. Argumentos usados: posicional 1: status.

```
0028                     pd.DataFrame({
```

Executa pd.DataFrame({'origin': ['2020-01-01'] * 7, 'timestamp': pd.date_range('2020-01-01', periods=7).strftime('%Y-%m-%d'), 'lead': range(1, 8), 'y_true': [10.0, 20.0, 30.0, 40.0, 50.0, 60.0, 70.0], 'y_pred': [10.0, 20.0, 40.0, 50.0, 60.0, 70.0, 80.0], 'initial_context': [initial] * 7, 'context': [15 if model == 'qwen' else initial] * 7, 'history_start': ['2018-01-01'] * 7, 'horizon': [7] * 7, 'elapsed_seconds_window': [1.0] * 7}).to_csv(folder / 'predictions.csv', index=False). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

pd.DataFrame({'origin': ['2020-01-01'] * 7, 'timestamp': pd.date_range('2020-01-01', periods=7).strftime('%Y-%m-%d'), 'lead': range(1, 8), 'y_true': [10.0, 20.0, 30.0, 40.0, 50.0, 60.0, 70.0], 'y_pred': [10.0, 20.0, 40.0, 50.0, 60.0, 70.0, 80.0], 'initial_context': [initial] * 7, 'context': [15 if model == 'qwen' else initial] * 7, 'history_start': ['2018-01-01'] * 7, 'horizon': [7] * 7, 'elapsed_seconds_window': [1.0] * 7}).to_csv — Serializa DataFrame como CSV. Argumentos usados: posicional 1: folder / 'predictions.csv'; index: False.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: {'origin': ['2020-01-01'] * 7, 'timestamp': pd.date_range('2020-01-01', periods=7).strftime('%Y-%m-%d'), 'lead': range(1, 8), 'y_true': [10.0, 20.0, 30.0, 40.0, 50.0, 60.0, 70.0], 'y_pred': [10.0, 20.0, 40.0, 50.0, 60.0, 70.0, 80.0], 'initial_context': [initial] * 7, 'context': [15 if model == 'qwen' else initial] * 7, 'history_start': ['2018-01-01'] * 7, 'horizon': [7] * 7, 'elapsed_seconds_window': [1.0] * 7}.

```
0029                         'origin': ['2020-01-01']*7,
```

Define 'origin': data de início do horizonte previsto; recebe ['2020-01-01'] * 7. Os valores são avaliados nesta janela/iteração.

```
0030                         'timestamp': pd.date_range('2020-01-01',
    periods=7).strftime('%Y-%m-%d'),
```

Define 'timestamp': campo do registro/coleção em construção; recebe pd.date_range('2020-01-01', periods=7).strftime('%Y-%m-%d'). Os valores são avaliados nesta janela/iteração.

pd.date_range('2020-01-01', periods=7).strftime — Formata datas como texto. Argumentos usados: posicional 1: '%Y-%m-%d'.

pd.date_range — Produz uma sequência ordenada de timestamps. Argumentos usados: posicional 1: '2020-01-01'; periods: 7.

```
0031                         'lead': range(1,8), 'y_true': [10.,20.,30.,40.,50.,60.,70.],
```

Define 'lead': posição de 1 a 7 dentro do horizonte; recebe range(1, 8); 'y_true': demanda observada usada somente na avaliação; recebe [10.0, 20.0, 30.0, 40.0, 50.0, 60.0, 70.0]. Os valores são avaliados nesta janela/iteração.

range — Gera inteiros com limite final exclusivo. Argumentos usados: posicional 1: 1; posicional 2: 8.

```
0032                         'y_pred': [10.,20.,40.,50.,60.,70.,80.],
```

Define 'y_pred': valor retornado pelo modelo para a data correspondente; recebe [10.0, 20.0, 40.0, 50.0, 60.0, 70.0, 80.0]. Os valores são avaliados nesta janela/iteração.

```
0033                         'initial_context': [initial]*7,
```

Define 'initial_context': origem inicial do protocolo de avaliação; recebe [initial] * 7. Os valores são avaliados nesta janela/iteração.

```
0034                         'context': [15 if model == 'qwen' else initial]*7,
```

Define 'context': número de linhas históricas efetivamente fornecidas; recebe [15 if model == 'qwen' else initial] * 7. Os valores são avaliados nesta janela/iteração.

```
0035                         'history_start': ['2018-01-01']*7,
```

Define 'history_start': primeira data histórica efetivamente fornecida; recebe ['2018-01-01'] * 7. Os valores são avaliados nesta janela/iteração.

```
0036                          'horizon': [7]*7, 'elapsed_seconds_window': [1.]*7
```

Define 'horizon': campo do registro/coleção em construção; recebe [7] * 7; 'elapsed_seconds_window': duração da janela, repetida em seus sete registros; recebe [1.0] * 7. Os valores são avaliados nesta janela/iteração.

```
0037                      }).to_csv(folder/'predictions.csv', index=False)
```

Continua os argumentos nomeados: index = False. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0038              with contextlib.redirect_stdout(io.StringIO()):
```

Abre o contexto gerenciado contextlib.redirect_stdout(io.StringIO()). Ao sair, libera o recurso ou restaura a simulação/redirecionamento, inclusive se ocorrer erro.

contextlib.redirect_stdout — Redireciona print temporariamente. Argumentos usados: posicional 1: io.StringIO().

io.StringIO — Buffer de texto em memória com interface de arquivo. Argumentos usados: nenhum argumento explícito.

```
0039                  compare(root)
```

Executa compare(root). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

compare — output: pasta com resultados dos três modelos/cidades. Sem retorno útil; valida e grava métricas globais e por lead. Argumentos usados: posicional 1: root.

```
0040              aggregate = pd.read_csv(root/'metrics_comparison.csv')
```

Atribui a aggregate o resultado de pd.read_csv(root / 'metrics_comparison.csv'). A expressão à direita é avaliada antes da atribuição.

pd.read_csv — Lê CSV em um DataFrame, inferindo tipos; datas só são convertidas posteriormente no validador. Argumentos usados: posicional 1: root / 'metrics_comparison.csv'.

```
0041              by_day = pd.read_csv(root/'metrics_by_horizon.csv')
```

Atribui a by_day o resultado de pd.read_csv(root / 'metrics_by_horizon.csv'). A expressão à direita é avaliada antes da atribuição.

pd.read_csv — Lê CSV em um DataFrame, inferindo tipos; datas só são convertidas posteriormente no validador. Argumentos usados: posicional 1: root / 'metrics_by_horizon.csv'.

```
0042              self.assertEqual(len(aggregate), 6)
```

Executa self.assertEqual(len(aggregate), 6). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: len(aggregate); posicional 2: 6.

len — Conta elementos. Argumentos usados: posicional 1: aggregate.

```
0043              self.assertEqual(len(by_day), 42)
```

Executa self.assertEqual(len(by_day), 42). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: len(by_day); posicional 2: 42.

len — Conta elementos. Argumentos usados: posicional 1: by_day.

```
0044              np.testing.assert_allclose(aggregate.rmse, np.sqrt(500/7))
```

Executa np.testing.assert_allclose(aggregate.rmse, np.sqrt(500 / 7)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

np.testing.assert_allclose — Asserção numérica vetorial tolerante. Argumentos usados: posicional 1: aggregate.rmse; posicional 2: np.sqrt(500 / 7).

np.sqrt — Raiz quadrada elemento a elemento ou escalar. Argumentos usados: posicional 1: 500 / 7.

```
0045              np.testing.assert_allclose(by_day.loc[by_day.lead <= 2, 'rmse'], 0.)
```

Executa np.testing.assert_allclose(by_day.loc[by_day.lead <= 2, 'rmse'], 0.0). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

np.testing.assert_allclose — Asserção numérica vetorial tolerante. Argumentos usados: posicional 1: by_day.loc[by_day.lead <= 2, 'rmse']; posicional 2: 0.0.

```
0046             np.testing.assert_allclose(by_day.loc[by_day.lead >= 3, 'rmse'], 10.)
```

Executa np.testing.assert_allclose(by_day.loc[by_day.lead >= 3, 'rmse'], 10.0). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

np.testing.assert_allclose — Asserção numérica vetorial tolerante. Argumentos usados: posicional 1: by_day.loc[by_day.lead >= 3, 'rmse']; posicional 2: 10.0.

```
0047             self.assertEqual(set(by_day.lead), set(range(1,8)))
```

Executa self.assertEqual(set(by_day.lead), set(range(1, 8))). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: set(by_day.lead); posicional 2: set(range(1, 8)).

set — Cria conjunto de elementos únicos. Argumentos usados: posicional 1: by_day.lead.

set — Cria conjunto de elementos únicos. Argumentos usados: posicional 1: range(1, 8).

range — Gera inteiros com limite final exclusivo. Argumentos usados: posicional 1: 1; posicional 2: 8.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

37. experiments/qwen_context_comparison/audit/audit_run.py

Auditoria offline das respostas já salvas. Reconstrói prompts, hashes e alvos; aplica mutação sentinela nos alvos futuros; recalcula erros e produz relatórios por janela. Não altera os resultados originais nem chama Ollama.

SHA-256: 6def8de2666618f7058ddbdd9648447dfef7ce79e71fc75c0a7c6340dbe15e85

Funções, contratos e pontos de conexão

```
L21: check(condition, name, **where)
```

condition: condição auditada; name: nome da checagem; **where: dataset, policy e origin. Acrescenta resultado à lista checks; falha não interrompe o restante da auditoria.

APIs externas usadas neste arquivo

Path.cwd, all, any, append, asdict, bool, check, copy, date, dict, duplicated, encode, equals, float, get, groupby, hashlib.sha256, head, hexdigest, items, itertuples, json.dumps, json.loads, len, max, mean, min, np.abs, np.allclose, np.array_equal, np.diff, np.dot, np.isclose, np.mean, np.sqrt, pd.DataFrame, pd.read_csv, print, query, read_bytes, read_text, reset_index, sort_values, sorted, str, strftime, sum, tail, to_csv, to_numpy, to_string, tolist, write_text. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Audit saved runs without invoking models or modifying inference sources."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  from pathlib import Path
```

Importa Path de pathlib. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0003  import hashlib
```

Importa hashlib. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0004  import json
```

Importa json. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  from dataclasses import asdict
```

Importa asdict de dataclasses. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0006  import numpy as np
```

Importa numpy como np. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0007  import pandas as pd
```

Importa pandas como pd. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0008  from src.pipelines.data_pipeline import prepare_dataset
```

Importa prepare_dataset de src.pipelines.data_pipeline. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0009  from src.pipelines.forecasting_pipeline import fingerprint, source_fingerprint
```

Importa fingerprint, source_fingerprint de src.pipelines.forecasting_pipeline. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0010  from src.data.features.covariates import model_inputs
```

Importa model_inputs de src.data.features.covariates. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0011  from src.data.preprocessing.generate_prompts import build_tabular_prompt
```

Importa build_tabular_prompt de src.data.preprocessing.generate_prompts. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0012  from src.models.promptcast import parse_prediction
```

Importa parse_prediction de src.models.promptcast. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0013  from src.data.ingestion.files import read_series
```

Importa read_series de src.data.ingestion.files. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0014  from src.data.preprocessing.resampling import daily_grid
```

Importa daily_grid de src.data.preprocessing.resampling. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0015  from src.data.preprocessing.missing_values import fill_past_only
```

Importa fill_past_only de src.data.preprocessing.missing_values. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0017  ROOT=Path.cwd()
```

Atribui a ROOT o resultado de Path.cwd(). A expressão à direita é avaliada antes da atribuição.

Path.cwd — Obtém o diretório de trabalho atual do processo. Argumentos usados: nenhum argumento explícito.

```
0018  BASE=ROOT/'experiments/qwen_context_comparison'
```

Atribui a BASE o resultado de ROOT / 'experiments/qwen_context_comparison'. A expressão à direita é avaliada antes da atribuição.

```
0019  OUT=BASE/'audit'
```

Atribui a OUT o resultado de BASE / 'audit'. A expressão à direita é avaliada antes da atribuição.

```
0020  checks=[]; window_rows=[]; summary=[]; examples=[]
```

Atribui a checks o resultado de []. A expressão à direita é avaliada antes da atribuição. Atribui a window_rows o resultado de []. A expressão à direita é avaliada antes da atribuição. Atribui a summary o resultado de []. A expressão à direita é avaliada antes da atribuição. Atribui a examples o resultado de []. A expressão à direita é avaliada antes da atribuição.

```
0021  def check(condition, name, **where):
```

Define check(condition, name, **where). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0022      checks.append(dict(check=name, passed=bool(condition), **where))
```

Executa checks.append(dict(check=name, passed=bool(condition), **where)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

checks.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: dict(check=name, passed=bool(condition), **where).

dict — Cria dicionário. Argumentos usados: check: name; passed: bool(condition); **desempacotamento: where.

bool — Converte condição escalar em booleano Python. Argumentos usados: posicional 1: condition.

```
0024  for dataset in ('joinville','guaratuba'):
```

Percorre ('joinville', 'guaratuba'); em cada iteração associa o item a dataset. O corpo indentado aplica a operação a esse item/grupo.

```
0025      paired={}
```

Atribui a paired o resultado de {}. A expressão à direita é avaliada antes da atribuição.

```
0026      for policy in ('15_days','expanding'):
```

Percorre ('15_days', 'expanding'); em cada iteração associa o item a policy. O corpo indentado aplica a operação a esse item/grupo.

```
0027          folder=BASE/policy/dataset/'qwen'
```

Atribui a folder o resultado de BASE / policy / dataset / 'qwen'. A expressão à direita é avaliada antes da atribuição.

```
0028          run=json.loads((folder/'run.json').read_text()); config=run['config'];
    spec=config['datasets'][dataset]
```

Atribui a run o resultado de json.loads((folder / 'run.json').read_text()). A expressão à direita é avaliada antes da atribuição. Atribui a config o resultado de run['config']. A expressão à direita é avaliada antes da atribuição. Atribui a spec o resultado de config['datasets'][dataset]. A expressão à direita é avaliada antes da atribuição.

json.loads — Decodifica uma string JSON em objetos Python. Argumentos usados: posicional 1: (folder / 'run.json').read_text().

(folder / 'run.json').read_text — Lê todo o arquivo como string. Argumentos usados: nenhum argumento explícito.

```
0029
    frame=pd.read_csv(folder/'predictions.csv').sort_values(['origin','lead']).reset_index(drop=Tru
    e)
```

Atribui a frame o resultado de pd.read_csv(folder / 'predictions.csv').sort_values(['origin', 'lead']).reset_index(drop=True). A expressão à direita é avaliada antes da atribuição.

pd.read_csv(folder / 'predictions.csv').sort_values(['origin', 'lead']).reset_index — Transforma o índice atual em coluna ou o descarta e cria índice sequencial. Argumentos usados: drop: True.

pd.read_csv(folder / 'predictions.csv').sort_values — Ordena linhas pelos valores de colunas. Argumentos usados: posicional 1: ['origin', 'lead'].

pd.read_csv — Lê CSV em um DataFrame, inferindo tipos; datas só são convertidas posteriormente no validador. Argumentos usados: posicional 1: folder / 'predictions.csv'.

```
0030          paired[policy]=frame
```

Atribui a paired[policy] o resultado de frame. A expressão à direita é avaliada antes da atribuição.

```
0031          df,windows=prepare_dataset(ROOT/spec['path'],spec['context'],horizon=7,step=7)
```

Atribui a (df, windows) o resultado de prepare_dataset(ROOT / spec['path'], spec['context'], horizon=7, step=7). A expressão à direita é avaliada antes da atribuição.

prepare_dataset — path: CSV; context: primeira origem; horizon=7 e step=7 nomeados. Retorna df e windows depois da preparação. Argumentos usados: posicional 1: ROOT / spec['path']; posicional 2: spec['context']; horizon: 7; step: 7.

```
0032          raw=read_series(ROOT/spec['path'])
```

Atribui a raw o resultado de read_series(ROOT / spec['path']). A expressão à direita é avaliada antes da atribuição.

read_series — path: caminho CSV; retorno: DataFrame validado. Encadeia pandas.read_csv e validate_series. Argumentos usados: posicional 1: ROOT / spec['path'].

```
0033          where=dict(dataset=dataset,policy=policy)
```

Atribui a where o resultado de dict(dataset=dataset, policy=policy). A expressão à direita é avaliada antes da atribuição.

dict — Cria dicionário. Argumentos usados: dataset: dataset; policy: policy.

```
0034          check(run['source_sha256']==source_fingerprint(),'source_hash',**where)
```

Executa check(run['source_sha256'] == source_fingerprint(), 'source_hash', **where). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

check — Registra uma checagem da auditoria sem interromper o laço. Argumentos usados: posicional 1: run['source_sha256'] == source_fingerprint(); posicional 2: 'source_hash'; **desempacotamento: where.

source_fingerprint — Sem parâmetros. Retorna digest incremental de nomes e bytes de todos os .py de src, em ordem fixa. Argumentos usados: nenhum argumento explícito.

```
0035          check(run['dataset_sha256']==hashlib.sha256((ROOT/spec['path']).read_bytes()).hexdige
    st(),'dataset_hash',**where)
```

Executa check(run['dataset_sha256'] == hashlib.sha256((ROOT / spec['path']).read_bytes()).hexdigest(), 'dataset_hash', **where). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

check — Registra uma checagem da auditoria sem interromper o laço. Argumentos usados: posicional 1: run['dataset_sha256'] == hashlib.sha256((ROOT / spec['path']).read_bytes()).hexdigest(); posicional 2: 'dataset_hash'; **desempacotamento: where.

hashlib.sha256((ROOT / spec['path']).read_bytes()).hexdigest — Retorna a representação hexadecimal do digest acumulado. Argumentos usados: nenhum argumento explícito.

hashlib.sha256 — Cria calculador de digest SHA-256. Argumentos usados: posicional 1: (ROOT / spec['path']).read_bytes().

(ROOT / spec['path']).read_bytes — Lê bytes integrais do arquivo. Argumentos usados: nenhum argumento explícito.

```
0036          check(run['run_id']==fingerprint({k:v for k,v in run.items() if
    k!='run_id'}),'run_hash',**where)
```

Executa check(run['run_id'] == fingerprint({k: v for k, v in run.items() if k != 'run_id'}), 'run_hash', **where). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

check — Registra uma checagem da auditoria sem interromper o laço. Argumentos usados: posicional 1: run['run_id'] == fingerprint({k: v for k, v in run.items() if k != 'run_id'}); posicional 2: 'run_hash'; **desempacotamento: where.

fingerprint — value: objeto serializável. Retorna SHA-256 hexadecimal do JSON com chaves ordenadas. Argumentos usados: posicional 1: {k: v for k, v in run.items() if k != 'run_id'}.

run.items — Percorre pares (chave, valor) do dicionário. Argumentos usados: nenhum argumento explícito.

```
0037          check(len(frame)==7*len(windows) and not
    frame.duplicated(['origin','lead']).any(),'complete_unique_predictions',**where)
```

Executa check(len(frame) == 7 * len(windows) and (not frame.duplicated(['origin', 'lead']).any()), 'complete_unique_predictions', **where). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

check — Registra uma checagem da auditoria sem interromper o laço. Argumentos usados: posicional 1: len(frame) == 7 * len(windows) and (not frame.duplicated(['origin', 'lead']).any()); posicional 2: 'complete_unique_predictions'; **desempacotamento: where.

len — Conta elementos. Argumentos usados: posicional 1: frame.

frame.duplicated(['origin', 'lead']).any — Reduz a existência de pelo menos um True. Argumentos usados: nenhum argumento explícito.

len — Conta elementos. Argumentos usados: posicional 1: windows.

frame.duplicated — Marca repetições. Argumentos usados: posicional 1: ['origin', 'lead'].

```
0038          for w in windows:
```

Percorre windows; em cada iteração associa o item a w. O corpo indentado aplica a operação a esse item/grupo.

```
0039              origin=str(df.timestamp.iloc[w.origin].date()); loc=dict(**where,origin=origin)
```

Atribui a origin o resultado de str(df.timestamp.iloc[w.origin].date()). A expressão à direita é avaliada antes da atribuição. Atribui a loc o resultado de dict(**where, origin=origin). A expressão à direita é avaliada antes da atribuição.

str — Produz representação textual. Argumentos usados: posicional 1: df.timestamp.iloc[w.origin].date().

dict — Cria dicionário. Argumentos usados: **desempacotamento: where; origin: origin.

df.timestamp.iloc[w.origin].date — Extrai a parte civil da data de um Timestamp. Argumentos usados: nenhum argumento explícito.

```
0040              saved=json.loads((folder/'responses'/f'{origin}.json').read_text())
```

Atribui a saved o resultado de json.loads((folder / 'responses' / f'{origin}.json').read_text()). A expressão à direita é avaliada antes da atribuição.

json.loads — Decodifica uma string JSON em objetos Python. Argumentos usados: posicional 1: (folder / 'responses' / f'{origin}.json').read_text().

(folder / 'responses' / f'{origin}.json').read_text — Lê todo o arquivo como string. Argumentos usados: nenhum argumento explícito.

```
0041              h,f=model_inputs(df,w,spec,history_days=config['qwen_history_days'])
```

Atribui a (h, f) o resultado de model_inputs(df, w, spec, history_days=config['qwen_history_days']). A expressão à direita é avaliada antes da atribuição.

model_inputs — df: série completa; window: ForecastWindow; dataset: contrato; history_days: None para expansivo ou inteiro para últimos N dias. Retorna (history, future), excluindo alvo do futuro. Argumentos usados: posicional 1: df; posicional 2: w; posicional 3: spec; history_days: config['qwen_history_days'].

```
0042              p=build_tabular_prompt(h,7,future_covariates=f)
```

Atribui a p o resultado de build_tabular_prompt(h, 7, future_covariates=f). A expressão à direita é avaliada antes da atribuição.

build_tabular_prompt — history: DataFrame histórico; horizon: dias; future_covariates: obrigatório e nomeado. Retorna CSV histórico unificado e tabela futura, com prefixo estável para história expansiva. Argumentos usados: posicional 1: h; posicional 2: 7; future_covariates: f.

```
0043              request=saved['audit']['request'];
    answer=saved['audit']['attempts'][-1]['response']
```

Atribui a request o resultado de saved['audit']['request']. A expressão à direita é avaliada antes da atribuição. Atribui a answer o resultado de saved['audit']['attempts'][-1]['response']. A expressão à direita é avaliada antes da atribuição.

```
0044              observed=frame[frame.origin==origin].sort_values('lead')
```

Atribui a observed o resultado de frame[frame.origin == origin].sort_values('lead'). A expressão à direita é avaliada antes da atribuição.

frame[frame.origin == origin].sort_values — Ordena linhas pelos valores de colunas. Argumentos usados: posicional 1: 'lead'.

```
0045              check(p==(BASE/policy/dataset/'prompts'/f'{origin}.txt').read_text()==request['pr
    ompt'],'exact_prompt_reconstruction',**loc)
```

Executa check(p == (BASE / policy / dataset / 'prompts' / f'{origin}.txt').read_text() == request['prompt'], 'exact_prompt_reconstruction', **loc). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

check — Registra uma checagem da auditoria sem interromper o laço. Argumentos usados: posicional 1: p == (BASE / policy / dataset / 'prompts' / f'{origin}.txt').read_text() == request['prompt']; posicional 2: 'exact_prompt_reconstruction'; **desempacotamento: loc.

(BASE / policy / dataset / 'prompts' / f'{origin}.txt').read_text — Lê todo o arquivo como string. Argumentos usados: nenhum argumento explícito.

```
0046              check(saved['request_id']==fingerprint(dict(run_id=run['run_id'],window=asdict(w)
    ,prompt=p)),'request_cache_identity',**loc)
```

Executa check(saved['request_id'] == fingerprint(dict(run_id=run['run_id'], window=asdict(w), prompt=p)), 'request_cache_identity', **loc). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

check — Registra uma checagem da auditoria sem interromper o laço. Argumentos usados: posicional 1: saved['request_id'] == fingerprint(dict(run_id=run['run_id'], window=asdict(w), prompt=p)); posicional 2: 'request_cache_identity'; **desempacotamento: loc.

fingerprint — value: objeto serializável. Retorna SHA-256 hexadecimal do JSON com chaves ordenadas. Argumentos usados: posicional 1: dict(run_id=run['run_id'], window=asdict(w), prompt=p).

dict — Cria dicionário. Argumentos usados: run_id: run['run_id']; window: asdict(w); prompt: p.

asdict — Converte dataclass em dicionário de seus campos. Argumentos usados: posicional 1: w.

```
0047              check(saved['window']==asdict(w) and
    saved['history_rows']==len(h),'window_record',**loc)
```

Executa check(saved['window'] == asdict(w) and saved['history_rows'] == len(h), 'window_record', **loc). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

check — Registra uma checagem da auditoria sem interromper o laço. Argumentos usados: posicional 1: saved['window'] == asdict(w) and saved['history_rows'] == len(h); posicional 2: 'window_record'; **desempacotamento: loc.

asdict — Converte dataclass em dicionário de seus campos. Argumentos usados: posicional 1: w.

len — Conta elementos. Argumentos usados: posicional 1: h.

```
0048             check(h.timestamp.max()<f.timestamp.min() and 'water_produced' not in
    f,'history_disjoint_future_target_absent',**loc)
```

Executa check(h.timestamp.max() < f.timestamp.min() and 'water_produced' not in f, 'history_disjoint_future_target_absent', **loc). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

check — Registra uma checagem da auditoria sem interromper o laço. Argumentos usados: posicional 1: h.timestamp.max() < f.timestamp.min() and 'water_produced' not in f; posicional 2: 'history_disjoint_future_target_absent'; **desempacotamento: loc.

h.timestamp.max — Retorna maior valor. Argumentos usados: nenhum argumento explícito.

f.timestamp.min — Retorna menor valor. Argumentos usados: nenhum argumento explícito.

```
0049             check(observed.timestamp.tolist()==df.iloc[w.target].timestamp.dt.strftime('%Y-
    %m-%d').tolist(),'forecast_dates_order',**loc)
```

Executa check(observed.timestamp.tolist() == df.iloc[w.target].timestamp.dt.strftime('%Y-%m-%d').tolist(), 'forecast_dates_order', **loc). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

check — Registra uma checagem da auditoria sem interromper o laço. Argumentos usados: posicional 1: observed.timestamp.tolist() == df.iloc[w.target].timestamp.dt.strftime('%Y-%m-%d').tolist(); posicional 2: 'forecast_dates_order'; **desempacotamento: loc.

observed.timestamp.tolist — Converte array/índice/Series em lista Python. Argumentos usados: nenhum argumento explícito.

df.iloc[w.target].timestamp.dt.strftime('%Y-%m-%d').tolist — Converte array/índice/Series em lista Python. Argumentos usados: nenhum argumento explícito.

df.iloc[w.target].timestamp.dt.strftime — Formata datas como texto. Argumentos usados: posicional 1: '%Y-%m-%d'.

```
0050
    check(np.array_equal(observed.y_true,df.iloc[w.target].water_produced),'ground_truth_exact',**l
    oc)
```

Executa check(np.array_equal(observed.y_true, df.iloc[w.target].water_produced), 'ground_truth_exact', **loc). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

check — Registra uma checagem da auditoria sem interromper o laço. Argumentos usados: posicional 1: np.array_equal(observed.y_true, df.iloc[w.target].water_produced); posicional 2: 'ground_truth_exact'; **desempacotamento: loc.

np.array_equal — Compara exatamente valores e formas de arrays. Argumentos usados: posicional 1: observed.y_true; posicional 2: df.iloc[w.target].water_produced.

```
0051             check(np.array_equal(observed.y_pred,saved['predictions']) and
    saved['predictions']==parse_prediction(answer['response'],7),'raw_response_equals_csv',**loc)
```

Executa check(np.array_equal(observed.y_pred, saved['predictions']) and saved['predictions'] == parse_prediction(answer['response'], 7), 'raw_response_equals_csv', **loc). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

check — Registra uma checagem da auditoria sem interromper o laço. Argumentos usados: posicional 1: np.array_equal(observed.y_pred, saved['predictions']) and saved['predictions'] == parse_prediction(answer['response'], 7); posicional 2: 'raw_response_equals_csv'; **desempacotamento: loc.

np.array_equal — Compara exatamente valores e formas de arrays. Argumentos usados: posicional 1: observed.y_pred; posicional 2: saved['predictions'].

parse_prediction — text: string JSON do modelo; horizon: quantidade exata de valores. Retorna list[float] ou lança erro; não remove prose nem aceita resposta parcial. Argumentos usados: posicional 1: answer['response']; posicional 2: 7.

```
0052              check(answer['done'] and answer.get('done_reason')!='length' and answer['prompt_e
    val_count']+answer['eval_count']<request['options']['num_ctx'],'generation_complete_below_limit
    ',**loc)
```

Executa check(answer['done'] and answer.get('done_reason') != 'length' and (answer['prompt_eval_count'] + answer['eval_count'] < request['options']['num_ctx']), 'generation_complete_below_limit', **loc). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

check — Registra uma checagem da auditoria sem interromper o laço. Argumentos usados: posicional 1: answer['done'] and answer.get('done_reason') != 'length' and (answer['prompt_eval_count'] + answer['eval_count'] < request['options']['num_ctx']); posicional 2: 'generation_complete_below_limit'; **desempacotamento: loc.

answer.get — Busca chave de dicionário com padrão se ausente. Argumentos usados: posicional 1: 'done_reason'.

```
0053
    check(len(p.encode())+768<=request['options']['num_ctx'],'full_prompt_budget',**loc)
```

Executa check(len(p.encode()) + 768 <= request['options']['num_ctx'], 'full_prompt_budget', **loc). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

check — Registra uma checagem da auditoria sem interromper o laço. Argumentos usados: posicional 1: len(p.encode()) + 768 <= request['options']['num_ctx']; posicional 2: 'full_prompt_budget'; **desempacotamento: loc.

len — Conta elementos. Argumentos usados: posicional 1: p.encode().

p.encode — Codifica string em bytes. Argumentos usados: nenhum argumento explícito.

```
0054              mutated=df.copy();
    mutated.loc[mutated.index>=w.origin,'water_produced']=987654321.123
```

Atribui a mutated o resultado de df.copy(). A expressão à direita é avaliada antes da atribuição. Atribui a mutated.loc[mutated.index >= w.origin, 'water_produced'] o resultado de 987654321.123. A expressão à direita é avaliada antes da atribuição.

df.copy — Copia uma tabela/dicionário para não alterar diretamente o objeto anterior. Argumentos usados: nenhum argumento explícito.

```
0055              hm,fm=model_inputs(mutated,w,spec,history_days=config['qwen_history_days'])
```

Atribui a (hm, fm) o resultado de model_inputs(mutated, w, spec, history_days=config['qwen_history_days']). A expressão à direita é avaliada antes da atribuição.

model_inputs — df: série completa; window: ForecastWindow; dataset: contrato; history_days: None para expansivo ou inteiro para últimos N dias. Retorna (history, future), excluindo alvo do futuro. Argumentos usados: posicional 1: mutated; posicional 2: w; posicional 3: spec; history_days: config['qwen_history_days'].

```
0056
    check(build_tabular_prompt(hm,7,future_covariates=fm)==p,'future_target_mutation_invariant',**l
    oc)
```

Executa check(build_tabular_prompt(hm, 7, future_covariates=fm) == p, 'future_target_mutation_invariant', **loc). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

check — Registra uma checagem da auditoria sem interromper o laço. Argumentos usados: posicional 1: build_tabular_prompt(hm, 7, future_covariates=fm) == p; posicional 2: 'future_target_mutation_invariant'; **desempacotamento: loc.

build_tabular_prompt — history: DataFrame histórico; horizon: dias; future_covariates: obrigatório e nomeado. Retorna CSV histórico unificado e tabela futura, com prefixo estável para história expansiva. Argumentos usados: posicional 1: hm; posicional 2: 7; future_covariates: fm.

```
0057              # Independent preprocessing on the prefix detects backfill/interpolation leakage.
```

Comentário de implementação: Independent preprocessing on the prefix detects backfill/interpolation leakage.. Não é executado; as operações efetivas são as linhas seguintes.

```
0058              past_only=fill_past_only(daily_grid(raw[raw.timestamp <
    df.timestamp.iloc[w.origin]]))
```

Atribui a past_only o resultado de fill_past_only(daily_grid(raw[raw.timestamp < df.timestamp.iloc[w.origin]])). A expressão à direita é avaliada antes da atribuição.

fill_past_only — df: grade com possíveis NaN. Retorno: cópia preenchida com passado. Não reconstrói atributos de calendário, limitação conhecida. Argumentos usados: posicional 1: daily_grid(raw[raw.timestamp < df.timestamp.iloc[w.origin]]).

daily_grid — df: tabela com timestamp. Retorno: grade diária com observed. Dias ausentes são expostos, não removidos. Argumentos usados: posicional 1: raw[raw.timestamp < df.timestamp.iloc[w.origin]].

```
0059              expected_history=past_only.tail(len(h))[h.columns].reset_index(drop=True)
```

Atribui a expected_history o resultado de past_only.tail(len(h))[h.columns].reset_index(drop=True). A expressão à direita é avaliada antes da atribuição.

past_only.tail(len(h))[h.columns].reset_index — Transforma o índice atual em coluna ou o descarta e cria índice sequencial. Argumentos usados: drop: True.

past_only.tail — Seleciona as últimas n linhas. Argumentos usados: posicional 1: len(h).

len — Conta elementos. Argumentos usados: posicional 1: h.

```
0060              check(h.reset_index(drop=True).equals(expected_history),'independent_past_only_pr
    eprocessing',**loc)
```

Executa check(h.reset_index(drop=True).equals(expected_history), 'independent_past_only_preprocessing', **loc). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

check — Registra uma checagem da auditoria sem interromper o laço. Argumentos usados: posicional 1: h.reset_index(drop=True).equals(expected_history); posicional 2: 'independent_past_only_preprocessing'; **desempacotamento: loc.

h.reset_index(drop=True).equals — Compara conteúdo e estrutura do receptor com outro objeto pandas. Argumentos usados: posicional 1: expected_history.

h.reset_index — Transforma o índice atual em coluna ou o descarta e cria índice sequencial. Argumentos usados: drop: True.

```
0061              pred=observed.y_pred.to_numpy(); actual=observed.y_true.to_numpy(); err=pred-
    actual
```

Atribui a pred o resultado de observed.y_pred.to_numpy(). A expressão à direita é avaliada antes da atribuição. Atribui a actual o resultado de observed.y_true.to_numpy(). A expressão à direita é avaliada antes da atribuição. Atribui a err o resultado de pred - actual. A expressão à direita é avaliada antes da atribuição.

observed.y_pred.to_numpy — Converte Series/DataFrame em array NumPy. Argumentos usados: nenhum argumento explícito.

observed.y_true.to_numpy — Converte Series/DataFrame em array NumPy. Argumentos usados: nenhum argumento explícito.

```
0062              diffs=np.diff(pred)
```

Atribui a diffs o resultado de np.diff(pred). A expressão à direita é avaliada antes da atribuição.

np.diff — Diferenças entre elementos consecutivos. Argumentos usados: posicional 1: pred.

```
0063              row=dict(**loc,context=len(h),history_start=str(h.timestamp.iloc[0].date()),histo
    ry_end=str(h.timestamp.iloc[-1].date()),
```

Atribui a row o resultado de dict(**loc, context=len(h), history_start=str(h.timestamp.iloc[0].date()), history_end=str(h.timestamp.iloc[-1].date()), rmse=float(np.sqrt(np.dot(err, err) / len(err))), mae=float(np.abs(err).mean()), bias=float(err.mean()), sse=float(np.dot(err, err)), actual_mean=float(actual.mean()), predicted_mean=float(pred.mean()), last7_mean=float(h.water_produced.tail(7).mean()), history_mean=float(h.water_produced.mean()), nondecreasing=bool(np.all(diffs >= 0)), strictly_increasing=bool(np.all(diffs > 0)), arithmetic_sequence=bool(np.allclose(diffs, diffs[0], atol=1e-06, rtol=0)), prompt_tokens=answer['prompt_eval_count'], output_tokens=answer['eval_count'], response_created_at=answer.get('created_at'), attempts=len(saved['audit']['attempts'])). A expressão à direita é avaliada antes da atribuição.

dict — Cria dicionário. Argumentos usados: **desempacotamento: loc; context: len(h); history_start: str(h.timestamp.iloc[0].date()); history_end: str(h.timestamp.iloc[-1].date()); rmse: float(np.sqrt(np.dot(err, err) / len(err))); mae: float(np.abs(err).mean()); bias: float(err.mean()); sse: float(np.dot(err, err)); actual_mean: float(actual.mean()); predicted_mean: float(pred.mean()); last7_mean: float(h.water_produced.tail(7).mean()); history_mean: float(h.water_produced.mean()); nondecreasing: bool(np.all(diffs >= 0)); strictly_increasing: bool(np.all(diffs > 0)); arithmetic_sequence: bool(np.allclose(diffs, diffs[0], atol=1e-06, rtol=0)); prompt_tokens: answer['prompt_eval_count']; output_tokens: answer['eval_count']; response_created_at: answer.get('created_at'); attempts: len(saved['audit']['attempts']).

len — Conta elementos. Argumentos usados: posicional 1: h.

str — Produz representação textual. Argumentos usados: posicional 1: h.timestamp.iloc[0].date().

str — Produz representação textual. Argumentos usados: posicional 1: h.timestamp.iloc[-1].date().

h.timestamp.iloc[0].date — Extrai a parte civil da data de um Timestamp. Argumentos usados: nenhum argumento explícito.

h.timestamp.iloc[-1].date — Extrai a parte civil da data de um Timestamp. Argumentos usados: nenhum argumento explícito.

```
0064
    rmse=float(np.sqrt(np.dot(err,err)/len(err))),mae=float(np.abs(err).mean()),bias=float(err.mean
    ()),
```

Continua os argumentos nomeados: rmse = float(np.sqrt(np.dot(err, err) / len(err))); mae = float(np.abs(err).mean()); bias = float(err.mean()). Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

float — Converte escalar para float Python. Argumentos usados: posicional 1: np.sqrt(np.dot(err, err) / len(err)).

float — Converte escalar para float Python. Argumentos usados: posicional 1: np.abs(err).mean().

float — Converte escalar para float Python. Argumentos usados: posicional 1: err.mean().

np.sqrt — Raiz quadrada elemento a elemento ou escalar. Argumentos usados: posicional 1: np.dot(err, err) / len(err).

np.abs(err).mean — Calcula média dos valores do objeto receptor. Argumentos usados: nenhum argumento explícito.

err.mean — Calcula média dos valores do objeto receptor. Argumentos usados: nenhum argumento explícito.

np.dot — Produto interno dos vetores. Argumentos usados: posicional 1: err; posicional 2: err.

len — Conta elementos. Argumentos usados: posicional 1: err.

np.abs — Módulo elemento a elemento. Argumentos usados: posicional 1: err.

```
0065
    sse=float(np.dot(err,err)),actual_mean=float(actual.mean()),predicted_mean=float(pred.mean()),
```

Continua os argumentos nomeados: sse = float(np.dot(err, err)); actual_mean = float(actual.mean()); predicted_mean = float(pred.mean()). Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

float — Converte escalar para float Python. Argumentos usados: posicional 1: np.dot(err, err).

float — Converte escalar para float Python. Argumentos usados: posicional 1: actual.mean().

float — Converte escalar para float Python. Argumentos usados: posicional 1: pred.mean().

np.dot — Produto interno dos vetores. Argumentos usados: posicional 1: err; posicional 2: err.

actual.mean — Calcula média dos valores do objeto receptor. Argumentos usados: nenhum argumento explícito.

pred.mean — Calcula média dos valores do objeto receptor. Argumentos usados: nenhum argumento explícito.

```
0066
    last7_mean=float(h.water_produced.tail(7).mean()),history_mean=float(h.water_produced.mean()),
```

Continua os argumentos nomeados: last7_mean = float(h.water_produced.tail(7).mean()); history_mean = float(h.water_produced.mean()). Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

float — Converte escalar para float Python. Argumentos usados: posicional 1: h.water_produced.tail(7).mean().

float — Converte escalar para float Python. Argumentos usados: posicional 1: h.water_produced.mean().

h.water_produced.tail(7).mean — Calcula média dos valores do objeto receptor. Argumentos usados: nenhum argumento explícito.

h.water_produced.mean — Calcula média dos valores do objeto receptor. Argumentos usados: nenhum argumento explícito.

h.water_produced.tail — Seleciona as últimas n linhas. Argumentos usados: posicional 1: 7.

```
0067
    nondecreasing=bool(np.all(diffs>=0)),strictly_increasing=bool(np.all(diffs>0)),
```

Continua os argumentos nomeados: nondecreasing = bool(np.all(diffs >= 0)); strictly_increasing = bool(np.all(diffs > 0)). Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

bool — Converte condição escalar em booleano Python. Argumentos usados: posicional 1: np.all(diffs >= 0).

bool — Converte condição escalar em booleano Python. Argumentos usados: posicional 1: np.all(diffs > 0).

np.all — Exige que todos os elementos sejam True. Argumentos usados: posicional 1: diffs >= 0.

np.all — Exige que todos os elementos sejam True. Argumentos usados: posicional 1: diffs > 0.

```
0068                arithmetic_sequence=bool(np.allclose(diffs,diffs[0],atol=1e-6,rtol=0)),
```

Continua os argumentos nomeados: arithmetic_sequence = bool(np.allclose(diffs, diffs[0], atol=1e-06, rtol=0)); atol = 1e-06; rtol = 0. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

bool — Converte condição escalar em booleano Python. Argumentos usados: posicional 1: np.allclose(diffs, diffs[0], atol=1e-06, rtol=0).

np.allclose — Exige proximidade em todos os elementos. Argumentos usados: posicional 1: diffs; posicional 2: diffs[0]; atol: 1e-06; rtol: 0.

```
0069                prompt_tokens=answer['prompt_eval_count'],output_tokens=answer['eval_count'],
```

Continua os argumentos nomeados: prompt_tokens = answer['prompt_eval_count']; output_tokens = answer['eval_count']. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0070
    response_created_at=answer.get('created_at'),attempts=len(saved['audit']['attempts']))
```

Continua os argumentos nomeados: response_created_at = answer.get('created_at'); attempts = len(saved['audit']['attempts']). Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

answer.get — Busca chave de dicionário com padrão se ausente. Argumentos usados: posicional 1: 'created_at'.

len — Conta elementos. Argumentos usados: posicional 1: saved['audit']['attempts'].

```
0071            window_rows.append(row)
```

Executa window_rows.append(row). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

window_rows.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: row.

```
0072            examples.append(dict(**loc,actual=actual.tolist(),predicted=pred.tolist()))
```

Executa examples.append(dict(**loc, actual=actual.tolist(), predicted=pred.tolist())). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

examples.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: dict(**loc, actual=actual.tolist(), predicted=pred.tolist()).

dict — Cria dicionário. Argumentos usados: **desempacotamento: loc; actual: actual.tolist(); predicted: pred.tolist().

actual.tolist — Converte array/índice/Series em lista Python. Argumentos usados: nenhum argumento explícito.

pred.tolist — Converte array/índice/Series em lista Python. Argumentos usados: nenhum argumento explícito.

```
0073        err=frame.y_pred.to_numpy()-frame.y_true.to_numpy(); y=frame.y_true.to_numpy()
```

Atribui a err o resultado de frame.y_pred.to_numpy() - frame.y_true.to_numpy(). A expressão à direita é avaliada antes da atribuição. Atribui a y o resultado de frame.y_true.to_numpy(). A expressão à direita é avaliada antes da atribuição.

frame.y_true.to_numpy — Converte Series/DataFrame em array NumPy. Argumentos usados: nenhum argumento explícito.

frame.y_pred.to_numpy — Converte Series/DataFrame em array NumPy. Argumentos usados: nenhum argumento explícito.

frame.y_true.to_numpy — Converte Series/DataFrame em array NumPy. Argumentos usados: nenhum argumento explícito.

```
0074        ws=[r for r in window_rows if r['dataset']==dataset and r['policy']==policy]
```

Atribui a ws o resultado de [r for r in window_rows if r['dataset'] == dataset and r['policy'] == policy]. A expressão à direita é avaliada antes da atribuição.

```
0075          metric=dict(**where,n=len(err),rmse=float(np.sqrt(np.dot(err,err)/len(err))),bias=flo
    at(err.mean()),
```

Atribui a metric o resultado de dict(**where, n=len(err), rmse=float(np.sqrt(np.dot(err, err) / len(err))), bias=float(err.mean()), mae=float(np.abs(err).mean()), r2=float(1 - np.dot(err, err) / np.dot(y - y.mean(), y - y.mean())), mape=float(np.mean(np.abs(err / y)) * 100), overprediction_fraction=float(np.mean(err > 0)), bias_fraction_mse=float(err.mean() ** 2 / np.mean(err ** 2)), nondecreasing_windows=sum((r['nondecreasing'] for r in ws)), arithmetic_windows=sum((r['arithmetic_sequence'] for r in ws)), windows=len(ws), tokens_min=min((r['prompt_tokens'] for r in ws)), tokens_max=max((r['prompt_tokens'] for r in ws)), top5_sse_fraction=float(sum(sorted([r['sse'] for r in ws], reverse=True)[:5]) / np.dot(err, err))). A expressão à direita é avaliada antes da atribuição.

dict — Cria dicionário. Argumentos usados: **desempacotamento: where; n: len(err); rmse: float(np.sqrt(np.dot(err, err) / len(err))); bias: float(err.mean()); mae: float(np.abs(err).mean()); r2: float(1 - np.dot(err, err) / np.dot(y - y.mean(), y - y.mean())); mape: float(np.mean(np.abs(err / y)) * 100); overprediction_fraction: float(np.mean(err > 0)); bias_fraction_mse: float(err.mean() ** 2 / np.mean(err ** 2)); nondecreasing_windows: sum((r['nondecreasing'] for r in ws)); arithmetic_windows: sum((r['arithmetic_sequence'] for r in ws)); windows: len(ws); tokens_min: min((r['prompt_tokens'] for r in ws)); tokens_max: max((r['prompt_tokens'] for r in ws)); top5_sse_fraction: float(sum(sorted([r['sse'] for r in ws], reverse=True)[:5]) / np.dot(err, err)).

len — Conta elementos. Argumentos usados: posicional 1: err.

float — Converte escalar para float Python. Argumentos usados: posicional 1: np.sqrt(np.dot(err, err) / len(err)).

float — Converte escalar para float Python. Argumentos usados: posicional 1: err.mean().

np.sqrt — Raiz quadrada elemento a elemento ou escalar. Argumentos usados: posicional 1: np.dot(err, err) / len(err).

err.mean — Calcula média dos valores do objeto receptor. Argumentos usados: nenhum argumento explícito.

np.dot — Produto interno dos vetores. Argumentos usados: posicional 1: err; posicional 2: err.

len — Conta elementos. Argumentos usados: posicional 1: err.

```
0076
    mae=float(np.abs(err).mean()),r2=float(1-np.dot(err,err)/np.dot(y-y.mean(),y-y.mean())),
```

Continua os argumentos nomeados: mae = float(np.abs(err).mean()); r2 = float(1 - np.dot(err, err) / np.dot(y - y.mean(), y - y.mean())). Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

float — Converte escalar para float Python. Argumentos usados: posicional 1: np.abs(err).mean().

float — Converte escalar para float Python. Argumentos usados: posicional 1: 1 - np.dot(err, err) / np.dot(y - y.mean(), y - y.mean()).

np.abs(err).mean — Calcula média dos valores do objeto receptor. Argumentos usados: nenhum argumento explícito.

np.abs — Módulo elemento a elemento. Argumentos usados: posicional 1: err.

np.dot — Produto interno dos vetores. Argumentos usados: posicional 1: err; posicional 2: err.

np.dot — Produto interno dos vetores. Argumentos usados: posicional 1: y - y.mean(); posicional 2: y - y.mean().

y.mean — Calcula média dos valores do objeto receptor. Argumentos usados: nenhum argumento explícito.

y.mean — Calcula média dos valores do objeto receptor. Argumentos usados: nenhum argumento explícito.

```
0077
    mape=float(np.mean(np.abs(err/y))*100),overprediction_fraction=float(np.mean(err>0)),
```

Continua os argumentos nomeados: mape = float(np.mean(np.abs(err / y)) * 100); overprediction_fraction = float(np.mean(err > 0)). Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

float — Converte escalar para float Python. Argumentos usados: posicional 1: np.mean(np.abs(err / y)) * 100.

float — Converte escalar para float Python. Argumentos usados: posicional 1: np.mean(err > 0).

np.mean — Calcula média aritmética. Argumentos usados: posicional 1: err > 0.

np.mean — Calcula média aritmética. Argumentos usados: posicional 1: np.abs(err / y).

np.abs — Módulo elemento a elemento. Argumentos usados: posicional 1: err / y.

```
0078             bias_fraction_mse=float(err.mean()**2/np.mean(err**2)),
```

Continua os argumentos nomeados: bias_fraction_mse = float(err.mean() ** 2 / np.mean(err ** 2)). Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

float — Converte escalar para float Python. Argumentos usados: posicional 1: err.mean() ** 2 / np.mean(err ** 2).

np.mean — Calcula média aritmética. Argumentos usados: posicional 1: err ** 2.

err.mean — Calcula média dos valores do objeto receptor. Argumentos usados: nenhum argumento explícito.

```
0079             nondecreasing_windows=sum(r['nondecreasing'] for r in
    ws),arithmetic_windows=sum(r['arithmetic_sequence'] for r in ws),
```

Continua os argumentos nomeados: nondecreasing_windows = sum((r['nondecreasing'] for r in ws)); arithmetic_windows = sum((r['arithmetic_sequence'] for r in ws)). Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

sum — Soma elementos de iterável. Argumentos usados: posicional 1: (r['nondecreasing'] for r in ws).

sum — Soma elementos de iterável. Argumentos usados: posicional 1: (r['arithmetic_sequence'] for r in ws).

```
0080             windows=len(ws),tokens_min=min(r['prompt_tokens'] for r in
    ws),tokens_max=max(r['prompt_tokens'] for r in ws),
```

Continua os argumentos nomeados: windows = len(ws); tokens_min = min((r['prompt_tokens'] for r in ws)); tokens_max = max((r['prompt_tokens'] for r in ws)). Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

len — Conta elementos. Argumentos usados: posicional 1: ws.

min — Retorna menor valor. Argumentos usados: posicional 1: (r['prompt_tokens'] for r in ws).

max — Retorna maior valor. Argumentos usados: posicional 1: (r['prompt_tokens'] for r in ws).

```
0081             top5_sse_fraction=float(sum(sorted([r['sse'] for r in
    ws],reverse=True)[:5])/np.dot(err,err)))
```

Continua os argumentos nomeados: top5_sse_fraction = float(sum(sorted([r['sse'] for r in ws], reverse=True)[:5]) / np.dot(err, err)); reverse = True. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

float — Converte escalar para float Python. Argumentos usados: posicional 1: sum(sorted([r['sse'] for r in ws], reverse=True)[:5]) / np.dot(err, err).

sum — Soma elementos de iterável. Argumentos usados: posicional 1: sorted([r['sse'] for r in ws], reverse=True)[:5].

np.dot — Produto interno dos vetores. Argumentos usados: posicional 1: err; posicional 2: err.

sorted — Devolve uma lista ordenada. Argumentos usados: posicional 1: [r['sse'] for r in ws]; reverse: True.

```
0082         summary.append(metric)
```

Executa summary.append(metric). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

summary.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: metric.

```
0083         stored=pd.read_csv(BASE/'metrics.csv').query('dataset == @dataset and context_policy
    == @policy').iloc[0]
```

Atribui a stored o resultado de pd.read_csv(BASE / 'metrics.csv').query('dataset == @dataset and context_policy == @policy').iloc[0]. A expressão à direita é avaliada antes da atribuição.

pd.read_csv(BASE / 'metrics.csv').query — Filtra linhas por expressão textual pandas. Argumentos usados: posicional 1: 'dataset == @dataset and context_policy == @policy'.

pd.read_csv — Lê CSV em um DataFrame, inferindo tipos; datas só são convertidas posteriormente no validador. Argumentos usados: posicional 1: BASE / 'metrics.csv'.

```
0084         check(np.isclose(stored.rmse,metric['rmse'],rtol=1e-12),'independent_rmse',**where)
```

Executa check(np.isclose(stored.rmse, metric['rmse'], rtol=1e-12), 'independent_rmse', **where). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

check — Registra uma checagem da auditoria sem interromper o laço. Argumentos usados: posicional 1: np.isclose(stored.rmse, metric['rmse'], rtol=1e-12); posicional 2: 'independent_rmse'; **desempacotamento: where.

np.isclose — Compara com tolerância numérica. Argumentos usados: posicional 1: stored.rmse; posicional 2: metric['rmse']; rtol: 1e-12.

```
0085      a,b=paired['15_days'],paired['expanding']
```

Atribui a (a, b) o resultado de (paired['15_days'], paired['expanding']). A expressão à direita é avaliada antes da atribuição.

```
0086      check(a[['origin','timestamp','lead','y_true']].equals(b[['origin','timestamp','lead','y_
    true']]),'paired_same_targets',dataset=dataset)
```

Executa check(a[['origin', 'timestamp', 'lead', 'y_true']].equals(b[['origin', 'timestamp', 'lead', 'y_true']]), 'paired_same_targets', dataset=dataset). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

check — Registra uma checagem da auditoria sem interromper o laço. Argumentos usados: posicional 1: a[['origin', 'timestamp', 'lead', 'y_true']].equals(b[['origin', 'timestamp', 'lead', 'y_true']]); posicional 2: 'paired_same_targets'; dataset: dataset.

a[['origin', 'timestamp', 'lead', 'y_true']].equals — Compara conteúdo e estrutura do receptor com outro objeto pandas. Argumentos usados: posicional 1: b[['origin', 'timestamp', 'lead', 'y_true']].

```
0088  pd.DataFrame(checks).to_csv(OUT/'checks.csv',index=False)
```

Executa pd.DataFrame(checks).to_csv(OUT / 'checks.csv', index=False). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

pd.DataFrame(checks).to_csv — Serializa DataFrame como CSV. Argumentos usados: posicional 1: OUT / 'checks.csv'; index: False.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: checks.

```
0089  wdf=pd.DataFrame(window_rows)
```

Atribui a wdf o resultado de pd.DataFrame(window_rows). A expressão à direita é avaliada antes da atribuição.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: window_rows.

```
0090  wdf.to_csv(OUT/'windows.csv',index=False)
```

Executa wdf.to_csv(OUT / 'windows.csv', index=False). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

wdf.to_csv — Serializa DataFrame como CSV. Argumentos usados: posicional 1: OUT / 'windows.csv'; index: False.

```
0091  pd.DataFrame(summary).to_csv(OUT/'summary.csv',index=False)
```

Executa pd.DataFrame(summary).to_csv(OUT / 'summary.csv', index=False). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

pd.DataFrame(summary).to_csv — Serializa DataFrame como CSV. Argumentos usados: posicional 1: OUT / 'summary.csv'; index: False.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: summary.

```
0092  worst=wdf.query("policy ==
    'expanding'").sort_values('rmse',ascending=False).groupby('dataset',sort=False).head(5)
```

Atribui a worst o resultado de wdf.query("policy == 'expanding'").sort_values('rmse', ascending=False).groupby('dataset', sort=False).head(5). A expressão à direita é avaliada antes da atribuição.

wdf.query("policy == 'expanding'").sort_values('rmse', ascending=False).groupby('dataset', sort=False).head — Seleciona as primeiras n linhas. Argumentos usados: posicional 1: 5.

wdf.query("policy == 'expanding'").sort_values('rmse', ascending=False).groupby — Cria grupos sem calcular métricas por si só. Argumentos usados: posicional 1: 'dataset'; sort: False.

wdf.query("policy == 'expanding'").sort_values — Ordena linhas pelos valores de colunas. Argumentos usados: posicional 1: 'rmse'; ascending: False.

wdf.query — Filtra linhas por expressão textual pandas. Argumentos usados: posicional 1: "policy == 'expanding'".

```
0093  worst.to_csv(OUT/'worst_windows.csv',index=False)
```

Executa worst.to_csv(OUT / 'worst_windows.csv', index=False). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

worst.to_csv — Serializa DataFrame como CSV. Argumentos usados: posicional 1: OUT / 'worst_windows.csv'; index: False.

```
0094  keys={(r.dataset,r.policy,r.origin) for r in worst.itertuples()}
```

Atribui a keys o resultado de {(r.dataset, r.policy, r.origin) for r in worst.itertuples()}. A expressão à direita é avaliada antes da atribuição.

worst.itertuples — Itera linhas como tuplas nomeadas. Argumentos usados: nenhum argumento explícito.

```
0095  (OUT/'worst_examples.json').write_text(json.dumps([r for r in examples if
    (r['dataset'],r['policy'],r['origin']) in keys],indent=2))
```

Executa (OUT / 'worst_examples.json').write_text(json.dumps([r for r in examples if (r['dataset'], r['policy'], r['origin']) in keys], indent=2)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

(OUT / 'worst_examples.json').write_text — Grava uma string em arquivo e retorna a quantidade de caracteres escrita. Argumentos usados: posicional 1: json.dumps([r for r in examples if (r['dataset'], r['policy'], r['origin']) in keys], indent=2).

json.dumps — Serializa objeto Python em string JSON. Argumentos usados: posicional 1: [r for r in examples if (r['dataset'], r['policy'], r['origin']) in keys]; indent: 2.

```
0096  print(pd.DataFrame(summary).to_string(index=False))
```

Executa print(pd.DataFrame(summary).to_string(index=False)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

print — Escreve representação textual na saída padrão. Argumentos usados: posicional 1: pd.DataFrame(summary).to_string(index=False).

pd.DataFrame(summary).to_string — Formata tabela como texto para terminal/log. Argumentos usados: index: False.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: summary.

```
0097  print('CHECKS',len(checks),'FAILED',sum(not c['passed'] for c in checks))
```

Executa print('CHECKS', len(checks), 'FAILED', sum((not c['passed'] for c in checks))). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

print — Escreve representação textual na saída padrão. Argumentos usados: posicional 1: 'CHECKS'; posicional 2: len(checks); posicional 3: 'FAILED'; posicional 4: sum((not c['passed'] for c in checks)).

len — Conta elementos. Argumentos usados: posicional 1: checks.

sum — Soma elementos de iterável. Argumentos usados: posicional 1: (not c['passed'] for c in checks).

```
0098  print(pd.DataFrame([c for c in checks if not c['passed']]).to_string(index=False))
```

Executa print(pd.DataFrame([c for c in checks if not c['passed']]).to_string(index=False)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

print — Escreve representação textual na saída padrão. Argumentos usados: posicional 1: pd.DataFrame([c for c in checks if not c['passed']]).to_string(index=False).

pd.DataFrame([c for c in checks if not c['passed']]).to_string — Formata tabela como texto para terminal/log. Argumentos usados: index: False.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: [c for c in checks if not c['passed']].

```
0099  print('WORST WINDOWS\n',worst[['dataset','origin','rmse','bias','actual_mean','predicted_mean
    ','last7_mean']].to_string(index=False))
```

Executa print('WORST WINDOWS\n', worst[['dataset', 'origin', 'rmse', 'bias', 'actual_mean', 'predicted_mean', 'last7_mean']].to_string(index=False)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

print — Escreve representação textual na saída padrão. Argumentos usados: posicional 1: 'WORST WINDOWS\n'; posicional 2: worst[['dataset', 'origin', 'rmse', 'bias', 'actual_mean', 'predicted_mean', 'last7_mean']].to_string(index=False).

worst[['dataset', 'origin', 'rmse', 'bias', 'actual_mean', 'predicted_mean', 'last7_mean']].to_string — Formata tabela como texto para terminal/log. Argumentos usados: index: False.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

Catálogo de funções e métodos de biblioteca

Esta referência explica as APIs efetivamente chamadas. Em cada ocorrência na listagem, os argumentos concretos foram mostrados, inclusive os posicionais. Aqui estão seu significado e os parâmetros relevantes. Métodos homônimos (como update de dict e de hash) dependem do objeto receptor. O catálogo não deve ser lido como lista de todos os parâmetros opcionais que a biblioteca admite.

ChronosForecaster

Constrói adaptador Chronos2 com pesos locais.

Parâmetros e uso neste projeto: context é contexto inicial registrado; dataset fornece nomes de covariáveis. predict depois recebe história crescente de verdade.

Demonstration

Constrói dataclass de exemplo few-shot.

Parâmetros e uso neste projeto: history e target são tuplas de números; target_end é posição final exclusiva dentro do prefixo autorizado.

ForecastWindow

Constrói dataclass de uma origem temporal.

Parâmetros e uso neste projeto: context_start=0, origin=posição do primeiro alvo, horizon=sete dias. history/target são properties que devolvem slices.

Mock

Cria objeto simulado para registrar chamadas.

Parâmetros e uso neste projeto: Sem argumentos neste teste. pipeline.predict_df.return_value é atribuído ao DataFrame artificial que predict receberá.

OllamaConfig

Constrói configuração tipada do cliente Qwen.

Parâmetros e uso neste projeto: Campos do JSON qwen e seed são argumentos nomeados. Os defaults legados só valem para campos omitidos; __post_init__ valida limites.

Path

Representa caminho de arquivo com operações portáveis.

Parâmetros e uso neste projeto: path posicional é string ou Path. O operador / combina diretórios, não faz divisão numérica nesse contexto.

Path.cwd

Obtém o diretório de trabalho atual do processo.

Parâmetros e uso neste projeto: Sem argumentos. Na auditoria, pressupõe execução a partir da raiz do repositório; diferente de ROOT derivado de __file__, depende da pasta de lançamento.

QwenForecaster

Constrói adaptador Qwen e consulta servidor local.

Parâmetros e uso neste projeto: config é OllamaConfig. Valida modelo/contexto e cria identity; não chama geração até predict.

RuntimeError

Cria exceção para estado de execução inesperado.

Parâmetros e uso neste projeto: Mensagem sinaliza caminho que deveria ser inalcançável depois das tentativas.

SystemExit

Solicita encerramento do processo Python.

Parâmetros e uso neste projeto: code=0 indica sucesso; 1 sinaliza falhas para scripts shell. Usada com raise no CLI.

TimesFMForecaster

Constrói adaptador TimesFM2.5 local.

Parâmetros e uso neste projeto: context é contexto inicial registrado; dataset define covariáveis. Previsão usa XReg + TimesFM.

ValueError

Cria exceção para valor/formato inválido.

Parâmetros e uso neste projeto: A string explica a violação: datas, dimensões, covariáveis, JSON ou contexto. Só é lançada quando precedida por raise ou usada por teste de exceção.

add_argument

Registra uma opção do CLI.

Parâmetros e uso neste projeto: Primeiro argumento é '--model' etc.; choices restringe valores; default define ausência; type=Path/int converte; help descreve a opção no --help. type=Path não verifica se o arquivo existe.

all

Exige que todos os elementos sejam True.

Parâmetros e uso neste projeto: Series/ndarray.all() reduz a máscara; builtin all(iterável) pode receber um gerador. Num conjunto vazio o resultado é True: outras verificações precisam excluir entradas vazias.

any

Reduz a existência de pelo menos um True.

Parâmetros e uso neste projeto: Series/ndarray.any() sem eixo reduz seus valores; DataFrame.any() reduz por coluna e o segundo .any() reduz as colunas. O builtin any(iterável) percorre um gerador de condições e para no primeiro True.

append

Adiciona um elemento ao final da lista.

Parâmetros e uso neste projeto: item pode ser dicionário de previsão, auditoria ou bloco textual. Modifica a lista e retorna None.

apply

Aplica uma função por grupo/objeto.

Parâmetros e uso neste projeto: No grupo de leads, func=lambda x: sorted(x)==list(range(1,8)) exige uma ocorrência de cada lead; .all() exige isso em toda janela.

argparse.ArgumentParser

Cria o parser da interface de linha de comando.

Parâmetros e uso neste projeto: description=__doc__ utiliza a docstring do módulo como texto de ajuda; parse_args vai interpretar os argumentos depois.

asdict

Converte dataclass em dicionário de seus campos.

Parâmetros e uso neste projeto: obj=config ou window. É usado na procedência e request_id; não converte uma classe qualquer sem dataclass.

asfreq

Reindexa a série para uma frequência temporal regular.

Parâmetros e uso neste projeto: freq='D' cria posições diárias entre começo e fim; dias ausentes ganham NaN. Não soma nem calcula média como resample(...).mean().

assertAlmostEqual

Compara números com aproximação decimal.

Parâmetros e uso neste projeto: first/second são métricas e valores esperados; tolerância padrão do unittest se não informada.

assertEqual

Asserção de igualdade do unittest.

Parâmetros e uso neste projeto: first/second são valores comparados; testa contratos discretos, como tamanho, datas ou listas de previsões.

assertFalse

Asserção de condição falsa.

Parâmetros e uso neste projeto: expr verifica, por exemplo, que um dia imputado não está marcado como observado.

assertGreaterEqual

Exige first >= second.

Parâmetros e uso neste projeto: Usada para origem nunca anteceder o contexto inicial.

assertIn

Exige item presente em container.

Parâmetros e uso neste projeto: Verifica datas ou chave de erro na auditoria de uma retentativa.

assertIsNone

Exige que o resultado seja o objeto None.

Parâmetros e uso neste projeto: Valida redução percentual indefinida para erro-base zero.

assertLess

Exige first < second.

Parâmetros e uso neste projeto: Compara a maior data histórica com a menor data de teste para detectar sobreposição.

assertNotIn

Exige item ausente de container.

Parâmetros e uso neste projeto: Usada para coluna-alvo não estar em future, e sentinela futura não estar no prompt.

assertRaises

Context manager que exige uma exceção.

Parâmetros e uso neste projeto: expected_exception é classe ou tupla de classes; o trecho dentro de with precisa falhar de modo esperado.

assertRaisesRegex

Exige exceção e mensagem compatível com expressão regular.

Parâmetros e uso neste projeto: expected_exception e expected_regex distinguem a razão de falha, como mudança no modelo da comparação.

assertTrue

Asserção de condição verdadeira.

Parâmetros e uso neste projeto: expr é uma condição já calculada; falha se falsa. Não corrige a condição.

astype

Converte o tipo dos valores.

Parâmetros e uso neste projeto: dtype=str transforma categorias em texto para TimesFM. Não faz one-hot encoding; a biblioteca recebe as categorias para processar.

bool

Converte condição escalar em booleano Python.

Parâmetros e uso neste projeto: Usado para serializar resultados de reduções NumPy; não usar em array com vários elementos sem .all/.any.

check

Registra uma checagem da auditoria sem interromper o laço.

Parâmetros e uso neste projeto: condition vira bool; name identifica teste; **where fornece cidade, política e origem. Lista checks preserva evidência de sucesso/falha.

choice

Sorteia elementos do gerador NumPy.

Parâmetros e uso neste projeto: a=origins, size=count e replace=False selecionam origens diferentes; não sorteia do período de teste.

compile

Prepara a configuração de previsão do TimesFM.

Parâmetros e uso neste projeto: forecast_config é ForecastConfig. O nome não significa treinamento ou alteração dos pesos. É chamado novamente por origem para capacidade compatível com história

crescente.

contextlib.redirect_stdout

Redireciona print temporariamente.

Parâmetros e uso neste projeto: new_target=io.StringIO() captura saída em memória, para testes não poluírem o terminal.

copy

Copia uma tabela/dicionário para não alterar diretamente o objeto anterior.

Parâmetros e uso neste projeto: DataFrame.copy() usa deep=True por padrão; para objetos Python dentro de células não significa cópia recursiva de tudo. dict.copy é superficial. Não recebe argumentos nas chamadas deste projeto.

dataclass

Decora a classe para gerar métodos a partir dos campos anotados.

Parâmetros e uso neste projeto: frozen=True impede reatribuição comum após construção. Não executa treinamento, nem valida sozinho os tipos anotados.

date

Extrai a parte civil da data de um Timestamp.

Parâmetros e uso neste projeto: Sem argumentos. O retorno é date, que a f-string representa como YYYY-MM-DD, sem horário.

dict

Cria dicionário.

Parâmetros e uso neste projeto: Pode receber mapeamento posicional, campos nomeados e **outro_dict. dict(config) é cópia superficial; pop remove só a chave nessa cópia.

drop

Retorna tabela sem as linhas/colunas indicadas.

Parâmetros e uso neste projeto: columns='water_produced' nos testes remove o alvo do futuro; inplace padrão False mantém o original intacto.

drop_duplicates

Retém uma linha por chave repetida.

Parâmetros e uso neste projeto: subset='origin' retém o primeiro registro de cada janela. Serve para não somar sete vezes a duração, repetida nas sete linhas do CSV.

duplicated

Marca repetições.

Parâmetros e uso neste projeto: Em DataFrame, subset=['origin','lead'] define a chave; em Series de datas não recebe subset. keep padrão 'first' preserva a primeira ocorrência como não duplicada.

encode

Codifica string em bytes.

Parâmetros e uso neste projeto: encoding='utf-8' no HTTP e orçamento do prompt; sem argumento, str.encode usa UTF-8. Bytes são usados por SHA-256 e corpo HTTP.

enumerate

Percorre iterável com contador.

Parâmetros e uso neste projeto: iterable e start=1 nos leads/janelas; o contador é separado do índice temporal do dataset.

eq

Compara igualdade elemento a elemento.

Parâmetros e uso neste projeto: other é uma Series compatível ou escalar, como 50 no teste da redução de RMSE; retorna máscara, que .all() agrega.

equals

Compara conteúdo e estrutura do receptor com outro objeto pandas.

Parâmetros e uso neste projeto: other é DataFrame ou DatetimeIndex compatível. Retorna bool; datas/ordem precisam coincidir. Não alinha ou reordena automaticamente para fazer as tabelas parecerem iguais.

error

Em ArgumentParser, mostra mensagem e encerra com erro de CLI.

Parâmetros e uso neste projeto: message é o motivo, por exemplo limite negativo. Não é o erro numérico de uma previsão.

exists

Verifica existência do caminho.

Parâmetros e uso neste projeto: Sem argumentos. Define se vale tentar ler cache já gravado.

extend

Adiciona cada elemento de um iterável à lista.

Parâmetros e uso neste projeto: iterable é lista de blocos de prompt. Diferente de append, não cria uma lista aninhada.

ffill

Propaga o último valor válido anterior para lacunas subsequentes.

Parâmetros e uso neste projeto: Sem argumentos. Não usa uma observação futura, mas não reconstrói calendário: copiar is_weekend/day para uma nova data é incorreto.

float

Converte escalar para float Python.

Parâmetros e uso neste projeto: x é valor numérico ou NumPy. Facilita serialização em JSON; não normaliza nem muda a unidade.

forecast_with_covariates

Executa regressão auxiliar e TimesFM com variáveis exógenas.

Parâmetros e uso neste projeto: inputs é lista de séries-alvo históricas; dynamic_numerical_covariates e dynamic_categorical_covariates são dicts com listas por série cobrindo passado+futuro; xreg_mode='xreg + timesfm' combina regressão e previsão residual. Retorna pontos e saídas adicionais; o segundo retorno é ignorado aqui.

from_pretrained

Carrega pesos/configuração de um checkpoint.

Parâmetros e uso neste projeto: Primeiro argumento é ID do modelo; local_files_only=True exige cache local. Em Chronos, device_map seleciona dispositivo e torch_dtype=torch.float32 força pesos nessa precisão. Em TimesFM usa a classe própria TimesFM_2p5_200M_torch.

get

Busca chave de dicionário com padrão se ausente.

Parâmetros e uso neste projeto: key é done/response/status; default é 0 ou '' quando informado. Evita KeyError, mas padrão 0 nos contadores não prova que servidor os forneceu.

groupby

Cria grupos sem calcular métricas por si só.

Parâmetros e uso neste projeto: by='lead' separa antecedências; by='origin' separa janelas; sort=False preserva ordem encontrada na auditoria. É necessário iterar, apply ou agregar depois.

hashlib.sha256

Cria calculador de digest SHA-256.

Parâmetros e uso neste projeto: data opcional são bytes, não texto. Na versão incremental inicia vazio e recebe update; no hash de configuração recebe JSON codificado.

head

Seleciona as primeiras n linhas.

Parâmetros e uso neste projeto: n=5 na auditoria limita as cinco piores janelas de cada grupo após ordenação.

hexdigest

Retorna a representação hexadecimal do digest acumulado.

Parâmetros e uso neste projeto: Sem argumentos. É texto para comparar identidades; não criptografa conteúdo para confidencialidade.

importlib.metadata.version

Lê versão instalada de uma distribuição Python.

Parâmetros e uso neste projeto: distribution_name é 'chronos-forecasting' ou 'timesfm'; não é necessariamente o mesmo nome do import. Pode falhar se a distribuição não estiver instalada.

int

Converte para inteiro, truncando parte fracionária numérica.

Parâmetros e uso neste projeto: x é contexto, tamanho da validação ou contagem; int(training_size*fraction) arredonda para baixo para positivos.

io.StringIO

Buffer de texto em memória com interface de arquivo.

Parâmetros e uso neste projeto: initial_value=table permite pd.read_csv sem arquivo físico; vazio serve para capturar prints.

isinstance

Verifica se um objeto é instância de um tipo ou tupla de tipos.

Parâmetros e uso neste projeto: parsed,dict e values,list validam formato do JSON. Para números o código prefere type(v) para excluir bool.

isna

Retorna máscara booleana de valores ausentes.

Parâmetros e uso neste projeto: Sem argumentos. Em DataFrame a máscara tem o mesmo formato; em Series tem uma entrada por linha.

issubset

Verifica se todos os elementos estão contidos em outro conjunto/iterável.

Parâmetros e uso neste projeto: other são colunas do DataFrame; valida presença do contrato sem exigir que não existam colunas extras.

items

Percorre pares (chave, valor) do dicionário.

Parâmetros e uso neste projeto: Sem argumentos. main obtém name,dataset da configuração.

iterrows

Itera pares (índice, Series da linha).

Parâmetros e uso neste projeto: Sem argumentos. O pipeline ignora o índice com _ e usa observation.timestamp/water_produced para registrar cada lead.

itertuples

Itera linhas como tuplas nomeadas.

Parâmetros e uso neste projeto: Sem argumentos, inclui índice e atributos pelos nomes das colunas válidos. Na auditoria permite r.dataset, r.policy e r.origin ao identificar piores janelas.

join

Une textos com o separador que recebe o método.

Parâmetros e uso neste projeto: '\\n'.join(parts) organiza prompt; ','.join(...) organiza datas/valores; '|'.join(...) registra nomes de covariáveis. Todos os elementos devem ser strings.

json.dumps

Serializa objeto Python em string JSON.

Parâmetros e uso neste projeto: obj=value; sort_keys=True estabiliza hashes; ensure_ascii=False preserva acentos; indent=2 melhora leitura; allow_nan=False rejeita NaN/inf na persistência. Nem todos esses argumentos aparecem em toda chamada.

json.load

Decodifica JSON lido de um arquivo/response.

Parâmetros e uso neste projeto: fp é o fluxo retornado por urlopen. Diferente de loads, não recebe diretamente a string do modelo.

json.loads

Decodifica uma string JSON em objetos Python.

Parâmetros e uso neste projeto: s=text; JSON null vira None, true vira True, arrays viram listas. Pode aceitar NaN no parser padrão, por isso parse_prediction testa math.isfinite depois.

len

Conta elementos.

Parâmetros e uso neste projeto: Objeto pode ser histórico, previsões ou texto/bytes. len(prompt) não é contagem de tokens; len(history) conta linhas.

list

Materializa um iterável em lista.

Parâmetros e uso neste projeto: iterável pode ser colunas, range ou tupla; pode também representar tipo em isinstance.

map

Aplica uma função a cada valor da Series.

Parâmetros e uso neste projeto: arg=lambda value: f'{float(value):.2f}' converte somente a representação da demanda em texto com duas casas. A demanda original fora dessa cópia conserva precisão.

math.isfinite

Confere se um único número é finito.

Parâmetros e uso neste projeto: x=v; combina-se com checagem de tipo e sinal para rejeitar NaN, inf e valores negativos.

max

Retorna maior valor.

Parâmetros e uso neste projeto: max(context_start,origin-history_days) impede começo negativo; max(std,1e-8) evita divisão por zero; método de Series reduz valores.

mean

Calcula média dos valores do objeto receptor.

Parâmetros e uso neste projeto: Sem argumentos nas Series de duração/demanda e arrays de erro; o receptor já foi filtrado para a janela ou subconjunto correto.

min

Retorna menor valor.

Parâmetros e uso neste projeto: Recebe limites para validar positividade, tempos de espera ou timestamps. Método Series.min opera sobre a coluna.

mkdir

Cria diretório.

Parâmetros e uso neste projeto: parents=True cria pais ausentes; exist_ok=True aceita diretório já existente. Sem exist_ok, colisão pode lançar FileExistsError.

normalize

Zera o horário de cada timestamp, preservando sua data.

Parâmetros e uso neste projeto: Series.dt.normalize() não recebe argumentos aqui. A comparação com a coluna original rejeita horários diferentes de meia-noite.

notna

Retorna True onde o valor não é ausente.

Parâmetros e uso neste projeto: Sem argumentos. Na grade diária, define observed antes de preencher water_produced, preservando quais alvos eram reais.

np.abs

Módulo elemento a elemento.

Parâmetros e uso neste projeto: x pode ser erro ou alvo; remove o sinal para MAE/MAPE/sMAPE, mas o viés auditado mantém o sinal.

np.allclose

Exige proximidade em todos os elementos.

Parâmetros e uso neste projeto: a=diffs,b=diffs[0], atol=1e-6,rtol=0 detectam progressão aritmética tolerando arredondamento absoluto mínimo.

np.arange

Gera progressão de valores com fim exclusivo.

Parâmetros e uso neste projeto: stop único produz 0...stop-1; start,stop definem origens válidas. dtype=float nos testes força floats; passo padrão é 1.

np.argsort

Retorna índices que ordenariam os valores.

Parâmetros e uso neste projeto: a=distances; kind='stable' preserva ordem de empates. [:count] seleciona os exemplos mais semelhantes.

np.array_equal

Compara exatamente valores e formas de arrays.

Parâmetros e uso neste projeto: a/b são alvos CSV versus dataset, ou previsão CSV versus resposta. Retorna bool e não aplica tolerância.

np.asarray

Representa dados como ndarray, evitando cópia quando possível.

Parâmetros e uso neste projeto: a é o vetor/lista recebido; dtype=float converte para ponto flutuante. Não valida finitude sozinho.

np.diff

Diferenças entre elementos consecutivos.

Parâmetros e uso neste projeto: a=pred; n padrão 1. Sete previsões geram seis incrementos para detectar rampas.

np.divide

Divide vetores com máscara e destino explícitos.

Parâmetros e uso neste projeto: x1=2*abs(error), x2=denom, out=zeros_like(error), where=denom!=0. Posições de denominador zero mantêm zero; evita NaN no sMAPE quando real e previsto são zero.

np.dot

Produto interno dos vetores.

Parâmetros e uso neste projeto: a=err,b=err soma quadrados independentemente de deterministic_metrics; divisão por N e sqrt reconstroem RMSE.

np.isclose

Compara com tolerância numérica.

Parâmetros e uso neste projeto: a/b são valores; rtol=1e-12 é tolerância relativa na auditoria do RMSE; atol padrão permanece se não informado.

np.isfinite

Testa finitude elemento a elemento.

Parâmetros e uso neste projeto: x é array numérico; retorna False para NaN e infinitos. .all() transforma em condição única.

np.mean

Calcula média aritmética.

Parâmetros e uso neste projeto: a é o vetor; axis omitido considera todos os elementos. Em métricas deve receber erros já alinhados, nunca listas filtradas separadamente.

np.random.default_rng

Cria um gerador pseudoaleatório local.

Parâmetros e uso neste projeto: seed=42 torna a seleção de demonstrações repetível sem depender do estado global de np.random.

np.sqrt

Raiz quadrada elemento a elemento ou escalar.

Parâmetros e uso neste projeto: x é MSE; produz RMSE nas mesmas unidades do alvo.

np.std

Calcula desvio-padrão.

Parâmetros e uso neste projeto: a é janela histórica; ddof padrão 0. É usado para padronizar forma na seleção por similaridade, não para transformar o alvo de teste.

np.sum

Soma elementos.

Parâmetros e uso neste projeto: a é erro ao quadrado ou desvios do alvo; axis omitido soma tudo. Resultados NumPy são convertidos por float para JSON.

np.testing.assert_allclose

Asserção numérica vetorial tolerante.

Parâmetros e uso neste projeto: actual e desired são resultados e referência; usa tolerâncias padrão se não especificadas. Lança AssertionError e é usada para testar métricas e fidelidade do prompt.

np.zeros_like

Cria zeros com formato e tipo semelhantes ao array.

Parâmetros e uso neste projeto: a=error fornece o formato do destino de divide. Não altera error.

nunique

Conta valores distintos.

Parâmetros e uso neste projeto: Sem argumentos. history_start.nunique()==1 verifica que todas as janelas começam na mesma data; origin.nunique() conta janelas.

object.__new__

Cria instância sem executar seu __init__.

Parâmetros e uso neste projeto: cls é QwenForecaster ou ChronosForecaster. Exclusivo dos testes para evitar servidor/download/GPU; é preciso preencher atributos manualmente depois.

parse_args

Interpreta argumentos do processo e retorna Namespace.

Parâmetros e uso neste projeto: Sem args explícitos, lê sys.argv[1:]. main acessa args.model/config/dataset/output/limit.

patch

Substitui temporariamente uma referência durante o bloco with.

Parâmetros e uso neste projeto: target é nome completo no módulo consumidor; side_effect=answers entrega respostas sequenciais; patch(time.sleep) evita esperas nos testes. Não altera permanentemente o arquivo-fonte.

pd.DataFrame

Constrói uma tabela a partir de registros ou colunas.

Parâmetros e uso neste projeto: data posicional pode ser lista de dicts (linhas) ou dict de listas (colunas); os comprimentos devem ser compatíveis. Colunas escalares em testes são propagadas pelo índice definido pelas demais.

pd.DatetimeIndex

Constrói um índice temporal para comparação.

Parâmetros e uso neste projeto: data é a coluna timestamp futura; é comparada com dates para exigir as mesmas datas e ordem.

pd.Timedelta

Representa uma duração temporal.

Parâmetros e uso neste projeto: days=1 adiciona um dia à última observação; essa expressão desloca a primeira data prevista para o dia seguinte.

pd.concat

Concatena tabelas.

Parâmetros e uso neste projeto: objs=[história, futuro] e axis padrão 0 empilham linhas; ignore_index=True renumera o índice. Aqui só concatena covariáveis, nunca alvos futuros.

pd.date_range

Produz uma sequência ordenada de timestamps.

Parâmetros e uso neste projeto: start é a primeira data; periods é a quantidade, não a data final; freq='D' incrementa um dia por elemento.

pd.read_csv

Lê CSV em um DataFrame, inferindo tipos; datas só são convertidas posteriormente no validador.

Parâmetros e uso neste projeto: Primeiro argumento: caminho Path/string ou buffer StringIO. Nesta base não há parse_dates na leitura; timestamp inicialmente pode ser texto.

pd.testing.assert_frame_equal

Verifica igualdade detalhada entre dois DataFrames e lança AssertionError se diferirem.

Parâmetros e uso neste projeto: left/right são tabelas com colunas de alinhamento ou covariáveis; o código usa padrões da biblioteca para checar também índices e tipos. É uma asserção, não retorna uma tabela corrigida.

pd.to_datetime

Converte valores em datas pandas.

Parâmetros e uso neste projeto: arg é a coluna/lista de datas; errors='raise' interrompe se algo não puder ser interpretado. Não inventa datas para entradas inválidas.

pd.to_numeric

Converte texto/objetos em números.

Parâmetros e uso neste projeto: arg é a Series; errors='raise' rejeita texto não numérico, em vez de convertê-lo silenciosamente para NaN.

pop

Remove e devolve valor de chave.

Parâmetros e uso neste projeto: key='qwen_history_days' é removida da cópia da configuração antes de comparar os outros campos. Sem default, chave ausente lança KeyError.

predict

Interface comum dos adaptadores do projeto.

Parâmetros e uso neste projeto: history é tabela histórica; horizon é quantidade prevista; future_covariates é tabela sem alvo; prompt é texto já construído para Qwen. Retorna lista de floats; cada classe escolhe backend.

predict_df

Chronos: prevê uma tabela temporal com covariáveis.

Parâmetros e uso neste projeto: frame é histórico; future_df é tabela futura sem alvo; prediction_length=horizon; freq='D'; batch_size=1; id_column, timestamp_column e target identificam colunas; quantile_levels=[0.1,0.5,0.9]; context_length=len(frame) pede todo o histórico. Retorno é DataFrame; coluna '0.5' é mediana.

print

Escreve representação textual na saída padrão.

Parâmetros e uso neste projeto: flush=True força envio imediato ao log de progresso; index=False pertence a to_string, não a print.

query

Filtra linhas por expressão textual pandas.

Parâmetros e uso neste projeto: expr contém nomes de colunas; @dataset/@policy referem-se às variáveis Python do escopo. Não é consulta SQL nem eval arbitrário de dados do modelo.

range

Gera inteiros com limite final exclusivo.

Parâmetros e uso neste projeto: start, stop, step definem origens; range(retries+1) inclui tentativa inicial e retentativas; range(1,8) gera sete leads.

read_bytes

Lê bytes integrais do arquivo.

Parâmetros e uso neste projeto: Sem argumentos. O hash do dataset usa bytes originais, incluindo quebras de linha.

read_text

Lê todo o arquivo como string.

Parâmetros e uso neste projeto: Sem encoding explícito, usa a configuração padrão do ambiente. JSON e prompts esperam texto compatível; não retorna dict sem json.loads.

relative_to

Calcula caminho relativo a um ancestral de um Path.

Parâmetros e uso neste projeto: other=ROOT; o resultado entra no hash de fonte para considerar nome/local do arquivo sem depender do prefixo absoluto da máquina.

rename_axis

Nomeia o eixo do índice.

Parâmetros e uso neste projeto: mapper='timestamp' permite que reset_index restaure uma coluna com esse nome.

replace

Substitui o destino pelo arquivo representado pelo Path.

Parâmetros e uso neste projeto: target=path final. No write_json, fonte e destino estão no mesmo diretório; reduz chance de deixar JSON parcial se o processo cair. Não é bloqueio contra dois escritores simultâneos.

reset_index

Transforma o índice atual em coluna ou o descarta e cria índice sequencial.

Parâmetros e uso neste projeto: drop=True descarta o índice anterior na ordenação/comparação; sem drop, a data do índice volta a ser coluna após asfreq.

resolve

Resolve caminho absoluto e links simbólicos.

Parâmetros e uso neste projeto: Sem argumentos aqui. Path(__file__).resolve().parents[2] encontra a raiz a partir do módulo.

rglob

Percorre recursivamente caminhos compatíveis com padrão.

Parâmetros e uso neste projeto: pattern='*.py' encontra fontes dentro de src; sorted fixa a ordem antes de compor o hash.

set

Cria conjunto de elementos únicos.

Parâmetros e uso neste projeto: iterável são colunas/chaves. Subtração detecta ausentes, interseção detecta nomes proibidos; não preserva ordem como contrato.

set_index

Transforma coluna em índice do DataFrame.

Parâmetros e uso neste projeto: keys='timestamp'; drop padrão True remove a coluna enquanto ela é índice. É necessário para asfreq temporal.

slice

Representa intervalo, não materializa dados.

Parâmetros e uso neste projeto: start é inclusivo e stop exclusivo; df.iloc[slice(...)] efetua a seleção depois.

sort_values

Ordena linhas pelos valores de colunas.

Parâmetros e uso neste projeto: by é 'timestamp', 'rmse' ou ['origin','lead']; ascending=False coloca erros maiores primeiro. Não modifica inplace nestas chamadas: devolve um novo objeto.

sorted

Devolve uma lista ordenada.

Parâmetros e uso neste projeto: iterável são caminhos, origens ou leads; key/reverse alteram critério/ordem quando fornecidos. Estabilidade ajuda hash e ordem de demonstrações.

split

Divide uma string por separador.

Parâmetros e uso neste projeto: sep='\\nEND HISTORY:' ou marcador CSV identifica segmentos do prompt no teste; a indexação [0]/[1] escolhe trecho esperado.

startswith

Testa prefixo textual exato.

Parâmetros e uso neste projeto: prefix é a parte histórica do prompt anterior mais uma quebra de linha. Confirma que o novo prompt expansivo começa com os mesmos bytes textuais.

str

Produz representação textual.

Parâmetros e uso neste projeto: x pode ser exceção, timestamp ou caminho. Usado em logs/hashes; não faz validação de conteúdo.

strftime

Formata datas como texto.

Parâmetros e uso neste projeto: format='%Y-%m-%d' produz ano-mês-dia e é usado em nomes de arquivos, datas dos prompts e CSVs. Pode ser método de Timestamp ou de índice/Series.dt.

strip

Remove espaços e quebras nas bordas do texto.

Parâmetros e uso neste projeto: Sem argumentos. Tira a quebra final de to_csv para manter o layout previsível, sem alterar os campos internos.

subTest

Identifica um caso dentro de um loop de teste.

Parâmetros e uso neste projeto: text=text rotula a resposta inválida corrente para diagnóstico individual.

sum

Soma elementos de iterável.

Parâmetros e uso neste projeto: No builtin, conta condições True como 1; no método de Series soma duração ou máscara. Não calcula média por si só.

tail

Seleciona as últimas n linhas.

Parâmetros e uso neste projeto: n=len(h) reconstrói o histórico restrito; n=7 calcula a média dos sete dias recentes. Não inclui futuro, pois o objeto já é história.

tempfile.TemporaryDirectory

Cria diretório temporário gerenciado.

Parâmetros e uso neste projeto: Sem argumentos; with remove ao sair. Permite simular resultados sem sobrescrever experimentos reais.

time.monotonic

Relógio crescente para medir duração.

Parâmetros e uso neste projeto: Sem argumentos. A diferença final-inicial mede segundos transcorridos, sem depender da data civil.

time.sleep

Suspende o processo por uma duração.

Parâmetros e uso neste projeto: secs=min(2**attempt,4) aplica espera de 1,2,4... segundos entre tentativas, limitada a quatro.

timesfm.ForecastConfig

Configura os limites e opções de inferência do TimesFM.

Parâmetros e uso neste projeto: max_context arredonda histórico para múltiplo de 32; max_horizon arredonda horizonte para múltiplo de 128. normalize_inputs normaliza internamente; use_continuous_quantile_head ativa cabeça contínua; force_flip_invariance estende invariância de sinal; infer_is_positive preserva não negatividade para entrada não negativa; fix_quantile_crossing corrige cruzamentos; return_backcast disponibiliza estimativas históricas usadas no caminho com regressão.

to_csv

Serializa DataFrame como CSV.

Parâmetros e uso neste projeto: path_or_buf ausente retorna string; um Path grava arquivo. index=False evita coluna extra de índice; float_format='%.6g' usa seis algarismos significativos. Não muda o DataFrame original.

to_numpy

Converte Series/DataFrame em array NumPy.

Parâmetros e uso neste projeto: dtype=float força formato numérico quando fornecido; sem dtype conserva tipo inferido. Não ordena linhas e não corrige alinhamento.

to_string

Formata tabela como texto para terminal/log.

Parâmetros e uso neste projeto: index=False omite índice auxiliar, conservando colunas. Não grava CSV.

tolist

Converte array/índice/Series em lista Python.

Parâmetros e uso neste projeto: Sem argumentos. É usado para serialização e para compatibilizar o retorno dos modelos com o pipeline.

torch.cuda.is_available

Consulta disponibilidade do backend CUDA.

Parâmetros e uso neste projeto: Sem argumentos. Seleciona device_map='cuda' ou 'cpu' para Chronos; não instala drivers.

torch.manual_seed

Define semente do gerador PyTorch.

Parâmetros e uso neste projeto: seed=42. Ajuda reprodutibilidade, mas não habilita sozinho todos os modos determinísticos de GPU.

torch.set_float32_matmul_precision

Configura precisão permitida para multiplicações de matrizes float32.

Parâmetros e uso neste projeto: precision='high' escolhe política do backend para acelerar sob tolerância; não muda a dtype declarada de todos os tensores.

tuple

Materializa sequência imutável.

Parâmetros e uso neste projeto: iterável é histórico/alvo de demonstração. A imutabilidade da tupla ajuda o contrato da dataclass.

type

Obtém a classe concreta do valor.

Parâmetros e uso neste projeto: type(v) not in (int,float) rejeita bool apesar de herdar de int; type(exc).__name__ registra nome da exceção.

unittest.main

Executa testes quando o arquivo é ponto de entrada.

Parâmetros e uso neste projeto: Sem argumentos no código; discover também pode localizar as classes sem executar esse bloco.

update

Acrescenta conteúdo ao objeto mutável.

Parâmetros e uso neste projeto: Em hash.update(data), data são bytes incorporados ao digest; em dict.update(...), argumentos nomeados adicionam/substituem campos como status e audit. O receptor distingue as duas APIs.

urllib.request.Request

Monta uma requisição HTTP.

Parâmetros e uso neste projeto: url é endpoint; data=json.dumps(payload).encode('utf-8') é corpo em bytes e induz POST; headers declara Content-Type application/json. Não envia a rede até urlopen.

urllib.request.urlopen

Abre conexão HTTP e devolve resposta como fluxo.

Parâmetros e uso neste projeto: url ou Request é primeiro argumento; timeout em segundos limita operações de espera da conexão. with fecha a resposta. GET ocorre quando recebe URL sem corpo, como /api/tags.

values

Percorre valores do dicionário.

Parâmetros e uso neste projeto: Sem argumentos. Comparador percorre procedências; testes percorrem datasets/covariáveis.

with_suffix

Produz outro Path trocando a extensão.

Parâmetros e uso neste projeto: suffix='.tmp' transforma run.json em run.tmp, usado antes da substituição atômica.

write_text

Grava uma string em arquivo e retorna a quantidade de caracteres escrita.

Parâmetros e uso neste projeto: data é a string; não cria pais automaticamente. Nestes usos, encoding não é explícito. Substitui conteúdo anterior.

zip

Emparelha iteráveis pela posição.

Parâmetros e uso neste projeto: truth.iterrows() e previsões precisam da mesma ordem. zip padrão para no menor comprimento, então tamanho é validado antes.

Como executar e localizar o resultado

```
bash experiments/start_ollama_context_server.sh
```

Em um terminal, inicia o servidor local separado; use ambientes já disponíveis, sem instalá-los ou atualizá-los.

```
conda run -n promptcast python -m src.pipelines.forecasting_pipeline \
  --model qwen --config configs/qwen_context_expanding.json \
  --output experiments/qwen_context_comparison/expanding
```

Executa Qwen3 4B expansivo nas duas cidades. Reutiliza somente caches com identidade exata. Não execute duplicatas escrevendo na mesma pasta.

```
conda run -n promptcast python -m src.pipelines.compare_qwen_contexts \
  --output experiments/qwen_context_comparison
```

Depois dos braços 15_days e expanding completos, gera métricas e comparação controlada.

```
conda run -n promptcast python -m unittest discover -s tests -v
```

Executa testes de contrato sem baixar modelos; testes mockados não substituem execução real do Ollama.

```
PYTHONPATH=. conda run -n promptcast python \
  experiments/qwen_context_comparison/audit/audit_run.py
```

Audita respostas salvas e escreve checks/windows/summary na pasta audit. Não faz uma nova inferência.

O ambiente selecionado precisa conter dependências já instaladas. O usuário exige consulta antes de modificar ambientes conda. O comando instalado qwen_forecast depende de instalação do pacote naquele ambiente; python -m funciona pela localização de src quando executado a partir da raiz.

Para estudar, não é necessário executar novamente centenas de janelas. Abra uma resposta em responses, compare audit.request.prompt com o .txt correspondente e associe predictions[i] à linha lead=i+1 do CSV. Para mudar código, use validação anterior ao teste; não escolha uma formulação de prompt por desempenho em janelas finais já inspecionadas.

Cobertura, fontes e reprodução do documento

Foram incluídos 28 arquivos de código/configuração/script e 10 inicializadores vazios. Todas as 1384 linhas físicas estão cobertas: linhas com conteúdo recebem código e anotação; linhas em branco são separadores registrados no manifesto. A extração usa os arquivos reais, não trechos reescritos.

A referência primária é o próprio repositório. Para semântica específica de TimesFM, também foi consultada a implementação instalada de timesfm/configs.py no ambiente timesfm, incluindo a docstring de ForecastConfig. Bibliotecas possuem versões instaladas distintas por ambiente; os parâmetros documentados correspondem às chamadas deste código, não a uma promessa para qualquer versão futura.

Arquivos gerados: guia_qwen_forecast.pdf, guia_qwen_forecast.html, guide_source/manifest.json e guide_source/line_coverage.json. O conteúdo editorial e o gerador estão em guide_source/content.py e guide_source/build_guide.py, fora de src para não invalidar hashes de inferência. O gerador documenta o produto, não integra sua execução.

Para reconstruir neste ambiente, execute python experiments/reports/guide_source/build_guide.py com o Python de sistema, que já possui ReportLab. Isso não altera ambientes conda. O manifesto é a referência para confirmar a qual versão de cada arquivo uma explicação pertence.